

**HashiCorp Certified Terraform Associate 2026**

# PPT Version

PPT Release Date = 4th January 2026

We regularly release new version of PPT when we update this course.

Please check regularly that you are using the latest version.

The Latest Version Details are mentioned in the PPT Lecture in Section 1.

# Understanding the Need

My personal journey with Terraform began with implementing the “AWS Hardening” guidelines.

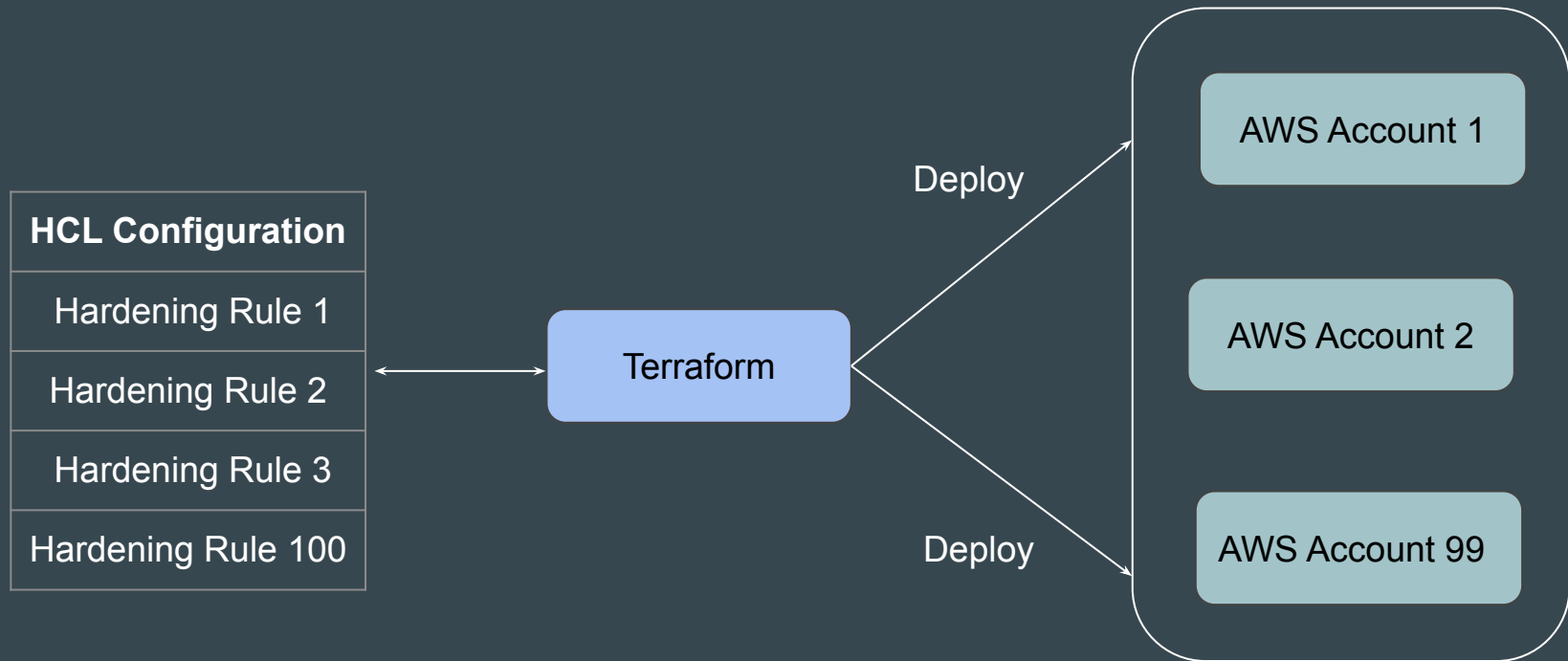
Back then, the guidelines spanned over 100 pages and required 2-3 days to apply to a single account.

Today, these guidelines have grown to more than 250 pages.



# Challenge that Terraform Solves

Terraform allows us to create reusable code that can deploy an identical set of infrastructure resources in a repeatable fashion.



# Amazing Terraform

One of the great benefits of Terraform is that it supports thousands of providers.

Once you learn Terraform Core concepts, you can write code to create and manage infrastructure across all the providers.



# Overview of Terraform Certification

Terraform has become one of the most popular and widely used tools for creating and managing infrastructure, and a de facto IaC tool for DevOps.

HashiCorp has introduced an official Terraform certification designed to validate students' knowledge and skills in core Terraform concepts.



# What Does this Course Cover?

We start this Terraform course from absolute scratch. Once core concepts are covered, we move on to advanced topics.

We cover *ALL* the topics of the official exams.



# Long Term Vision

Due to immense popularity, HashiCorp has also released a Terraform Professional certification.

A long, difficult, practical-based exam of 4 hours

We launched the first Terraform Professional certification course in the world.





# About Me

- DevSecOps Engineer - Defensive Security.
- Teaching is one of my passions.
- I have total of 17 courses, and around 440,000+ students now.

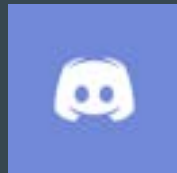
## Something about me :-



- HashiCorp Certified Terraform [Associate and Professional]
- Certified Ethical Hacker
- AWS Certified [DevOps Pro, SA Pro, Advanced Networking, Security Specialty ...]
- RedHat Certified Architect (RHCA) + 13 more Certifications
- Part time Security Consultant

# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)



[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **Exploring Exam BluePrint**

# Understanding the Basics

This is a **certification specific course** and we cover all the pointers that are part of the official exam blueprint.

| Exam Objectives |                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------|
| <b>1</b>        | <b>Infrastructure as Code (IaC) with Terraform</b>                                      |
| 1a              | Explain what IaC is                                                                     |
| 1b              | Describe the advantages of IaC patterns                                                 |
| 1c              | Explain how Terraform manages multi-cloud, hybrid cloud, and service-agnostic workflows |
| <b>2</b>        | <b>Terraform fundamentals</b>                                                           |
| 2a              | Install and version Terraform providers                                                 |
| 2b              | Describe how Terraform uses providers                                                   |
| 2c              | Write Terraform configuration using multiple providers                                  |
| 2d              | Explain how Terraform uses and manages state                                            |

## Point to Note

The arrangement of topics in this course is a little different from the exam blueprint to ensure this course remains beginner friendly and topics are covered in a step by step manner.

# **About the Course and Important Resources**

# Understanding the Basics

This is a **certification specific course** and we cover all the pointers that are part of the official exam blueprint.

| Certification   |                                                    | Overview | Certifications |
|-----------------|----------------------------------------------------|----------|----------------|
| Exam objectives |                                                    |          |                |
| 1               | Understand infrastructure as code (IaC) concepts   |          |                |
| 1a              | Explain what IaC is                                |          |                |
| 1b              | Describe advantages of IaC patterns                |          |                |
| 2               | Understand the purpose of Terraform (vs other IaC) |          |                |
| 2a              | Explain multi-cloud and provider-agnostic benefits |          |                |
| 2b              | Explain the benefits of state                      |          |                |

## 2 Choice for Instructor

1. Cover Theory and Basic Demos -- Course Length Maintained
2. Cover Practically -- Course Length will be Longer





# Our Choice

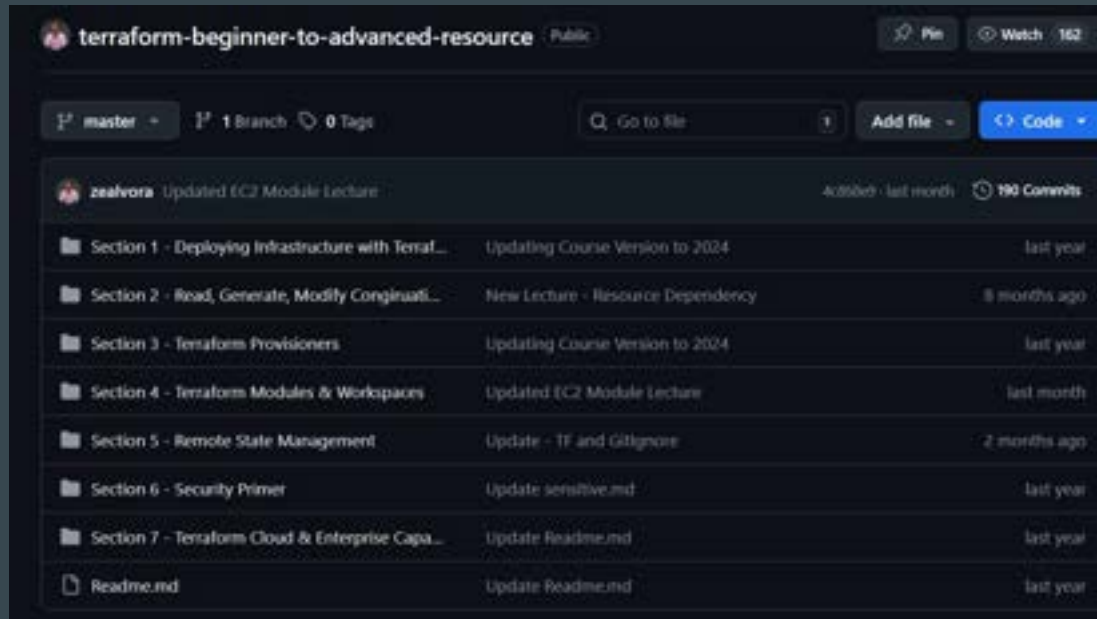
This course follows practical based approach.

The course will be lengthy but worth it.



# Course Resource - GitHub

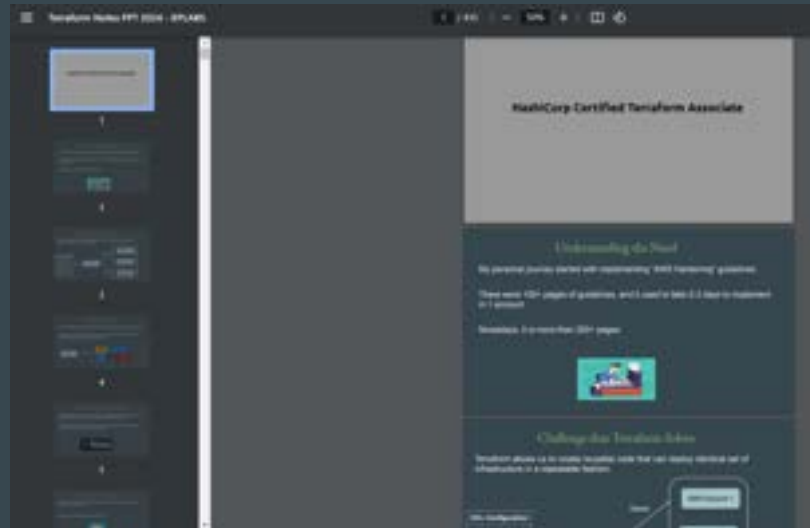
All the code that we use during practicals have been added to our GitHub page.



# Course Resource - PPT Slides

All the slides we use in this course are available to download as PDFs.

The PDF is attached as part of the lecture titled “Central PPT Notes”.



# Our Community (Optional)

We also have a Discord community where individuals preparing for the same certification can connect, discuss, and seek technical support.

<https://kplabs.in/chat>

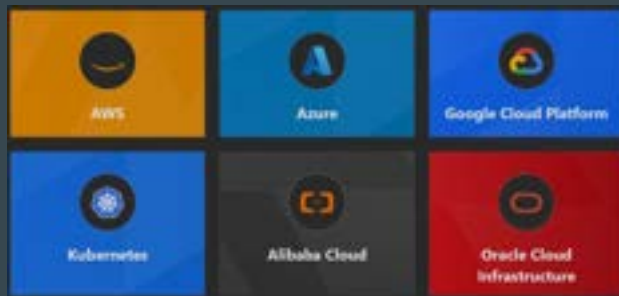


# Important Note - Platform for This Course

Terraform supports hundreds of platforms like AWS, Azure, GCP, etc.

To learn Terraform concepts, we have to choose 1 platform for our testing.

For this course, we have chosen AWS.



# Clarification Regarding AWS Platform

Aim of this course is to learn Core Concepts of Terraform and not AWS.

We use very basic AWS services like Virtual Machine, AWS users to demonstrate and Learn the Core Terraform concepts.

The Terraform structure and concepts remain SAME irrespective of platform.

We have hundreds of students from diverse platform backgrounds, such as Azure and GCP, who have completed this course and are actively implementing Terraform across platforms.

# **Infrastructure as Code (IAC)**

# Understanding the Basics

There are two ways in which you can create and manage your infrastructure:

- Manually approach.
- Through Automation





# Work Requirement: Database Backup

I was assigned a task to take database backup every day at 10 PM and the backup had to be stored in Amazon S3 Storage with appropriate timestamp.

- db-backup-01-01-2024.sql
- db-backup-02-01-2024.sql

Initially due to lack of time, I used to manually take DB backup at 10 PM and upload it to S3.



# Learning from this Work Requirement

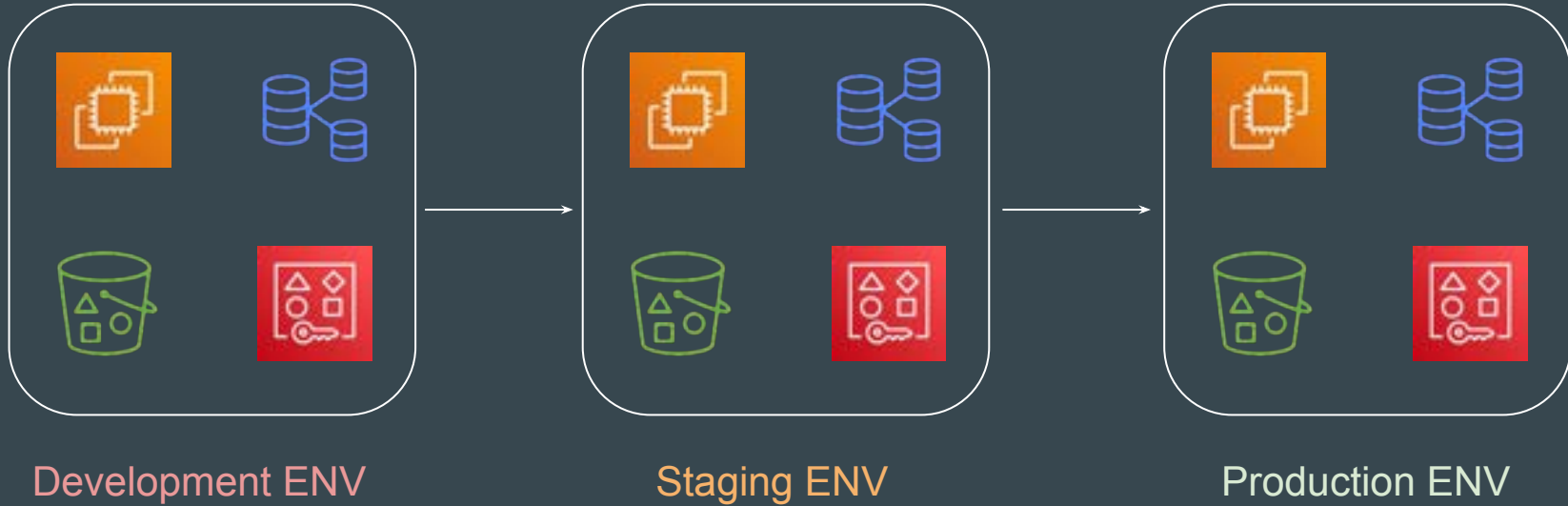
If a particular task has to be done in an repeatable manner, it **MUST** be automated.

## Points to Note:

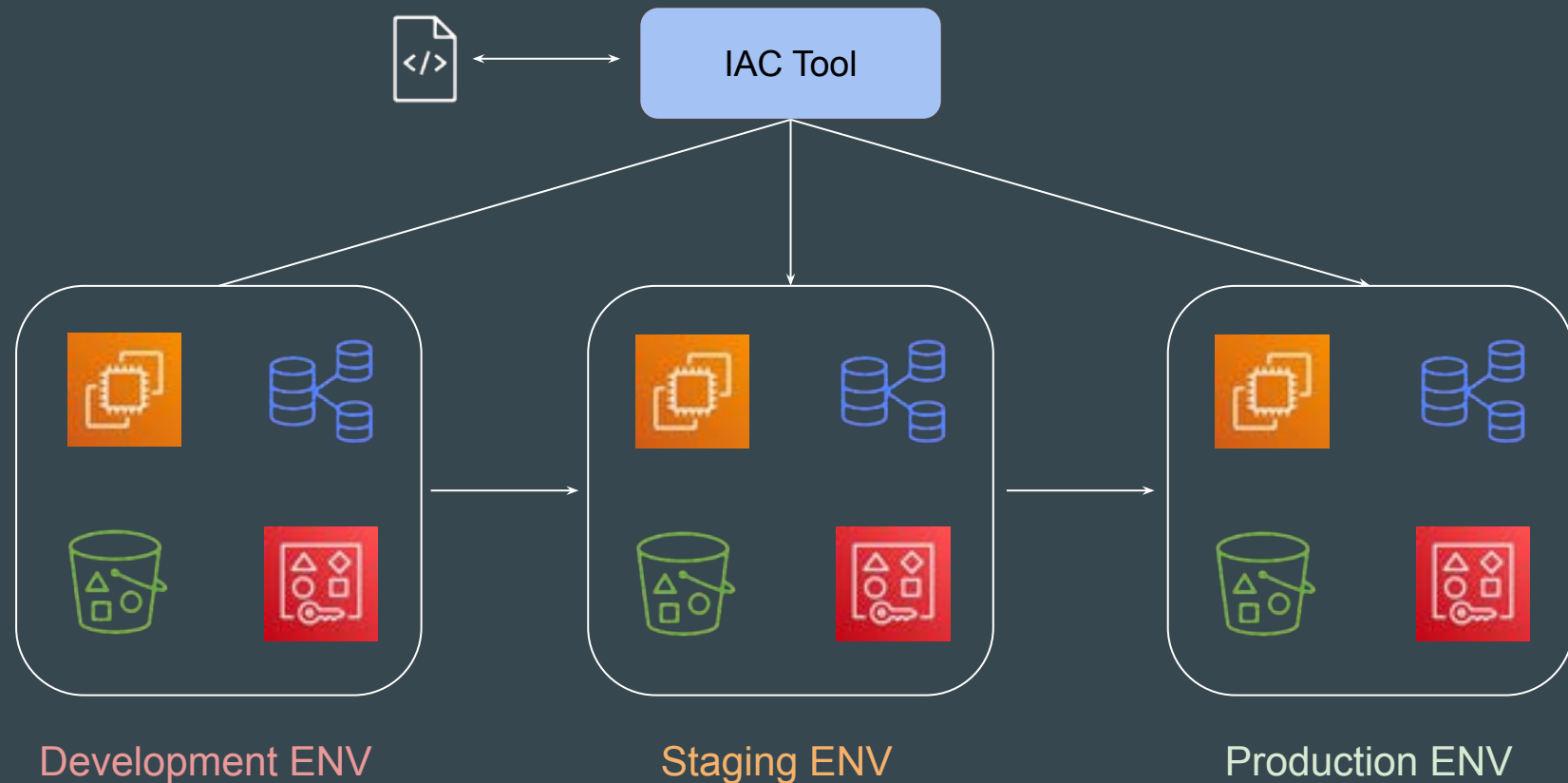
1. Depending on the type of task, the tools for automation will change.
2. There are wide variety of Tools & Technologies used for Automation like Ansible, CloudFormation, Terraform, Python etc.

# Example of a Single Service

Set of resources (Virtual Machine, Database, S3, AWS Users) must be created with exact similar configuration in Dev, Stage and Production environment.



# Example of a Single Service - Automated Way



# Basics of Infrastructure as Code

Infrastructure as Code (IaC) is the managing and provisioning of infrastructure through code instead of through manual processes.

```
! test.yaml
AWS::CloudFormation::Template
  AWSTemplateFormatVersion: 2010-09-09
  Description: Simple EC2

  Resources:
    WebAppInstance:
      Type: AWS::EC2::Instance
      Properties:
        ImageId: ami-079db87dc4c10ac91
        InstanceType: t2.micro
```

```
vm.tf > ...
resource "aws_instance" "myec2" {
  ami = "ami-079db87dc4c10ac91"
  instance_type = "t2.micro"
}
```

# Benefits of Infrastructure As Code

There are several benefits of designing your infrastructure as code:

- Speed of Infrastructure Management.
- Low Risk of Human Errors.
- Version Control.
- Easy collaboration between Teams.

# **Choosing Right IAC Tool**

# Available Tools

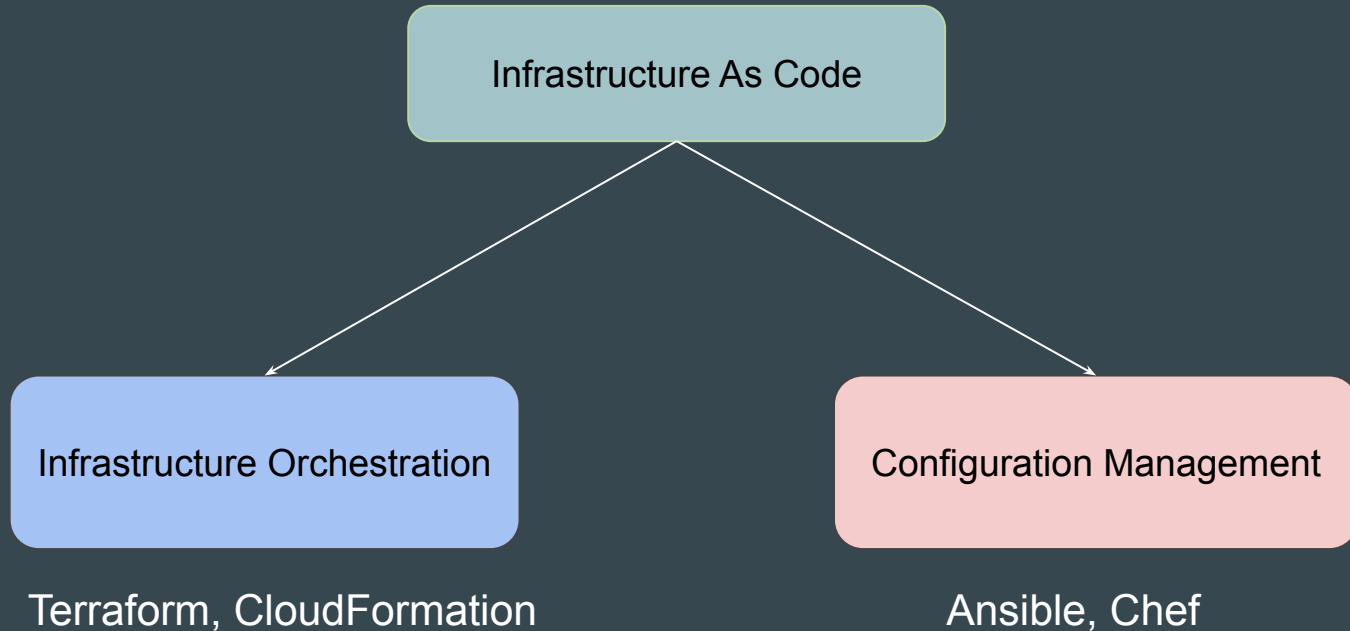
There are various types of tools that can allow you to deploy infrastructure as code :

- Terraform
- CloudFormation
- Heat
- Ansible
- SaltStack
- Chef, Puppet and others



# Categories of Tools

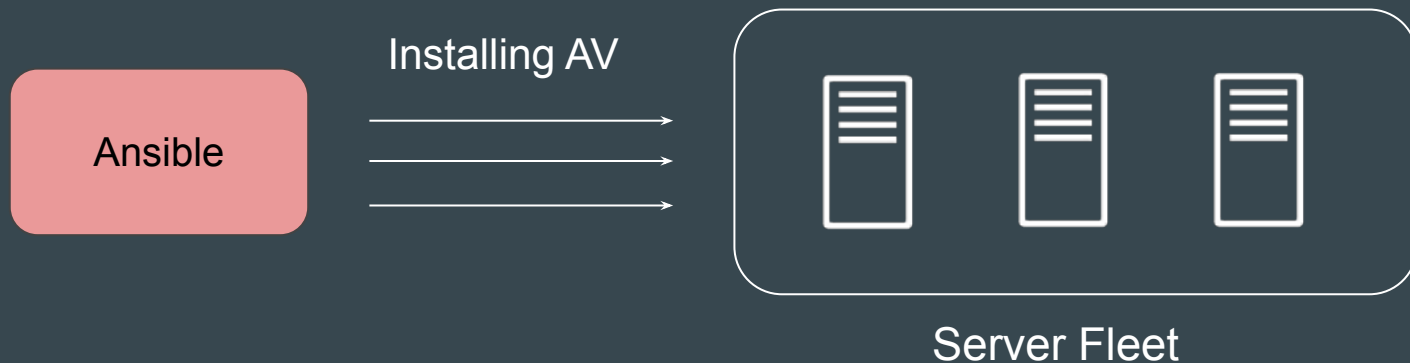
The tools are widely divided into two major categories



# Configuration Management

Configuration Management tools are primarily used to maintain desired configuration of systems (inside servers)

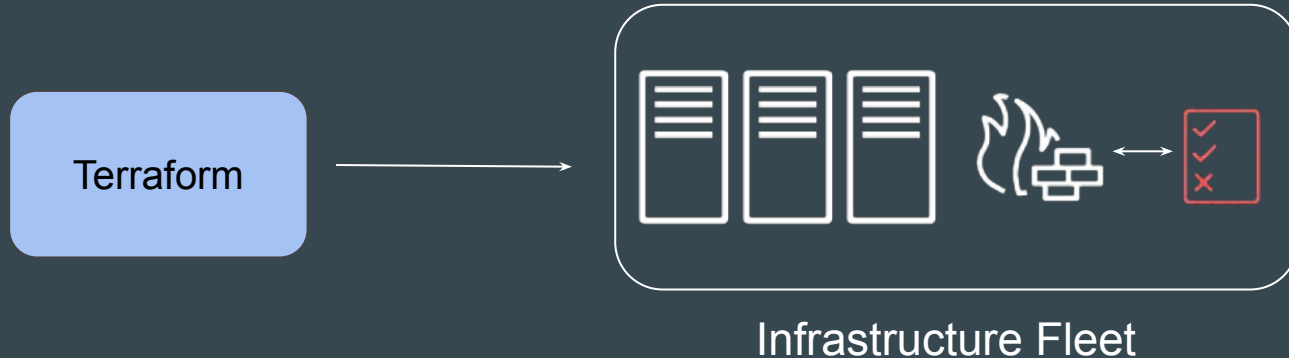
Example: ALL servers should have Antivirus installed with version 10.0.2



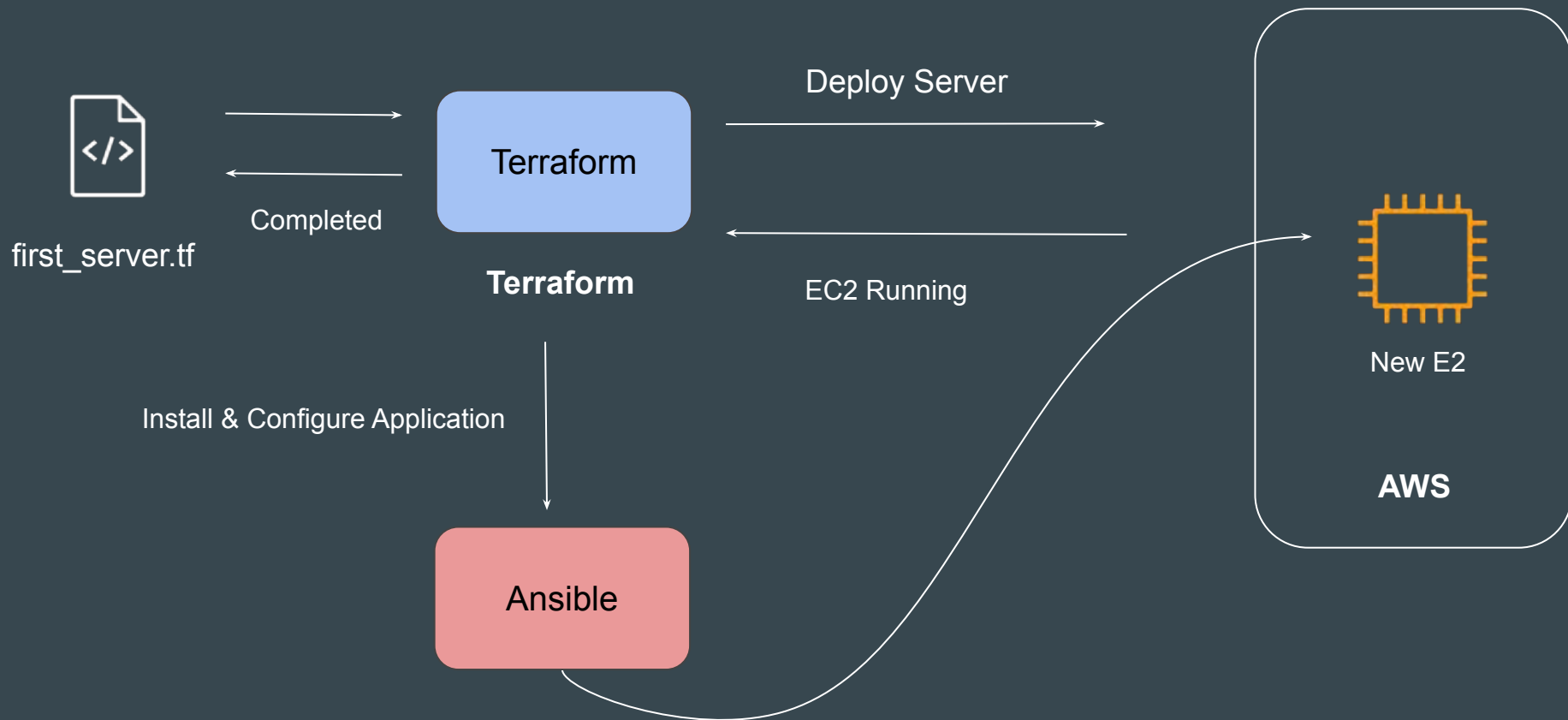
# Infrastructure Orchestration

Infrastructure Orchestration is primarily used to create and manage infrastructure environments.

Example: Create 3 Servers with 4 GB RAM, 2 vCPUs. Each server should have firewall rule to allow SSH connection from Office IPs.



# IAC & Configuration Management = Friends



# How to choose IAC Tool?

- i) Is your infrastructure going to be vendor specific in longer term ? Example AWS.
- ii) Are you planning to have multi-cloud / hybrid cloud based infrastructure ?
- iii) How well does it integrate with configuration management tools ?
- iv) Price and Support

# Use-Case 1 - Requirement of Organization 1

1. Organization is going to be based on AWS for next 25 years.
2. Official support is required in-case if team face any issue related to IAC tool or code itself.
3. They want some kind of GUI interface that supports automatic code generation.

## Use-Case 2 - Requirement of Organization 2

1. Organization is based on Hybrid Solution. They use VMware for on-premise setup; AWS, Azure and GCP for Cloud.
2. Official support is required in-case if IAC tool has any issues.

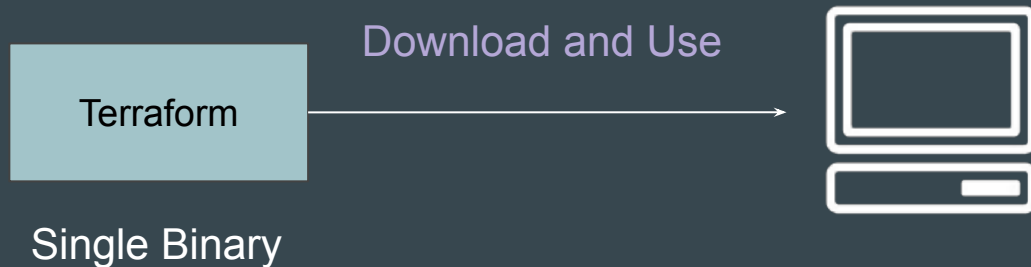
# Installing Terraform



# Overview of Installation Process

Terraform installation process is very simple.

You have a single binary file, download and use it.



# Supported Operating Systems

Terraform supports major operating systems including:

- Windows
- macOS
- Linux
- FreeBSD
- OpenBSD
- Solaris

## Point to Note

If you are manually downloading the Terraform binary rather than installing it via a package manager, you will need to ensure that the directory containing the Terraform executable is included in your **system's PATH environment variable**.

---

# Choosing IDE For Terraform

Terraform in detail

---

# Terraform Code in Notepad!

You can write Terraform code in Notepad and it will not have any impact.

## Downsides:

- Slower Development
- Limited Features



HashiCorp

# Terraform



```
Notepad - Notepad
File Edit Format View Help
variable "elb_names" {
  type = list
  default = ["dev-loadbalancer"]
}

resource "aws_iam_user" "lb" {
  name = var.elb_names[count.index]
  count = 2
  path = "/system/"
}
```

# Need of a Better Software

There is a need of a better application that allows us to develop code faster.



```
test.tf — C:\Users\Zeal Vora\Desktop\samp-code — Atom

Project
└─ samp-code
  └─ .terraform
     ├── terraform.lock.hcl
     ├── kms-use-case-01.json
     ├── kms-use-case-02.json
     ├── kms-use-case-03.json
     ├── terraform.tfstate
     ├── terraform.tfstate.backup
     └── test.tf

test.tf
1  variable "elb_names" {
2      type = list
3      default = ["dev-loadbalancer"]
4  }
5
6  resource "aws_iam_user" "lb" {
7      name = var.elb_names[count.index]
8      count = 2
9      path = "/system/"
10 }
```

# What are the Options!

There are many popular source code editors available in the market.

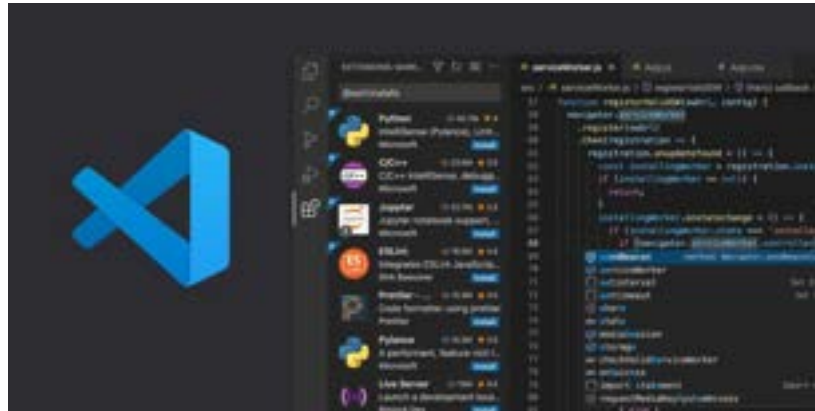


# Editor for This Course

We are going to make use of [Visual Studio Code](#) as primary editor in this course.

## Advantages:

1. Supports Windows, Mac, Linux
2. Supports Wide variety of programming languages.
3. Many Extensions.







Using  
Notepad

---



Using Visual  
Studio Code

# **Visual Studio Code Extensions**

# Understanding the Basics

Extensions are add-ons that allow you to customize and enhance your experience in Visual Studio by adding new features or integrating existing tools

They offer wide range of functionality related to colors, auto-complete, report spelling errors etc.



# Terraform Extension

HashiCorp also provides extension for Terraform for Visual Studio Code.

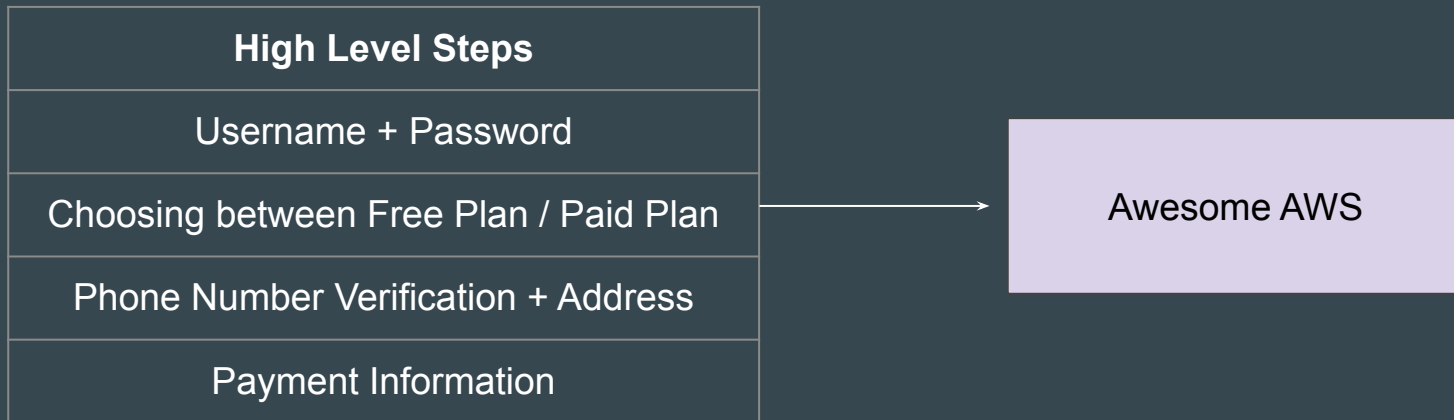
```
demo.tf
resource "aws_instance" "myec2" {
  ami = "ami-00c39f71452c08778"
  instance_type = "t2.micro"
}
```

```
demo.tf > ...
resource "aws_instance" "myec2" {
  ami = "ami-00c39f71452c08778"
  instance_type = "t2.micro"
}
```

# **AWS Account Sign-Up Process**

# Signing Up for AWS Account

Creating a new AWS account is a simple and straightforward process.



# Setting the Base

When you sign up for your AWS account, you can choose between Free plan or Paid account plan.

### Choose your account plan



**Free (6 months)**  
Learn, experiment, and build prototypes

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✗ Workloads scale beyond credit thresholds
- ✗ Access to all AWS services and features

ⓘ After the 6 month free period or when all credits are used, you can choose to upgrade to a paid plan. Otherwise, your account closes automatically.

Choose free plan



**Paid**  
Develop production-ready workloads

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✓ Workloads scale beyond credit thresholds
- ✓ Access to all AWS services and features

ⓘ After all of your credits are used, you are charged using pay-as-you-go pricing.

Choose paid plan

# Free Plan vs Paid Plan

| Free Plan                                                                   | Paid Plan                                                                   |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Receive USD \$100 sign up credit and earn up to \$100 in additional credits | Receive USD \$100 sign up credit and earn up to \$100 in additional credits |
| Access to Always free services                                              | Access to Always free services and short-term trial offers                  |
| Access to select AWS services and features                                  | Access to all AWS services and features                                     |
| No charges incur during usage                                               | Pay for charges that exceed the credit balance                              |
| Account closes when credits are depleted or when the plan duration ends     | Account doesn't close when credits are depleted                             |
| Not eligible for other promotional credits and discounts                    | Eligible for other promotional credits and discounts                        |



# Reference Screenshot - Email Address



**Try AWS at no cost for up to 6 months**

Start with USD \$100 in AWS credits, plus earn up to USD \$100 by completing various activities.



## Sign up for AWS

Root user email address  
Used for account recovery and as described in the [AWS Privacy Notice](#)

AWS account name  
Choose a name for your account. You can change this name in your account settings after you sign up.

**Verify email address**

OR

**Sign in to an existing AWS account**

This site uses essential cookies. See our [Cookie Notice](#) for more information.

# Reference Screenshot - Password

## Try AWS at no cost for up to 6 months

Start with USD \$100 in AWS credits, plus earn up to USD \$100 by completing various activities.



## Sign up for AWS

### Create your password

✔ It's you! Your email address has been successfully verified. ✕

Your password provides you with sign in access to AWS, so it's important we get it right.

Root user password

••••••••

Confirm root user password

••••••••

☐ Show password

Matches

Continue (step 1 of 5)

OR

Sign in to an existing AWS account

This site uses essential cookies. See our [Cookie Notice](#) for more information.

# Reference Screenshot - Plans

**Choose your account plan**



**Free (6 months)**  
Learn, experiment, and build prototypes

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✗ Workloads scale beyond credit thresholds
- ✗ Access to all AWS services and features

ⓘ After the 6 month free period or when all credits are used, you can choose to upgrade to a paid plan. Otherwise, your account closes automatically.

Choose free plan



**Paid**  
Develop production-ready workloads

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✓ Workloads scale beyond credit thresholds
- ✓ Access to all AWS services and features

ⓘ After all of your credits are used, you are charged using pay-as-you-go pricing.

Choose paid plan

# Reference Screenshot - Contact Information

### Earn additional AWS credits

Complete various activities to earn up to an additional USD \$100 in credits, such as creating your first AWS budget to monitor cloud costs.



## Sign up for AWS

### Contact Information

How do you plan to use AWS?

☐ Business - for your work, school, or organization

☐ Personal - for your own projects

Who should we contact about this account?

Full Name

Country Code Phone Number

 +1  222-333-4444

Country or Region

United States 

Address line 1

Address line 2

Apartment, suite, unit, building, floor, etc.

City

# Reference Screenshot - Payment

**Why is this required?**

Our verification process holds USD \$1 (or equivalent) for 3-5 days to verify your account and prevent fraud.

For the free plan, no charges occur until upgrade to a paid plan. Providing your billing information now enables a seamless upgrade to a paid plan.



## Sign up for AWS

### Billing Information

**Billing country**  
Your billing country determines the payment methods available to you to pay for AWS services.

India ▼

### Payment method type

☒ **UPI AutoPay**  
Set up automatic payments using Unified Payments Interface (UPI).

☐ **Credit or debit card**  
AWS accepts all major credit and debit cards.

### UPI AutoPay information

Use your preferred UPI app to setup automatic payments. You can cancel AutoPay at any time. [Learn more](#) ↗

UPI

**Automatic payment limit**  
₹15,000

**UPI ID**

# **MFA for AWS Account**

# Multi-Factor Authentication is Important

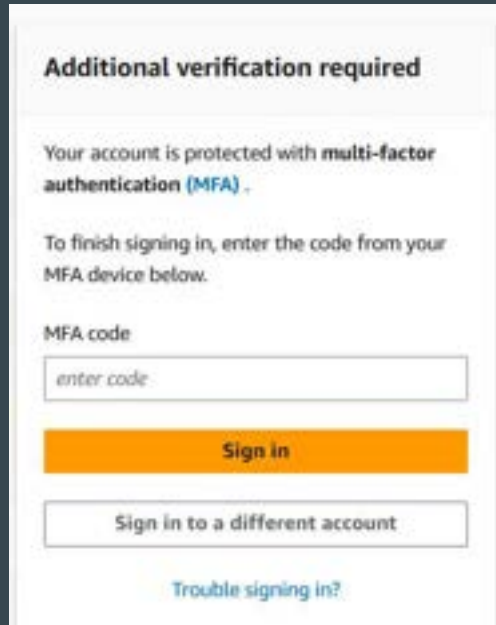
Your AWS account is connected to your payment methods (such as debit or credit cards).

If your login credentials are compromised, an attacker could deploy hundreds of servers for activities like Bitcoin mining, resulting in significant charges to your account.



# Setting the Base

With MFA enabled, when a user signs in to the AWS Management Console, they are prompted for their username and password—something they know—and an authentication code from their MFA device—something they have



**Additional verification required**

Your account is protected with **multi-factor authentication (MFA)**.

To finish signing in, enter the code from your MFA device below.

MFA code

**Sign in**

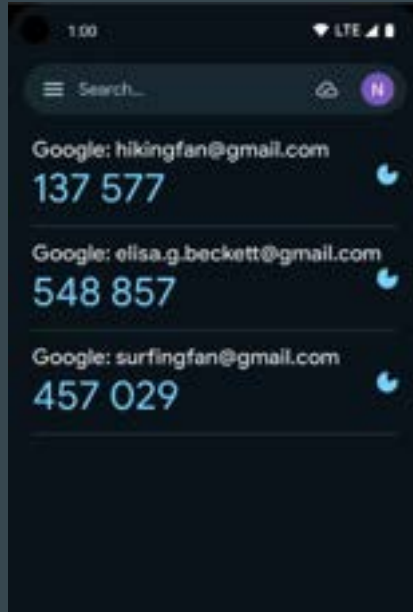
Sign in to a different account

[Trouble signing in?](#)



# Multiple Ways to Set MFA - Authenticator Apps

There are several options available for setting up multi-factor authentication (MFA), such as Google Authenticator and Authy.



# Device Options for MFA

You can use Passkey, Authenticator app, and Hardware TOTP token for setting up MFA

**MFA device**

**Device options**  
In addition to username and password, you will use this device to authenticate into your account.

- ☒  **Passkey or security key**  
Authenticate using your fingerprint, face, or screen lock. Create a passkey on this device or use another device, like a FIDO2 security key.
- ☐  **Authenticator app**  
Authenticate using a code generated by an app installed on your mobile device or computer.
- ☐  **Hardware TOTP token**  
Authenticate using a code generated by Hardware TOTP token or other hardware devices.

# Registering an AWS Account

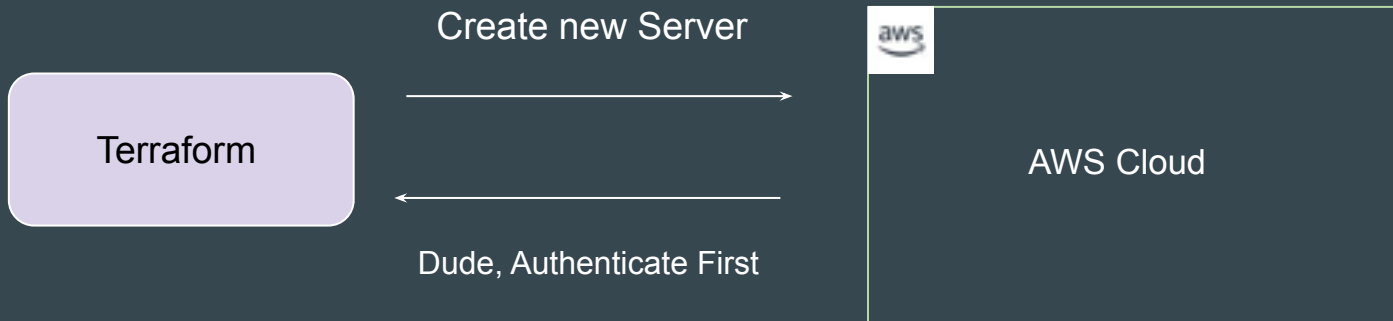


# Authentication and Authorization



# Understanding the Basics

Before we start working on managing environments through Terraform, the first important step is related to Authentication and Authorization.



# Basics of Authentication and Authorization

Authentication is the process of verifying who a user is.

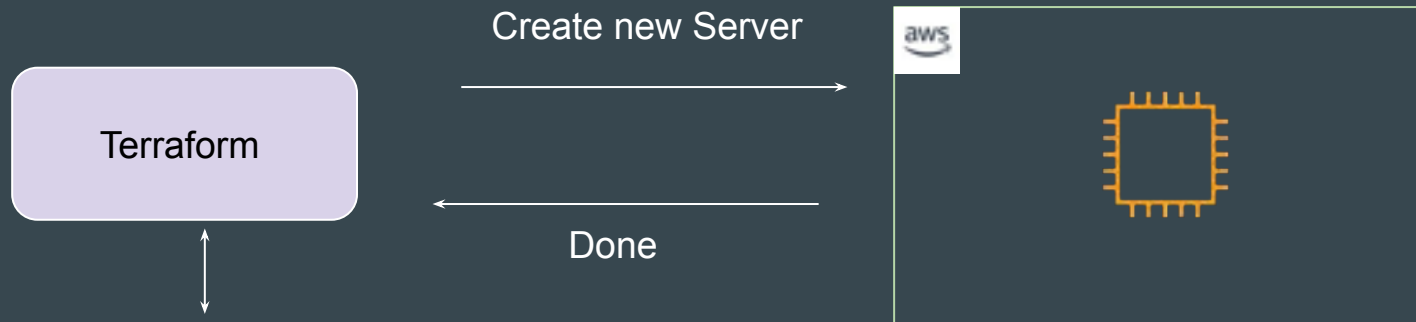
Authorization is the process of verifying what they have access to

Example:

Alice is a user in AWS with no access to any service.

# Learning for Today's Video

Terraform needs **access credentials with relevant permissions** to create and manage the environments.



| username | password |
|----------|----------|
| Bob      | pwd928#  |

# Access Credentials

Depending on the provider, the type of access credentials would change.

| Provider      | Access Credentials                  |
|---------------|-------------------------------------|
| AWS           | Access Keys and Secret Keys         |
| GitHub        | Tokens                              |
| Kubernetes    | Kubeconfig file, Credentials Config |
| Digital Ocean | Tokens                              |



# First Virtual Machine Through Terraform



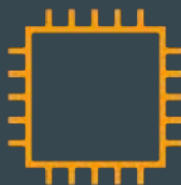
# Revising the Basics of EC2

EC2 stands for Elastic Compute Cloud.

In-short, it's a name for a virtual server that you launch in AWS.



VM



EC2 Instance

# Available Regions

Cloud providers offers multiple regions in which we can create our resource.

You need to decide the region in which Terraform would create the resource.



# Virtual Machine Configuration

A Virtual Machine would have it's own set of configurations.

- CPU
- Memory
- Storage
- Operating System

While creating VM through Terraform, you will need to define these.

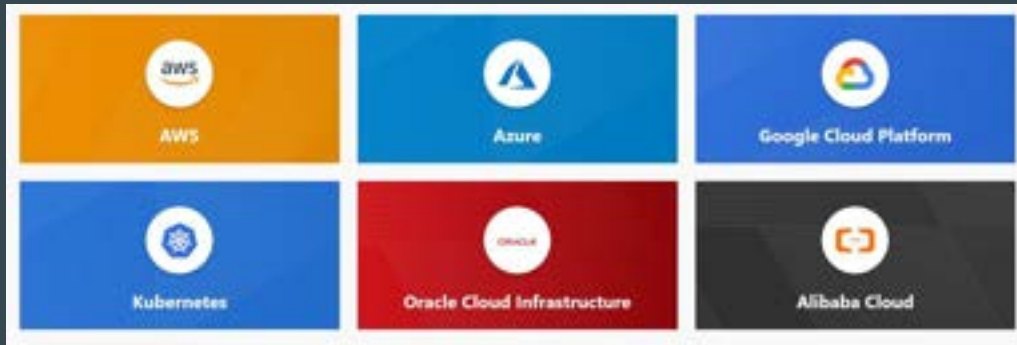
# Providers and Resources



# Basics of Providers

Terraform supports multiple providers.

Depending on what type of infrastructure we want to launch, we have to use appropriate providers accordingly.



# Learning 1 - Provider Plugins

A provider is a plugin that lets Terraform manage an external API.

When we run `terraform init`, plugins required for the provider are automatically downloaded and saved locally to a `.terraform` directory.

```
C:\Users\realv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

## Learning 2 - Resource

Resource block describes one or more infrastructure objects

### Example:

- resource aws\_instance
- resource aws\_alb
- resource iam\_user
- resource digitalocean\_droplet

```
resource "aws_instance" "myec2" {  
    ami = "ami-082b5a644766e0e6f"  
    instance_type = "t2.micro"  
}
```



## Learning 3 - Resource Blocks

A resource block declares a resource of a given type ("aws\_instance") with a given local name ("myec2").

Resource type and Name together serve as an identifier for a given resource and so must be unique.

```
resource "aws_instance" "myec2" {  
  ami = "ami-082b5a644766e0e6f"  
  instance_type = "t2.micro"  
}
```

EC2 Instance Number 1

```
resource "aws_instance" "web" {  
  ami          = ami-123  
  instance_type = "t2.micro"  
}
```

EC2 Instance Number 2

## Point to Note

You can only use the resource that are supported by a specific provider.

In the below example, provider of Azure is used with resource of aws\_instance

```
provider "azurerm" {}

resource "aws_instance" "web" {
  ami          = ami-123
  instance_type = "t2.micro"
}
```

# Important Question

The core concepts, standard syntax remains similar across all providers.

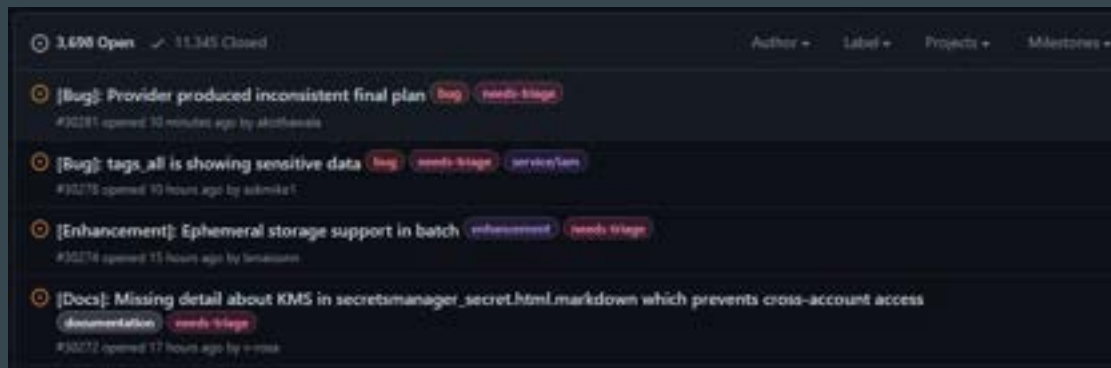
If you learn the basics, you should be able to work with all providers easily.



# Issues and Bugs with Providers

A provider that is maintained by HashiCorp does not mean it has no bugs.

It can happen that there are inconsistencies from your output and things mentioned in documentation. You can raise issue at Provider page.



# Relax and Have a Meme Before Proceeding

That stupid walk you do when someone's mopping a floor and you know you're gonna walk over it but you want them to see how sorry you are to be walking over it so you make yourself look like you're walking over hot lava.



# Provider Tiers



# Provider Maintainers

There are 3 primary type of provider tiers in Terraform.

| Provider Tiers | Description                                                                                  |
|----------------|----------------------------------------------------------------------------------------------|
| Official       | Owned and Maintained by HashiCorp.                                                           |
| Partner        | Owned and Maintained by Technology Company that maintains direct partnership with HashiCorp. |
| Community      | Owned and Maintained by Individual Contributors.                                             |

# Provider Namespace

**Namespaces** are used to help users identify the organization or publisher responsible for the integration

| Tier      | Description                                                              |
|-----------|--------------------------------------------------------------------------|
| Official  | hashicorp                                                                |
| Partner   | Third-party organization<br>e.g. mongodb/mongodbatlas                    |
| Community | Maintainer's individual or organization account, e.g.<br>DeviaVir/gsuite |



# Important Learning

Terraform requires explicit source information for any providers that are not HashiCorp-maintained, using a new syntax in the `required_providers` nested block inside the terraform configuration block

```
provider "aws" {  
  region      = "us-west-2"  
  access_key  = "PUT-YOUR-ACCESS-KEY-HERE"  
  secret_key  = "PUT-YOUR-SECRET-KEY-HERE"  
}
```

HashiCorp Maintained

```
terraform {  
  required_providers {  
    digitalocean = {  
      source = "digitalocean/digitalocean"  
    }  
  }  
}  
  
provider "digitalocean" {  
  token = "PUT-YOUR-TOKEN-HERE"  
}
```

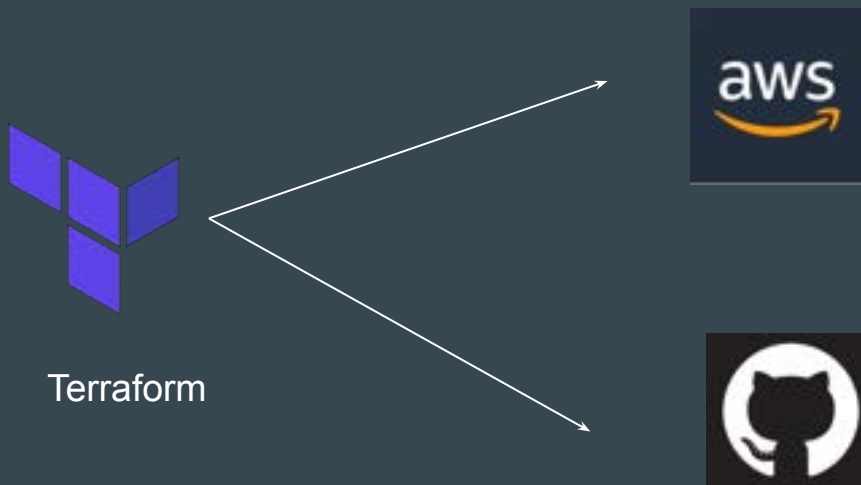
Non-HashiCorp Maintained

# **Terraform Destroy**

# Learning to Destroy Resources

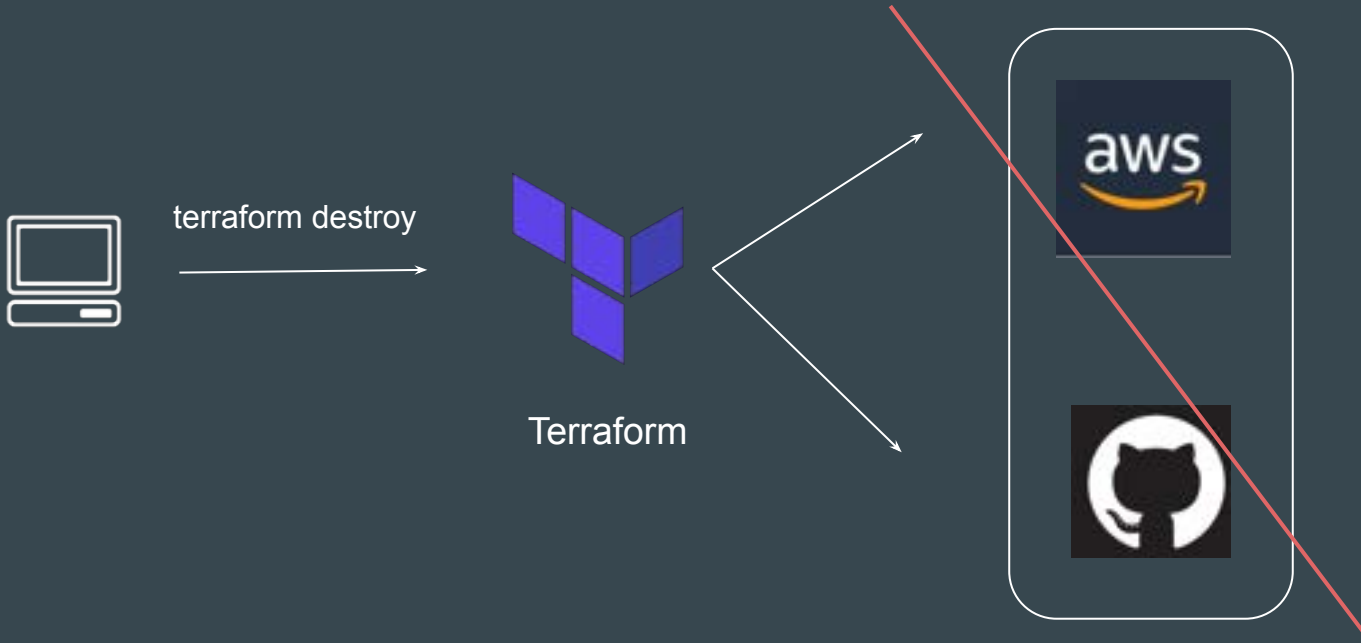
If you keep the infrastructure running, you will get charged for it.

Hence it is important for us to also know on how we can delete the infrastructure resources created via terraform.



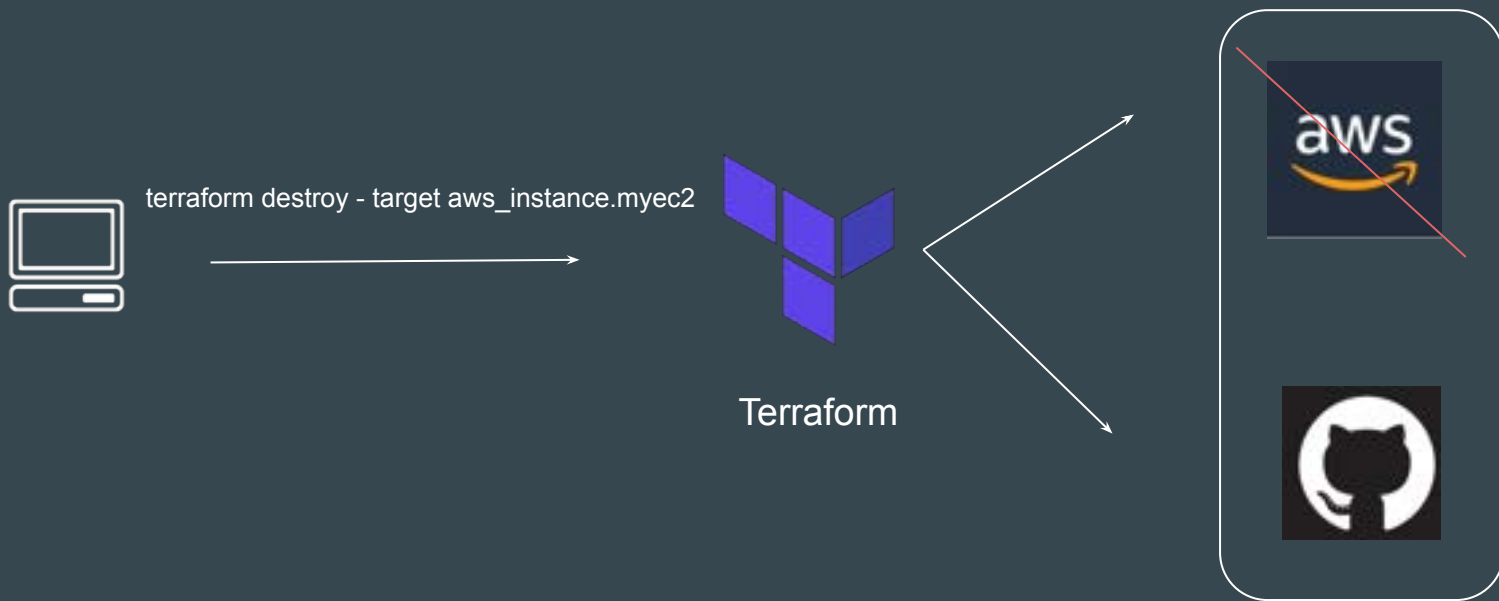
# Approach 1 - Destroy ALL

**terraform destroy** allows us to destroy all the resource that are created within the folder.



## Approach 2 - Destroy Some

terraform destroy with **-target** flag allows us to destroy specific resource.



# Terraform Destroy with Target

The **-target** option can be used to focus Terraform's attention on only a subset of resources.

Combination of : Resource Type + Local Resource Name

| Resource Type     | Local Resource Name |
|-------------------|---------------------|
| aws_instance      | myec2               |
| github_repository | example             |

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```

```
resource "github_repository" "example" {  
  name      = "example"  
  description = "My awesome codebase"  
  visibility = "public"  
}
```

# **AWS Provider - Authentication Configuration**

# Understanding the Basics

At this stage, we have been manually hardcoding the access / secret keys within the provider block.

Although a working solution, but it is **not optimal from security point of view**.

```
aws-provider-config.tf > ...  
provider "aws" {  
  region    = "us-east-1"  
  access_key = "AKIAQTSHLD4OASDZ5QJI"  
  secret_key = "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"  
}  
  
resource "aws_elb" "lb" {  
  domain = "vpc"  
}
```



## Better Way

We want our code to run successfully without hardcoding the secrets in the provider block.

```
aws-provider-config.tf > ...  
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_eip" "lb" {  
  domain = "vpc"  
}
```

# Better Approach

The AWS Provider can source credentials and other settings from the shared configuration and credentials files.

```
aws-provider-config.tf > ...  
provider "aws" {  
  shared_config_files    = ["/Users/tf_user/.aws/conf"]  
  shared_credentials_files = ["/Users/tf_user/.aws/creds"]  
  profile                = "customprofile"  
}  
  
resource "aws_eip" "lb" {  
  domain = "vpc"  
}
```

# Default Configurations

If shared files lines are not added to provider block, by default, Terraform will locate these files at `$HOME/.aws/config` and `$HOME/.aws/credentials` on Linux and macOS.

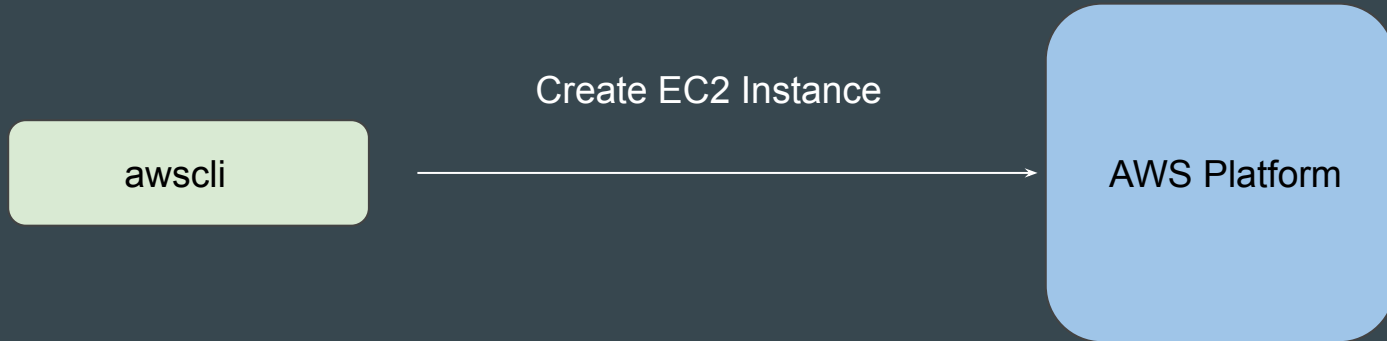
`"%USERPROFILE%\aws\config"` and `"%USERPROFILE%\aws\credentials"` on Windows.

```
aws-provider-config.tf > ...  
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_eip" "lb" {  
    domain = "vpc"  
}
```

# AWS CLI

AWS CLI allows customers to manage AWS resources directly from CLI.

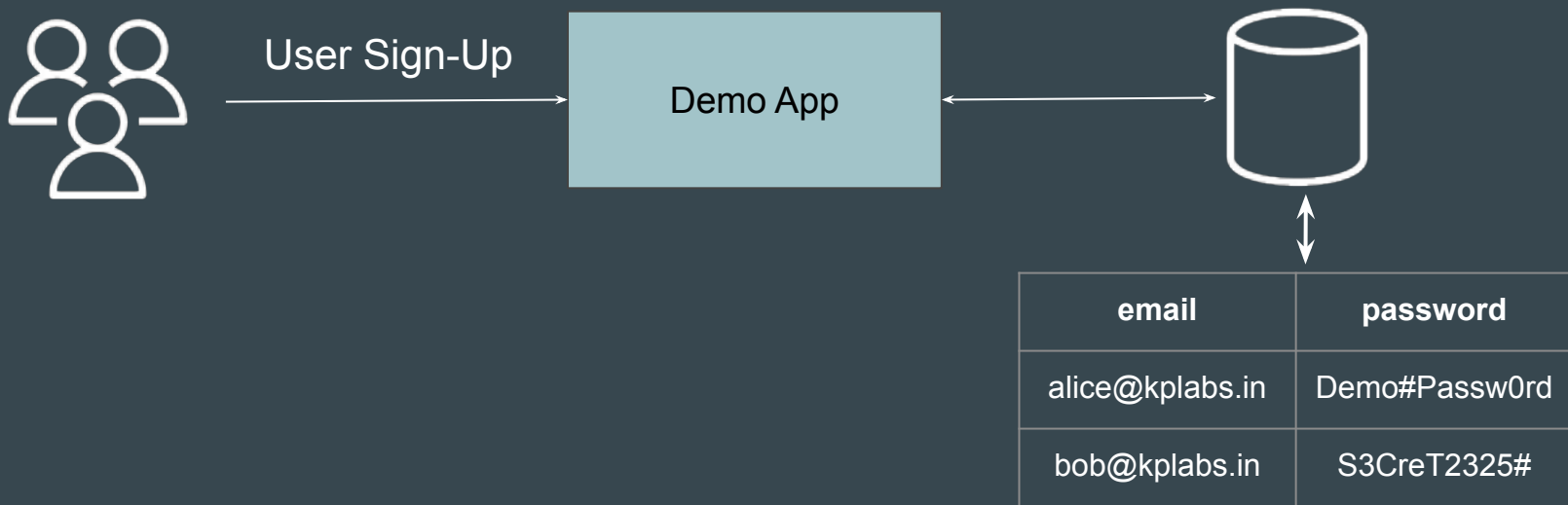
When you configure Access/Secret keys in AWS CLI, the location in which these credentials are stored is the same default location that Terraform searches the credentials from.



# **Terraform State File**

# Simple Analogy

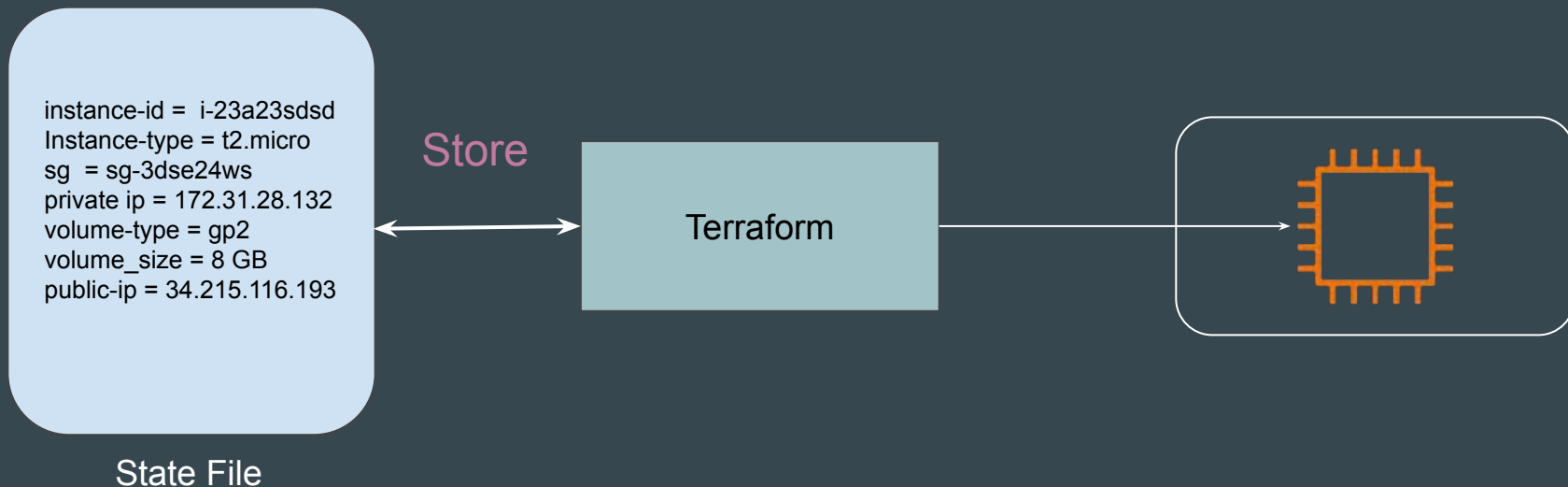
The demo application provides a frontend interface for users and stores all the data in a backend database.



# Terraform State File

Terraform stores information about managed infrastructure in a state file.

This state file keeps track of resources created by your configuration.



## Point to Note

1. By default, the state information is stored in file named `terraform.tfstate`
2. Format of state file is JSON
3. Never modify the Terraform State file manually. Any mistake can cause corruption of state file.



## **Desired State and Current State**

# Desired State (The "What You Want")

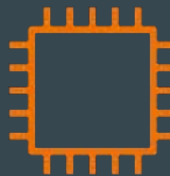
The **Desired State** is the configuration you write in your Terraform files (usually ending in .tf).

Your desired state is EC2 instance of t2.micro and OS associated with "ami-123"

```
resource "aws_instance" "myec2" {  
  ami = "ami-123"  
  instance_type = "t2.micro"  
}
```

Desired State

Terraform

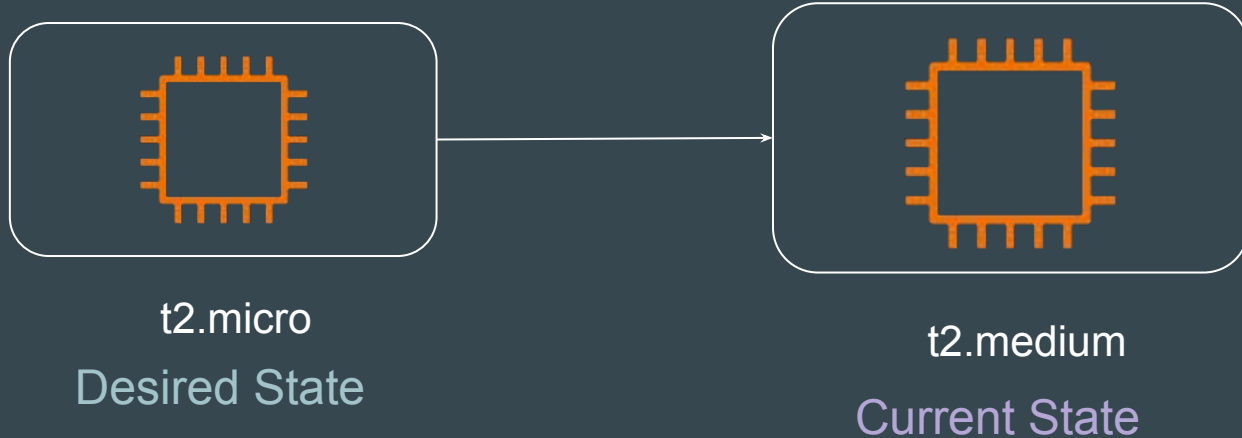


t2.micro

# Current State (The "What Actually Exists")

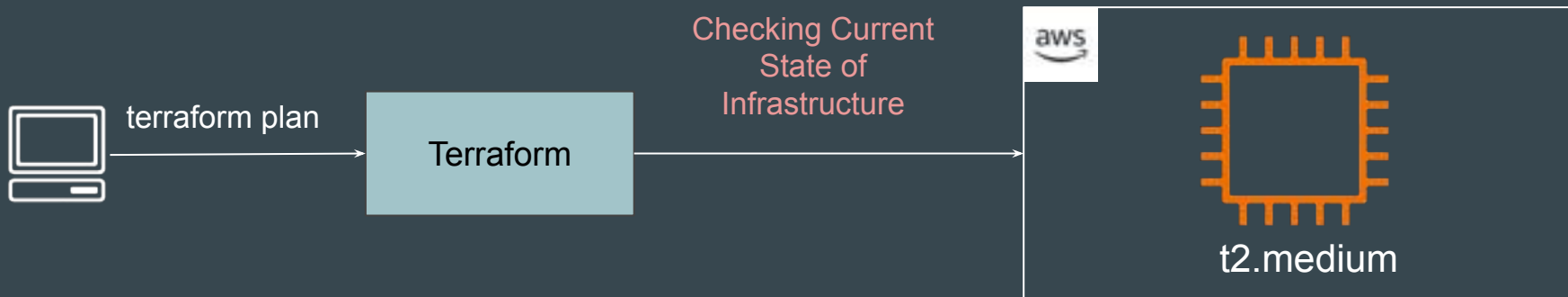
The **Current State** represents the **actual reality of your infrastructure**.

Example: Bob manually modified the instance size from t2.micro to t2.medium



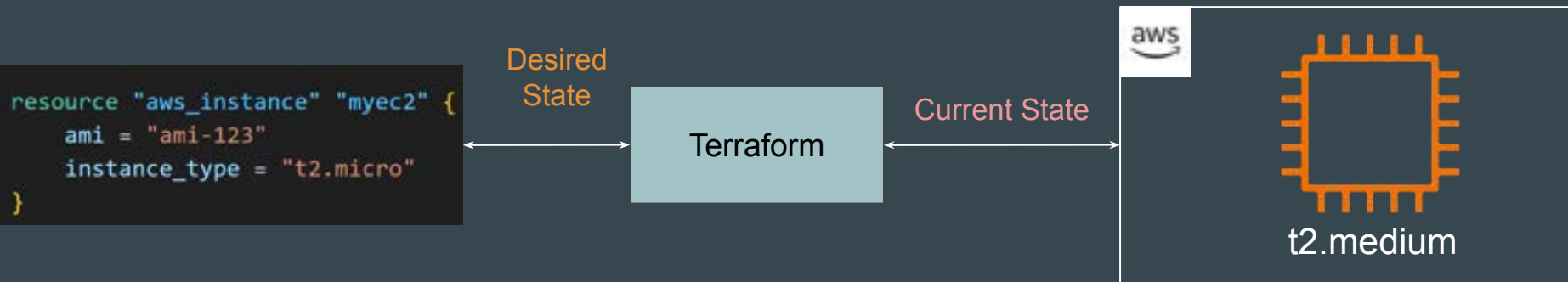
# The Reconciliation Process (Part 1)

When you run `terraform plan` or `terraform apply`, Terraform first reaches out to the Cloud Provider API to check the Current State of Infrastructure.



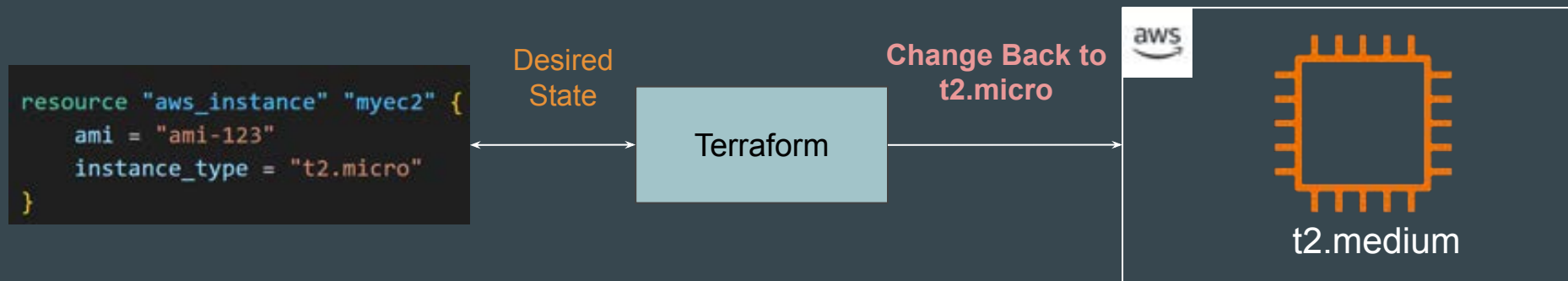
# The Reconciliation Process (Part 2)

Terraform compares the Current State (what it just found in the cloud) against your Desired State (your .tf files).



# The Reconciliation Process (Part 3)

Based on the difference, Terraform determines **what actions are necessary to make the Current State match the Desired State.**



# Generic Actions

Based on the difference, Terraform determines what actions are necessary to make the Current State match the Desired State.

|        |                                                                                               |
|--------|-----------------------------------------------------------------------------------------------|
| Create | If Desired exists but Current does not.                                                       |
| Update | If both exist, but a property differs.                                                        |
| Delete | If Current exists (and is tracked in the state file) but Desired (the code) has been removed. |
| No-Op  | If Current matches Desired exactly.                                                           |

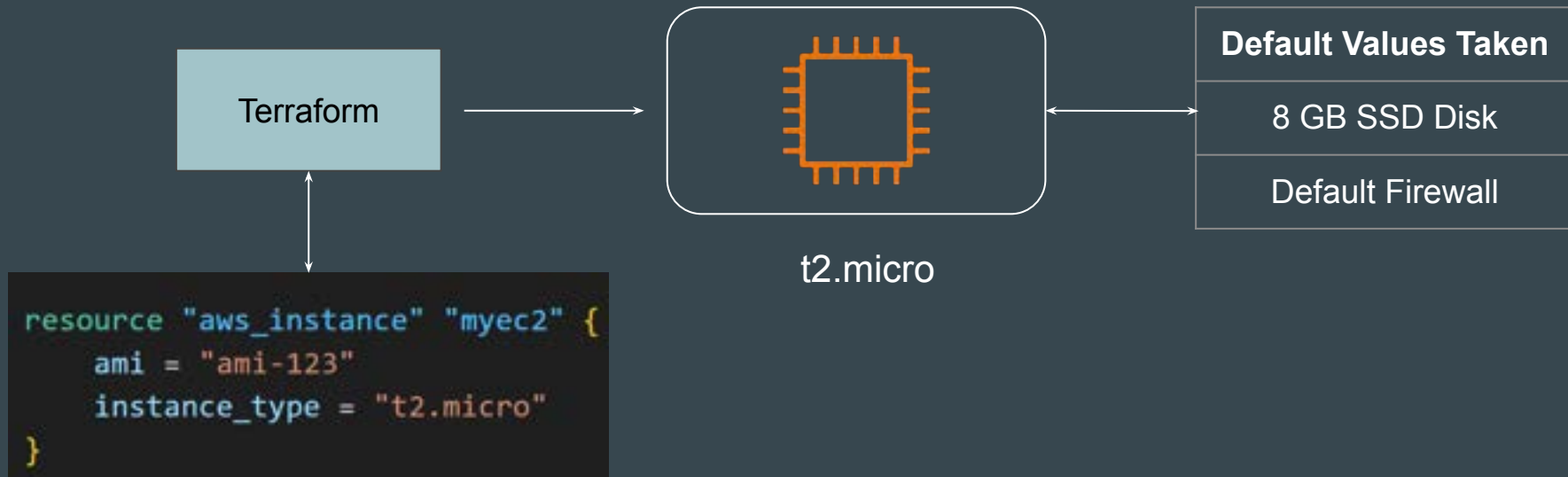




**More Clarity - Desired State and Current State**

# Setting the Base

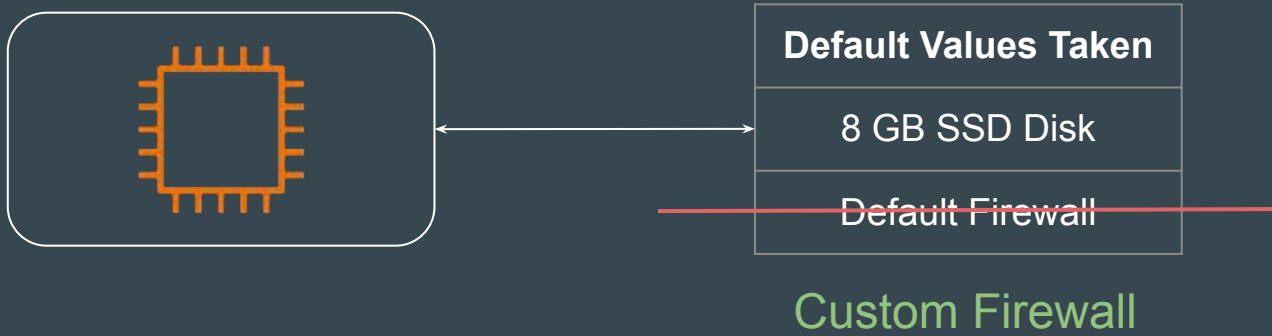
When you launch an EC2 instance without specifying certain configuration parameters, AWS automatically assigns default values to those unspecified settings.



## Point to Note

These default values are generally not considered to be your desired state.

If you manually change these default values, it will have no impact in next terraform plan and apply stages.



# Best Practice

Explicitly Define All Desired Attributes in Your Configuration.

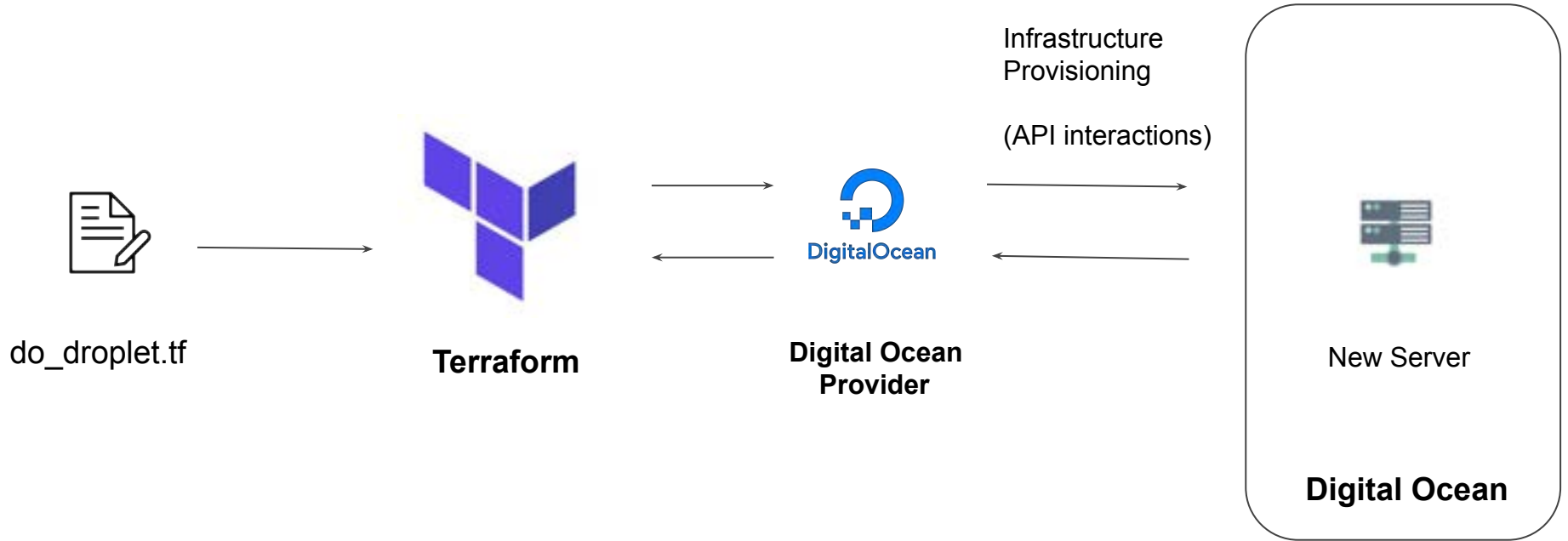
---

# Provider Versioning

Terraform in detail

---

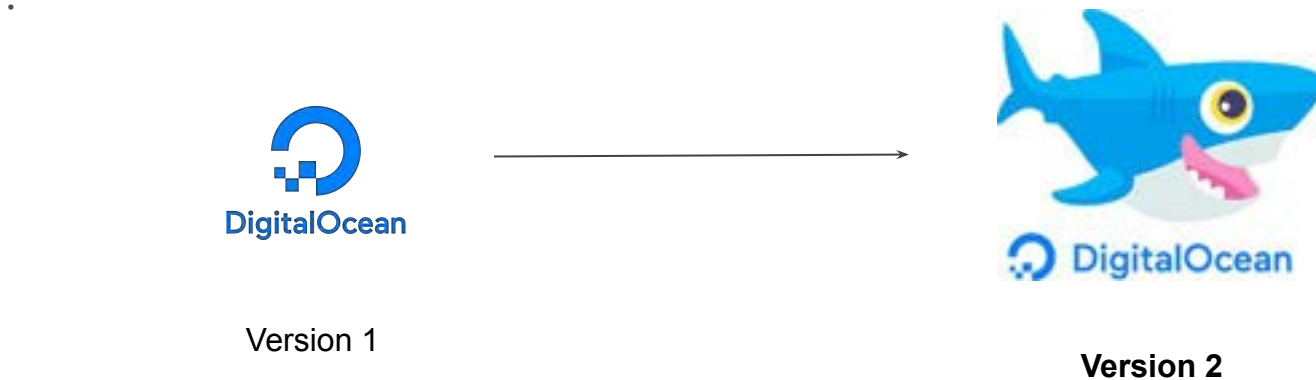
# Provider Architecture



# Overview of Provider Versioning

Provider plugins are released separately from Terraform itself.

They have different set of version numbers.



# Explicitly Setting Provider Version

During terraform init, if version argument is not specified, the most recent provider will be downloaded during initialization.

For production use, you should constrain the acceptable provider versions via configuration, to ensure that new versions with breaking changes will not be automatically installed.

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 3.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}
```



# Arguments for Specifying provider

There are multiple ways for specifying the version of a provider.

| Version Number Arguments | Description                       |
|--------------------------|-----------------------------------|
| $\geq 1.0$               | Greater than equal to the version |
| $\leq 1.0$               | Less than equal to the version    |
| $\sim > 2.0$             | Any version in the 2.X range.     |
| $\geq 2.10, \leq 2.30$   | Any version between 2.10 and 2.30 |

# Dependency Lock File

Terraform dependency lock file allows us to lock to a specific version of the provider.

If a particular provider already has a selection recorded in the lock file, Terraform will always re-select that version for installation, even if a newer version has become available.

You can override that behavior by adding the `-upgrade` option when you run `terraform init`,

```
provider "registry.terraform.io/hashicorp/aws" {  
  version      = "2.70.0"  
  constraints = ">= 2.31.0, <= 2.70.0"  
  hashes = [  
    "h1:fx8tbGVwK1YIDI6UdHLnorC9PA1ZP5WEeW3V3aDCdWY=",  
    "zh:01a5f351146434b418f9ff8d8cc956ddc801110f1cc8b139e01be2ff8c544605",  
    "zh:1ec08abbaf09e3e0547511d48f77a1e2c89face2d55886b23f643011c76cb247",  
    "zh:606d134fef7c1357c9d155aadbee6826bc22bc0115b6291d483bc1444291c3e1",  
    "zh:67e31a71a5ecbbc96a1a6708c9cc300bbfe921c322320cdbb95b9002026387e1",  
  ]  
}
```

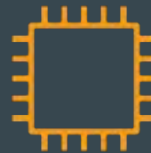
# **Terraform Refresh**

# Understanding the Challenge

Terraform can create an infrastructure based on configuration you specified.

It can happen that the infrastructure gets modified manually.

```
resource "aws_instance" "web" {  
  ami      = ami-123  
  instance_type = "t2.micro"  
}
```



t2.micro

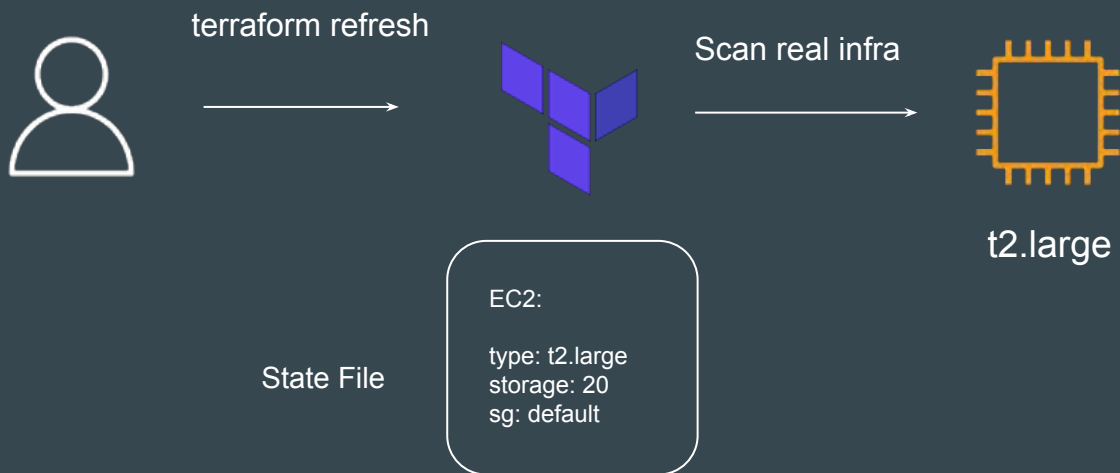
State File

EC2:

type: t2.micro  
storage: 20  
sg: default

# Understanding the Challenge

The **terraform refresh** command will check the latest state of your infrastructure and update the state file accordingly.



## Points to Note

You shouldn't typically need to use this command, because Terraform automatically performs the same refreshing actions as a part of creating a plan in both the `terraform plan` and `terraform apply` commands.

# Understanding the Usage

The terraform refresh command is deprecated in newer version of terraform.

The -refresh-only option for terraform plan and terraform apply was introduced in Terraform v0.15.4.

---

# Lecture Format - Terraform Course

Terraform in detail

---



# Overview of the Format

We tend to use a different folder for each practical that we do in the course.

This allows us to be more systematic and allows easier revisit in-case required.

| Lecture Name              | Folder Names |
|---------------------------|--------------|
| Create First EC2 Instance | folder1      |
| Tainting resource         | folder2      |
| Conditional Expression    | folder3      |

# Find the appropriate code from GitHub

Code in GitHub is arranged according to sections that are matched to the domains in the course.

Every section in GitHub has easy Readme file for quick navigation.

## Video-Document Mapper

| Sr No | Document Link                                                           |
|-------|-------------------------------------------------------------------------|
| 1     | <a href="#">Understanding Attributes and Output Values in Terraform</a> |
| 2     | <a href="#">Referencing Cross-Account Resource Attributes</a>           |
| 3     | <a href="#">Terraform Variables</a>                                     |
| 4     | <a href="#">Approaches for Variable Assignment</a>                      |
| 5     | <a href="#">Data Types for Variables</a>                                |

# Destroy Resource After Practical

We know how to destroy resources by now

`terraform destroy`

After you have completed your practical, make sure you destroy the resource before moving to the next practical.

This is easier if you are maintaining separate folder for each practical.

# Relax and Have a Meme Before Proceeding



**alcohol**  
@Mandac5

What is an extreme sport?



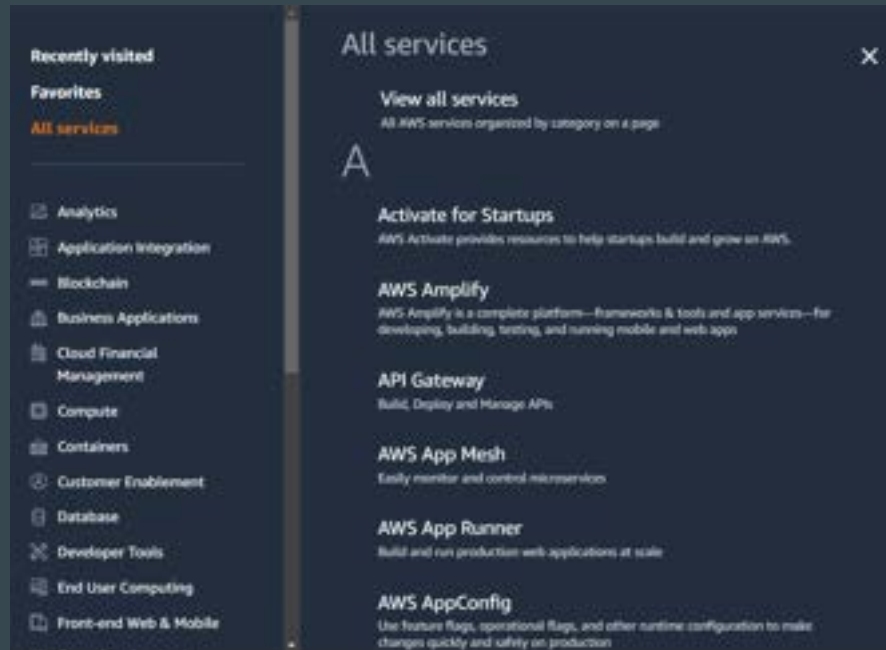
**allison**  
@amazaleax

Doing your homework while the  
teacher is collecting it

# **Learning Scope - AWS Services for Terraform Course**

# Understanding the Basics

AWS has more than 200 services available.



# Aim of the Course

Primary aim of this course is to master the core concepts of Terraform.

Terraform = Infrastructure as Code Tool.

To learn Terraform, we need to create infrastructure somewhere.



## Services that we Choose

Throughout the course, we use very **basic AWS services** to demonstrate Terraform concepts.

- Virtual Machine (EC2)
- Firewall (Security Groups)
- AWS Users (IAM Users)
- IP Address (Elastic IP)



## Basics of These Services are Covered

We have 100,000+ students from different background who are learning Terraform.

Some are AWS Pros, Some are from Azure/GCP, Some are students

To align everyone on same page, we also cover basics of the AWS service that we use throughout the course.

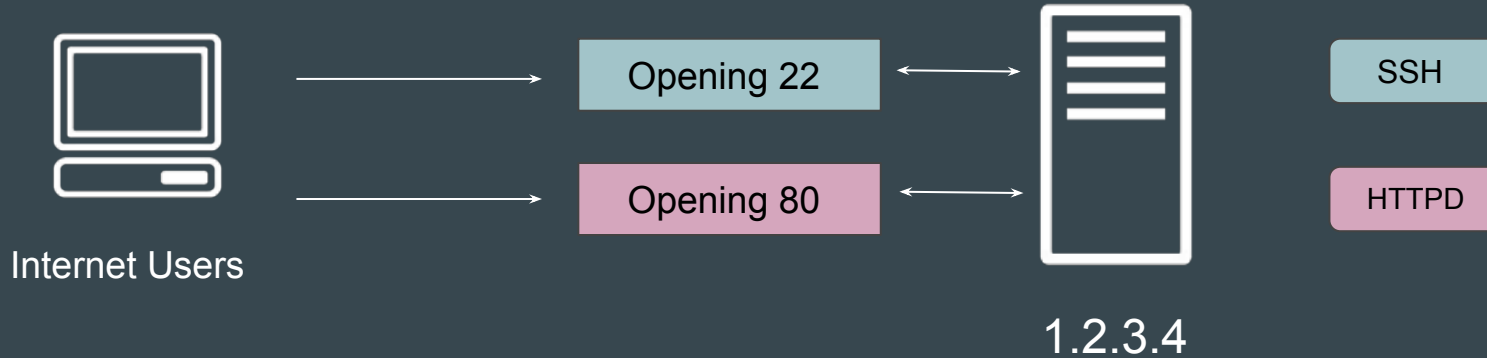
# Example - Creating Firewall Through Terraform

1. Basics of Firewalls in AWS
2. Firewall Practical in AWS (GUI Console)
3. Creating Firewall Rules Through Terraform.

# **Basics of Firewalls**

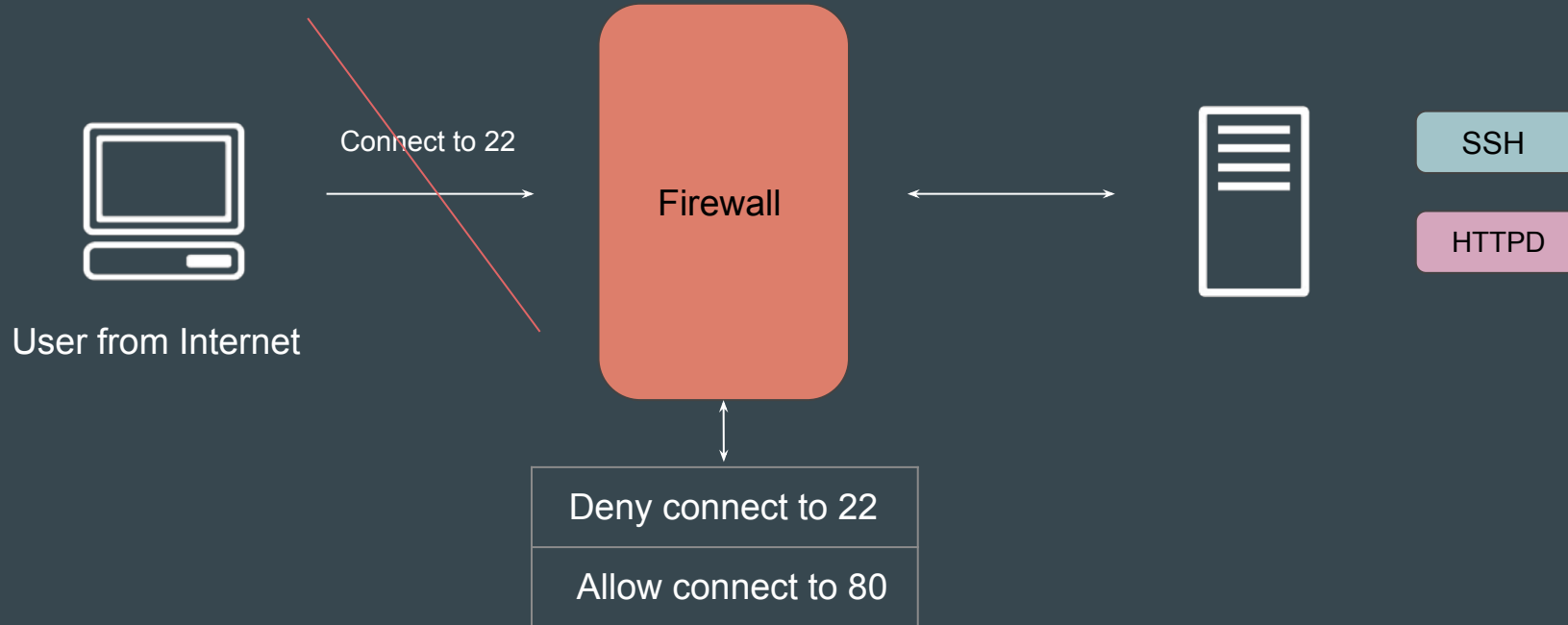
# Basics of Ports

A port acts as a **endpoint of communication** to identify a given application or process on an Linux operating system



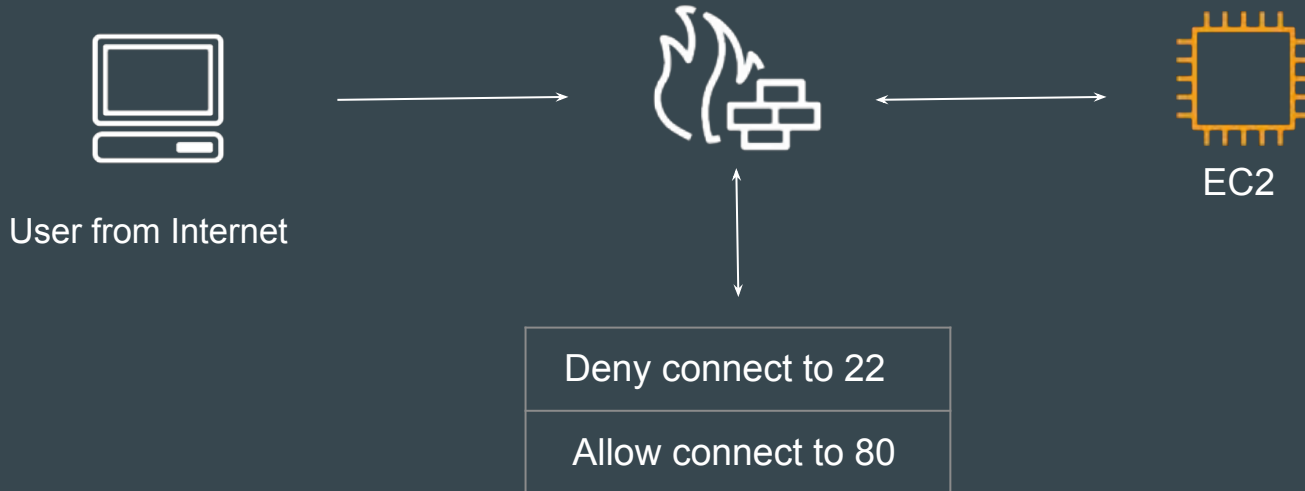
# Basics of Firewall

Firewall is a **network security system** that monitors and controls incoming and outgoing network traffic based on predetermined security rules.



# Firewall in AWS

A **security group** acts as a virtual firewall for your instance to control inbound and outbound traffic.



# Sample Security Group with Rules

## Details

|                                      |                                             |                                            |                                                   |
|--------------------------------------|---------------------------------------------|--------------------------------------------|---------------------------------------------------|
| Security group name<br>demo-firewall | Security group ID<br>sg-0fcf94c6ff13977b2   | Description<br>Demo Purpose                | VPC ID<br>vpc-050f2228461400045 <a href="#">🔗</a> |
| Owner<br>042025557788                | Inbound rules count<br>3 Permission entries | Outbound rules count<br>1 Permission entry |                                                   |

Inbound rulesOutbound rulesTags

### Inbound rules (3)

Manage tags Edit inbound rules

< 1 >

| IP version | Type         | Protocol | Port range | Source         |
|------------|--------------|----------|------------|----------------|
| IPv4       | SSH          | TCP      | 22         | 172.31.0.0/16  |
| IPv4       | HTTP         | TCP      | 80         | 0.0.0.0/0      |
| IPv4       | MySQL/Aurora | TCP      | 3306       | 172.31.0.10/32 |

# Inbound and Outbound Rules

Firewalls control both inbound and outbound connections to and from the server.



EC2

| Inbound                 | Outbound                   |
|-------------------------|----------------------------|
| Allow 80 from 0.0.0.0/0 | Allow 3306 to 172.31.10.50 |



# **Creating Firewall Rules with Terraform**

# Architecture of Today's Video

We will create a Firewall (Security Group) in AWS with following configuration



terraform-firewall



| Inbound                 | Outbound  |
|-------------------------|-----------|
| Allow 80 from 0.0.0.0/0 | Allow ALL |

# Reference - Final Code in Video

```
firewall.tf X
firewall.tf > resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4"

resource "aws_security_group" "allow_tls" {
  name          = "terraform-firewall"
  description   = "Managed from Terraform"
}

resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "0.0.0.0/0"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}

resource "aws_vpc_security_group_egress_rule" "allow_all_traffic_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1" # semantically equivalent to all ports
}
```

# Dealing with Documentation Code Updates - Terraform

# Understanding the Challenge

Occasionally in the newer version of Providers, you will see some changes in the way you create a resource.

```
resource "aws_security_group" "allow_tls" {  
  name      = "terraform-firewall"  
  description = "Managed from Terraform"  
}  
  
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "0.0.0.0/0"  
  from_port         = 80  
  ip_protocol       = "tcp"  
  to_port           = 80  
}
```

New Approach

```
resource "aws_security_group" "old_approach" {  
  name      = "allow_tls"  
  description = "Allow TLS inbound traffic"  
  
  ingress {  
    description      = "TLS from VPC"  
    from_port        = 443  
    to_port          = 443  
    protocol         = "tcp"  
    cidr_blocks      = ["10.77.32.50/32"]  
  }  
}
```

Old Approach

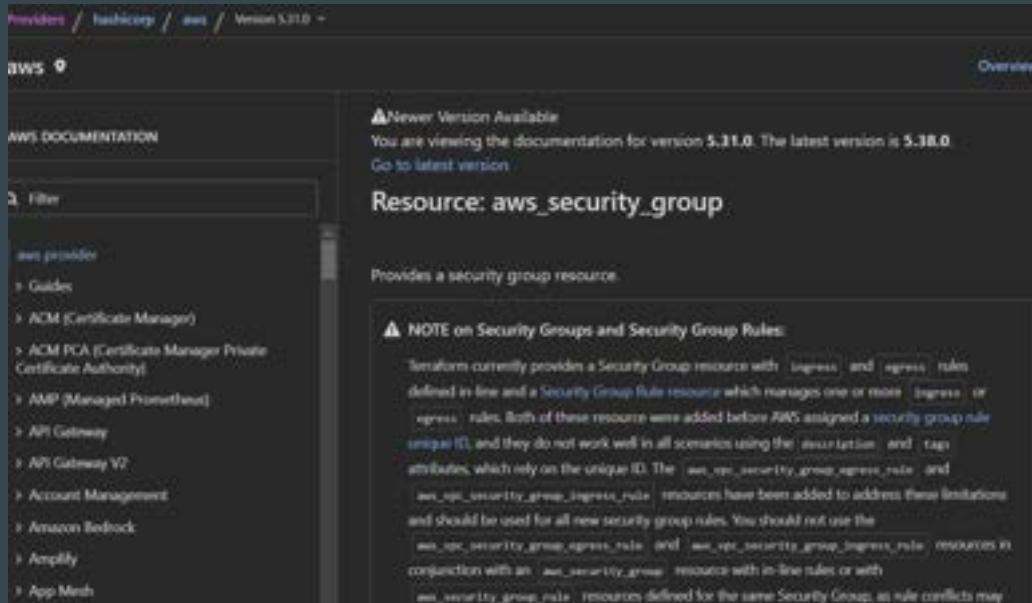
## Points to Note

Just because a better approach is recommended, does NOT always mean that the older approach will stop working.

Organizations can continue to use the approach that suits best in it's environment.

# Switching to Older Provider Doc

You can always switch to the older version of provider documentation page to understand the changes.



## Closing Pointers

For larger enterprises, it becomes difficult to upgrade their code base to the newer approach that provider recommends.

In such case, they stick with the appropriate provider version that supports the older approach of creating the resource.



# Create Elastic IP with Terraform

# Basics of Elastic IP in AWS

An Elastic IP address is a static IPv4 address in AWS.

You can create it and associate it with EC2 instance.



# Aim of Today's Video

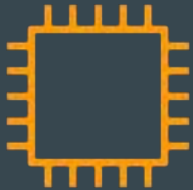
We are going to use Terraform to generate Elastic IP resource in AWS.

# Attributes

# Basics of Attributes

Each resource has its associated set of attributes.

Attributes are the fields in a resource that hold the values that end up in state.



| Attributes  | Values                        |
|-------------|-------------------------------|
| ID          | i-abcd                        |
| public_ip   | 52.74.32.50                   |
| private_ip  | 172.31.10.50                  |
| private_dns | ip-172-31-10-50-.ec2.internal |

# Points to Note

Each resource type has a predefined set of attributes determined by the provider.

## Attribute Reference

This resource exports the following attributes in addition to the arguments above:

- `arn` - ARN of the instance.
- `capacity_reservation_specification` - Capacity reservation specification of the instance.
- `id` - ID of the instance.
- `instance_state` - State of the instance. One of: `pending`, `running`, `shutting-down`, `terminated`, `stopping`, `stopped`. See [Instance Lifecycle](#) for more information.
- `outpost_arn` - ARN of the Outpost the instance is assigned to.
- `password_data` - Base-64 encoded encrypted password data for the instance. Useful for getting the administrator password for instances running Microsoft Windows. This attribute is only exported if `get_password_data` is true. Note that this encrypted value will be stored in the state file, as with all exported attributes. See [GetPasswordData](#) for more information.
- `primary_network_interface_id` - ID of the instance's primary network interface.

## **Cross-Resource Attribute References**

# Typical Challenge

It can happen that in a single terraform file, you are defining two different resources.

However Resource 2 might be dependent on some value of Resource 1.



Elastic IP Address

Security group

Allow 443 from Elastic IP



# Understanding The Workflow

```
eip.tf > ...  
resource "aws_eip" "lb" {  
  domain = "vpc"  
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "HOW-TO-ADD-ELASTIC-IP-ADDRESS-HERE"  
  from_port         = 80  
  ip_protocol       = "tcp"  
  to_port           = 80  
}
```

Elastic IP



52.72.30.50



Security group

Allow 443 from 52.72.30.50

# Analyzing the Attributes of EIP

We have to find which attribute stores the Public IP associated with EIP Resource.

## Attribute Reference

This resource exports the following attributes in addition to the arguments above:

- `allocation_id` - ID that AWS assigns to represent the allocation of the Elastic IP address for use with instances in a VPC.
- `association_id` - ID representing the association of the address with an instance in a VPC.
- `carrier_ip` - Carrier IP address.
- `customer_owned_ip` - Customer owned IP.
- `id` - Contains the EIP allocation ID.
- `private_dns` - The Private DNS associated with the Elastic IP address (if in VPC).
- `private_ip` - Contains the private IP address (if in VPC).
- `public_dns` - Public DNS associated with the Elastic IP address.
- `public_ip` - Contains the public IP address.

# Referencing Attribute in Other Resource

We have to find a way in which attribute value of “public\_ip” is referenced to the cidr\_ipv4 block of security group rule resource.



Elastic IP

| Attribute | Value       |
|-----------|-------------|
| public_ip | 52.72.52.72 |

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "REFERENCE-PUBLIC-IP-ATTRIBUTE-HERE"  
  from_port         = 80  
  ip_protocol       = "tcp"  
  to_port           = 80  
}
```

# Cross Referencing Resource Attribute

Terraform allows us to reference the attribute of one resource to be used in a different resource.

Overall syntax:

`<RESOURCE TYPE>.<NAME>.<ATTRIBUTE>`

# Cross Referencing Resource Attribute

We can specify the resource address with attribute for cross-referencing.



Elastic IP

| Attribute | Value       |
|-----------|-------------|
| public_ip | 52.72.52.72 |

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "aws_eip.lb.public_ip"  
  from_port         = 80  
  ip_protocol        = "tcp"  
  to_port           = 80  
}
```

# String Interpolation in Terraform

`${...}`): This syntax indicates that Terraform will replace the expression inside the curly braces with its calculated value.

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "${aws_eip.lb.public_ip}/32"  
  from_port         = 80  
  ip_protocol       = "tcp"  
  to_port           = 80  
}
```

# Joke Time

Why did the Terraform attribute take a break?

...It was feeling over-referenced.

---

How did the Terraform attribute become a detective?

...It followed the resource trail.

**Output Values**



# Understanding the Basics

**Output values** make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use.



# Sample Example

## Use-Case:

Create a Elastic IP (Public IP) resource in AWS and output the value of the EIP.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + demo = (known after apply)
aws_eip.lb: Creating...
aws_eip.lb: Creation complete after 3s [id=eipalloc-0680508decfe8c252]

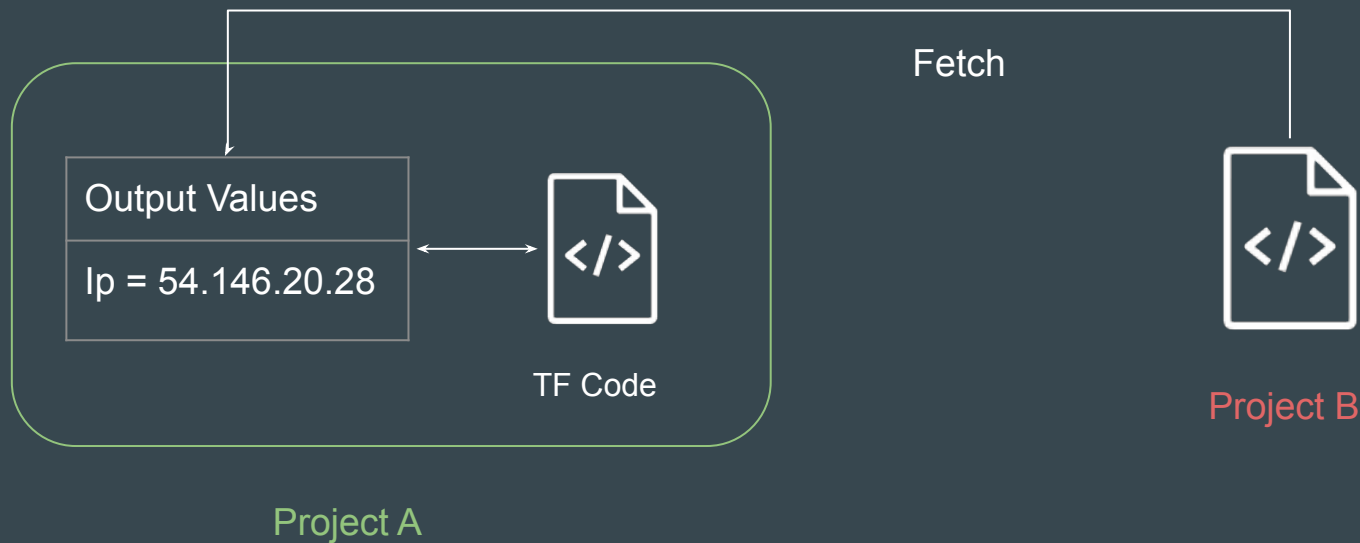
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

demo = "54.146.20.18"
```

## Point to Note

Output values defined in Project A can be referenced from code in Project B as well.



# **Terraform Variables**

# Understanding the Challenge

Repeated static values in the code can create more work in the future.

Example: VPN IP needs to be whitelisted for 5 ports through Firewall Rules.

| Port Number | CIDR Block      | Description      |
|-------------|-----------------|------------------|
| 80          | 101.0.62.210/32 | VPN IP Whitelist |
| 443         | 101.0.62.210/32 | VPN IP Whitelist |
| 22          | 101.0.62.210/32 | VPN IP Whitelist |
| 21          | 101.0.62.210/32 | VPN IP Whitelist |
| 8080        | 101.0.62.210/32 | VPN IP Whitelist |

# Reference Screenshot

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "101.0.62.210/32"  
  from_port         = 80  
  ip_protocol       = "tcp"  
  to_port           = 80  
}
```

Firewall Rule 1

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "101.0.62.210/32"  
  from_port         = 443  
  ip_protocol       = "tcp"  
  to_port           = 443  
}
```

Firewall Rule 2

# Better Approach

A better solution would be to **define repeated static value in one central place.**

| Key    | Value           |
|--------|-----------------|
| vpn_ip | 101.0.62.210/32 |

Central Location

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "fetch-from-central-location"  
  from_port         = 443  
  ip_protocol       = "tcp"  
  to_port           = 443  
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "fetch-from-central-location"  
  from_port         = 8080  
  ip_protocol       = "tcp"  
  to_port           = 8080  
}
```

# Basics of Variables

Terraform input variables are used to pass certain values from outside of the configuration

| Name     | Value           |
|----------|-----------------|
| vpn_ip   | 101.0.62.210/32 |
| app_port | 8080            |

Variable File

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = var.vpn_ip  
  from_port         = var.app_port  
  ip_protocol       = "tcp"  
  to_port           = var.app_port  
}
```





# Benefits of Variables

1. Update important values in one central place instead of searching and replacing them throughout your code, saving time and potential mistakes.
2. No need to touch the core Terraform configuration file. This can avoid human mistakes while editing.



# **Variable Definitions File (TFVars)**

# Understanding the Base

Managing variables in production environment is one of the very important aspect to keep code clean and reusable.

HashiCorp recommends creating a separate file with name of `*.tfvars` to define all variable value in a project.

# How Recommended Folder Structure Looks Like

1. Main Terraform Configuration File.
2. **variables.tf** file that defines all the variables.
3. **terraform.tfvars** file that defines value to all the variables.

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = var.vpn_ip  
  from_port         = var.app_port  
  ip_protocol       = "tcp"  
  to_port           = var.app_port  
}
```

Main Configuration File

```
variables.tf X  
  
variables.tf > ...  
variable "vpn_ip" {}  
  
variable "app_port" {}
```

variables.tf File

```
terraform.tfvars ●  
  
terraform.tfvars > ...  
vpn_ip = "101.0.62.210/32"  
  
app_port = "8080"
```

terraform.tfvars file

# Configuration for Different Environments

Organizations can have wide set of environments: Dev, Stage, Prod

```
resource "aws_instance" "web" {  
  ami          = "ami-1234"  
  instance_type = var.instance_type  
}
```

Main Configuration File

```
variables.tf > ...  
variable "vpn_ip" {}  
  
variable "app_port" {}  
  
variable "instance_type" {}
```

variables.tf file

```
dev.tfvars > ...  
vpn_ip = "52.0.62.30/32"  
  
app_port = "8080"  
  
instance_type = "t2.micro"
```

Dev

tfvars file

```
prod.tfvars > ...  
vpn_ip = "101.0.62.30/32"  
  
app_port = "8080"  
  
instance_type = "m5.large"
```

Prod

# Selecting tfvars File

If you have multiple variable definitions file (\*.tfvars) file, you can manually define the file to use during command line.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"
```

```
Terraform used the selected providers to generate the following execution plan.  
following symbols:
```

```
+ create
```

```
Terraform will perform the following actions:
```

```
# aws_instance.web will be created
```

```
+ resource "aws_instance" "web" {  
  + ami              = "ami-1234"  
  + arn              = (known after apply)  
  + associate_public_ip_address = (known after apply)  
  + availability_zone = (known after apply)  
  + cpu_core_count    = (known after apply)  
  + cpu_threads_per_core = (known after apply)  
  + disable_api_stop   = (known after apply)  
  + disable_api_termination = (known after apply)  
  + ebs_optimized      = (known after apply)
```

## Point to Note

If file name is terraform.tfvars → Terraform will automatically load values from it.

If file name is different like prod.tfvars → You have to explicitly define the file during plan / apply operation.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"

Terraform used the selected providers to generate the following execution plan,
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-1234"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
```

# **Approach to Variable Assignment**



# Understanding the Base

By default, whenever you define a variable, you must also set a value associated with it.

```
resource "aws_instance" "web" {  
  ami          = "ami-1234"  
  instance_type = var.instance_type  
}
```

Main Configuration File

```
variables.tf > ...  
  
variable "instance_type" {}
```

variables.tf

**VARIABLE IS DEFINED**



**BUT WHERE IS THE VALUE**

## Add a Value in CLI

If you have not defined a value for a variable, Terraform will ask you to input the value in CLI Prompt when you run terraform plan / apply operation.

```
C:\Users\zealv\kplabs-terraform>terraform plan
var.instance_type
  Enter a value:
```

# Declaring Variable Values

When variables are declared in your configuration, they can be set in a number of ways:

1. Variable Defaults.
2. Variable Definition File (\*.tfvars)
3. Environment Variables
4. Setting Variables in the Command Line.

# Variable Defaults

You can set a default value for a variable.

If there is no value supplied, the default value will be taken.



variables.tf > ...

```
variable "app_port" {  
    default = "8080"  
}
```

# Variable Definition File (\*.tfvars)

Variable Values can be defined in \*.tfvars file.

```
prod.tfvars > ...  
vpn_ip = "101.0.62.30/32"  
  
app_port = "8080"  
  
instance_type = "m5.large"
```

# Setting Variable in Command Line

To specify individual variables on the command line, use the `-var` option when running the `terraform plan` and `terraform apply` commands:

```
C:\Users\zealiv\kplabs-terraform>terraform plan -var="instance_type=m5.large"

Terraform used the selected providers to generate the following execution plan. Resource
following symbols:
  - create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-1234"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + get_password_data      = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "m5.large"
```

# Setting Variable through Environment Variables

Terraform searches the environment of its own process for environment variables named **TF\_VAR\_** followed by the name of a declared variable.

```
C:\Users\zealv\kplabs-terraform>echo %TF_VAR_instance_type%
t2.large

C:\Users\zealv\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following execution plan
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
    + ami                        = "ami-1234"
    + arn                       = (known after apply)
    + associate_public_ip_address = (known after apply)
    + availability_zone         = (known after apply)
```

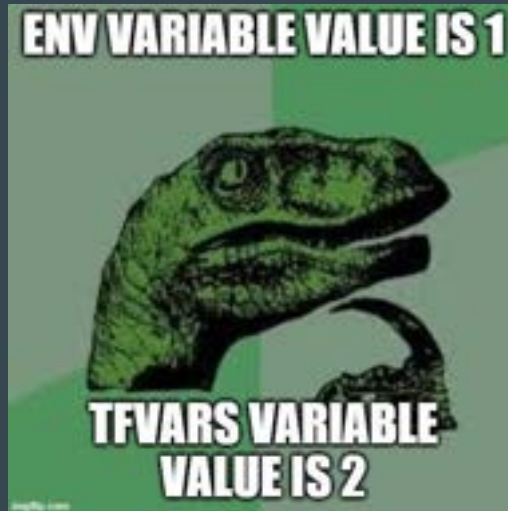


# **Variable Definition Precedence**

# Understanding the Base

Values for a variable can be defined at multiple different places.

What if values for a variable are different?



# Simple Example

```
variable "instance_type" {}
```

1. Default Value is t2.micro
2. Terraform.tfvars value is "t2.small"
3. Environment Variable TF\_VAR\_instance\_type = "t2.large"

Which value will Terraform take?

# Variable Definition Precedence

Terraform loads variables in the following order, with later sources taking precedence over earlier ones:

1. Environment variables
2. The terraform.tfvars file, if present.
3. The terraform.tfvars.json file, if present.
4. Any \*.auto.tfvars or \*.auto.tfvars.json files, processed in lexical order of their filenames.
5. Any -var and -var-file options on the command line

## Example 1

ENV Variable of TF\_VAR\_instance\_type = "t2.micro"

Value in terraform.tfvars = "t2.large"

Final Result = "t2.large"

## Example 2

1. ENV Variable of `TF_VAR_instance_type` = "t2.micro"
2. Value in `terraform.tfvars` = "t2.large"
3. `terraform plan -var="instance_type=m5.large"`

Final Result = "m5.large"

# Data Types

# Setting the Base

Data type refers to the **type of value**.

Depending on the requirement, you can use wide variety of values in Terraform configuration.

| Example Data Type | Data Type |
|-------------------|-----------|
| "Hello World"     | String    |
| 7575              | Number    |



# Restricting Variable Value to Data Type

We can restrict the value of a variable to a data type.

Example:

Only numbers should be allowed in AWS Usernames.

```
variable "username" {  
    type = number  
}
```

# Data Types in Terraform

| Data Types | Description                                                                           |
|------------|---------------------------------------------------------------------------------------|
| string     | a sequence of Unicode characters representing some text, like "hello".                |
| number     | A Numeric value                                                                       |
| bool       | a boolean value, either true or false                                                 |
| list       | a sequence of values, like ["us-west-1a", "us-west-1c"]                               |
| set        | a collection of unique values that do not have any secondary identifiers or ordering. |
| map        | a group of values identified by named labels, like {name = "Mabel", age = 52}.        |
| null       | a value that represents absence or omission.                                          |

# **Data Type - List**

# List Data Type

Allows us to store **collection of values** for a single variable / argument.

Represented by a pair of square brackets containing a comma-separated sequence of values, like ["a", 15, true].

Useful when multiple values needs to be added for a specific argument

```
variable "my-list" {  
  type = list  
  default = ["mumbai", "bangalore", "delhi"]  
}
```

# Data Type and Documentation

Arguments for a resource requires specific data types.

Some argument requires list, some requires map and so on.

The details of data type expected for an argument is mentioned in documentation.

- `volume_tags` - (Optional) Map of tags to assign, at instance-creation time, to root and EBS volumes.

**NOTE:**

Do not use `volume_tags` if you plan to manage block device tags outside the `aws_instance` configuration, such as using `tags` in an `aws_ebs_volume` resource attached via `aws_volume_attachment`. Doing so will result in resource cycling and inconsistent behavior.

- `vpc_security_group_ids` - (Optional, VPC only) List of security group IDs to associate with.

## Use-Case 1: List Data Type

EC2 instance can have one or more security groups.

Requirement:

Create EC2 instance with 2 security groups attached.

# Specify the Type of Values in List

We can also specify the type of values expected in a list.

```
variable "my-list" {  
  type = list(number)  
  default = ["1", "2", "3"]  
}
```

# **Map - Data Type**



# Map Data Type

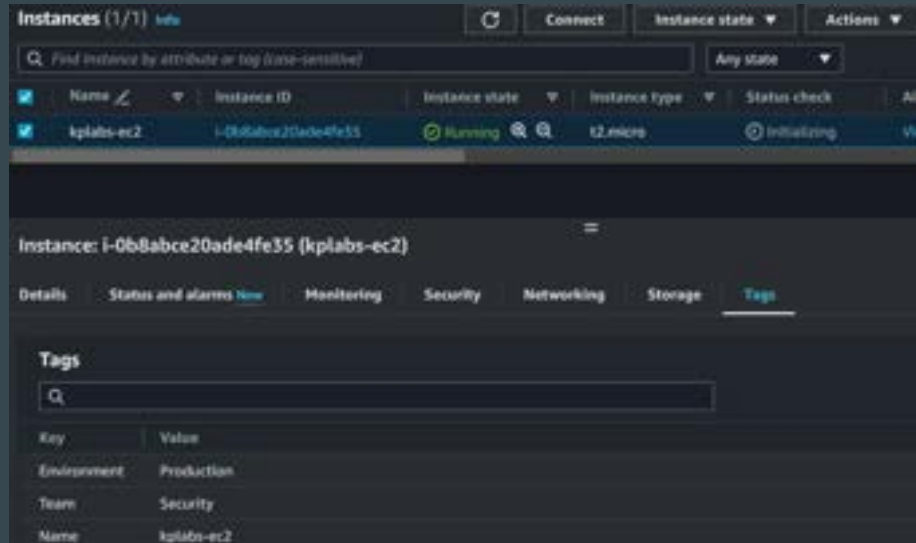
A map data type represents a collection of key-value pair elements

```
variable "instance_tags" {  
  type = map  
  default = {  
    Name = "app-server"  
    Environment = "development"  
    Team = "payments"  
  }  
}
```

# Use-Case of Map

We can add multiple tags to AWS resources.

These tags are key-value pairs.

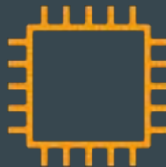


# **The COUNT Meta-Argument**

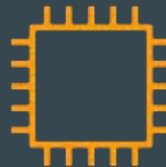
# Understanding the Challenge

By default, a resource block configures one real infrastructure object.

```
resource "aws_instance" "first_ec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```



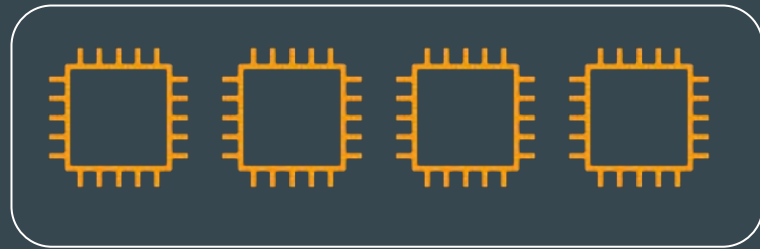
```
resource "aws_instance" "second_ec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```



# Understanding the Use-Case

Sometimes you want to manage several similar objects (like a fixed pool of compute instances) without writing a separate block for each one.

```
resource "aws_instance" "second_ec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  <something-logic-here>  
}
```

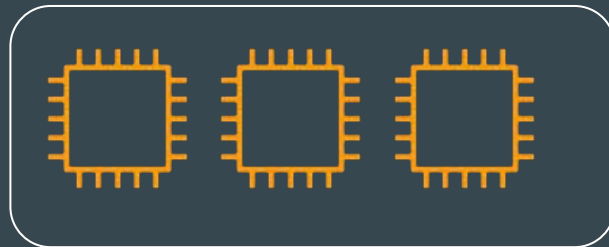


Pool of Servers

# Introducing Count Argument

The count argument accepts a whole number, and creates that many instances of the resource.

```
resource "aws_instance" "second_ec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  count = 3  
}
```

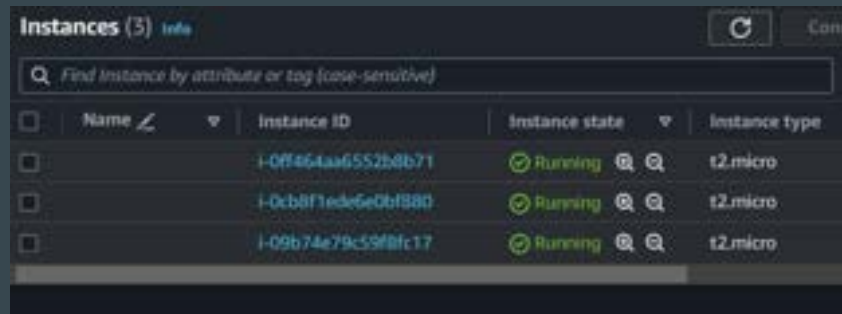


Pool of Servers

# Challenges with Count

The instances created through count and identical copies, but you might want to customize certain properties for each one.

```
resource "aws_instance" "first_ec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  count = 3  
}
```



Instances (3) Info

Find Instance by attribute or tag (case-sensitive)

| <input type="checkbox"/> | Name ↙ | Instance ID         | Instance state | Instance type |
|--------------------------|--------|---------------------|----------------|---------------|
| <input type="checkbox"/> |        | i-0ff464aa6552b8b71 | Running        | t2.micro      |
| <input type="checkbox"/> |        | i-0cbbf1ede5e0bf880 | Running        | t2.micro      |
| <input type="checkbox"/> |        | i-09b74e79c59f8fc17 | Running        | t2.micro      |

## Example - IAM User

For many resources, exact identical copies are not required and will not work.

Example: You cannot have multiple AWS Users with exact same name.

```
resource "aws_iam_user" "lb" {  
  name = "developer-user"  
  count = 3  
}
```



```
Error: creating IAM User (developer-user): operation error IAM: CreateUser, https response error Status  
: 44c3ff43-0bd4-430e-a838-ba5e43aa569a, EntityAlreadyExists: User with name developer-user already exists  
  
with aws_iam_user.lb[0],  
on iam.tf line 2, in resource "aws_iam_user" "lb":  
  2: resource "aws_iam_user" "lb" {
```



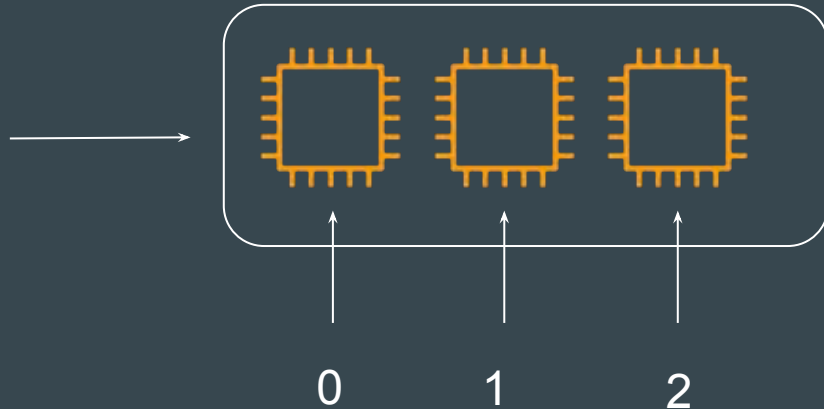
**COUNT.INDEX**

# Introducing Count Index

When using count, you can also make use of `count.index` which allows better flexibility.

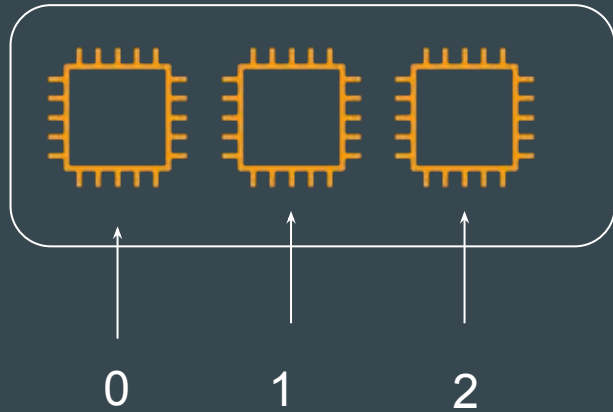
This attribute holds a distinct index number, starting from 0, that uniquely identifies each instance created by the count meta-argument.

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  count = 3  
}
```



# Tabular Representation

Following representation shows each EC2 instance's resource address that contains the index.



| Resource Address                   | Description         |
|------------------------------------|---------------------|
| <code>aws_instance.myec2[0]</code> | First EC2 Instance  |
| <code>aws_instance.myec2[1]</code> | Second EC2 Instance |
| <code>aws_instance.myec2[2]</code> | Third EC2 Instance  |

# CLI Output

Within CLI output, you will be able to see the index value of resource.

```
# aws_instance.first_ec2[0] will be created
+ resource "aws_instance" "first_ec2" {
  + ami                    = "ami-00c39f71452c08778"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
```

First EC2

```
# aws_instance.first_ec2[1] will be created
+ resource "aws_instance" "first_ec2" {
  + ami                    = "ami-00c39f71452c08778"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
```

Second EC2

## Example - IAM User Use-Case

The `${count.index}` is dynamic expression that utilizes the `count.index` attribute so that each username will be unique.

```
resource "aws_iam_user" "lb" {  
  name = "developer-user.${count.index}"  
  count = 3  
}
```



**Users (7)** [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search:  3 matches

| <input type="checkbox"/> | User name ▲                      | Path ▼ | Groups ▼ | Last activity |
|--------------------------|----------------------------------|--------|----------|---------------|
| <input type="checkbox"/> | <a href="#">developer-user-0</a> | /      | 0        | -             |
| <input type="checkbox"/> | <a href="#">developer-user-1</a> | /      | 0        | -             |
| <input type="checkbox"/> | <a href="#">developer-user-2</a> | /      | 0        | -             |

# Enhancing with Count Index

You can use `count.index` to iterate through the list to have more customization.

```
variable "dev_names" {  
  type = list  
  default = ["alice", "bob", "johncorner"]  
}  
  
resource "aws_iam_user" "lb" {  
  name = var.dev_names[count.index]  
  count = 3  
}
```



**Users (7)** [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

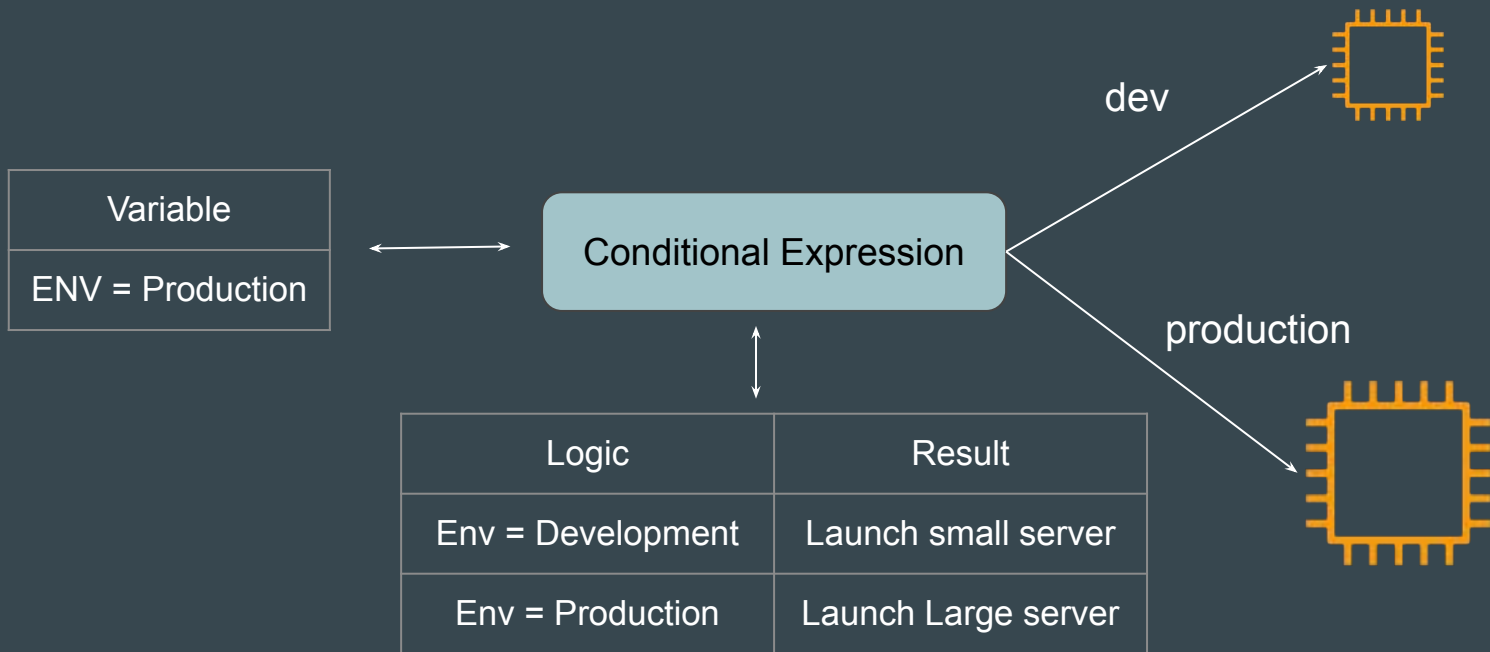
Q Search

| <input type="checkbox"/> | User name ▾                | Path ▾ | Groups ▾ | Last activity |
|--------------------------|----------------------------|--------|----------|---------------|
| <input type="checkbox"/> | <a href="#">alice</a>      | /      | 0        | -             |
| <input type="checkbox"/> | <a href="#">bob</a>        | /      | 0        | -             |
| <input type="checkbox"/> | <a href="#">johncorner</a> | /      | 0        | -             |

# **Conditional Expressions**

# Setting the Base

Conditional expressions in Terraform allow you to choose between two values based on a condition





# Syntax of Conditional Expression

The syntax of a conditional expression is as follows:

`condition ? true_val : false_val`

If condition is true then the result is true\_val. If condition is false then the result is false\_val.

# Conditional Expression Based on Use-Case

If Environment is Development, t2.micro instance type should be used.

If Environment is NOT development, m5.large instance type should be used.

```
variable "environment" {  
    default = "development"  
}  
  
resource "aws_instance" "example" {  
    instance_type = var.environment == "development" ? "t2.micro" : "m5.large"  
    ami           = "ami-12345678"  
}
```

# Conditional Expression with Multiple Variables

In the following example, **only if** env=production and region=us-east-1, the larger instance type of m5.large can be used.

```
variable "environment" {
    default = "production"
}

variable "region" {
    default = "ap-south-1"
}

resource "aws_instance" "example" {
    instance_type = var.environment == "production" && var.region == "us-east-1" ? "m5.large" : "t2.micro"
    ami           = "ami-12345678"
}
```

# **Terraform Functions**

# Basics of Function

A function is a block of code that performs a specific task.



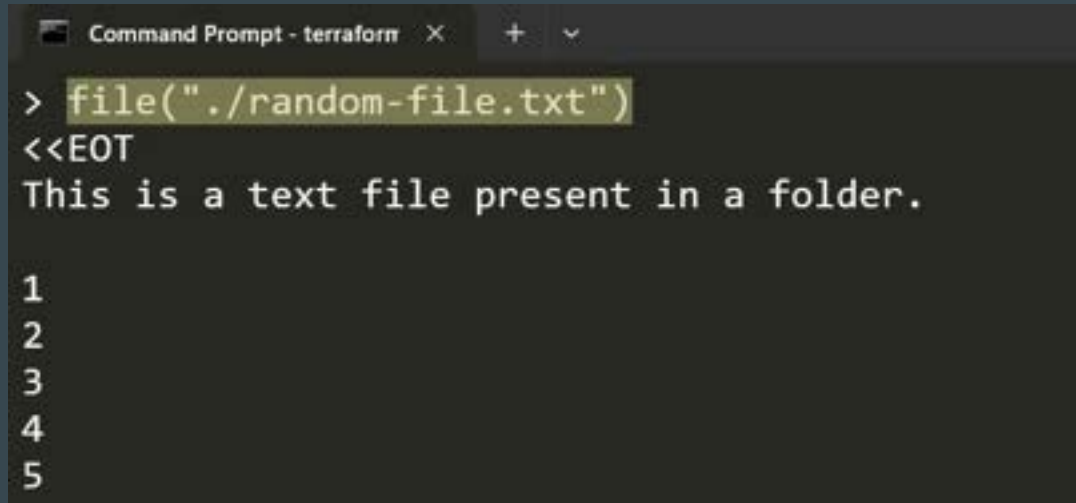
# Function 1 - MAX

`max ()` takes one or more numbers and returns the greatest number.

```
C:\Users\zealv\kplabs-terraform>terraform console  
> max(10,20,30)  
30
```

## Function 2 - FILE

`file ()` reads the contents of a file at the given path and returns them as a string.



```
Command Prompt - terraform X + v
> file("./random-file.txt")
<<EOT
This is a text file present in a folder.

1
2
3
4
5
```

# Introducing Terraform Console

Terraform Console provides an interactive environment specifically **designed to test functions** and experiment with expressions before integrating them into your main code.

```
C:\Users\zealv\kplabs-terraform>terraform console  
> max(10,20,30)  
30
```



# Importance of File Function

file reads the contents of a file at the given path and returns them as a string.

```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": "ec2:*",
        "Effect": "Allow",
        "Resource": "*"
      },
      {
        "Effect": "Allow",
        "Action": "elasticloadbalancing:*",
        "Resource": "*"
      }
    ]
  })
}
```

Before



```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = file("./ec2-policy.json")
}
```

After

# Functions in Terraform

Terraform has wide variety of functions available to achieve different set of use-cases.

Functions are grouped into categories. Some of these include:

| Function Categories   | Functions Available                     |
|-----------------------|-----------------------------------------|
| Numeric Functions     | abs, ceil, floor, max, min              |
| String Functions      | concat, replace, split, tolower,toupper |
| Collection Functions  | element, keys, length, merge, sort      |
| Filesystem Functions: | file, filebase64, dirname               |

## Important Point to Note

The Terraform language **does not support user-defined functions**, and so only the functions built in to the language are available for use

The documentation includes a page for all of the available built-in functions.

# **Challenge - Analyzing Code Containing Functions**

## Setting the Base

As part of this challenge, you will be given a code that contains multiple sets of Terraform Functions.

You have to analyze what this code does without running the “apply” operation.

```
variable "ami" {
  type = map
  default = {
    "us-east-1" = "ami-08a0d1e16fc3f61ea"
    "us-west-2" = "ami-0d6621c01e8c2de2c"
    "ap-south-1" = "ami-0470e33cd681b2476"
  }
}

resource "aws_instance" "app-dev" {
  ami = lookup(var.ami,var.region)
  instance_type = "t2.micro"
  count = length(var.tags)

  tags = {
    Name = element(var.tags,count.index)
    CreationDate = formatdate("DD MMM YYYY hh:mm ZZ",timestamp())
  }
}
```

# Overall Workflow

1. Analyze what exactly the given code in GitHub will do without running the “apply operation”.
2. Analyze the outcome by applying function using Terraform Console and reading the documentation.
3. Make a note of it.
4. Run the “terraform apply” operation to verify if it matches your findings.

# **Solution - Analyzing Code Containing Functions**

# 1- Analyzing Lookup Function

lookup retrieves the value of a single element from a map, given its key.

Format: lookup(map, key, default)

```
> lookup({a="ay", b="bee"}, "a", "what?")  
ay  
> lookup({a="ay", b="bee"}, "c", "what?")  
what?
```



# Testing Lookup Function

To test lookup function, add the details that are part of the map associated with variable of ami and the default value of variable of region.

```
variable "ami" {  
  type = map  
  default = {  
    "us-east-1" = "ami-08a0d1e16fc3f61ea"  
    "us-west-2" = "ami-0d6621c01e8c2de2c"  
    "ap-south-1" = "ami-0470e33cd681b2476"  
  }  
}
```

```
variable "region" {  
  default = "us-east-1"  
}
```

terraform console

```
> lookup({"us-east-1" = "ami-08a0d1e16fc3f61ea", "us-west-2" = "ami-0d6621c01e8c2de2c", "ap-south-1" = "ami-0470e33cd681b2476"},  
"us-east-1")  
"ami-08a0d1e16fc3f61ea"
```

## 2 - Analyzing Length Function

`length` determines the length of a given list, map, or string.

```
> length([])
0
> length(["a", "b"])
2
> length({"a" = "b"})
1
> length("hello")
5
```

# Testing Length Function

Code: `count = length(var.tags)`

```
variable "tags" {  
  type = list  
  default = ["firstec2", "secondec2"]  
}
```



```
> length(["firstec2", "secondec2"])  
2
```

### 3 - Analyzing Element Function

element retrieves a single element from a list.

Format: `element(list, index)`

```
> element(["a", "b", "c"], 1)  
b
```

# Testing Element Function

Code:    Name = element(var.tags,count.index)

```
variable "tags" {  
  type = list  
  default = ["firstec2","secondec2"]  
}
```



```
> element(["firstec2","secondec2"],0)  
"firstec2"
```

```
> element(["firstec2","secondec2"],1)  
"secondec2"
```

## 4 - Analyzing TimeStamp Function

timestamp returns a UTC timestamp string in RFC 3339 format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```

# Testing TimeDate Function

A simple call to the timestamp () returns the timestamp value

```
> timestamp()  
"2024-06-16T13:43:13Z"
```

## 5 - Analyzing Formatdate Function

formatdate converts a timestamp into a different time format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```



## 5 - Testing Formatdate Function

Code Block:

```
CreationDate = formatdate("DD MMM YYYY hh:mm ZZZ",timestamp())
```

```
> formatdate("DD MMM YYYY hh:mm ZZZ",timestamp())  
"16 Jun 2024 13:46 UTC"
```

# Final Result

1. Two set of EC2 instances will be created.
2. Name of EC2 will be “firstec2”, and “secondec2”
3. EC2 will have a tag of creation date with the timestamp value

# You are Awesome

Learning “Terraform Function” is a longer learning journey compared to other topics.

In today’s video, we learnt the practical aspect of Function in Terraform Code.



# Local Values

# Understanding the Challenge

Various resources in your project can have common values like tags.

Repeating these values across multiple resource blocks increases the code length and makes it difficult to manage in larger projects.

```
resource "aws_instance" "myec2" {  
    ami = "ami-00c39f71452c08778"  
    instance_type = "t2.micro"  
    tags = {  
        Team = "payments-team"  
    }  
}
```

```
resource "aws_security_group" "allow_tls" {  
    name = "firewall_sg"  
    tags = {  
        Team = "payments-system"  
    }  
}
```

# Solution using Variables

One solution is to centralize these common values using Variables

```
variable "payment_tag" {  
  type = map  
  default = {  
    Team = "payments-team"  
  }  
}
```

Variable

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  tags = var.payment_tag  
}
```

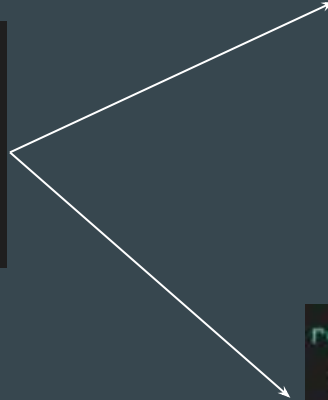
```
resource "aws_security_group" "allow_tls" {  
  name = "firewall_sg"  
  tags = var.payment_tag  
}
```

# Introducing Local Values

Local Values are similar to Variables in a sense that it allows you to store data centrally and that can be referenced in multiple parts of configuration.

```
locals {  
  common_tags = {  
    Team = "payments-team"  
  }  
}
```

Locals



```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  tags = local.common_tags  
}
```

```
resource "aws_security_group" "allow_tls" {  
  name = "firewall_sg"  
  tags = local.common_tags  
}
```

## Additional Benefit of Locals

You can add expressions to locals, which allows you to compute values dynamically

```
locals {  
  # Ids for multiple sets of EC2 instances, merged together  
  instance_ids = concat(aws_instance.blue.*.id, aws_instance.green.*.id)  
}  
  
locals {  
  # Common tags to be assigned to all resources  
  common_tags = {  
    Service = local.service_name  
    Owner   = local.owner  
  }  
}
```



# Locals vs Variables

Variable value can be defined in wide variety of places like terraform.tfvars, ENV Variables, CLI and so on.

Locals are more of a private resource. You have to directly modify the code.

Locals are used when you want to avoid repeating the same expression multiple times.

# Important Points to Note

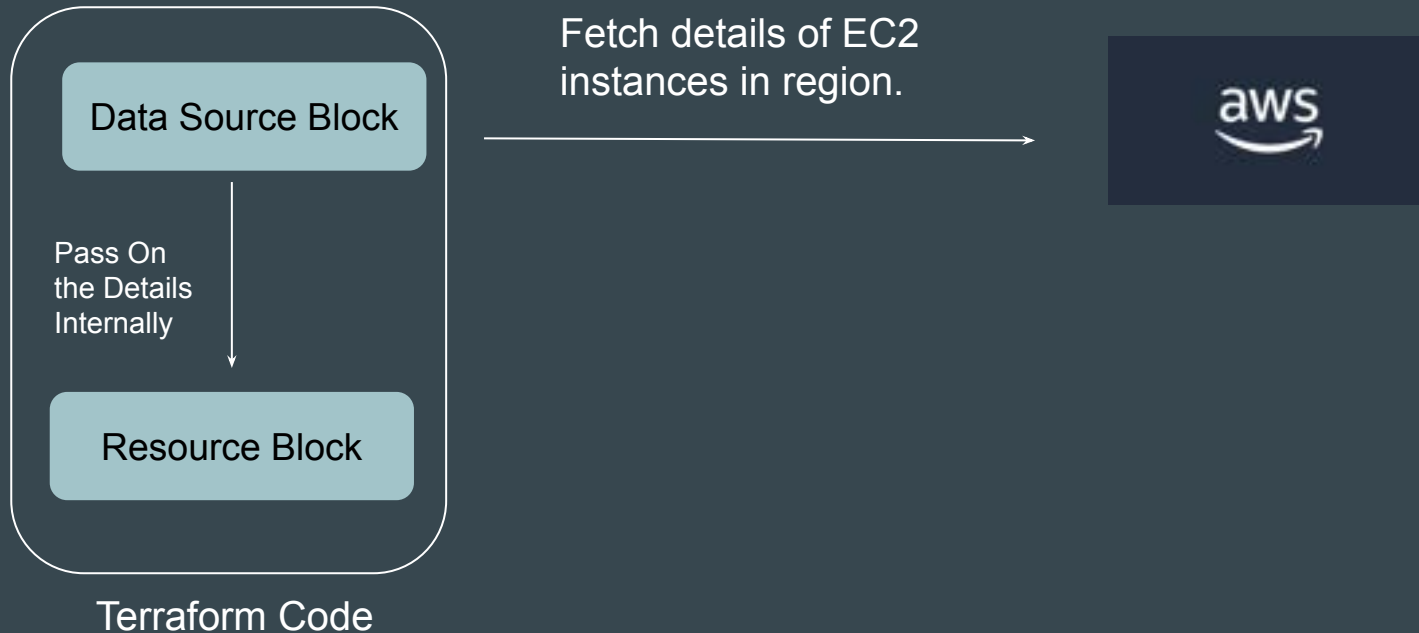
Local values are often referred to as just "locals"

Local values are created by a **locals** block (plural), but you reference them as attributes on an object named **local** (singular)

# **Data Sources**

# Introducing Data Sources

Data sources allow Terraform to **use / fetch** information defined outside of Terraform



## Example 1 - Reading Info of DO Account

Following data source code is used to get information on your DigitalOcean account.

```
data "digitalocean_account" "example" {}
```

## Example 2 - Reading a File

Following data source allows you to read contents of a file in your local filesystem.

```
data "local_file" "foo" {  
  filename = "${path.module}/demo.txt"  
}
```

# Clarity regarding path.module

`${path.module}` returns the current file system path where your code is located.

```
data "local_file" "foo" {  
  filename = "${path.module}/demo.txt"  
}
```

## Example 3 - Fetch EC2 Instance Details

Following data source code is used to fetch details about the EC2 instance in your AWS region.

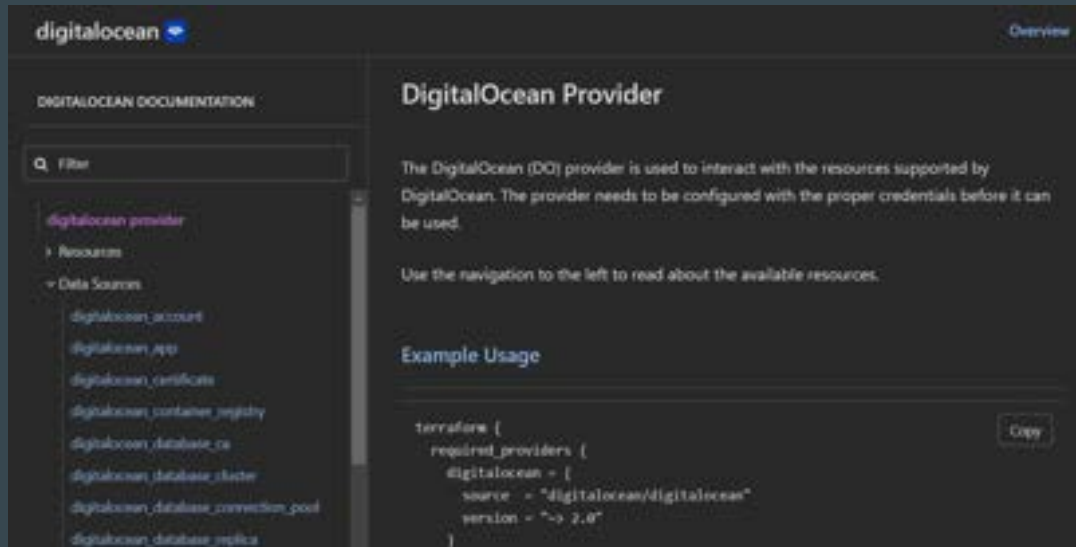
```
data "aws_instances" "example" {}
```



# **Data Sources Documentation Reference**

# Finding Available Data Sources

List of available data source are associated with each resource of a provider.



# **Data Sources Format**

# Understanding the Basic Structure

A data source is accessed via a special kind of resource known as a **data resource**, declared using a **data** block:

Following data block requests that Terraform read from a given data source ("aws\_instance") and export the result under the given local name ("foo").

```
data "aws_instance" "foo" {}
```

# Filter Structure

Within the block body (between { and }) are query constraints defined by the data source.

```
data "aws_instance" "foo" {  
  filter {  
    name      = "tag:Team"  
    values    = ["Production"]  
  }  
}
```

# Fetching Latest OS Image Using Data Sources

# Understanding the Requirement

You have been given a requirement to write a Terraform code that creates EC2 instance using latest OS Image of Amazon Linux.

# Approach that New User will Take

We want to use the latest OS image for creating server in AWS.

Steps that we typically follow:

1. Go to EC2 Console.
2. Fetch the latest AMI ID
3. Add that AMI ID in Terraform code.





# Sample Reference Code

```
resource "aws_instance" "web" {  
  ami          = "ami-0440d3b780d96b29d"  
  instance_type = "t2.micro"  
}
```

Hard Coded static value

# Static Information is Boring

Hardcoding static details in your Terraform code will lead you to repeatedly modify your code to meet changing requirements.



## Another Challenge with Static Values

In many of the cases, the static value changes depending on the region.

Example: AMI IDs are specific to region.

Hardcoded AMI in code will only work for single region.

Mumbai Region



ami-1234

Singapore Region



ami-5678

Tokyo Region

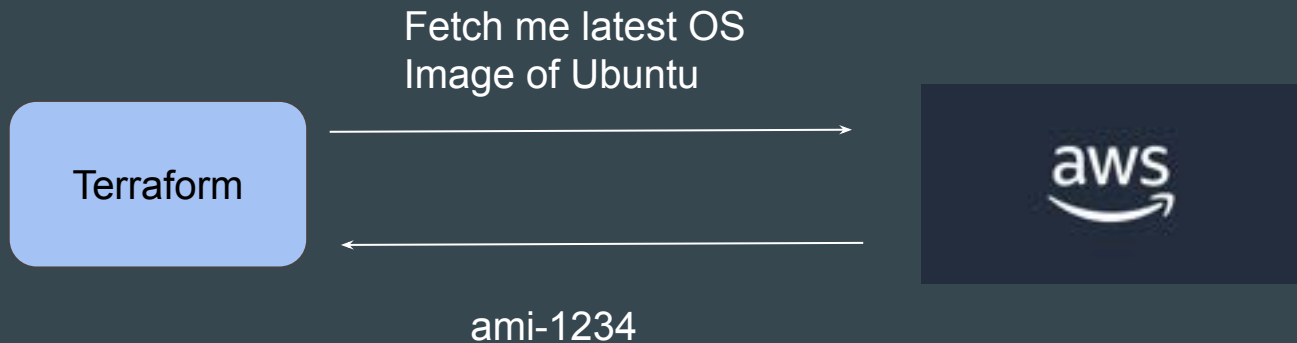


ami-9012

# Time to be Pros - Dynamic Configuration

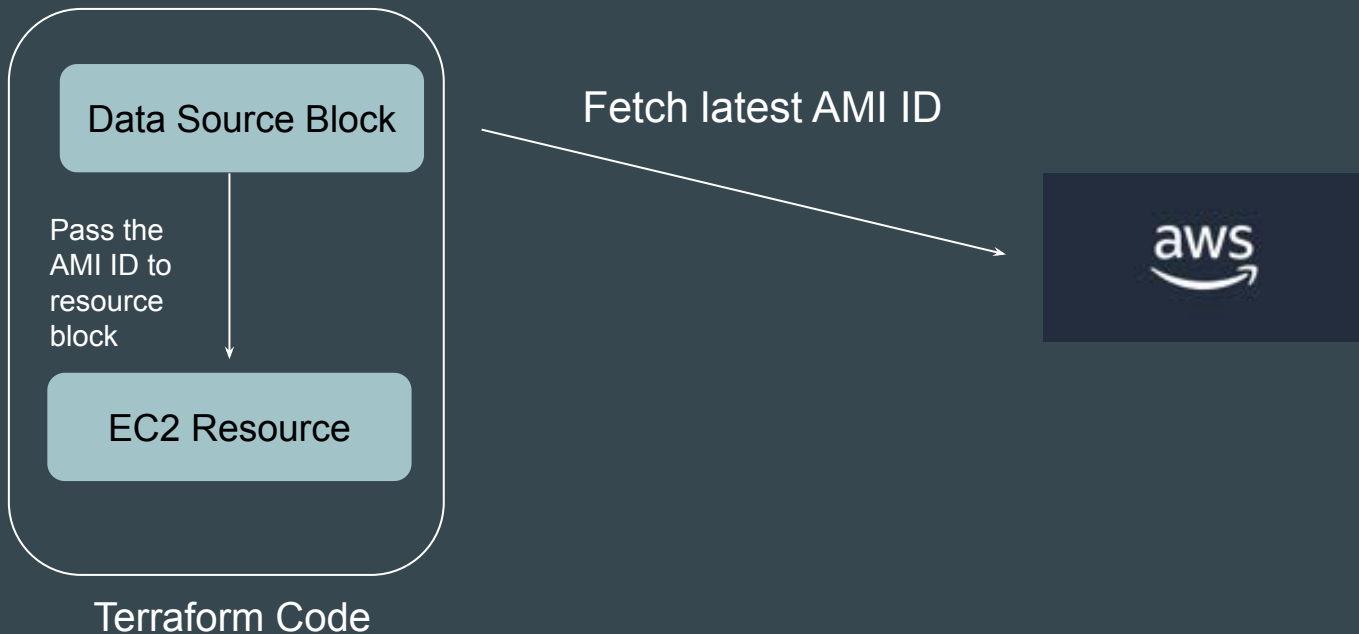
We want Terraform to automatically query the latest OS image in AWS or any other provide and use that for creating server.

We need code which works for all region without modification.



# Introducing Data Sources

Data sources allow Terraform to **use information defined outside of Terraform** and we can use that information to provision resources.



# Debugging Terraform

# Basics of Debugging

Debugging is the **process of finding the root cause** of a specific issue.

30-40% of the time of a System Administrator goes into Debugging.



# Example - SSH Verbosity

One of the important requirement in Debugging is getting detailed Log

Depending on the application, the approach to get detailed logs will differ.

```
C:\Users\zealv\OneDrive\Desktop>ssh -v -i dp_rsa root@64.227.154.149
OpenSSH_for_Windows_8.6p1, LibreSSL 3.4.3
debug1: Authenticator provider $SSH_SK_PROVIDER did not resolve; disabling
debug1: Connecting to 64.227.154.149 [64.227.154.149] port 22.
debug1: Connection established.
debug1: identity file dp_rsa type -1
debug1: identity file dp_rsa-cert type -1
debug1: Local version string SSH-2.0-OpenSSH_for_Windows_8.6
debug1: Remote protocol version 2.0, remote software version OpenSSH_9.6p1 Ubuntu-3ubuntu13.4
debug1: compat_banner: match: OpenSSH_9.6p1 Ubuntu-3ubuntu13.4 pat OpenSSH* compat 0x04000000
debug1: Authenticating to 64.227.154.149:22 as 'root'
debug1: load_hostkeys: fopen C:\\Users\\zealv\\.ssh\\known_hosts2: No such file or directory
debug1: load_hostkeys: fopen __PROGRAMDATA__\\ssh\\ssh_known_hosts: No such file or directory
debug1: load_hostkeys: fopen __PROGRAMDATA__\\ssh\\ssh_known_hosts2: No such file or directory
debug1: SSH2_MSG_KEXINIT sent
debug1: SSH2_MSG_KEXINIT received
debug1: kex: algorithm: curve25519-sha256
debug1: kex: host key algorithm: ssh-ed25519
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: kex: client->server cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: expecting SSH2_MSG_KEX_ECDH_REPLY
```



# Debugging in Terraform

Similar to SSH Verbosity, even Terraform allows us to set wide variety of log levels for getting detailed logs for debugging purpose.

```
root@test-kplabs:~# terraform plan
2024-08-24T05:01:48.844Z [INFO] Terraform version: 1.9.5
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/terraform-svchost v0.1.1
2024-08-24T05:01:48.845Z [DEBUG] using github.com/zclconf/go-cty v1.14.4
2024-08-24T05:01:48.845Z [INFO] Go runtime version: go1.22.5
2024-08-24T05:01:48.845Z [INFO] CLI args: []string{"terraform", "plan"}
2024-08-24T05:01:48.845Z [TRACE] Stdout is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stderr is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stdin is a terminal
2024-08-24T05:01:48.845Z [DEBUG] Attempting to open CLI config file: /root/.terraformrc
2024-08-24T05:01:48.845Z [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
2024-08-24T05:01:48.845Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /var/lib/snapd/desktop/terraform/plugins
2024-08-24T05:01:48.848Z [INFO] CLI command args: []string{"plan"}
```

# Understanding the Basics

Terraform has detailed logs that you can enable by setting the **TF\_LOG** environment variable to any value.

You can set TF\_LOG to one of the log levels (in order of decreasing verbosity)

| Log Level |
|-----------|
| TRACE     |
| DEBUG     |
| INFO      |
| WARN      |
| ERROR     |

# Storing the Logs to File

To persist logged output you can set **TF\_LOG\_PATH** in order to force the log to always be appended to a specific file when logging is enabled

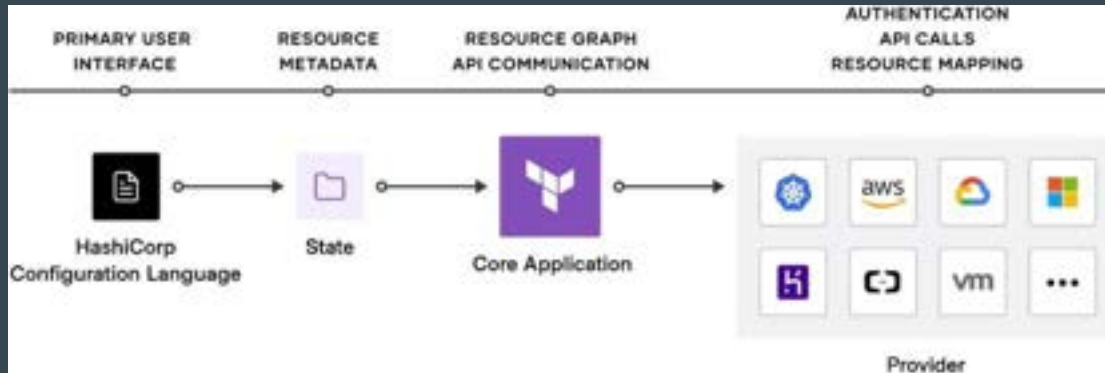
```
root@test-kplabs:~# terraform plan
2024-08-24T05:01:48.844Z [INFO] Terraform version: 1.9.5
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/terraform-svchost v0.1.1
2024-08-24T05:01:48.845Z [DEBUG] using github.com/zclconf/go-cty v1.14.4
2024-08-24T05:01:48.845Z [INFO] Go runtime version: go1.22.5
2024-08-24T05:01:48.845Z [INFO] CLI args: []string{"terraform", "plan"}
2024-08-24T05:01:48.845Z [TRACE] Stdout is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stderr is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stdin is a terminal
2024-08-24T05:01:48.845Z [DEBUG] Attempting to open CLI config file: /root/.terraformrc
2024-08-24T05:01:48.845Z [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
2024-08-24T05:01:48.845Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /var/lib/snapd/desktop/terraform/plugins
2024-08-24T05:01:48.848Z [INFO] CLI command args: []string{"plan"}
```



# **Terraform Troubleshooting Model**

# Terraform Troubleshooting Model

There are four potential types of issues that you could experience with Terraform Language, State, Core, and Provider Errors.



# 1 - Language Errors

In most of the cases, the errors that you will face will be related to this.

When Terraform encounters a syntax error in your configuration, it prints out the line numbers and an explanation of the error.

```
C:\kplabs-terraform>terraform plan
```

```
Error: Unclosed configuration block
```

```
on demo.tf line 11, in resource "aws_iam_user" "dev":  
11: resource "aws_iam_user" "dev" {
```

```
There is no closing brace for this block before the end of the file.  
elsewhere in this file.
```

## 2 - State Errors

If state is out of sync, Terraform may destroy or change your existing resources.

If state is locked, you will also be blocked from running write operations.

```
C:\kplabs-terraform>terraform plan
```

```
Error: Error acquiring the state lock
```

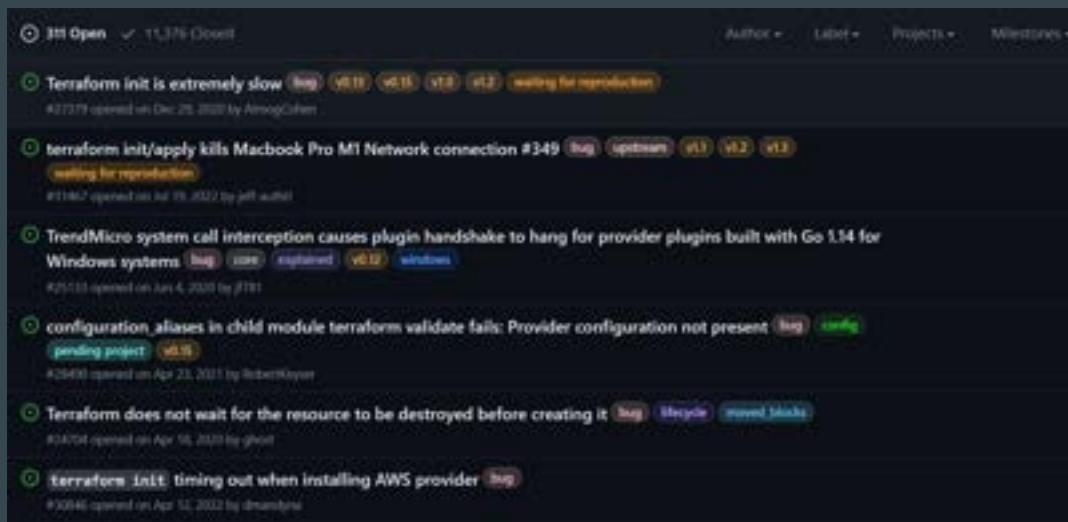
```
Error message: Failed to read state file: The state file could not be read: read terraform.tfstate  
access the file because another process has locked a portion of the file.
```

```
Terraform acquires a state lock to protect the state from being written  
by multiple users at the same time. Please resolve the issue above and try  
again. For most commands, you can disable locking with the "-lock=false"  
flag, but this is not recommended.
```

### 3 - Core errors

These errors are directly related to the main Terraform application.

Errors produced at this level may be a bug.

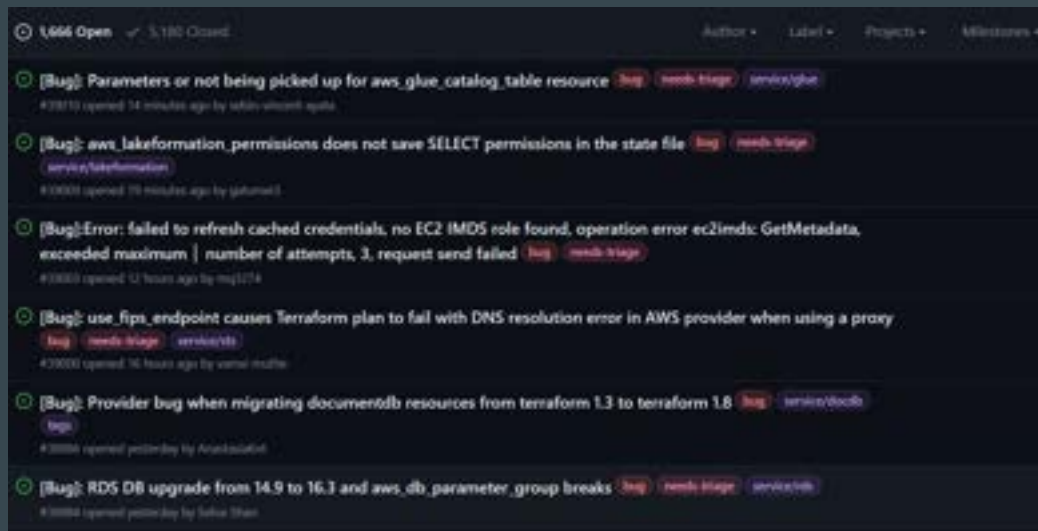




## 4 - Provider errors

These set of errors are primarily related to the provider plugins.

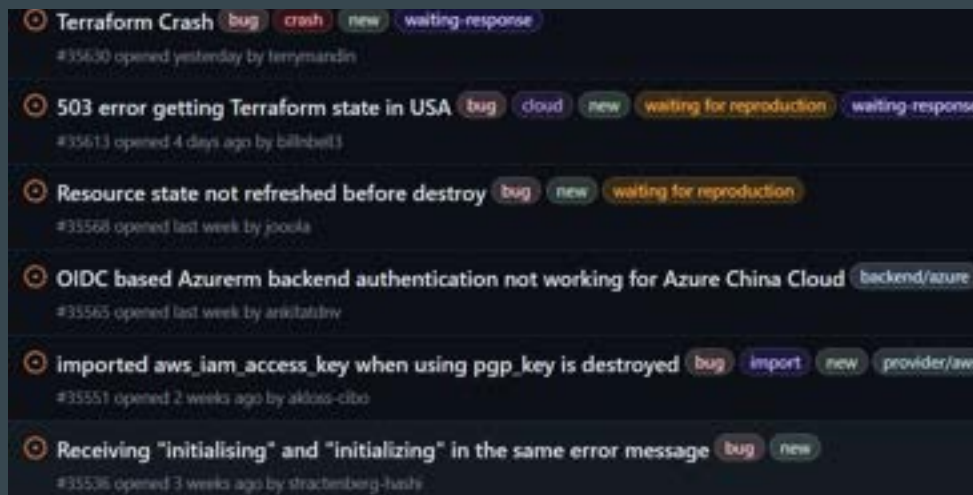
Use the Provider GitHub page for reporting and identifying the issue.



# **Reporting Terraform Bugs**

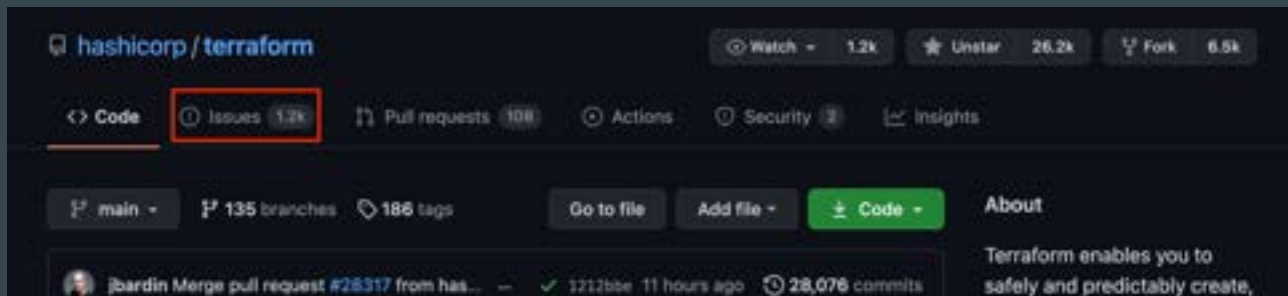
# Reporting Bugs

You can report bugs in the Terraform Core GitHub page or appropriate provider page.



# 1 - Navigate to Issues

First, navigate to the Terraform GitHub repository and choose "Issues" from the top tabs.



## 2 - Choose "New Issue".

The screenshot shows the GitHub repository page for `hashicorp/terraform`. The repository has 1.2k watches, 26.2k stars, and 6.5k forks. The 'Issues' tab is selected, showing 1.2k issues. The 'Pinned issues' section displays a pinned issue titled 'Terraform 0.15.0-rc2 released!' opened on Feb 24 by pkolyvas. Below the pinned issues, there is a search bar with the filter 'is:issue is:open'. To the right of the search bar are buttons for 'Labels' (247) and 'Milestones' (4). The 'New Issue' button is highlighted with a red box. Below the search bar, there is a table of issues with columns for checkboxes, status, counts, and filters. The first row shows '1,243 Open' and '15,597 Closed' issues. The second row shows an issue titled 'Modify an RDS Aurora cluster instance size independently from the cluster instance' with labels 'enhancement' and 'new'.

hashicorp / terraform

Watch 1.2k Unstar 26.2k Fork 6.5k

Code Issues 1.2k Pull requests 108 Actions Security 2 Insights

Pinned issues

Terraform 0.15.0-rc2 released! #27917 opened on Feb 24 by pkolyvas Open

Filters is:issue is:open Labels 247 Milestones 4 New Issue

1,243 Open 15,597 Closed Author Label Projects Milestones Assignee Sort

Modify an RDS Aurora cluster instance size independently from the cluster instance enhancement new

## 3 - Click “Get Started”



**Bug Report**  
Let us know about an unexpected error, a crash, or an incorrect behavior.

**Documentation Issue**  
Report an issue or suggest a change in the documentation.

**Feature Request**  
Suggest a new feature or other enhancement.

Get started

Get started

Get started

# 4 - Fill Core Terraform Template

**Terraform Version** \*

Run `terraform version` to show the version, and paste the result below. If you are not running the latest version of Terraform, please try upgrading because your issue may have already been fixed.

...output of `terraform version`...

**Terraform Configuration Files** \*

Paste the relevant parts of your Terraform configuration between the `""` marks below. For Terraform configs larger than a few resources, or that involve multiple files, please make a GitHub repository that we can clone, rather than copy-pasting multiple files in here.

""terraform  
\_\_terraform config\_\_  
""

**Debug Output** \*

Full debug output can be obtained by running Terraform with the environment variable `TF_LOG=TRACE`. Please create a GitHub Gist containing the debug output. Please do not paste the debug output in the issue, since debug output is long. Debug output may contain sensitive information. Please review it before posting publicly.

\_\_link to gist\_\_

**terraform fmt**



# Importance of Readability

When multiple engineers work on the same codebase, varying coding styles can make code reviews painful and debugging difficult.

The `terraform fmt` command formats Terraform configuration file contents so that it matches the canonical format and style

```
demo.tf > ...  
resource "local_file" "foo" {  
    content = "foo!"  
    filename = "${path.module}/demo.txt"  
}
```

Before fmt

```
demo.tf > ...  
resource "local_file" "foo" {  
    content = "foo!"  
    filename = "${path.module}/demo.txt"  
}
```

terraform fmt

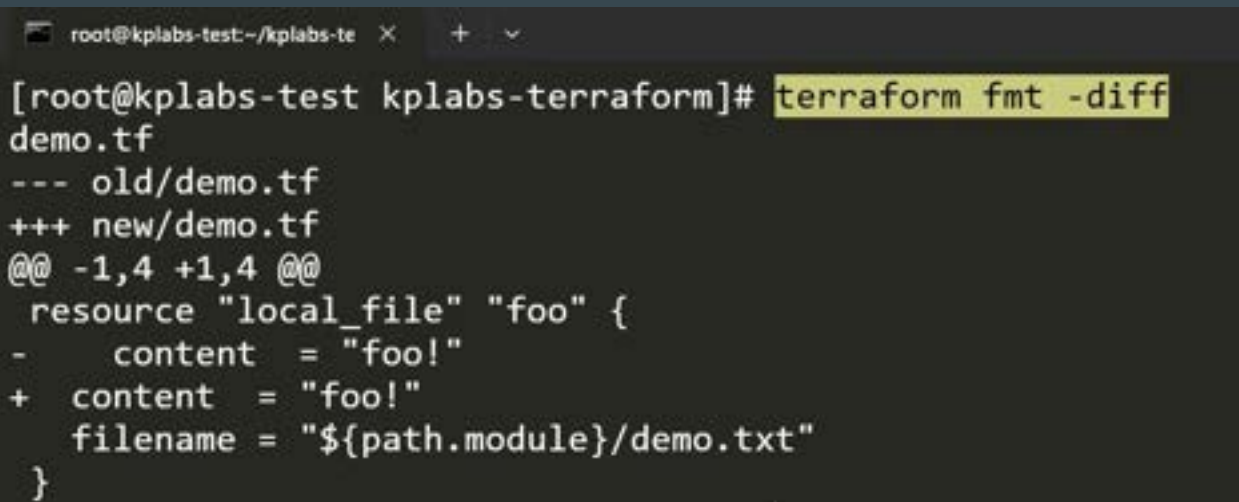
After fmt

```
demo.tf > ...  
resource "local_file" "foo" {  
    content = "foo!"  
    filename = "${path.module}/demo.txt"  
}
```

# Visualizing Changes

If you want to see exactly what terraform fmt is changing, use the **-diff** flag.

This is helpful for understanding the specific style adjustments being applied.

A terminal window with a dark background. The title bar shows 'root@kplabs-test:~/kplabs-te' and window controls. The prompt is '[root@kplabs-test kplabs-terraform]#'. The command 'terraform fmt -diff' is entered and highlighted in yellow. The output shows a diff for 'demo.tf' between 'old/demo.tf' and 'new/demo.tf'. It indicates a change at lines 1,4 to 1,4. The resource 'local\_file' named 'foo' is shown with a change in the 'content' attribute from 'foo!' to 'foo!' and the addition of a 'filename' attribute set to '\${path.module}/demo.txt'.

```
root@kplabs-test:~/kplabs-te x + - v
[root@kplabs-test kplabs-terraform]# terraform fmt -diff
demo.tf
--- old/demo.tf
+++ new/demo.tf
@@ -1,4 +1,4 @@
  resource "local_file" "foo" {
-     content  = "foo!"
+   content  = "foo!"
+   filename = "${path.module}/demo.txt"
  }
```

# Recursive Formatting

By default, terraform fmt only looks at the current directory.

If you have a complex project with subdirectories, use the **-recursive** flag.

```
C:\kplabs-terraform>terraform fmt -recursive  
demo.tf  
folder-1\01.tf
```

## Check Mode (Dry Run)

In CI/CD pipelines, you often want to verify if the code is formatted correctly without actually modifying it.

The `-check` flag returns 0 if all files are formatted correctly and 3 if any files require formatting.

```
C:\kplabs-terraform>terraform fmt -check  
demo.tf
```

## Point to Note

terraform fmt **does not change the behavior of your infrastructure**; it only changes how the code looks.

# **Terraform - Load Order and Semantics**

# Setting the Base

Terraform normally **loads all of the .tf and .tf.json files** within a directory

The files loaded must end in either .tf or .tf.json





## Point to Note

Since terraform loads all of the .tf and .tf.json files within a directory, **it expects each one to define a distinct set of configuration objects**. If two files attempt to define the same object, Terraform returns an error.

Example: The following two files are in same folder of “load-order”. It will result in error when terraform plan/apply operations run.

```
load-order > a_resource.tf > ...  
  
resource "local_file" "file_one" {  
  filename = "${path.module}/file_one.txt"  
  content  = "I am File One."  
}
```

a\_resource.tf

```
load-order > c_resource.tf > ...  
1  
2 resource "local_file" "file_one" {  
3   filename = "${path.module}/file_two.txt"  
4   content  = "I am File two."  
5 }
```

c\_resource.tf

## Point to Note

Terraform loads all configuration files within the directory specified in alphabetical order.

Terraform does not automatically read sub-directories unless explicitly called.

Organizing code into `networking.tf`, `compute.tf` is for humans, not Terraform.

# Dynamic Blocks

# Setting the Base

One of Terraform's powerful features for keeping your code DRY (Don't Repeat Yourself) is the Dynamic Block.

If you have ever find yourself copy-pasting the same configuration block (like ingress rules, etc) over and over again within a single resource, Dynamic Blocks are the solution.

```
resource "aws_security_group" "demo_sg" {  
  name      = "sample-sg"  
  
  ingress {  
    from_port = 8200  
    to_port   = 8200  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
resource "aws_security_group" "demo_sg" {  
  name      = "sample-sg"  
  
  ingress {  
    from_port = 8300  
    to_port   = 8300  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

# Introducing Dynamic Blocks

A dynamic block allows you to generate these nested blocks programmatically based on a variable (list or map), rather than writing them out manually.

```
resource "aws_security_group" "dynamicsg" {  
  name      = "dynamic-sg"  
  
  dynamic "ingress" {  
    for_each = var.sg_ports  
    content {  
      from_port   = ingress.value  
      to_port     = ingress.value  
      protocol    = "tcp"  
      cidr_blocks = ["0.0.0.0/0"]  
    }  
  }  
}
```

```
variable "sg_ports" {  
  type      = list(number)  
  default   = [8200, 8201, 8300, 9200, 9500]  
}
```



---

# Terraform Validate

Terraform in detail

---

# Overview of Terraform Validate

Terraform Validate primarily checks whether a configuration is syntactically valid.

It can check various aspects including unsupported arguments, undeclared variables and others.

```
resource "aws_instance" "myec2" {  
  ami          = "ami-082b5a644766e0e6f"  
  instance_type = "t2.micro"  
  sky = "blue"  
}
```

bash-4.2# terraform validate

Error: Unsupported argument

on validate.tf line 10, in resource "aws\_instance" "myec2":  
10: sky = "blue"

An argument named "sky" is not expected here.

# Terraform Taint



# Understanding the Use-Case

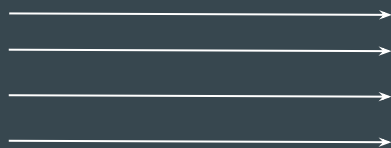
You have created a new resource via Terraform.

Users have made a lot of manual changes (both infrastructure and inside the server)

Two ways to deal with this: Import Changes to Terraform / Delete & Recreate the resource



Lots of manual changes

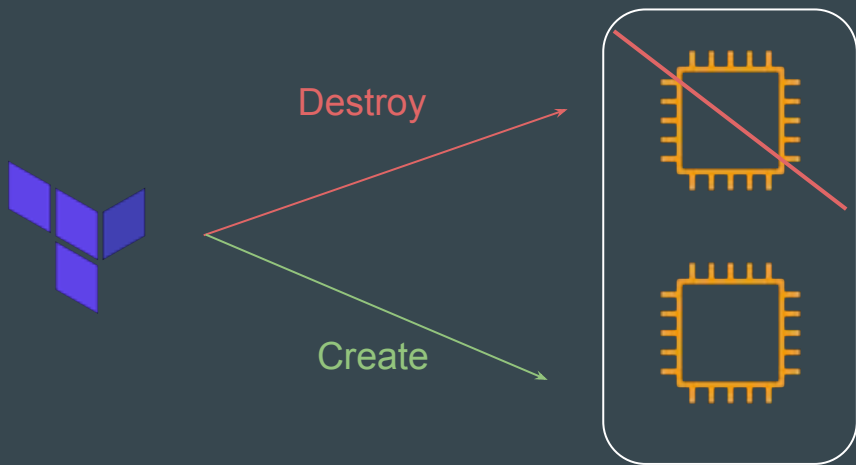


Terraform Managed Resource

## Recreating the Resource

The **-replace** option with terraform apply to force Terraform to replace an object even though there are no configuration changes that would require it.

```
terraform apply -replace="aws_instance.web"
```



## Points to Note

Similar kind of functionality was achieved using `terraform taint` command in older versions of Terraform.

For Terraform v0.15.2 and later, HashiCorp recommend using the `-replace` option with `terraform apply`

---

# Splat Expression

## Terraform Expressions

---

# Overview of Splat Expression

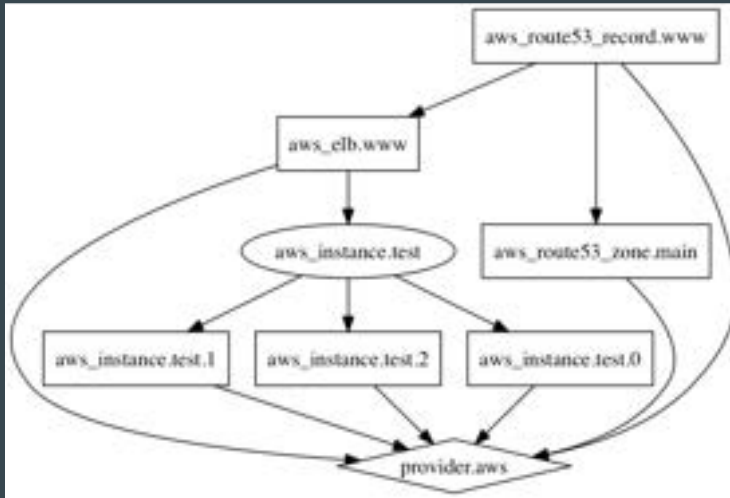
Splat Expression allows us to get a list of all the attributes.

```
resource "aws_iam_user" "lb" {  
  name = "iamuser.${count.index}"  
  count = 3  
  path = "/system/"  
}  
  
output "arns" {  
  value = aws_iam_user.lb[*].arn  
}
```

# Terraform Graph

# Understanding the Base Structure

Terraform graph refers to a **visual representation of the dependency relationships** between resources defined in your Terraform configuration.



# Summary and Conclusion

Terraform graphs are a valuable tool for visualizing and understanding the relationships between resources in your infrastructure defined with Terraform.

It can improve your overall workflow by aiding in planning, debugging, and managing complex infrastructure configurations.

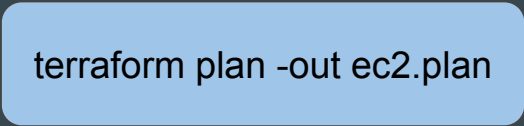


# **Saving Terraform Plan to File**

# Setting the Base

Terraform allows saving a plan to a file.

```
terraform plan -out ec2.plan
```



```
Saved the plan to: ec2.plan
```

```
To perform exactly these actions, run the following command to apply:  
terraform apply "ec2.plan"
```

# Apply from Plan File

You can run the terraform apply by referencing the plan file.

This ensures the infrastructure state remains exactly as shown in the plan to ensure consistency.

```
C:\Users\zealv\kplabs-terraform>terraform apply infra.plan
aws_security_group.allow_tls: Creating...
aws_security_group.allow_tls: Creation complete after 4s [id=sg-02ed88b1d80a484a6]
aws_vpc_security_group_ingress_rule.app_port: Creating...
aws_vpc_security_group_ingress_rule.ssh_port: Creating...
aws_vpc_security_group_ingress_rule.ftp_port: Creating...
aws_vpc_security_group_ingress_rule.app_port: Creation complete after 1s [id=sgr-0b8f10164824c9027]
aws_vpc_security_group_ingress_rule.ssh_port: Creation complete after 1s [id=sgr-042688b669ba3d824]
aws_vpc_security_group_ingress_rule.ftp_port: Creation complete after 2s [id=sgr-0962b3406710b00ba]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

# Exploring Terraform Plan File

The saved Terraform plan file will be a binary file.

You can use the **terraform show** command to read the contents in detail.

```
{
  "format_version": "1.2",
  "terraform_version": "1.0.1",
  "variables": {
    "ssh_port": {
      "value": "2222"
    },
    "ftp_port": {
      "value": "21"
    },
    "ssh_port": {
      "value": "22"
    },
    "ssh_ip": {
      "value": "200.20.30.50/32"
    }
  },
  "planned_values": {
    "root_module": {
      "resources": [
        {
          "address": "aws_security_group.allow_tls",
          "mode": "managed",
          "type": "aws_security_group",
          "name": "allow_tls",
          "provider_name": "registry.terraform.io/hashicorp/aws",
          "schema_version": 1,
          "values": {
            "description": "Managed from Terraform",
            "name": "terraform-firewall",

```

# Use-Cases of Saving Plan to a File

Many organizations require documented proof of planned changes before implementation.

These changes will further be reviewed and approved.

Running apply from plan ensures consistent desired outcome.

---

# Terraform Output

Terraform in detail

---

# Terraform Output

The terraform output command is used to extract the value of an output variable from the state file.

```
C:\Users\Zeal Vora\Desktop\terraform\terraform output>terraform output iam_names  
[  
  "iamuser.0",  
  "iamuser.1",  
  "iamuser.2",  
]
```

# Terraform Settings



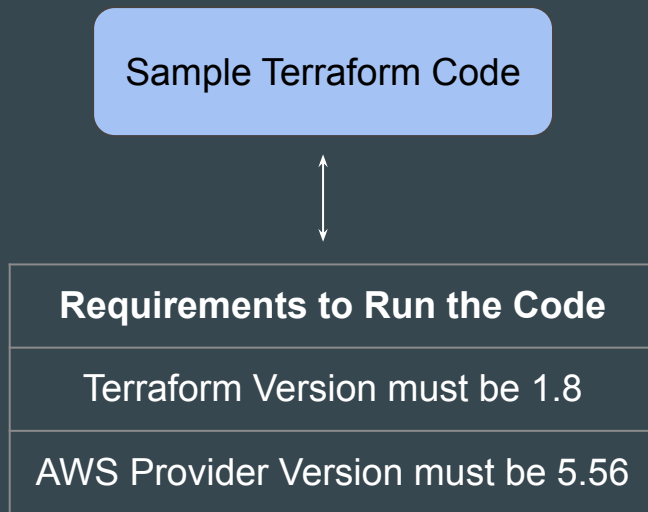
## Setting the Base

We can use the provider block to define various aspects of the provider, like region, credentials and so on.

```
provider "aws" {  
  region      = "us-east-1"  
  access_key  = "AKIAIOSFODNN7EXAMPLE"  
  secret_key  = "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"  
}  
  
resource "aws_security_group" "sg_01" {  
  name = "app_firewall"  
}
```

# Specific Version to Run Your Code

In a Terraform project, your code might require a very specific set of versions to run.



# Introducing Terraform Settings

Terraform Settings are used to configure project-specific Terraform behaviors, such as requiring a minimum Terraform version to apply to your configuration.

Terraform settings are gathered together into **terraform blocks**:

```
terraform {  
  # <setting-1>  
  # <setting-2>  
}
```

# 1 - Specifying a Required Terraform Version

If your code is compatible with specific versions of Terraform, you can use the `required_version` block to add your constraints.

```
terraform {  
  required_version = "1.8"  
}
```

## 2 - Specifying Provider Requirements

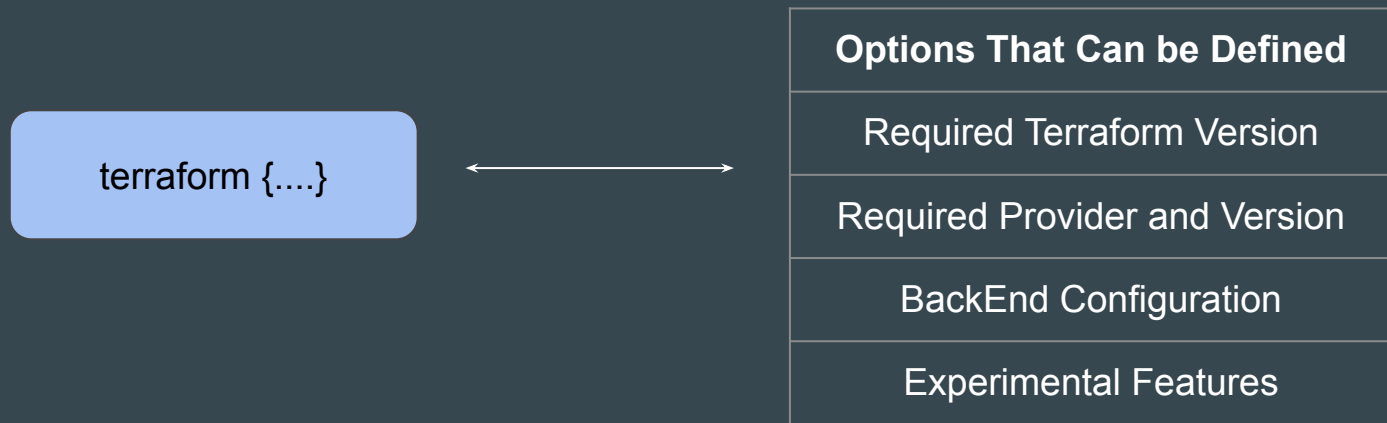
The `required_providers` block can be used to specify all of the providers required by your Terraform code.

You can further fine-tune to include a specific version of the provider plugins.

```
terraform {  
  required_providers {  
    aws = {  
      version = "5.56"  
      source  = "hashicorp/aws"  
    }  
  }  
}
```

# Flexibility in Settings Block

There are a wide variety of options that can be specified in the Terraform block.



## Point to Note

It is a good practice to include the `terraform { }` block to include details like `required_providers` as part of your project.

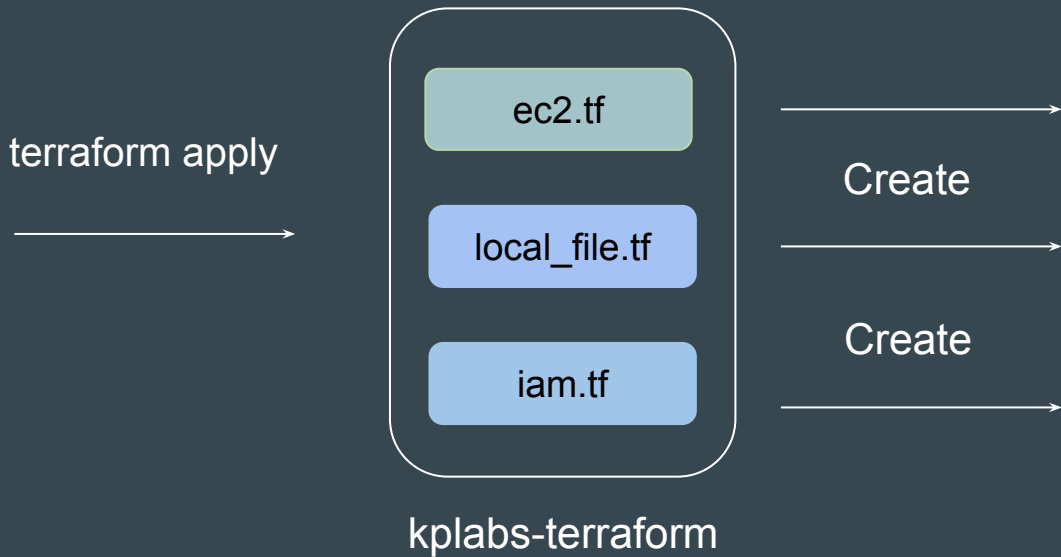
The `provider { }` block is still important to specify various other aspects like regions, credentials, alias and others.

# **Resource Targeting**



## Setting the Base

In a typical Terraform workflow, you apply the entire plan at once. This is also the default behaviour.



# Understanding Resource Targeting

Resource targeting in Terraform allows you to **apply changes to a specific subset of resources** rather than applying changes to your entire infrastructure.

terraform plan <local\_file>



```
resource "aws_iam_user" "dev" {  
  name = "kplabs-user-01"  
}  
  
resource "aws_security_group" "prod" {  
  name      = "terraform-firewalls"  
}  
  
resource "local_file" "foo" {  
  content  = "foo!"  
  filename = "${path.module}/foo.txt"  
}
```

## Using the -target flag

You can use Terraform's **-target** option to target specific resources, modules as part of operation.

terraform plan **-target** local\_file.foo



```
resource "aws_iam_user" "dev" {  
  name = "kplabs-user-01"  
}  
  
resource "aws_security_group" "prod" {  
  name      = "terraform-firewalls"  
}  
  
resource "local_file" "foo" {  
  content  = "foo!"  
  filename = "${path.module}/foo.txt"  
}
```

# Use-Cases

A project has 10 resources. Multiple team members are working on each resource's Terraform code to release some updates.

When you run `terraform plan`, you see Terraform trying to make changes to all 10 resources.

There is a critical issue and you need to add a port 80 in security group that is managed via terraform without having to update other 9 resources.

## Points to Note

Occasionally you may want to apply only part of a plan, such as when Terraform's state has become out of sync with your resources due to a network failure, a problem with the upstream cloud platform, or a bug in Terraform or its providers.

To support this, Terraform lets you target specific resources when you plan, apply, or destroy your infrastructure.

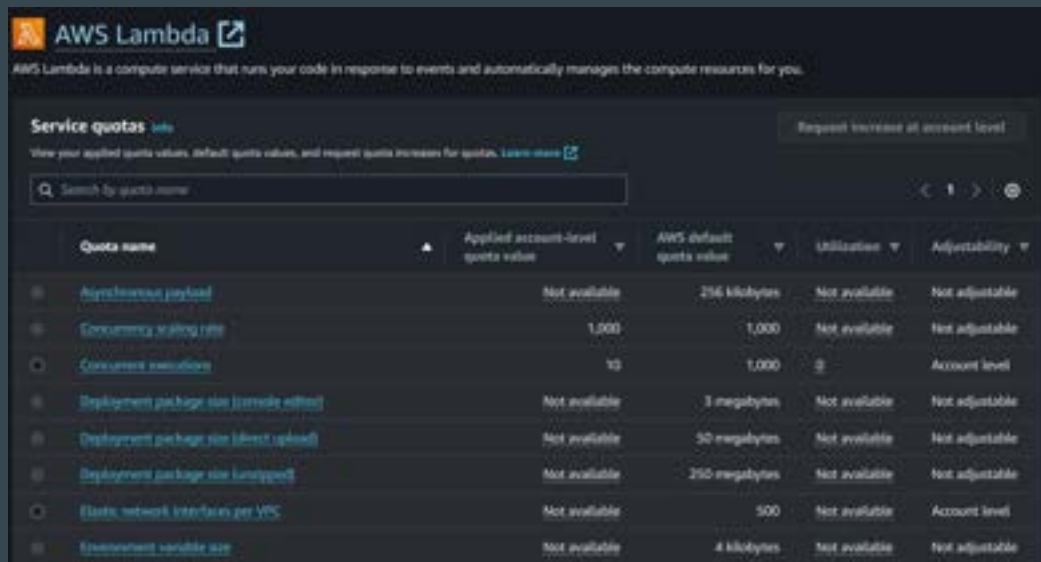
Targeting individual resources can be useful for troubleshooting errors, **but should not be part of your normal workflow.**

# **Dealing with Larger Infrastructure**

# Cloud does not mean Unlimited

Using a Cloud provider does not mean you have access to unlimited resources.

There are some limits that apply to all of the services.



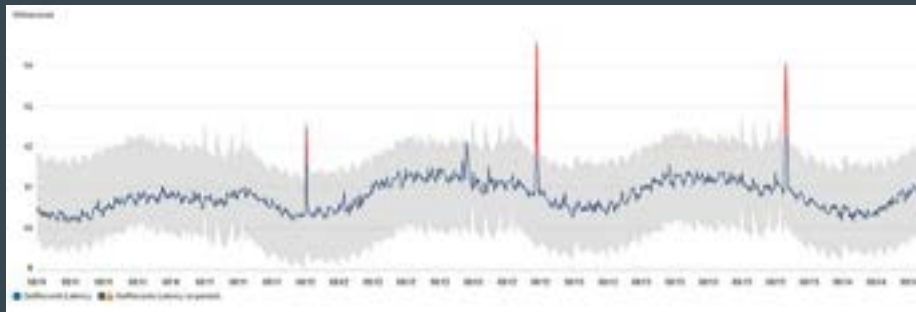
The screenshot shows the AWS Lambda console's 'Service quotas' page. At the top, there's a header for 'AWS Lambda' with a description: 'AWS Lambda is a compute service that runs your code in response to events and automatically manages the compute resources for you.' Below this is a 'Service quotas' section with a 'Request increase at account level' button. A search bar is present with the placeholder 'Search by quota name'. The main content is a table with columns: 'Quota name', 'Applied account-level quota value', 'AWS default quota value', 'Utilization', and 'Adjustability'. The table lists various quotas such as 'Asynchronous payload', 'Concurrency scaling rate', 'Concurrency execution', 'Deployment package size (console editor)', 'Deployment package size (direct upload)', 'Deployment package size (unzip)', 'Elastic network interfaces per VPC', and 'Environment variable size'.

| Quota name                                               | Applied account-level quota value | AWS default quota value | Utilization   | Adjustability  |
|----------------------------------------------------------|-----------------------------------|-------------------------|---------------|----------------|
| <a href="#">Asynchronous payload</a>                     | Not available                     | 256 kilobytes           | Not available | Not adjustable |
| <a href="#">Concurrency scaling rate</a>                 | 1,000                             | 1,000                   | Not available | Not adjustable |
| <a href="#">Concurrency execution</a>                    | 10                                | 1,000                   | 0             | Account level  |
| <a href="#">Deployment package size (console editor)</a> | Not available                     | 3 megabytes             | Not available | Not adjustable |
| <a href="#">Deployment package size (direct upload)</a>  | Not available                     | 30 megabytes            | Not available | Not adjustable |
| <a href="#">Deployment package size (unzip)</a>          | Not available                     | 250 megabytes           | Not available | Not adjustable |
| <a href="#">Elastic network interfaces per VPC</a>       | Not available                     | 500                     | Not available | Account level  |
| <a href="#">Environment variable size</a>                | Not available                     | 4 kilobytes             | Not available | Not adjustable |

## Less Known Limit - API Limit in AWS

When you're architecting for the cloud, you need to keep **API throttling** in mind, particularly the types of calls and the frequency with which they are called.

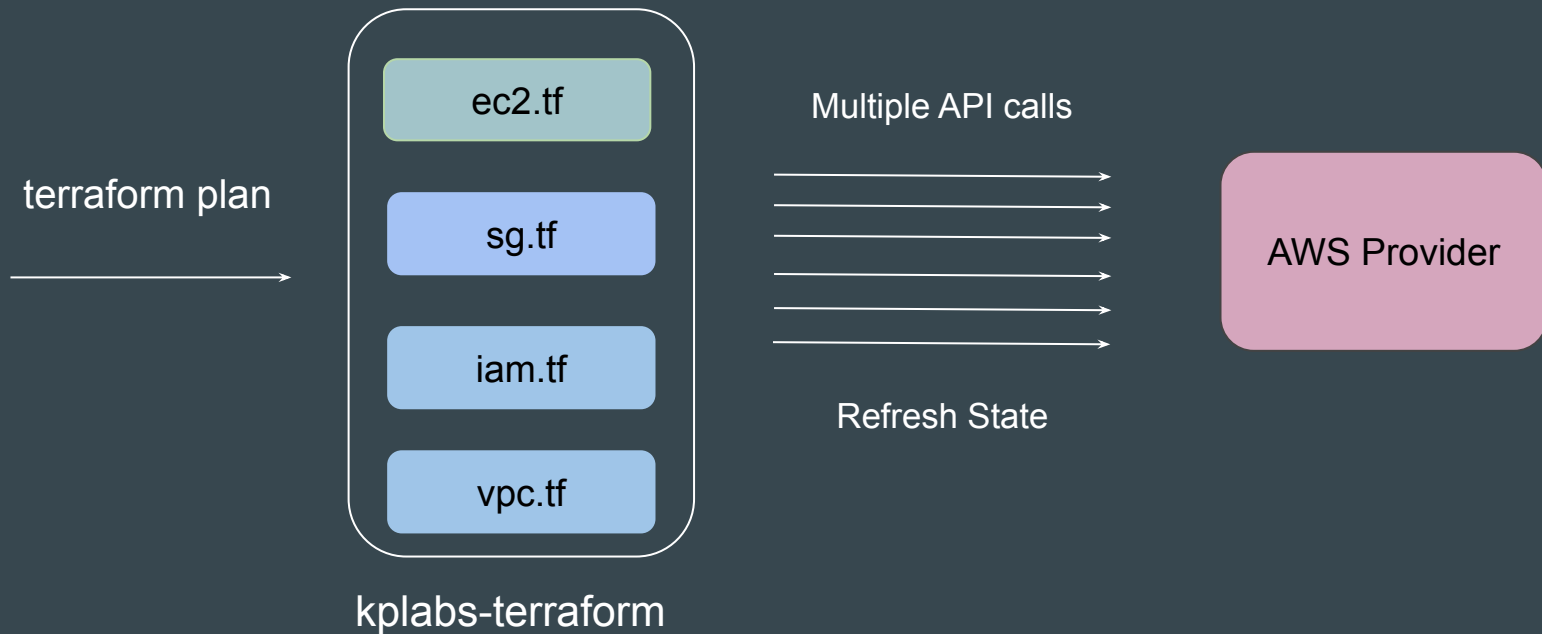
When the allotted rate limit for an API call is exceeded, you'll receive an error response and the call will be throttled.





# Key Consideration in Terraform

When a single project has large set of resources that are managed, even a simple terraform plan can lead to higher number of API calls to the provider.



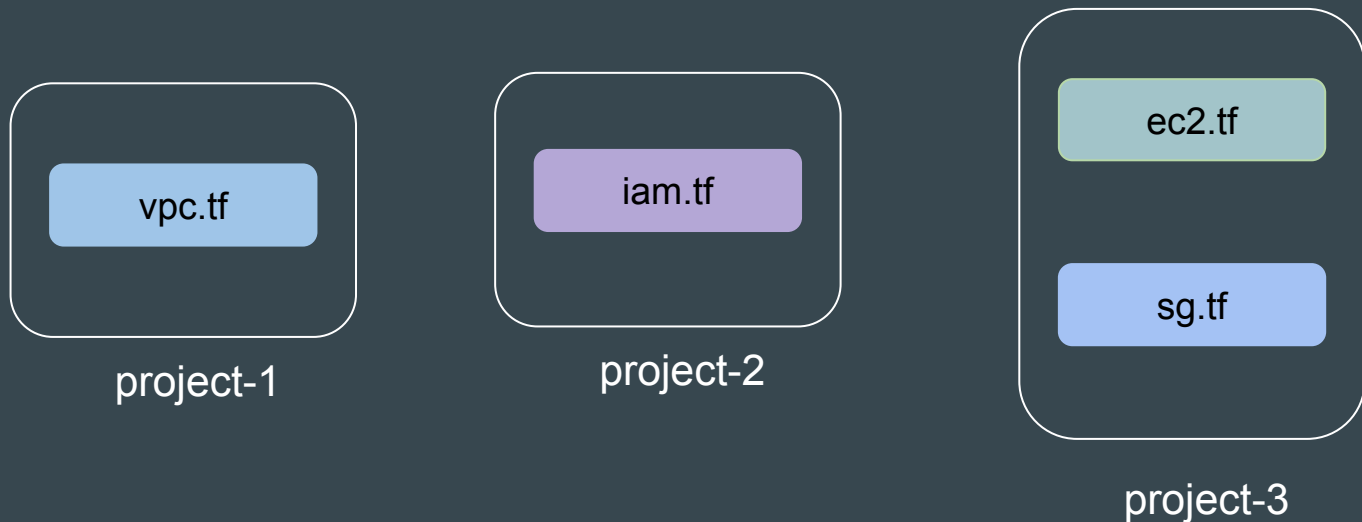
## Time to Slow Down

Whenever you are designing Terraform projects, you have to take in mind the API limits of the provider to avoid throttling that will further impact production systems.



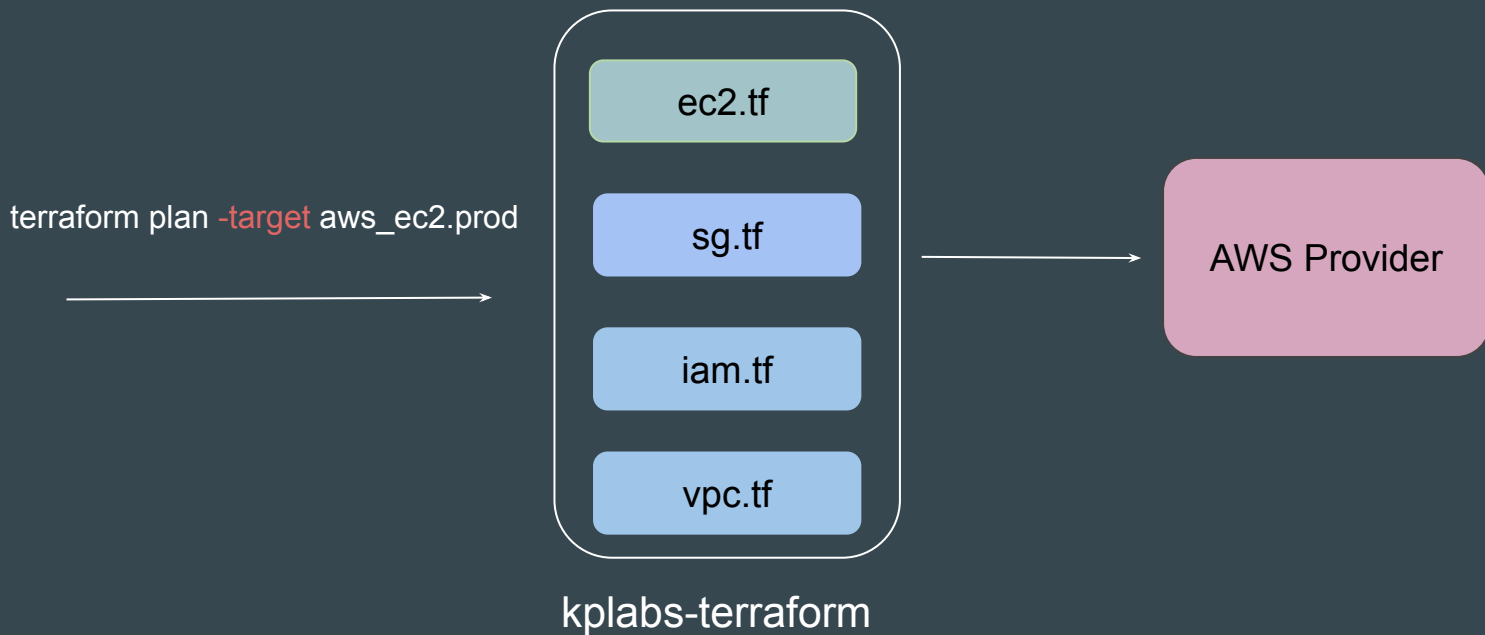
# 1 - Convert Big Project to Multiple Smaller Project

Whenever you are designing Terraform projects, you have to take in mind the API limits of the provider to avoid throttling that will further impact production systems.



## 2 - Use Resource Targeting

Using resource targeting, the number of calls that will be made to a provider will be much less.



### 3 - Setting refresh=false

The **-refresh=false** option is used in normal planning mode to skip the default behavior of refreshing Terraform state before checking for configuration changes.

By disabling automatic refresh, you can potentially speed up the planning and application process, especially if you're confident that the existing state is accurate.

```
C:\kplabs-terraform\larger-infra>terraform plan -refresh=false

Terraform used the selected providers to generate the following execution plan,
following symbols:
+ create

Terraform will perform the following actions:

# aws_security_group.allow_tl will be created
+ resource "aws_security_group" "allow_tl" {
+   arn                = (known after apply)
+   description        = "Managed from Terraform"
+   egress              = (known after apply)
+   id                 = (known after apply)
+   ingress            = (known after apply)
+   name               = "test-firewall"
+   name_prefix        = (known after apply)
+   owner_id            = (known after apply)
+   revoke_rules_on_delete = false
+   tags_all           = (known after apply)
+   vpc_id             = (known after apply)
+ }

Plan: 1 to add, 0 to change, 0 to destroy.
```

# Reference Screenshot

```
C:\kplabs-terraform\larger-infra>terraform plan
aws_security_group.allow_tls: Refreshing state... [id=sg-0f6ab960cd5974e3d]
module.vpc.aws_vpc.this[0]: Refreshing state... [id=vpc-0ab5b868189cc37f]
aws_vpc_security_group_ingress_rule.allow_tls_ipv6: Refreshing state... [id=sgr-02ba1b8105691d04]
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv4: Refreshing state... [id=sgr-0684d5de21661e90a]
module.vpc.aws_default_security_group.this[0]: Refreshing state... [id=sg-0e4a6df13518ee751]
module.vpc.aws_default_route_table.default[0]: Refreshing state... [id=rtb-0669b753720e5d5c3]
module.vpc.aws_subnet.public[2]: Refreshing state... [id=subnet-0cc048829889218ba]
module.vpc.aws_subnet.private[2]: Refreshing state... [id=subnet-0c6ffa119e73881cf]
module.vpc.aws_subnet.private[1]: Refreshing state... [id=subnet-0b3710a0a30a92e61]
module.vpc.aws_subnet.public[1]: Refreshing state... [id=subnet-0ef80e11f37e114fc]
module.vpc.aws_subnet.private[0]: Refreshing state... [id=subnet-010725dbbfadffec]
module.vpc.aws_internet_gateway.this[0]: Refreshing state... [id=igw-07acd64321b4c8ee]
module.vpc.aws_subnet.public[0]: Refreshing state... [id=subnet-079ea1df0674ab8e9]
module.vpc.aws_default_network_acl.this[0]: Refreshing state... [id=acl-0af44b9fd2dcd2899]
module.vpc.aws_route_table.private[2]: Refreshing state... [id=rtb-05c2f3592aedd257d]
module.vpc.aws_route_table.private[0]: Refreshing state... [id=rtb-0ee65dc36f2ba2769]
module.vpc.aws_route_table.private[1]: Refreshing state... [id=rtb-0e0117617010fd2eb]
module.vpc.aws_vpn_gateway.this[0]: Refreshing state... [id=vgw-056912b1df8e05812]
module.vpc.aws_route_table.public[0]: Refreshing state... [id=rtb-09e5cb134415c2b37]
module.vpc.aws_eip.nat[1]: Refreshing state... [id=eipalloc-0893b740cbd72963c]
module.vpc.aws_eip.nat[2]: Refreshing state... [id=eipalloc-0eb138eaa9980e42]
module.vpc.aws_eip.nat[0]: Refreshing state... [id=eipalloc-0940cf568ed9a4eta]
module.vpc.aws_route_table_association.private[1]: Refreshing state... [id=rtbassoc-064613d944a71f46]
module.vpc.aws_route_table_association.private[0]: Refreshing state... [id=rtbassoc-0d5464665a321b560]
module.vpc.aws_route_table_association.private[2]: Refreshing state... [id=rtbassoc-075364c0bf6f069d6]
module.vpc.aws_route_table_association.public[2]: Refreshing state... [id=rtbassoc-0e4962322428456cd]
module.vpc.aws_route_table_association.public[1]: Refreshing state... [id=rtbassoc-0663f72daafec7542]
module.vpc.aws_route_table_association.public[0]: Refreshing state... [id=rtbassoc-00232967b2084e45f]
module.vpc.aws_route.public_internet_gateway[0]: Refreshing state... [id=rtb-09e5cb134415c2b371080289404]
```

terraform plan

```
C:\kplabs-terraform\larger-infra>terraform plan -refresh=false
Terraform used the selected providers to generate the following execution plan,
following symbols:
+ create

Terraform will perform the following actions:

# aws_security_group.allow_tls will be created
+ resource "aws_security_group" "allow_tls" {
+   arn                = (known after apply)
+   description        = "Managed from Terraform"
+   egress              = (known after apply)
+   id                 = (known after apply)
+   ingress             = (known after apply)
+   name               = "test-firewall"
+   name_prefix         = (known after apply)
+   owner_id            = (known after apply)
+   revoke_rules_on_delete = false
+   tags_all            = (known after apply)
+   vpc_id              = (known after apply)
+ }

Plan: 1 to add, 0 to change, 0 to destroy.
```

terraform plan -refresh=false

---

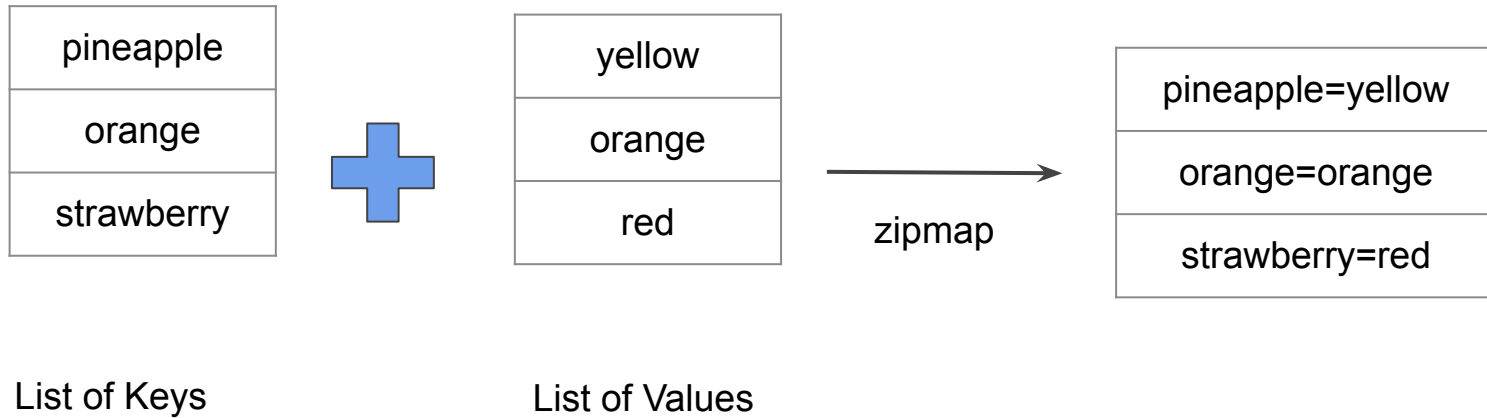
# Zipmap

Terraform Function

---

# Overview of Zipmap

The zipmap function constructs a map from a list of keys and a corresponding list of values.





# Sample Output of Zipmap Function

```
←[ ]> zipmap(["pineapple","oranges","strawberry"], ["yellow","orange","red"])  
{  
  "oranges" = "orange"  
  "pineapple" = "yellow"  
  "strawberry" = "red"  
}
```

# Simple Use-Case

You are creating multiple IAM users.

You need output which contains direct mapping of IAM names and ARNs

```
zipmap = {  
  "demo-user.0" = "arn:aws:iam::018721151861:user/system/demo-user.0"  
  "demo-user.1" = "arn:aws:iam::018721151861:user/system/demo-user.1"  
  "demo-user.2" = "arn:aws:iam::018721151861:user/system/demo-user.2"  
}
```

---

# Comments in Terraform Code

Commenting the Code!

---

# Overview of Comments

A comment is a text note added to source code to provide explanatory information, usually about the function of the code

```
'''In this program, we check if the number is positive or  
negative or zero and  
display an appropriate message'''  
  
num = 3.4  
  
# Try these two variations as well:  
# num = 0  
# num = -4.5  
  
if num > 0:  
    print("Positive number")  
elif num == 0:  
    print("Zero")  
else:  
    print("Negative number")
```

# Comments in Terraform

The Terraform language supports three different syntaxes for comments:

| Type      | Description                                                                     |
|-----------|---------------------------------------------------------------------------------|
| #         | begins a single-line comment, ending at the end of the line.                    |
| //        | also begins a single-line comment, as an alternative to #.                      |
| /* and */ | are start and end delimiters for a comment that might span over multiple lines. |

# **Resource Behavior and Meta-Argument**

# Understanding the Basics

A **resource block** declares that you want a particular infrastructure object to exist with the given settings

```
resource "aws_instance" "myec2" {  
    ami = "ami-00c39f71452c08778"  
    instance_type = "t2.micro"  
}
```

# How Terraform Applies a Configuration

Create resources that exist in the configuration but are not associated with a real infrastructure object in the state.

Destroy resources that exist in the state but no longer exist in the configuration.

Update in-place resources whose arguments have changed.

Destroy and re-create resources whose arguments have changed but which cannot be updated in-place due to remote API limitations.

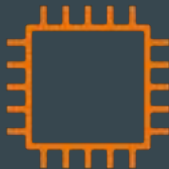


# Understanding the Limitations

What happens if we want to change the default behavior?

Example: Some modification happened in Real Infrastructure object that is not part of Terraform but you want to ignore those changes during terraform apply.

```
resource "aws_instance" "web" {  
  ami          = "ami-00c39f71452c08778"  
  instance_type = "t3.micro"  
  
  tags = {  
    Name = "HelloWorld"  
  }  
}
```



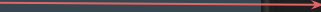
|      |            |
|------|------------|
| Name | HelloWorld |
|------|------------|

|     |            |
|-----|------------|
| Env | Production |
|-----|------------|

## Solution - Using Meta Arguments

Terraform allows us to include **meta-argument** within the resource block which allows some details of this standard resource behavior to be customized on a per-resource basis.

Inside resource block



```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    ignore_changes = [tags]  
  }  
}
```

# Different Meta-Arguments

| Meta-Argument | Description                                                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| depends_on    | Handle hidden resource or module dependencies that Terraform cannot automatically infer.                                                               |
| count         | Accepts a whole number, and creates that many instances of the resource                                                                                |
| for_each      | Accepts a map or a set of strings, and creates an instance for each item in that map or set.                                                           |
| lifecycle     | Allows modification of the resource lifecycle.                                                                                                         |
| provider      | Specifies which provider configuration to use for a resource, overriding Terraform's default behavior of selecting one based on the resource type name |

# **Meta Argument - LifeCycle**

# Basics of Lifecycle Meta-Argument

Some details of the default resource behavior can be customized using the special nested lifecycle block within a resource block body:

```
resource "aws_instance" "myec2" {  
  ami = "ami-0f34c5ae932e6f0e4"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "HelloEarth"  
  }  
  
  lifecycle {  
    ignore_changes = [tags]  
  }  
}
```

## Arguments Available

There are four argument available within lifecycle block.

| Arguments             | Description                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| create_before_destroy | New replacement object is created first, and the prior object is destroyed after the replacement is created.         |
| prevent_destroy       | Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource |
| ignore_changes        | Ignore certain changes to the live resource that does not match the configuration.                                   |
| replace_triggered_by  | Replaces the resource when any of the referenced items change                                                        |

# Replace Triggered By

Replaces the resource when any of the referenced items change.

```
resource "aws_appautoscaling_target" "ecs_target" {  
  # ...  
  lifecycle {  
    replace_triggered_by = [  
      # Replace 'aws_appautoscaling_target' each time this instance of  
      # the 'aws_ecs_service' is replaced.  
      aws_ecs_service.svc.id  
    ]  
  }  
}
```

# **Create Before Destroy Argument**



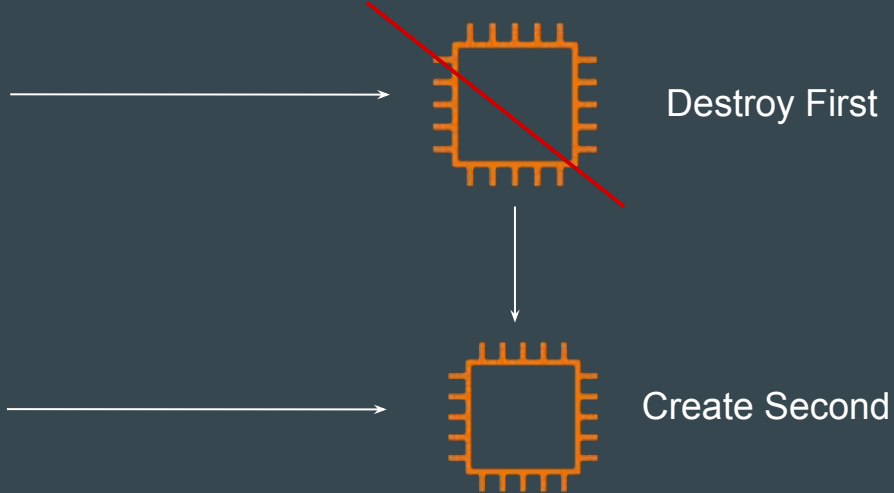
# Understanding the Default Behavior

By default, when Terraform must change a resource argument that cannot be updated in-place due to remote API limitations, Terraform will instead destroy the existing object and then create a new replacement object with the new configured arguments.

```
demo.tf > _  
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```

Changed AMI

```
resource "aws_instance" "myec2" {  
  ami      = "ami-123456789"  
  instance_type = "t2.micro"  
}
```



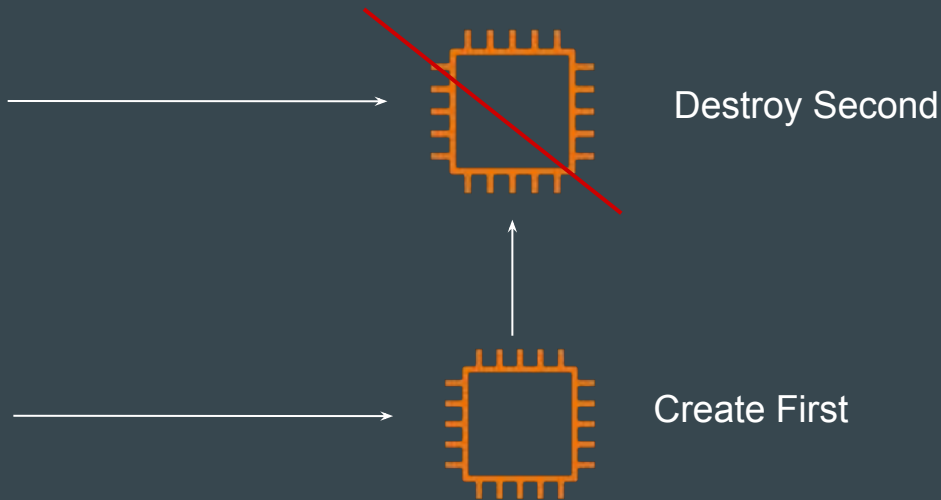
# Create Before Destroy Argument

The `create_before_destroy` meta-argument changes this behavior so that the new replacement object is created first, and the prior object is destroyed after the replacement is created.

```
resource "aws_instance" "myec2" {  
  ami = "ami-053b0d53c279acc90"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

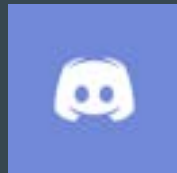
↓ Changed AMI

```
resource "aws_instance" "myec2" {  
  ami = "ami-123456789"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```



# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)



[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **LifeCycle - Prevent Destroy Argument**

## Prevent Destroy Argument

This meta-argument, when set to true, will cause Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource, as long as the argument remains present in the configuration.

```
resource "aws_instance" "myec2" {  
  ami          = "ami-123456789"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    prevent_destroy = true  
  }  
}
```

## Points to Note

This can be used as a measure of safety against the accidental replacement of objects that may be costly to reproduce, such as database instances.

Since this argument must be present in configuration for the protection to apply, note that this setting does not prevent the remote object from being destroyed if the resource block were removed from configuration entirely.

# **LifeCycle - Ignore Changes Argument**

# Ignore Changes

In cases where settings of a remote object is modified by processes outside of Terraform, the Terraform would attempt to "fix" on the next run.

In order to change this behavior and ignore the manually applied change, we can make use of `ignore_changes` argument under `lifecycle`.

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    ignore_changes = [tags]  
  }  
}
```



## Points to Note

Instead of a list, the special keyword **all** may be used to instruct Terraform to ignore all attributes, which means that Terraform can create and destroy the remote object but will never propose updates to it.

```
resource "aws_instance" "myec2" {  
  ami = "ami-0f34c5ae932e6f0e4"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "HelloEarth"  
  }  
  
  lifecycle {  
    ignore_changes = all  
  }  
}
```

---

# Challenges with Count

Meta-Argument

---

# Revising the Basics

Resource are identified by the index value from the list.

```
variable "iam_names" {  
  type = list  
  default = ["user-01", "user-02", "user-03"]  
}  
  
resource "aws_iam_user" "iam" {  
  name = var.iam_names[count.index]  
  count = 3  
  path = "/system/"  
}
```



| Resource Address    | Infrastructure |
|---------------------|----------------|
| aws_iam_user.iam[0] | user-01        |
| aws_iam_user.iam[1] | user-02        |
| aws_iam_user.iam[2] | user-03        |

# Challenge - 1

If the order of elements of index is changed, this can impact all of the other resources.

```
variable "iam_names" {  
  type = list  
  default = ["user-0", "user-01", "user-02", "user-03"]  
}  
  
resource "aws_iam_user" "iam" {  
  name = var.iam_names[count.index]  
  count = 4  
  path = "/system/"  
}
```



| Resource Address     | Infrastructure |
|----------------------|----------------|
| aws_iam_user.iam.[0] | user-01        |
| aws_iam_user.iam.[1] | user-02        |
| aws_iam_user.iam.[2] | user-03        |

# Important Note

If your resources are almost identical, count is appropriate.

If distinctive values are needed in the arguments, usage of `for_each` is recommended.

```
resource "aws_instance" "server" {  
  count = 4 # create four similar EC2 instances  
  ami      = "ami-a1b2c3d4"  
  instance_type = "t2.micro"  
}
```

# **Resource Dependencies**

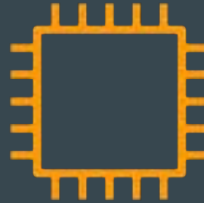
# Sample Requirement

An application deployed on an EC2 instance must store its data in external storage before it can initialize.

Resources Needed: S3 Bucket (for external storage) and EC2 Instance



Step 1: Create S3 Bucket



Step 2: Create EC2 Instance

# Understanding the Challenge

In a scenario where resources are defined independently, there is no guaranteed order stating which resource will be created first.

Sometimes the EC2 instance may be created first, and sometimes the S3 bucket may be created first.

```
resource "aws_instance" "example" {  
    ami          = "ami-0e449927258d45bc4"  
    instance_type = "t2.micro"  
}  
  
resource "aws_s3_bucket" "example" {  
    bucket = "kplabs-demo-s3-bucket-007"  
}
```



# Introducing depends\_on Meta Argument

The `depends_on` meta-argument instructs Terraform to complete all actions on the dependency object before performing actions on the object declaring the dependency.

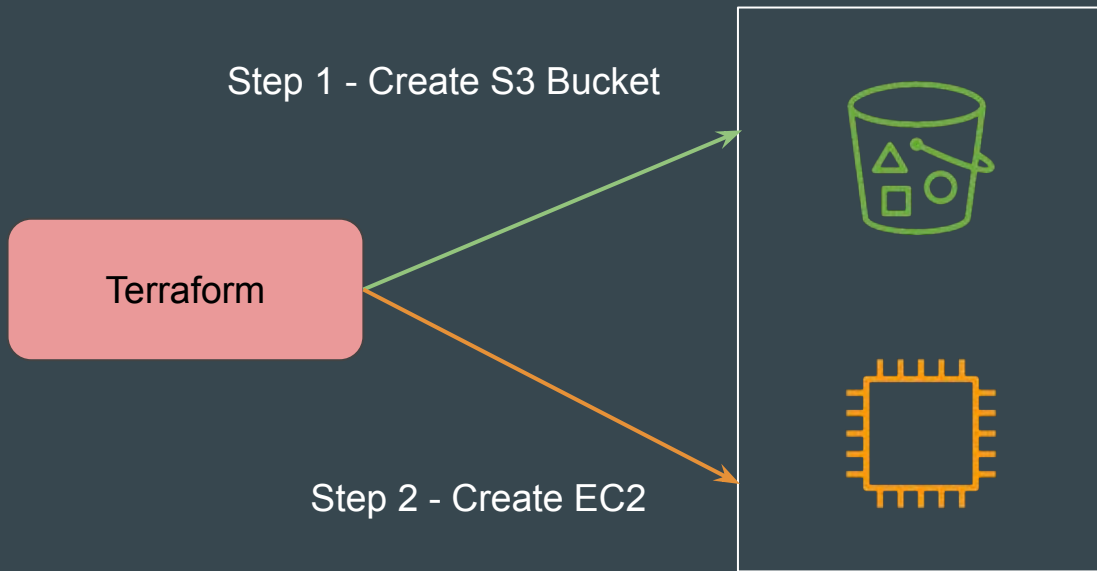


```
resource "aws_instance" "example" {
  ami           = "ami-0e449927258d45bc4"
  instance_type = "t2.micro"
  depends_on = [aws_s3_bucket.example]
}

resource "aws_s3_bucket" "example" {
  bucket = "kplabs-demo-s3-bucket-007"
}
```

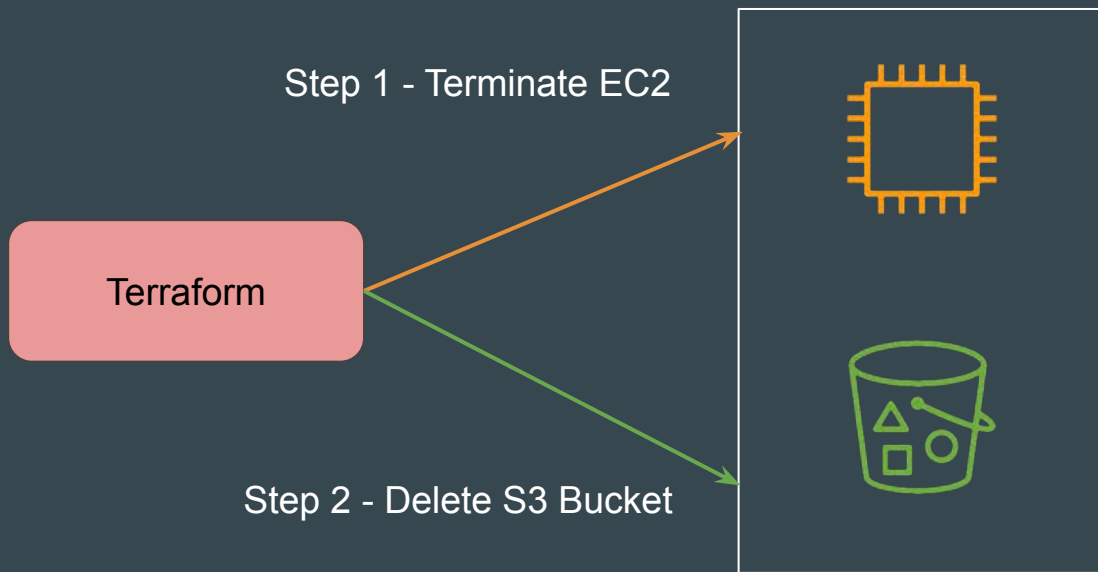
# Order of Creation

The **depends\_on** meta-argument explicitly tells Terraform that the `aws_instance.example` must be created after `aws_s3_bucket.example`.



# Order of Deletion

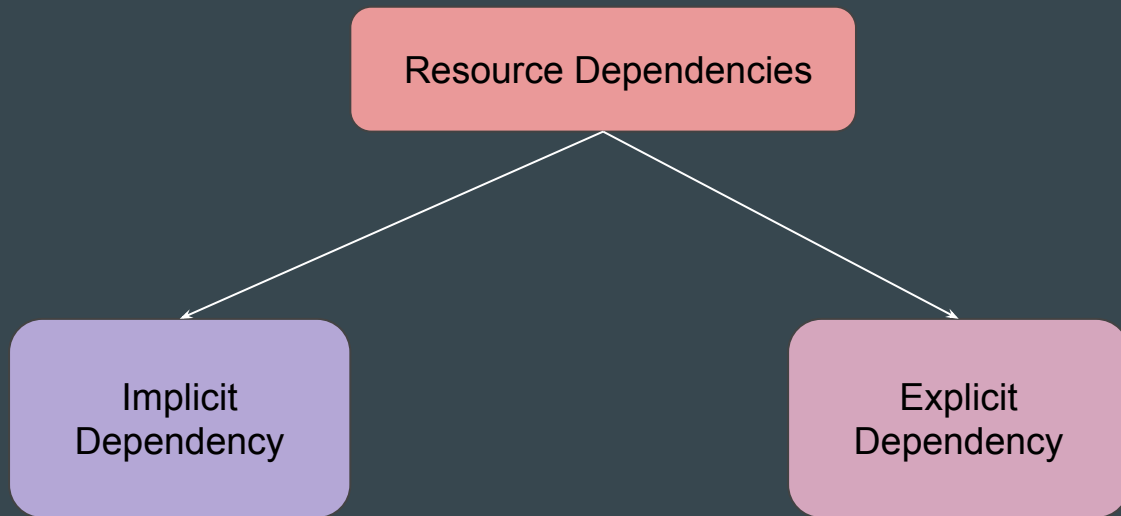
When you run `terraform destroy`, the order is reversed to ensure dependencies are not broken during deletion.



# **Implicit vs Explicit Dependencies**

# Setting the Base

There are two ways to define dependencies in Terraform.



# Explicit Dependency

Explicit dependencies are **declared using the depends\_on meta-argument**.

You **use this when there's no direct attribute reference**, but you still need to control the order of resource creation.

```
resource "aws_instance" "example" {  
    ami          = "ami-0e449927258d45bc4"  
    instance_type = "t2.micro"  
    depends_on = [aws_s3_bucket.example]  
}  
  
resource "aws_s3_bucket" "example" {  
    bucket = "kplabs-demo-s3-bucket-007"  
}
```

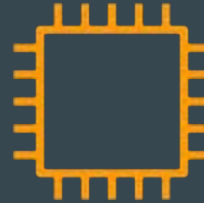
# Sample Requirement

EC2 instance should only allow communication from trusted set of IP addresses.

Resources Needed: EC2 Instance + Security Group (Firewall)



Step 1: Create Security Group



Step 2: Create EC2 Instance

# Introducing Implicit Dependency

Since in `aws_instance` resource there is a reference to the ID of the `aws_security_group` resource, Terraform automatically understands that the security group must be created before the EC2 instance



```
resource "aws_instance" "example" {
  ami           = "ami-0e449927258d45bc4"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.prod.id]
}

resource "aws_security_group" "prod" {
  name = "production-sg"
}
```



# Final Revision

## Explicit Dependency

aws\_instance.example

depends\_on

aws\_s3\_bucket.example

## Implicit Dependency

aws\_security\_group.prod

used in attribute

aws\_instance.example

# **SET - Data Type**

# Revising List Data Type

Lists are used to store multiple items in a single variable.

These items can be duplicates as well.

```
C:\Users\zealv\kplabs-terraform>terraform apply -auto-approve
var.my-list
  Enter a value: ["hello","world","hello"]

Changes to Outputs:
  + variable_output = [
    + "hello",
    + "world",
    + "hello",
  ]

You can apply this plan to save these new output values to the Terraform
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

variable_output = tolist([
  "hello",
  "world",
  "hello",
])
```

# List and Index

List items are indexed, the first item has index [0], the second item has index [1] etc.

```
variable "user_names" {  
  type = list  
  default = ["alice", "bob", "john"]  
}
```



0



1



2

# SET Data Type

Sets can only store unique elements.

Any duplicates are automatically removed.

```
C:\Users\zealv\kplabs-terraform>terraform apply -auto-approve
var.my-set
  Enter a value: ["hello","world","hello"]

Changes to Outputs:
  + variable_output = [
    + "hello",
    + "world",
  ]

You can apply this plan to save these new output values to the Terraform
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
variable_output = toset([
  "hello",
  "world",
])
```

## Point to Note

While defining a SET, you need to also define the type of value that is expected.

```
variable "my-set" {  
    type = set(string)  
}
```

# SET is Unordered

A set does not store the order of the elements.

Terraform only tracks the presence of elements, not their order.

If the elements in a set change order, Terraform won't detect that as a change. However, if an element is added or removed, Terraform will apply updates accordingly.

```
variable "user_names" {  
  type = list  
  default = ["alice", "bob", "john"]  
}
```

```
variable "user_names" {  
  type = set(string)  
  default = ["john", "bob", "alice"]  
}
```

# The `for_each` Meta-Argument



## Setting the Base

By default, a resource block configures one real infrastructure object.

However, sometimes you want to manage several similar objects (like a fixed pool of compute instances) without writing a separate block for each one.

Terraform has two ways to do this: `count` and `for_each`.

```
resource "aws_iam_user" "lb" {  
  name = "alice"  
}
```

# Creating 5 IAM Users

If we want to create multiple resources with different configuration, we have to add multiple different resource blocks.



```
iam.tf
iam.tf > ...

resource "aws_iam_user" "lb" {
  name = "alice"
}

resource "aws_iam_user" "lb" {
  name = "bob"
}

resource "aws_iam_user" "lb" {
  name = "john"
}

resource "aws_iam_user" "lb" {
  name = "james"
}

resource "aws_iam_user" "lb" {
  name = "will"
}
```

# Introducing for\_each

If a resource block includes a **for\_each** meta argument whose value is a map or a set of strings, Terraform creates one instance for each member of that map or set.

```
variable "user_names" {  
  type = set(string)  
  default = ["alice","bob"]  
}  
  
resource "aws_iam_user" "lb" {  
  for_each = var.user_names  
  name = each.value  
}
```



```
C:\kplabs-terraform>terraform plan  
  
Terraform used the selected providers to generate the following  
following symbols:  
+ create  
  
Terraform will perform the following actions:  
  
# aws_iam_user.lb["alice"] will be created  
+ resource "aws_iam_user" "lb" {  
  + arn          = (known after apply)  
  + force_destroy = false  
  + id           = (known after apply)  
  + name        = "alice"  
  + path        = "/"  
  + tags_all     = (known after apply)  
  + unique_id    = (known after apply)  
}  
  
# aws_iam_user.lb["bob"] will be created  
+ resource "aws_iam_user" "lb" {  
  + arn          = (known after apply)  
  + force_destroy = false  
  + id           = (known after apply)  
  + name        = "bob"  
  + path        = "/"  
  + tags_all     = (known after apply)  
  + unique_id    = (known after apply)  
}
```

## Point to Note

In blocks where `for_each` is set, an additional each object is available.

These object has two attributes:

| Each Object             | Description                                                 |
|-------------------------|-------------------------------------------------------------|
| <code>each.key</code>   | The map key (or set member) corresponding to this instance. |
| <code>each.value</code> | The map value corresponding to this instance                |

## Example - for\_each with Map

When for\_each is used with map, we can make use of each object to extract both key and value from the given map.

```
variable "mymap" {
  default = {
    dev = "ami-123"
    prod = "ami-456"
  }
}

resource "aws_instance" "web" {
  for_each      = var.mymap
  ami           = each.value
  instance_type = "t3.micro"

  tags = {
    Name = each.key
  }
}
```

# **Data Type - Object**

# Revising Map Data Type

In a basic setup, a **map is a collection of key-value pairs** where all values must be of the same type whereas keys are string.

Strict structure is not required explicitly.

```
variable "my-map" {  
  type = map(number)  
}  
  
output "variable_value" {  
  value = var.my-map  
}
```



```
C:\kplabs-terraform>terraform plan  
var.my-map  
  Enter a value: {"Name"="123","Location"="456"}  
  
Changes to Outputs:  
+ variable_value = {  
  + Location = 456  
  + Name     = 123  
}
```

# Introducing Object Data Type

An **object** is also a collection of key-value pairs, but each value can be of a different type.

A proper structure is generally required while defining object data type.

```
variable "my-object" {  
  type = object({Name = string, userID = number})  
}  
  
output "variable_value" {  
  value = var.my-object  
}
```



```
C:\kplabs-terraform>terraform plan  
var.my-object  
  Enter a value: {"Name"="Zeal","userID"=123}  
  
Changes to Outputs:  
+ variable_value = {  
  + Name      = "Zeal"  
  + userID    = 123  
}
```



# Proper Structure is Required

It is important to have a structure of attributes allowed as part of the object.

```
variable "my-map" {  
  type = map  
}
```

← This will work

```
variable "my-object" {  
  type = object  
}
```

← This will NOT work

## Keep in Mind the Syntax

`object(...)`: a collection of named attributes that each have their own type.

The schema for object types is `{ <KEY> = <TYPE>, <KEY> = <TYPE>, ... }` — a pair of curly braces containing a comma-separated series of `<KEY> = <TYPE>` pairs.

Extra attributes are discarded during type conversion.

## Example - Extra Attributes Provided

In the following example, an additional key=value pair is provided that is not part of the object structure. We see on how it gets discarded in the output.

```
variable "my-object" {  
  type = object({Name = string, userID = number})  
}  
  
output "variable_value" {  
  value = var.my-object  
}
```



```
C:\kplabs-terraform> terraform plan  
var.my-object  
  Enter a value: {"Name"="Zeal","userID"="123","email"="instructors@kplabs.in"}  
  
Changes to Outputs:  
+ variable_value = {  
  + Name      = "Zeal"  
  + userID    = 123  
}
```

# Relax and Have a Meme Before Proceeding



# **Input Variable Validation**

# Understanding the Challenge

Many times, the terraform plan would run properly without errors; however, as soon as you run “terraform apply”, you get an error.

```
C:\kplabs-terraform>terraform plan
```

```
Terraform used the selected providers to generate the following  
following symbols:
```

```
• create
```

```
Terraform will perform the following actions:
```

```
# aws_s3_bucket.examplebucket will be created
```

```
• resource "aws_s3_bucket" "examplebucket" {  
  • acceleration_status      = (known after apply)  
  • acl                      = (known after apply)  
  • arn                     = (known after apply)  
  • bucket                  = "hi"  
  • bucket_domain_name      = (known after apply)  
  • bucket_prefix           = (known after apply)  
  • bucket_regional_domain_name = (known after apply)  
  • force_destroy           = false
```

Plan Stage

```
aws_s3_bucket.this: Creating...
```

```
Errors: creating S3 Bucket (hi): operation error S3: CreateBucket, https response error  
T87GNG0E, HostID: W/rG0Hvr4EyBDQhKsGwMrrY9HLx/NteAND+eOUUn14g9G2r2HgyDJ3djNP8k9bX8zYlvir  
The specified bucket is not valid.
```

```
with aws_s3_bucket.this,  
on tf-logs.tf line 4, in resource "aws_s3_bucket" "this":  
  4: resource "aws_s3_bucket" "this" {
```

Apply Stage

# Reason is Due to Lack of Validation

Every service has some restrictions and limits on aspects like naming convention, capacity, etc.

## IAM name requirements

IAM names have the following requirements and restrictions:

- Policy documents can contain only the following Unicode characters: horizontal tab (U+0009), linefeed (U+000A), carriage return (U+000D), and characters in the range U+0020 to U+00FF.
- Names of users, groups, roles, policies, instance profiles, server certificates, and paths must be alphanumeric, including the following common characters: plus (+), equals (=), comma (,), period (.), at (@), underscore (\_), and hyphen (-). Path names must begin and end with a forward slash (/).
- Names of users, groups, roles, and instance profiles must be unique within the account. They aren't distinguished by case, for example, you can't create groups named both `ADMINS` and `admins`.

# Terraform and Provider Validation

For many of the resources, Terraform and its associated providers will validate the input so that any potential errors can be detected quickly.

```
resource "aws_iam_user" "dev" {  
  name = "kplabs-user-01#"  
}
```



```
C:\kplabs-terraform>terraform plan
```

```
Error: invalid value for name (must only contain alphanumeric characters, hyphens, underscores, commas, periods, @ symbols, plus and equals signs)
```

```
with aws_iam_user.dev,  
on tf-logs.tf line 5, in resource "aws_iam_user" "dev":  
5:   name = "kplabs-user-01#"
```



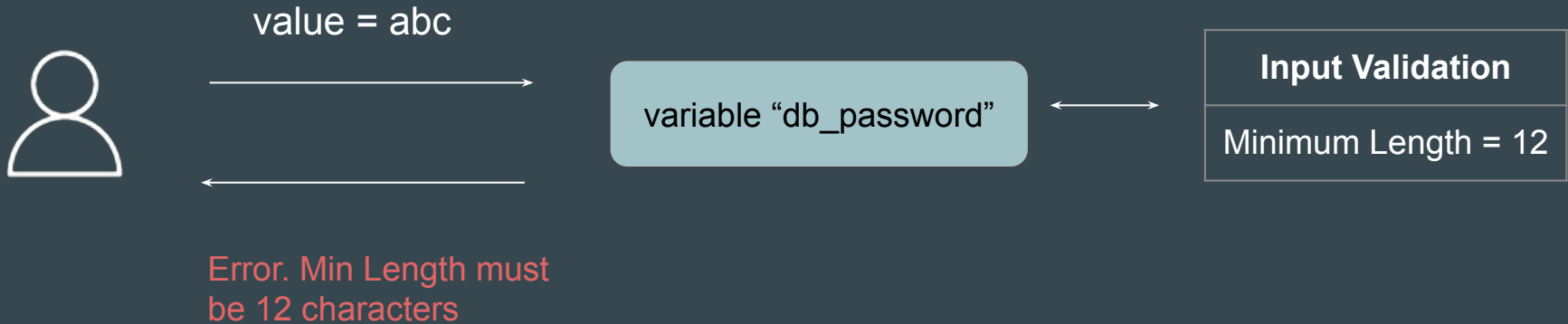
# Terraform and Provider Validation

The provider-side validation works for many of the resources but NOT all of the resources.

Hence, it is essential to have user-side validation to ensure consistency and conformance to the standards.

# Input Variable Validation

The **input variable validation** feature allows you to enforce certain rules or constraints on the values that can be assigned to input variables.



# Reference Screenshot

```
C:\kplabs-terraform>terraform plan
```

```
var.db_password
```

```
  Password for the database
```

```
  Enter a value: abc
```

```
Planning failed. Terraform encountered an error while generating this plan.
```

```
Error: Invalid value for variable
```

```
on tf-logs.tf line 3:
```

```
  3: variable "db_password" {
```

```
    | var.db_password is "abc"
```

```
Database password must be at least 12 characters long.
```

```
This was checked by the validation rule at tf-logs.tf:7,3-13.
```

# Importance of Validation

It allows organizations to ensure consistency and adhere to organizational best practices.

Allows organizations to make make your Terraform code more predictable.

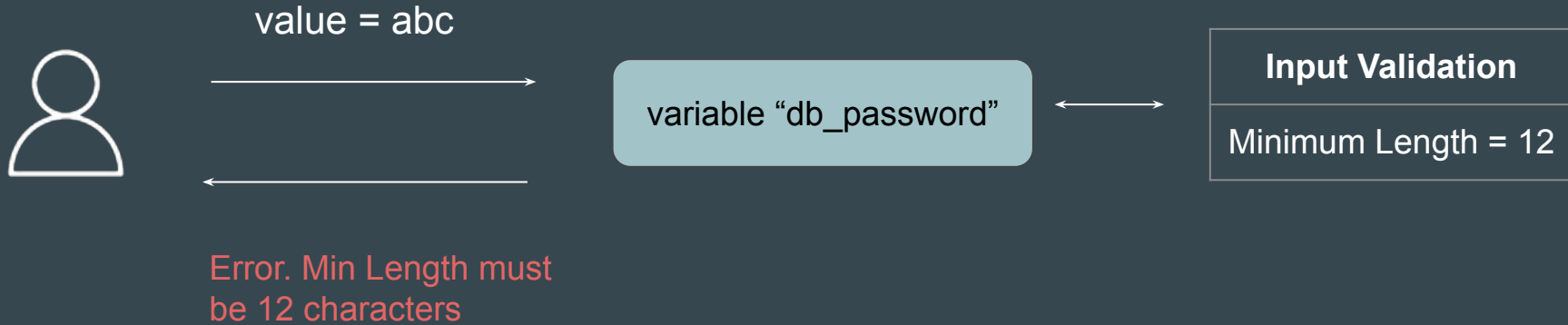
Allows organizations to catch misconfigurations at an early stage, saving time and potential infrastructure issues.

# **Input Variable Validation - Practical**

## Scenario To Consider for Practical

The value for “db\_password” variable must be of minimum 12 characters length.

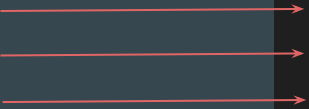
If user enters anything less than 12, Terraform should thrown an error.



# Understanding the Structure

There are three important components of input variable validation feature.

Validation Block, Condition, and Error Message



```
variable "db_password" {  
  type = string  
  description = "Password for the database"  
  
  validation {  
    condition      = length(var.db_password) >= 12  
    error_message = "Database password must be at least 12 characters long."  
  }  
}
```

# Understanding Each Components

**Validation block** is used within a variable declaration to define one or more conditions that the variable's value must satisfy.

**Condition** is a boolean expression that must evaluate to true for the validation to pass.

The **error message** is a string that is displayed when the condition fails.

```
variable "db_password" {  
    type = string  
    description = "Password for the database"  
  
    validation {  
        condition      = length(var.db_password) >= 12  
        error_message = "Database password must be at least 12 characters long."  
    }  
}
```

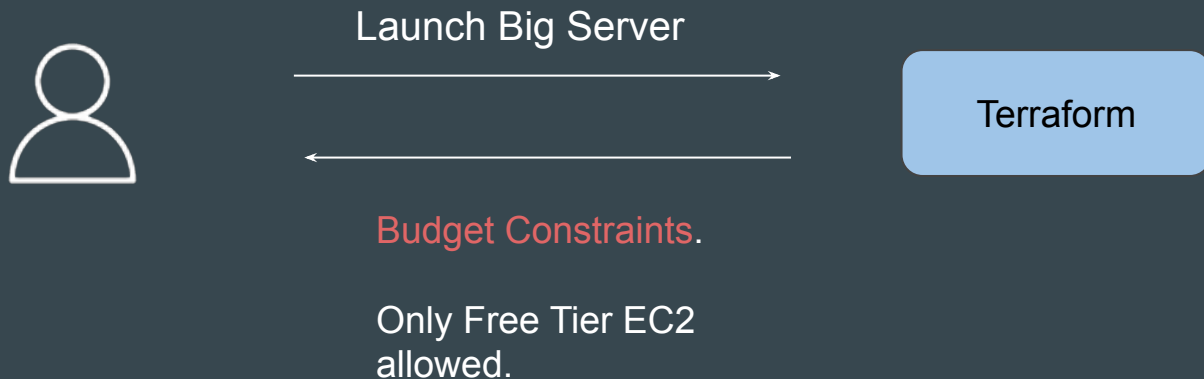


# **Preconditions and Postconditions**

# Introducing Pre-Conditions

Terraform preconditions are custom conditions that are checked **BEFORE** evaluating the object they are associated with.

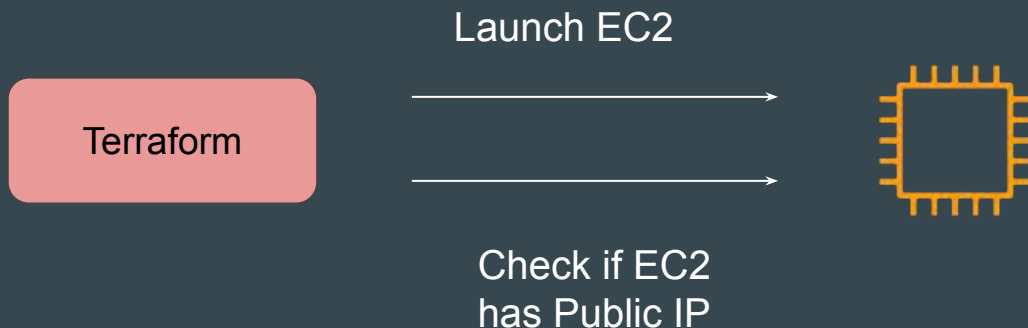
Example: Launch EC2 instance only if it is part of free tier



# Introducing Post-Conditions

Terraform postconditions are custom conditions that are checked **AFTER** evaluating the object they are associated with.

Example: Verify if EC2 has Public IP address after it has been created.



# Overall Syntax

The precondition and postcondition are defined inside the lifecycle block.

```
resource "aws_instance" "example" {  
  instance_type = "t3.micro"  
  ami           = "ami-123"  
  
  lifecycle {  
    precondition {  
      condition      = logic-here  
      error_message = "Error Message Here"  
    }  
  
    postcondition {  
      condition      = logic-here  
      error_message = "Error Message Here"  
    }  
  }  
}
```

# Reference Code

```
variable "instance_type" {}

data "aws_ec2_instance_type" "example" {
  instance_type = var.instance_type
}

resource "aws_instance" "example" {
  instance_type = var.instance_type
  ami           = "ami-066784287e358dad1"

  lifecycle {
    precondition {
      condition = data.aws_ec2_instance_type.example.free_tier_eligible
      error_message = "Instance Type is not part of free tier"
    }

    postcondition {
      condition = self.public_dns == ""
      error_message = "Public IPV4 or DNS is mandatory for this server"
    }
  }
}
```

## Point to Note

You can use the **self** object in postcondition blocks to refer to attributes of the instance under evaluation.

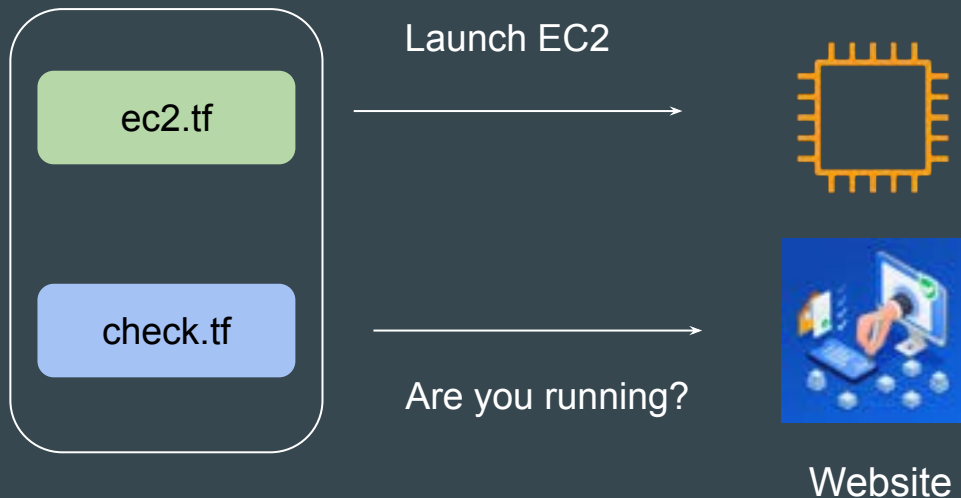
Preconditions and postconditions are available in Terraform v1.2.0 and later.

Preconditions and Postconditions are supported for resources, data sources, and outputs

**Check Blocks**

# Basics of Check Blocks

The check block can validate your infrastructure **outside** the usual resource lifecycle.





# Using Data Source + Instance Block

In default configuration, if you use data source block for checking website along with other resource block, if data block fails, Terraform will error out.

```
data "http" "example" {  
  url = "https://google1212.com"  
}  
  
resource "local_file" "foo" {  
  content = "Hi"  
  filename = "${path.module}/foo.txt"  
}
```



Plan: 1 to add, 0 to change, 0 to destroy.

Error: Error making request

with data.http.example,  
on check-block.tf line 14, in data "http" "example":  
14: data "http" "example" {

Error making request: GET https://google1212.com giving up after 1 attempt(s):  
lookup google1212.com: no such host

# Workflow of Check Block

Check block can include the nested data source block

If a scoped data source's provider raises any errors, they are masked as warnings and do not prevent Terraform from continuing operation execution.

```
check "website_check" {  
  data "http" "terraform_io" {  
    url = "https://google1212.com"  
  }  
}
```



```
Plan: 1 to add, 0 to change, 0 to destroy.  
local_file.foo: Creating...  
local_file.foo: Creation complete after 0s [id=94dd9e08c129c785f7f256e82fbe0a30e]  
data.http.terraform_io: Reading...  
  
Warning: Error making request  
  
with data.http.terraform_io,  
on check-block.tf line 25, in check "website_check":  
25:   data "http" "terraform_io" {  
  
Error making request: GET https://google1212.com giving up after 1 attempt(s):  
lookup google1212.com: no such host  
  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

# Format of Check Block

Each check block must have at least one **assert** blocks.

Each assert block has a **condition** attribute and an **error\_message** attribute.

```
check "website_check" {  
  data "http" "terraform_io" {  
    url = "https://google.com"  
  }  
  
  assert {  
    condition = your-condition-here  
    error_message = "your error message here."  
  }  
}
```

## Point to Note - Check Blocks

Assertions do not affect Terraform's execution of an operation. A failed assertion reports a warning without halting the ongoing operation.

This contrasts with other custom conditions, such as a postcondition, where Terraform produces an error immediately, halting the operation and blocking the application or planning of future resources.

# Sample Use-Case - AWS Budgets

Check the AWS Budget and warn if Budget has exceeded.

```
check "check_budget_exceeded" {  
  data "aws_budgets_budget" "example" {  
    name = aws_budgets_budget.example.name  
  }  
  
  assert {  
    condition = !data.aws_budgets_budget.example.budget_exceeded  
    error_message = "budet exceeded"  
  }  
}
```

```
| Warning: Check block assertion failed  
|  
| on main.tf line 43, in check "check_budget_exceeded":  
| 43:     condition = !data.aws_budgets_budget.example.budget_exceeded  
|     |  
|     | data.aws_budgets_budget.example.budget_exceeded is true  
|  
| AWS budget has been exceeded! Calculated spend: '1550.0' and budget limit: '1200.0'
```

# Point to Note

Checks becomes more useful when they are regularly run in automation and can alert us if they fail.

HCP Terraform can use check blocks to continuously monitor health and provide notifications using continuous validation feature.



| Name  | Run status    | Health checks succeeded | Health checks passed | Health checks failed | Health checks errored | Resources drifted | Resources undrifted |
|-------|---------------|-------------------------|----------------------|----------------------|-----------------------|-------------------|---------------------|
| admin | errored       | 0                       | 0                    | 0                    | 0                     | 0                 | 0                   |
| api   | errored       | 0                       | 0                    | 0                    | 0                     | 0                 | 0                   |
| file  | not_estimated | not                     | 1                    | 0                    | 0                     | 0                 | 0                   |
| web   | succeeded     | true                    | 0                    | 0                    | 0                     | 0                 | 0                   |

# Moved Blocks

## Setting the Base

In shared modules and long-lived configurations, you may eventually outgrow your initial module structure and resource names.

```
resource "aws_security_group" "database_firewall" {  
  name      = "db_firewall"  
}
```

Rename

```
resource "aws_security_group" "payments_database_firewall" {  
  name      = "db_firewall"  
}
```



# Understanding the Challenge

Terraform understands moving or renaming an object as an intent to destroy the object at the old address and to create a new object at the new address.

```
resource "aws_security_group" "database_firewall" {  
  name      = "db_firewall"  
}  
  
resource "aws_security_group" "payments_database_firewall" {  
  name      = "db_firewall"  
}
```



After rename



```
Terraform used the selected providers to generate the following execution plan,  
following symbols:  
  + create  
  - destroy  
  
Terraform will perform the following actions:  
  
# aws_security_group.database_firewall will be destroyed  
# (because aws_security_group.database_firewall is not in configuration)  
resource "aws_security_group" "database_firewall" {  
  arn              = "arn:aws:ec2:us-east-1:042025557788:security-g  
  description      = "Managed by Terraform" -> null  
  egress           = [] -> null  
  id               = "tg-02c7e7f0185c885f6" -> null  
  ingress          = [] -> null  
  name             = "db_firewall" -> null  
  owner_id         = "042025557788" -> null  
  revoke_rules_on_delete = false -> null  
  tags             = {} -> null  
  tags_all         = {} -> null  
  vpc_id           = "vpc-069367557b523ef5a" -> null  
  # (1 unchanged attribute hidden)  
}  
  
# aws_security_group.payments_database_firewall will be created  
+ resource "aws_security_group" "payments_database_firewall" {  
  + arn              = (known after apply)
```

# Introducing Moved Blocks

Using **moved block**, Terraform treats an existing object at the old address as if it now belongs to the new address.

```
resource "aws_security_group" "payments_database_firewall" {  
  name      = "db_firewall"  
}  
  
moved {  
  from = aws_security_group.database_firewall  
  to   = aws_security_group.payments_database_firewall  
}
```



```
C:\kplabs-terraform\1>terraform plan  
aws_security_group.payments_database_firewall: Refreshing state... [id=sg-02c7e7f0185c885f6]  
  
Terraform will perform the following actions:  
  
# aws_security_group.database_firewall has moved to aws_security_group.payments_database_firewall  
resource "aws_security_group" "payments_database_firewall" {  
  id      = "sg-02c7e7f0185c885f6"  
  name    = "db_firewall"  
  tags    = {}  
  # (9 unchanged attributes hidden)  
}  
  
Plan: 0 to add, 0 to change, 0 to destroy.
```

# Moved Block Syntax

A moved block expects no labels and contains only from and to arguments:

```
moved {  
    from = aws_instance.a  
    to   = aws_instance.b  
}
```

## Point to Note

Both `terraform state mv` and `moved block` allow us to achieve a similar set of use cases.

One benefit of `moved block` is that the blocks are more visible for all team members to know of.

`terraform state mv` can be used in more complex scenarios as it can also support scripting for bulk operations.

# **Terraform Provisioners**

## Setting the Base

We have been using Terraform to create and manage resources for a specific provider.

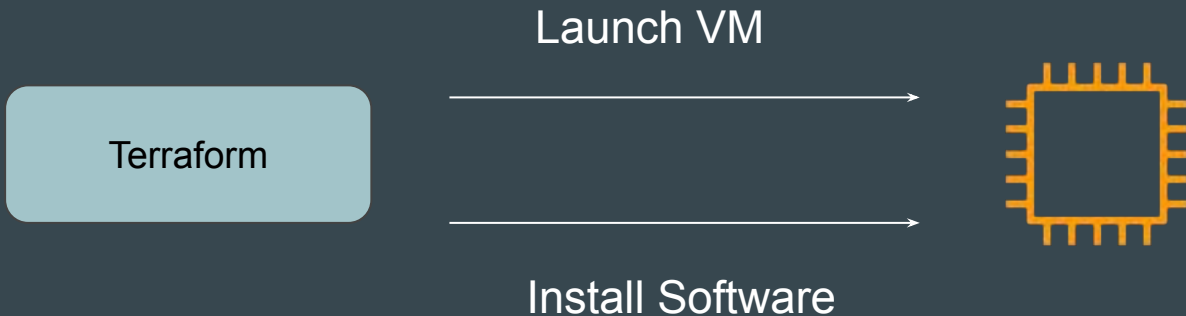
Organizations would want end-to-end solution for creation of infrastructure and configuring appropriate packages required for the application.



# Introducing Provisioners

Provisioners are used to **execute scripts on a local or remote machine** as part of resource creation or destruction.

Example: After VM is launched, install software package required for application.



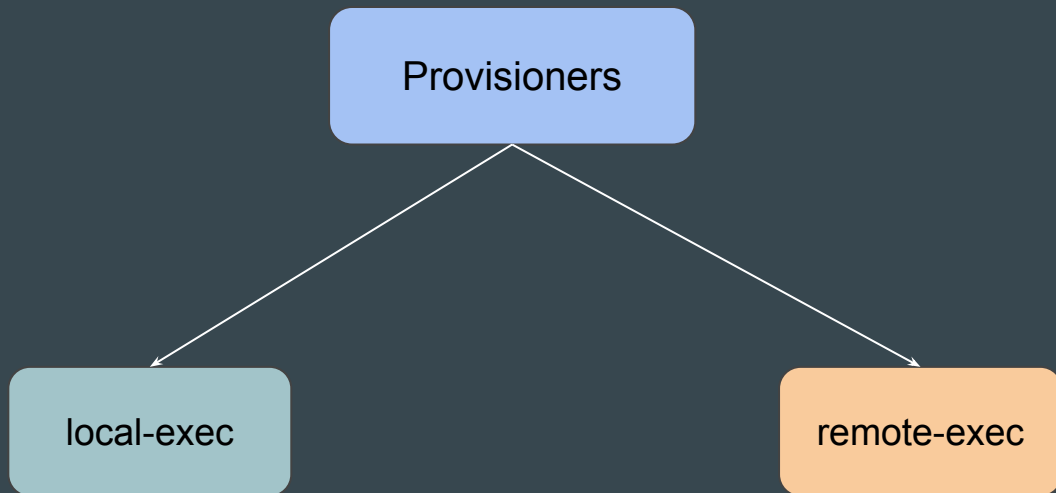
# **Types of Provisioners in Terraform**



# Setting the Base

Provisioners are used to **execute scripts on a local or remote machine** as part of resource creation or destruction.

There are 2 major types of provisioners available



## Type 1 - local-exec provisioner

The local-exec provisioner **invokes a local executable** after a resource is created.

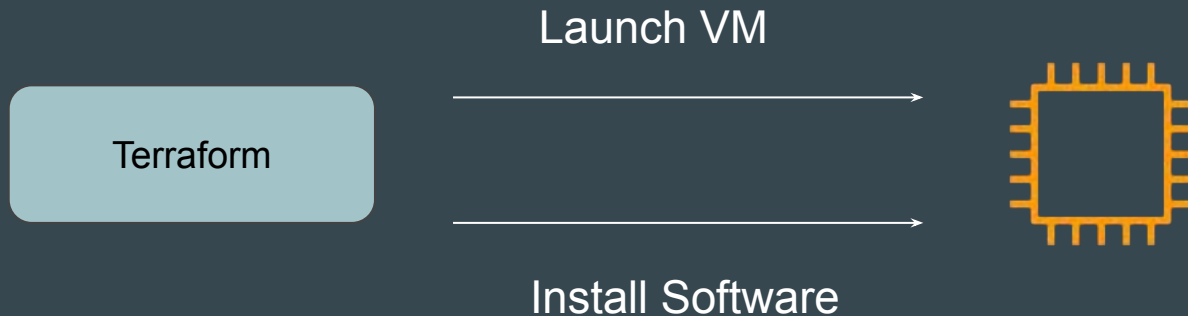
Example: After EC2 is launched, fetch the IP and store it in file server\_ip.txt



## Type 2 - remote-exec provisioner

remote-exec provisioners allow to invoke scripts or run commands directly on the remote server.

Example: After EC2 is launched, install “apache” software



# Today's Demo

For today's demo, the Terraform code will run two provisioners.

1. Remote-Exec will install Nginx software in EC2 to have basic website.
2. Local-Exec will fetch the Public IP of EC2 and store it in a new file.



# **Format of Provisioners**

# 1 - Defining Provisioners

Provisioners are defined inside a specific resource.

```
resource "aws_instance" "myec2" {  
  ami = "ami-001843b876406202a"  
  instance_type = "t2.micro"  
  
  <provisioners-need-to-be-defined-inside-resource>  
  
}
```

## 2 - Defining provisioner

Provisioners are defined by “provisioner” followed by type of provisioner

```
resource "aws_instance" "myec2" {  
  ami = "ami-001843b876406202a"  
  instance_type = "t2.micro"  
  
  provisioner "local-exec" {}  
  
  provisioner "remote-exec" {}  
  
}
```

### 3 - Local Provisioner Approach

For local provisioners, we have to specify command that needs to be run locally

```
resource "aws_instance" "example" {  
  ami = "ami-001843b876406202a"  
  instance_type = "t2.micro"  
  
  provisioner "local-exec" {  
    command = "echo Server has been created through Terraform"  
  }  
}
```



## 4 - Remote Exec Provisioner Approach

Since commands are executed on remote-server, we have to provide way for Terraform to connect to remote server.

Details to connect to the Server



Commands to Run on the Server



```
resource "aws_instance" "myec2" {  
  ami = "ami-001843b876406202a"  
  instance_type = "t2.micro"  
  
  connection {  
    type      = "ssh"  
    user      = "ec2-user"  
    private_key = file("../terraform-key.pem")  
    host      = self.public_ip  
  }  
  
  provisioner "remote-exec" {  
    inline = [  
      "sudo yum install -y nginx",  
      "sudo systemctl start nginx"  
    ]  
  }  
}
```

## **Points to Note - Provisioners**

# Provisioners are Defined inside Resource Block

It is not necessary to define a `aws_instance` resource block for provisioner to run.

They can be defined inside other resource types as well.

```
resource "aws_iam_user" "lb" {  
  name = "loadbalancer"  
  
  provisioner "local-exec" {  
    command = "echo local-exec provisioner is starting"  
  }  
}
```

# Multiple Provisioners Blocks for Single Resource

We can define multiple provisioners block in a single resource block.

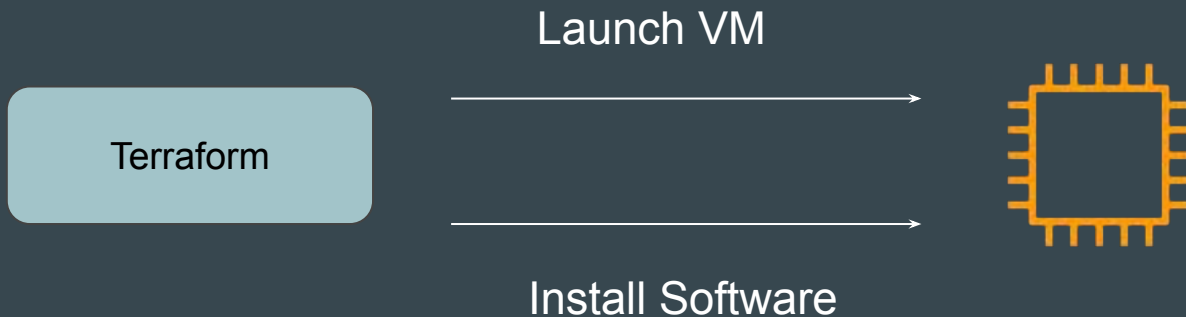
```
resource "aws_iam_user" "lb" {  
  name = "loadbalancer"  
  
  provisioner "local-exec" {  
    command = "echo local-exec provisioner is starting"  
  }  
  
  provisioner "local-exec" {  
    command = "echo local-exec-2 provisioner is starting"  
  }  
}
```

# **Creation-Time & Destroy-Time Provisioners**

# Basic of Creation-Time Provisioners

By default, provisioners run when the resource they are defined within is created.

Creation-time provisioners are **only run during creation**, not during updating or any other lifecycle.



# Destroy-Time Provisioner

Destroy provisioners are run before the resource is destroyed.

Example:

Remove and De-Link Anti-Virus software before EC2 gets terminated.

Define destroy-time  
provisioner



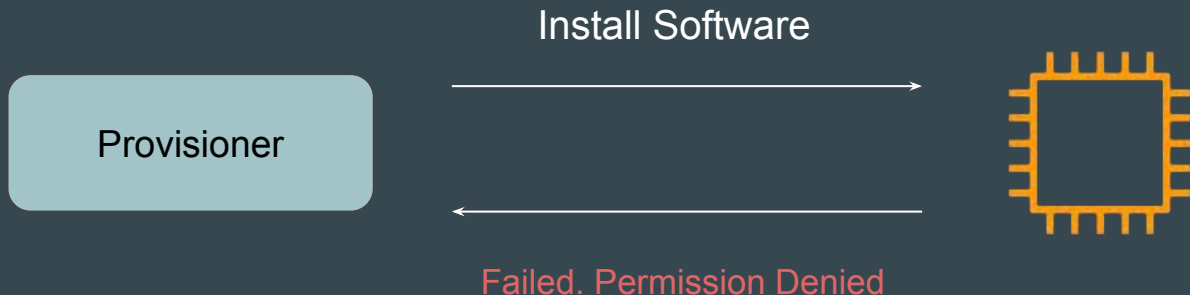
```
resource "aws_instance" "web" {  
  # ...  
  
  provisioner "local-exec" {  
    when      = destroy  
    command = "echo 'Destroy-time provisioner'"  
  }  
}
```

# Tainting Resource in Creation-Time Provisioners

If a creation-time provisioner fails, the resource is marked as **tainted**.

A tainted resource will be planned for destruction and recreation upon the next terraform apply.

Terraform does this because a failed provisioner can leave a resource in a semi-configured state.

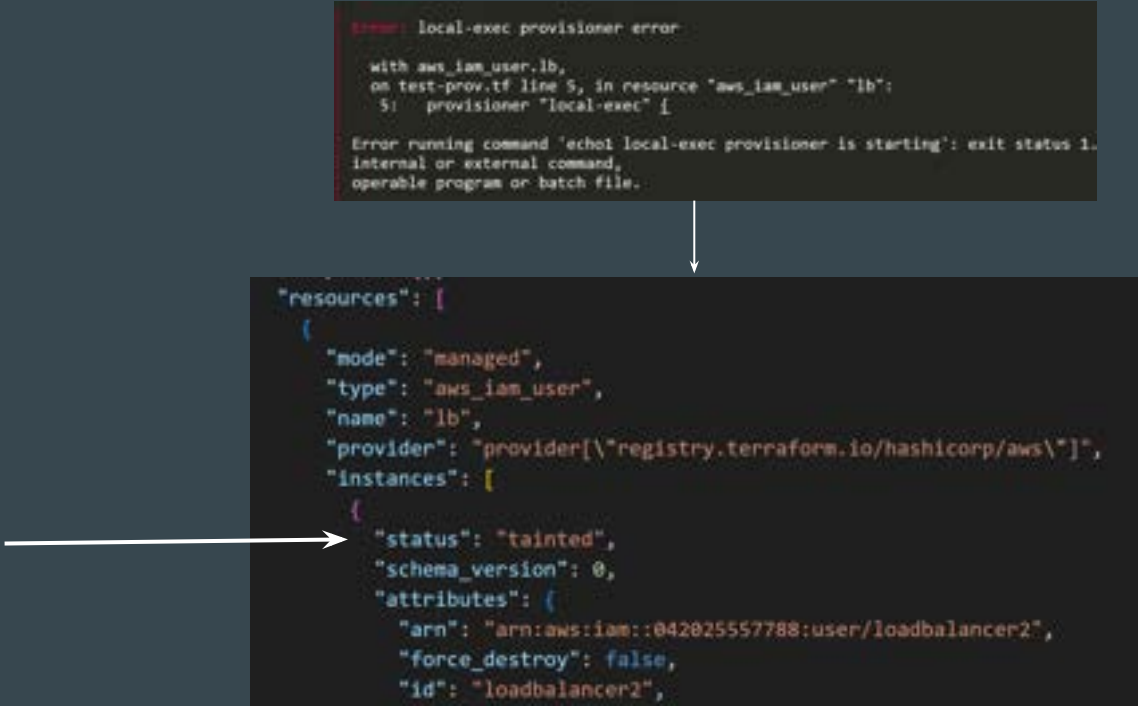




# Reference Screenshot - Resource Marked as Tainted

Following screenshot shows state file that has marked the resource as “tainted” because the provisioner had failed.

```
Error: local-exec provisioner error  
  
with aws_iam_user.lb,  
on test-prov.tf line 5, in resource "aws_iam_user" "lb":  
5:   provisioner "local-exec" {  
  
Error running command 'echo local-exec provisioner is starting': exit status 1.  
internal or external command,  
operable program or batch file.
```



A white arrow points from the error message above to the state file entry below. Another white arrow points from the left towards the "status": "tainted" line in the state file.

```
"resources": [  
  {  
    "mode": "managed",  
    "type": "aws_iam_user",  
    "name": "lb",  
    "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",  
    "instances": [  
      {  
        "status": "tainted",  
        "schema_version": 0,  
        "attributes": {  
          "arn": "arn:aws:iam::042025557788:user/loadbalancer2",  
          "force_destroy": false,  
          "id": "loadbalancer2",
```

# **Failure Behaviour in Provisioners**

# Understanding the Challenge

By default, provisioners that fail will also cause the terraform apply itself to fail.

This will lead to resource being tainted and we have to re-create the resource.

```
# aws_iam_user.lb will be created
+ resource "aws_iam_user" "lb" {
  + arn                = [known after apply]
  + force_destroy     = false
  + id                = [known after apply]
  + name              = "loadbalancer2"
  + path              = "/"
  + tags_all          = [known after apply]
  + unique_id         = [known after apply]
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_iam_user.lb: Creating...
aws_iam_user.lb: Provisioning with 'local-exec'...
aws_iam_user.lb (local-exec): Executing: ["cmd" "/C" "echo! local-exec provisioner is starting"]
aws_iam_user.lb (local-exec): 'echo!' is not recognized as an internal or external command,
aws_iam_user.lb (local-exec): operable program or batch file.

Error: local-exec provisioner error

   with aws_iam_user.lb,
   on test-prov.tf line 5, in resource "aws_iam_user" "lb":
    5:   provisioner "local-exec" {

Error running command 'echo! local-exec provisioner is starting': exit status 1. Output: 'echo!' is not recognized as an
internal or external command,
operable program or batch file.
```

# Basics of On Failure Setting

The `on_failure` setting can be used to change the default behaviour.

| Allowed Values | Description                                                                                                     |
|----------------|-----------------------------------------------------------------------------------------------------------------|
| continue       | Ignore the error and continue with creation or destruction.                                                     |
| fail           | Raise an error and stop applying (the default behavior). If this is a creation provisioner, taint the resource. |

## Reference Code - On-Failure

Following screenshot shows a reference code where `on_failure` is set to `continue`.

```
resource "aws_iam_user" "lb" {  
  name = "loadbalancer"  
  
  provisioner "local-exec" {  
    command = "echo1 local-exec provisioner is starting"  
    on_failure = continue  
  }  
}
```

# Reference Screenshot - Failed Provisioner

Following screenshot shows that the provisioner has failed but still the apply has completed successfully.

This is an example of `on_failure = continue`

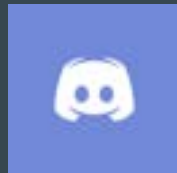


```
Plan: 1 to add, 0 to change, 1 to destroy.
aws_iam_user.lb: Destroying... [id=loadbalancer2]
aws_iam_user.lb: Destruction complete after 1s
aws_iam_user.lb: Creating...
aws_iam_user.lb: Provisioning with 'local-exec'...
aws_iam_user.lb (local-exec): Executing: ["cmd" "/C" "echo1 local-exec provisioner is starting"]
aws_iam_user.lb (local-exec): 'echo1' is not recognized as an internal or external command,
aws_iam_user.lb (local-exec): operable program or batch file.
aws_iam_user.lb: Creation complete after 1s [id=loadbalancer2]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)



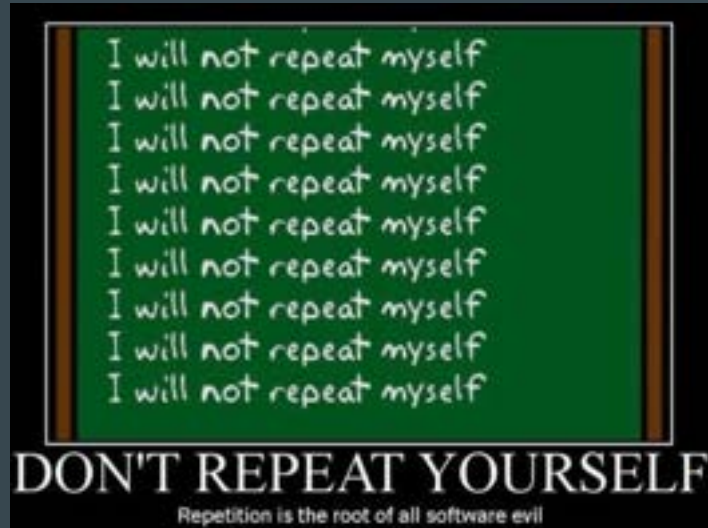
[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **Terraform Modules**



# Understanding the Basic

In software engineering, **don't repeat yourself** (DRY) is a principle of software development aimed at reducing repetition of software patterns.



# Understanding the Challenge

Let's assume there are 10 teams in your organization using Terraform to create and manage EC2 instances.

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 1

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 3

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 5

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 2

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 4

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
}
```

Team 6

# Challenge with the Previous Example

1. Repetition of Code.
2. Change in AWS Provider specific option will require change in EC2 code blocks of all the teams.
3. Lack of standardization.
4. Difficult to manage.
5. Difficult for developers to use.

# Better Approach

In this approach, the DevOps Team has defined standard EC2 template in a central location that all can use.

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
  associate_public_ip_address = true  
  instance_initiated_shutdown_behavior = "stop"  
  ebs_optimized = true  
  source_dest_check = false  
  hibernation = true  
  disable_api_termination = true  
}
```

Standard Template

Team 1 Code

Team 2 Code

Team 3 Code

Team 4 Code

# Introducing Terraform Modules

**Terraform Modules** allows us to centralize the resource configuration and it makes it easier for multiple projects to re-use the Terraform code for projects.

```
resource "aws_instance" "web" {  
  ami          = "ami-1234"  
  instance_type = "t3.micro"  
  key_name     = "user1"  
  monitoring   = true  
  vpc_security_group_ids = ["sg-12345678"]  
  associate_public_ip_address = true  
  instance_initiated_shutdown_behavior = "stop"  
  ebs_optimized = true  
  source_dest_check = false  
  hibernation = true  
  disable_api_termination = true  
}
```

Terraform Module

Team 1 Code

Team 2 Code

Team 3 Code

Team 4 Code

# Multiple Modules for a Single Project

Instead of writing code from scratch, we can use multiple ready-made modules available.

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
  associate_public_ip_address = true  
  instance_initiated_shutdown_behavior = "stop"  
  ebs_optimized = true  
  source_dest_check = false  
  hibernation = true  
  disable_api_termination = true  
}
```

EC2 Module

```
resource "aws_vpc" "vpc" {  
  cidr_block = local.cidr_block / 0 : 0  
  
  cidr_block = var.cidr_block / 0 : 0  
  ipam_pool_id = var.ipam_pool_id  
  ipam_pool_length = var.ipam_pool_length  
  
  enable_dns_hostnames = var.enable_dns_hostnames  
  enable_dns_support = var.enable_dns_support  
  ipam_pool_id = var.ipam_pool_id  
  ipam_pool_length = var.ipam_pool_length  
  ipam_pool_network_border_group = var.ipam_pool_network_border_group  
}
```

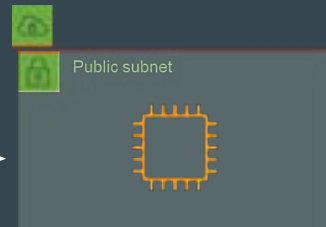
VPC Module

Team 1 Code

Team 2 Code

Team 3 Code

Team 4 Code



Infrastructure Created

## **Points to Note - Referencing Terraform Modules**

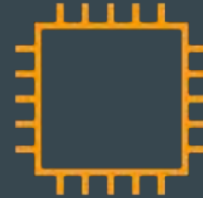
# Understanding the Base

For some infrastructure resources, you can directly use the module calling code, and the entire infrastructure will be created for you.

ec2-module.tf > ...

```
module "ec2-instance" {  
  source = "terraform-aws-modules/ec2-instance/aws"  
  version = "5.6.1"  
}
```

terraform apply





# Avoiding Confusion

Just by referencing any module, it is not always the case that the infrastructure resource will be created for you directly.

Some modules require specific inputs and values from the user side to be filled in before a resource gets created.

# Example Module - AWS EKS

If you try to use an AWS EKS Module directly and run “terraform apply”, it will throw an **error**.

```
eks-module.tf > ...  
module "eks" {  
  source = "terraform-aws-modules/eks/aws"  
  version = "20.11.1"  
}
```

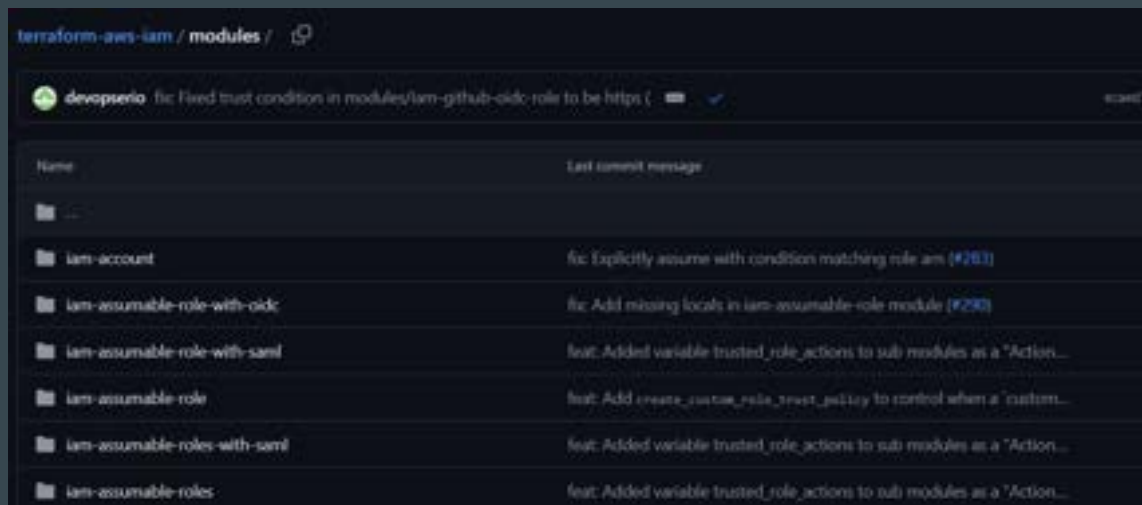
terraform apply

```
Plan: 21 to add, 0 to change, 0 to destroy.  
  
Error: Error in function call  
  
on .terraform/modules/eks/main.tf line 48, in resource "aws_eks_cluster" "this":  
48:   subnet_ids = coalesce(list(var.control_plane_subnet_ids, var.subnet_ids))  
    while calling coalesce(list(...))  
    var.control_plane_subnet_ids is empty list of string  
    var.subnet_ids is empty list of string  
  
Call to function "coalesce(list(...))" failed: no non-null arguments.
```

# Module Structure Can be Different

Some module pages in GitHub can contain multiple sets of modules together for different features.

In such cases, you have to reference the exact sub-module required.



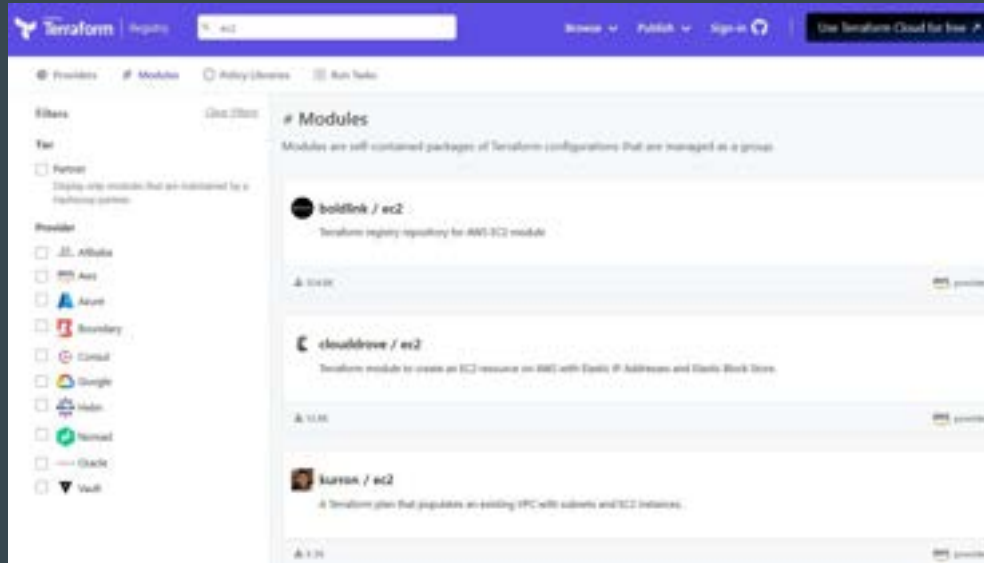
# Learnings for Today's Video

Always read the Module Documentation to understand the overall structure, important information, and what is expected from the user side when creating a resource.

# **Choosing the Right Terraform Module**

# Understanding the Base

Terraform Registry can contain multiple modules for a specific infrastructure resource maintained by different users



# 1 - Check Total Downloads

Module Downloads can provide early indication about level of acceptance by users in the Terraform community

The screenshot displays the Terraform Registry page for the `aws/ec2-instance` module. The page includes the AWS logo, the module name `ec2-instance`, and a description: "Terraform module to create AWS EC2 instance(s) resources via". It also shows the version `1.6.1 (latest)` and a list of download statistics:

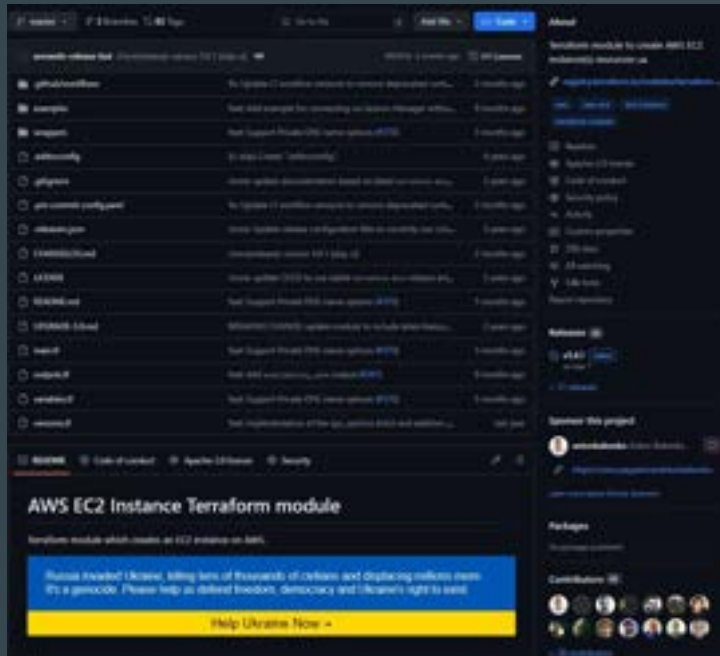
| Module Downloads        |         |
|-------------------------|---------|
| Downloads this week     | 114,191 |
| Downloads this month    | 476,217 |
| Downloads this year     | 3,868   |
| Downloads over all time | 14,466  |

Below the statistics, there is a section for "Provision Instructions" which includes a code snippet for the module configuration:

```
module "ec2-instance" {  
  source = "terraform-aws-modules/ec2-instance"   
  version = "1.6.1"  
}
```

## 2 - Check GitHub Page of Module

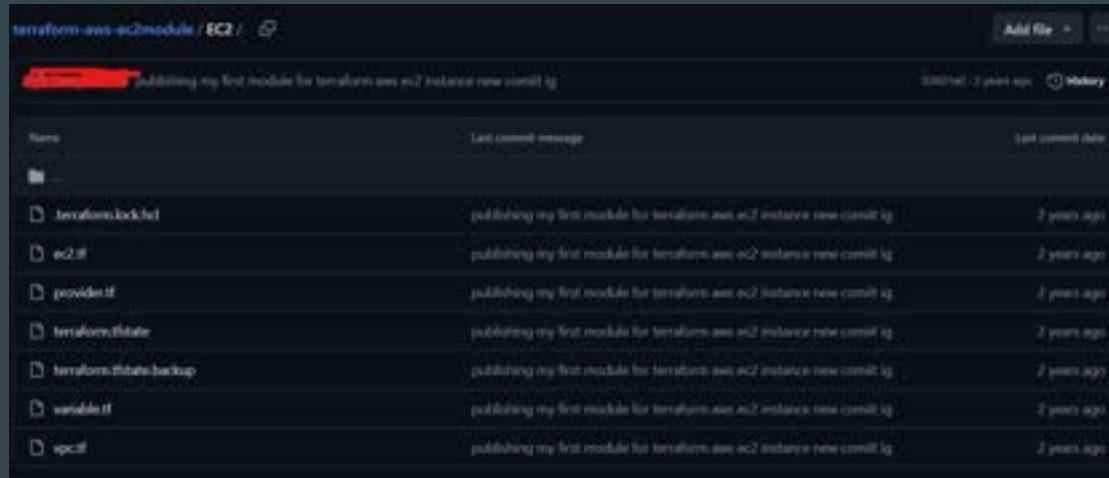
GitHub page can provide important information related to the Contributors, Reported Issues and other data.





# 3 - Avoid Modules Written by Individual Participant

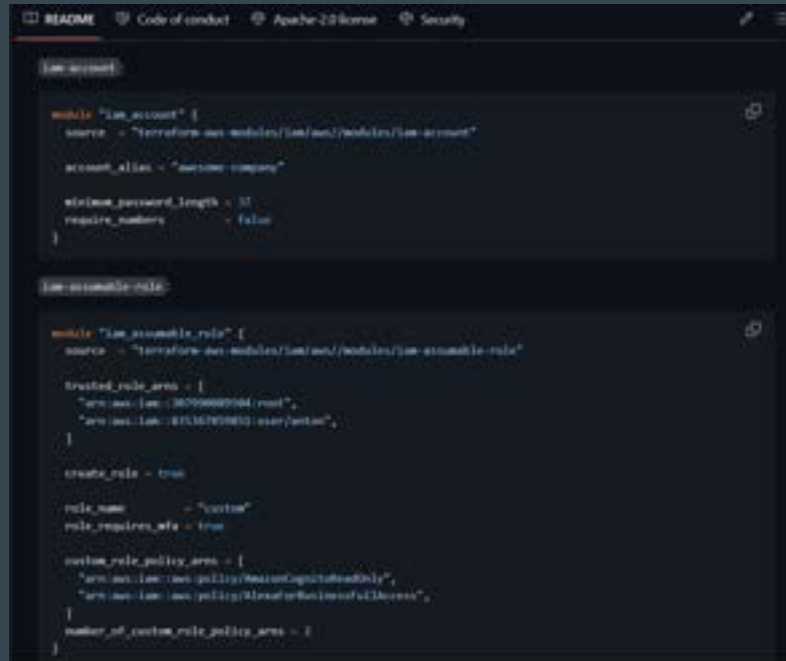
Avoid module that are maintained by a single contributor as regular updates, issues and other areas might not always be maintained.



| Name                                                                                                       | Last commit message                                                     | Last commit date |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------|
|                           |                                                                         |                  |
|  .terraform.lock.hcl      | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  ec2.tf                   | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  provider.tf              | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  terraform.tfstate        | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  terraform.tfstate.backup | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  variables.tf             | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |
|  ec2.tf                   | publishing my first module for terraform aws ec2 instance new commit ig | 2 years ago      |

## 4 - Analyze Module Documentation

Good documentation should include an overview, usage instructions, input and output variables, and examples.



```
README Code of conduct Apache 2.0 license Security

[iam-account]

module "iam_account" {
  source = "terraform-aws-modules/iam/aws//modules/iam-account"

  account_alias = "awsone-company"

  minimum_password_length = 11
  require_numbers         = false
}

[iam-assume-role]

module "iam_assume_role" {
  source = "terraform-aws-modules/iam/aws//modules/iam-assume-role"

  trusted_role_arns = [
    "arn:aws:iam::36996667064:root",
    "arn:aws:iam::41516705853:role/pentest",
  ]

  create_role = true

  role_name           = "curious"
  role_requires_mfa = true

  custom_role_policy_arns = [
    "arn:aws:iam::aws:policy/AWSReadOnlyAccess",
    "arn:aws:iam::aws:policy/AWSFullAccess",
  ]
  number_of_custom_role_policy_arns = 2
}
```

## 5 - Check Version History of Module

Look at the version history. Frequent updates and a clear versioning strategy suggest active maintenance.

The screenshot shows the Terraform Registry page for the 'iam' module. The page header includes the Terraform logo, a search bar, and navigation links for 'Browse', 'Publish', and 'Sign-in'. A button for 'Use Terraform Cloud for free' is also present.

The main content area displays the 'iam' module details, including the AWS logo and the description 'Terraform module to create AWS IAM resources via'. It also shows the publication date (May 15, 2024) and the source code repository (GitHub).

A dropdown menu for 'Version 5.38.1 (latest)' is open, showing a list of versions from 5.32.1 to 5.38.1. The versions are listed in descending order, with the latest version at the top.

To the right of the version list, there is a 'Module Downloads' section showing download statistics for the current version (5.38.1). The statistics are as follows:

| Module Downloads        | All versions |
|-------------------------|--------------|
| Downloads this week     | 1.9M         |
| Downloads this month    | 7.4M         |
| Downloads this year     | 36.1M        |
| Downloads over all time | 110.1M       |

Below the download statistics, there is a 'Provision Instructions' section. It contains a code block with the following Terraform configuration snippet:

```
module "iam" {
  source = "terraform-aws-modules/iam"
  version = "5.38.1"
}
```

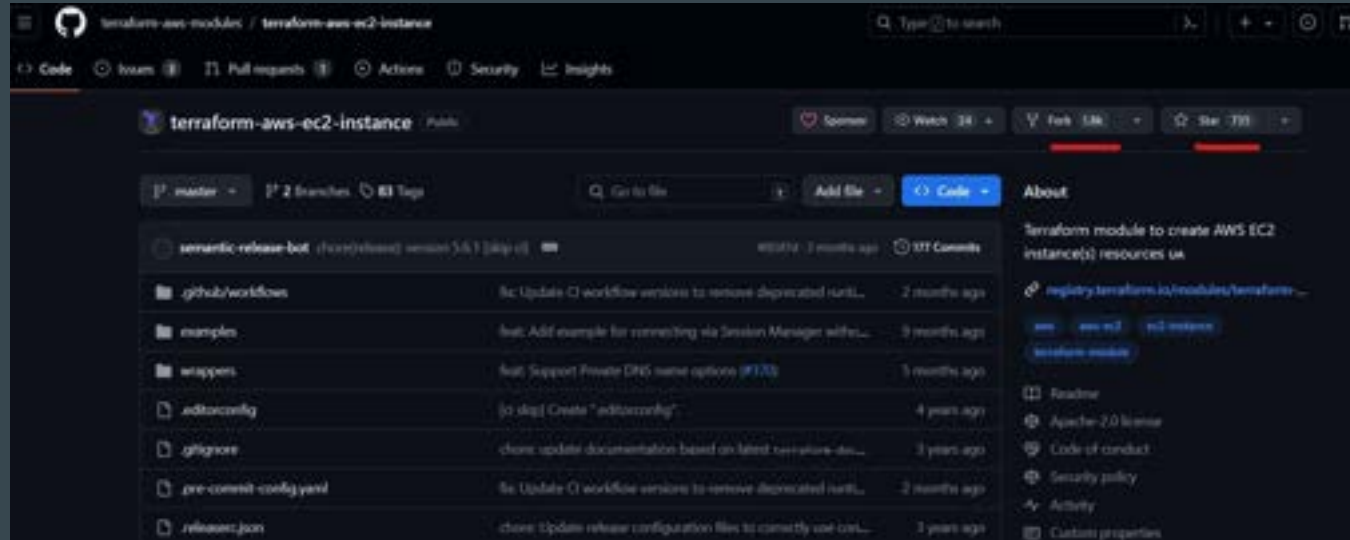
## 6 - Analyze the Code

Inspect the module's source code on GitHub or another platform. Clean, well-structured code is a good sign.

```
resource "aws_instance" "this" {  
  count = local.create && !var.ignore_ami_changes && !var.create_spot_instance ? 1 : 0  
  
  ami           = local.ami  
  instance_type = var.instance_type  
  cpu_core_count = var.cpu_core_count  
  cpu_threads_per_core = var.cpu_threads_per_core  
  hibernation    = var.hibernation  
  
  user_data          = var.user_data  
  user_data_base64   = var.user_data_base64  
  user_data_replace_on_change = var.user_data_replace_on_change  
  
  availability_zone = var.availability_zone  
  subnet_id        = var.subnet_id  
  vpc_security_group_ids = var.vpc_security_group_ids  
  
  key_name        = var.key_name  
  monitoring      = var.monitoring  
  get_password_data = var.get_password_data  
  iam_instance_profile = var.create_iam_instance_profile ? aws_iam_instance_profile.this[0].name : var.iam_instance_profile
```

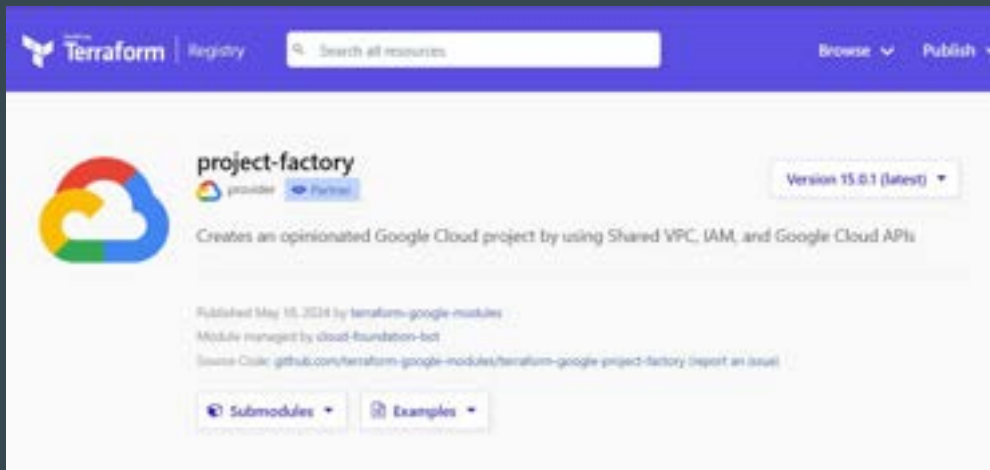
## 7 - Check the Community Feedback

The number of stars and forks on GitHub can indicate popularity and community interest.



## 8 - Modules Maintained by HashiCorp Partner

Search for modules that are maintained by HashiCorp Partners



## Important Point to Note

Avoid directly trying any random Terraform module that is not actively maintained and looks shady (primarily by sole individual contributors)

An attacker can include malicious code in a module that sends information about your environment to the attacker.

# Which Modules do Organizations Use?

In most of the scenarios, organizations maintain their own set of modules.

They might initially fork a module from the Terraform registry and modify it based on their use case.

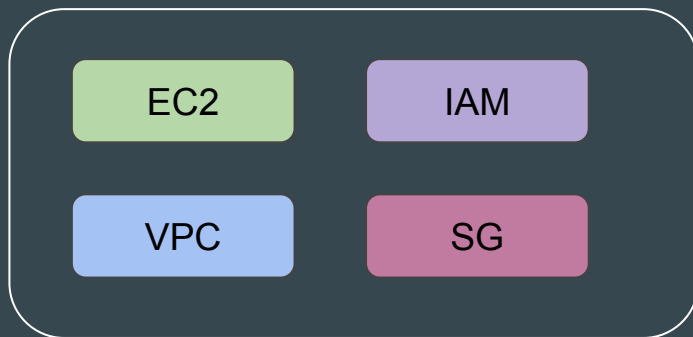


# **Creating Base Module Structure**

# Understanding the Base Structure

A base “modules” folder.

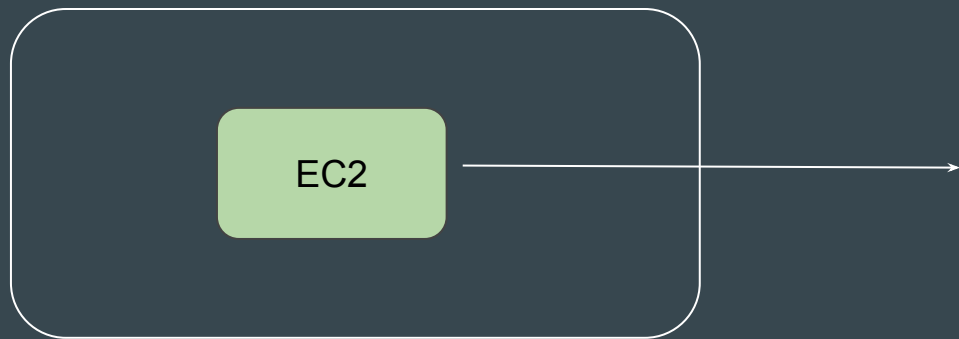
A sub-folder containing name for each modules that are available.



modules folder

# What is Inside the Sub-Folders

Each module's sub-folder contains the actual module Terraform code that other projects can reference from.



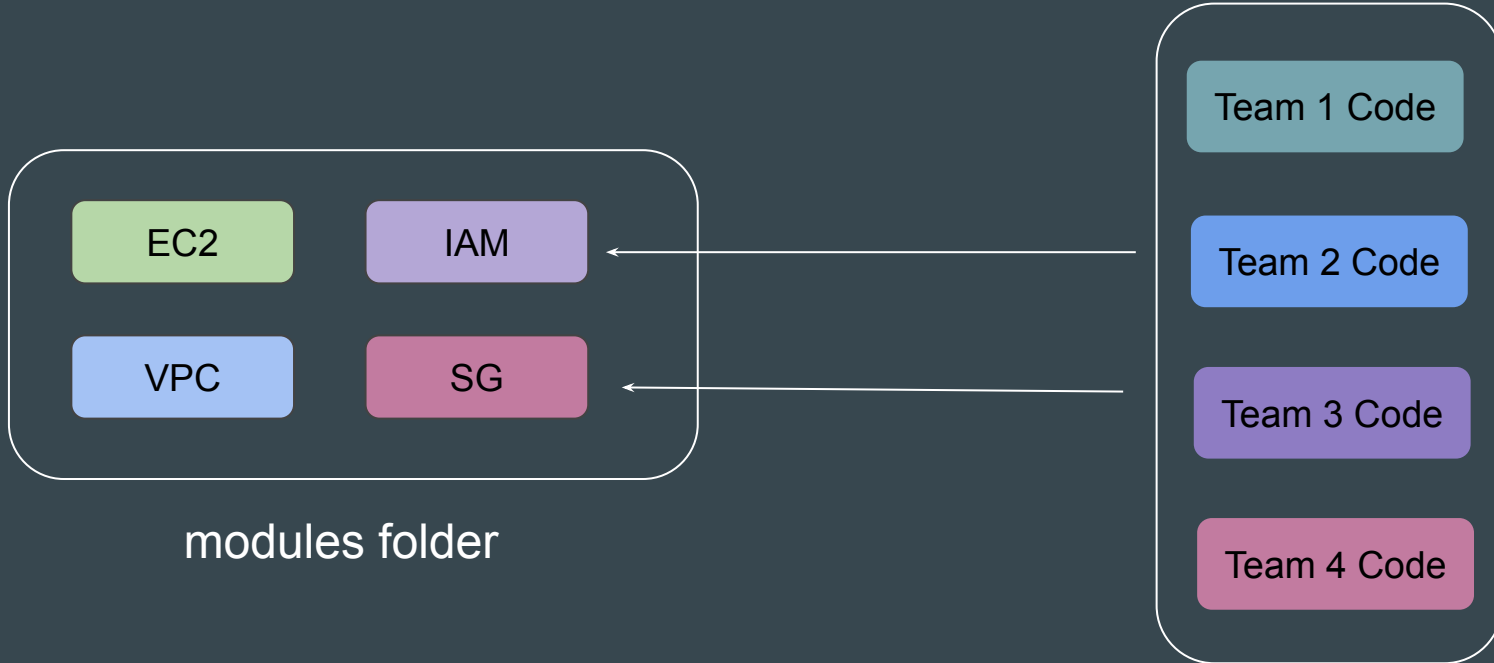
modules folder

```
resource "aws_instance" "web" {  
  ami           = "ami-1234"  
  instance_type = "t3.micro"  
  key_name      = "user1"  
  monitoring    = true  
  vpc_security_group_ids = ["sg-12345678"]  
  associate_public_ip_address = true  
  instance_initiated_shutdown_behavior = "stop"  
  ebs_optimized = true  
  source_dest_check = false  
  hibernation = true  
  disable_api_termination = true  
}
```

main.tf

# Calling the Module

Each Team can call various set of modules that are available in the modules folder based on their requirements.

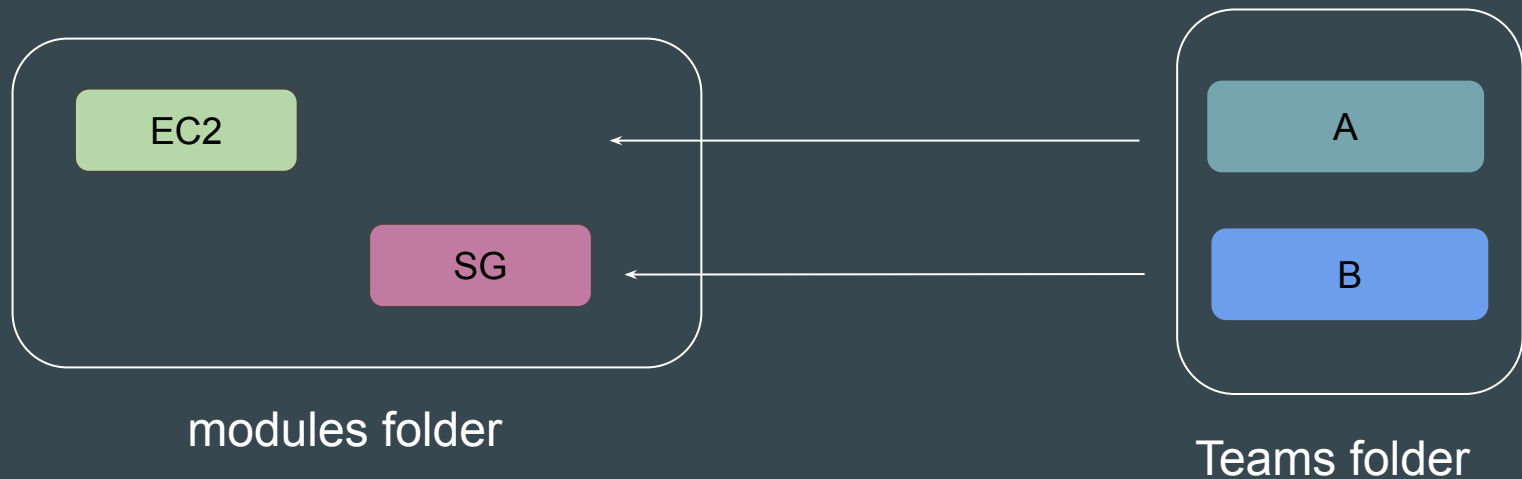


# Our Practical Structure

Our practical structure will include two main folders (modules and teams).

Modules sub-folder will contain sub-folders of modules that are available.

Teams sub-folder will contain list of teams that we want to be made available.



# **Module Sources - Calling a Module**

# Understanding the Base

Module source code can be present in wide variety of locations.

These includes:

1. GitHub
2. HTTP URLs
3. S3 Buckets
4. Terraform Registry
5. Local paths

## Base - Calling the Module

In order to reference to a module, you need to make use of **module** block

The module block must contain source argument that contains location to the referenced module.

```
module "ec2" {  
  source = "https://example.com/vpc-module.zip"  
}
```



## Example 1 - Local Paths

Local paths are used to reference to module that is available in local filesystem.

A local path must begin with either `./` or `../` to indicate that a local path

```
module "ec2" {  
  source = "../modules/ec2"  
}
```

## Example 2 - Generic Git Repository

Arbitrary Git repositories can be used by prefixing the address with the special `git::` prefix.

```
module "vpc" {  
  source = "git::https://example.com/vpc.git"  
}
```

# Module Version

A specific module can have multiple versions.

You can reference to specific version of module with the **version** block

```
module "eks" {  
  source  = "terraform-aws-modules/eks/aws"  
  version = "20.11.1"  
}
```

# **Improvements in Custom Module Code**

# Our Simple Module

We had created a very simple module that allows developers to launch an EC2 instance when calling the module.

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-0bb84b8ffd87024d8"  
  instance_type = "t2.micro"  
}
```

# Need to Analyze Shortcomings

Being a simplistic and a basic module code, there is a good room of improvements.

In today's video, we will be discussing about some of the important shortcomings with the code.



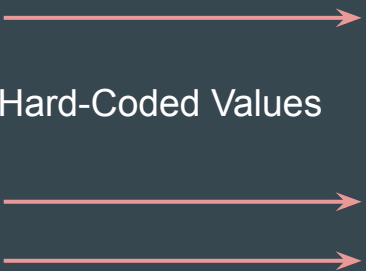
# Challenge 1 - Hardcoded Values

The values are hardcoded as part of the module.

If developer is calling the module, he will have to stick with same values.

Developer will not be able to override the hardcoded values of the module.

Hard-Coded Values



```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-0bb84b8ffd87024d8"  
  instance_type = "t2.micro"  
}
```

## Challenge 2 - Provider Improvements

Avoid hard-coding region in the Module code as much as possible.

A `required_provider` block with version constraints for module to work is important.

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-0bb84b8ffd87024d8"  
  instance_type = "t2.micro"  
}
```



```
terraform {  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = ">= 5.5"  
    }  
  }  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-0bb84b8ffd87024d8"  
  instance_type = "t2.micro"  
}
```



# **Variables in Terraform Modules**

## Point to Note

As much as possible, **avoid hardcoding values** as part of the Modules.

This will make the module less flexible.



# Convert Hard Coded Values to Variables

For modules, it is especially recommended to convert hard-coded values to **variables** so that it can be overridden based on user requirements.

```
resource "aws_instance" "myec2" {  
  ami = "ami-0bb84b8ffd87024d8"  
  instance_type = "t2.micro"  
}
```

Bad Approach



```
resource "aws_instance" "myec2" {  
  ami = var.ami  
  instance_type = var.instance_type  
}
```

Good Approach

# Advantages of Variables in Module Code

Variable based approach will allow the teams to override the values.

```
resource "aws_instance" "myec2" {  
  ami = var.ami  
  instance_type = var.instance_type  
}
```

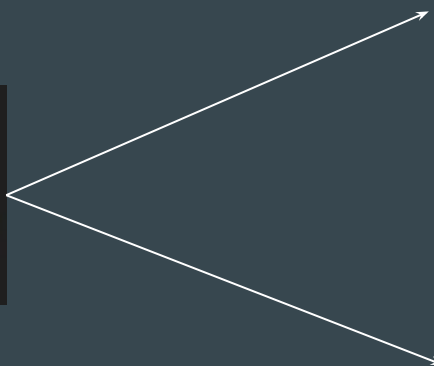
Main Module

Team 1

instance\_type = t2.micro

Team 2

instance\_type = m5.large



# Reviewing Professional EC2 Module Code

Reviewing an EC2 Module code that is professionally written, we see that the values associated with arguments are not hardcoded and variables are used extensively.

```
resource "aws_instance" "this" {  
  count = local.create ? var.ignore_ami_changes ? var.create_spot_instance ? 1 : 0  
  
  ami           = local.ami  
  instance_type = var.instance_type  
  cpu_core_count = var.cpu_core_count  
  cpu_threads_per_core = var.cpu_threads_per_core  
  hibernation    = var.hibernation  
  
  user_data           = var.user_data  
  user_data_base64    = var.user_data_base64  
  user_data_replace_on_change = var.user_data_replace_on_change  
  
  availability_zone = var.availability_zone  
  subnet_id        = var.subnet_id  
  vpc_security_group_ids = var.vpc_security_group_ids  
  
  key_name     = var.key_name  
  monitoring   = var.monitoring  
  get_password_data = var.get_password_data  
  iam_instance_profile = var.create_iam_instance_profile ? aws_iam_instance_profile.this[0].name : var.iam_instance_profile
```

# **Module Outputs**

# Revising Output Values

**Output values** make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use.



```
resource "aws_eip" "lb" {  
  domain = "vpc"  
}  
  
output "public-ip" {  
  value = aws_eip.lb.public_ip  
}
```

# Understanding the Challenge

If you want to create a resource that has a dependency on an infrastructure created through a module, you won't be able to implicitly call it without output values.

```
resource "aws_instance" "myec2" {  
  ami = "ami-123"  
  instance_type = "t2.micro"  
}
```



```
module "ec2" {  
  source = "../../modules/ec2"  
}  
  
resource "aws_eip" "lb" {  
  instance = module.ec2.instance_id  
  domain   = "vpc"  
}
```



# Accessing Child Module Outputs

Ensure to include output values in the module code for better flexibility and integration with other resources and projects.

**Format:** module.<MODULE NAME>.<OUTPUT NAME>

```
resource "aws_instance" "myec2" {  
    ami = var.ami  
    instance_type = var.instance_type  
}  
  
output "instance_id" {  
    value = aws_instance.myec2.id  
}
```



```
module "ec2" {  
    source = "../../modules/ec2"  
}  
  
resource "aws_eip" "lb" {  
    instance = module.ec2.instance_id  
    domain   = "vpc"  
}
```

# **Root and Child Modules**

# Root Module

Root Module resides in the **main working directory of your Terraform configuration**. This is the entry point for your infrastructure definition

```
module "ec2" {  
  source = "../modules/ec2"  
}
```

Root Module

# Child Module

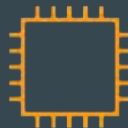
A **module that has been called by another module** is often referred to as a child module.

```
resource "aws_instance" "myec2" {  
  ami = var.ami  
  instance_type = var.instance_type  
}
```

Child Module

```
module "ec2" {  
  source = "../..//modules/ec2"  
}
```

Root Module



# **Standard Module Structure**

# Setting the Base

At this stage, we have been keeping the overall module structure very simple to understand the concepts.

In production environments, it is important to follow recommendations and best-practices set by HashiCorp.

# Basic of Standard Module Structure

The **standard module structure** is a file and directory layout HashiCorp recommends for reusable modules.

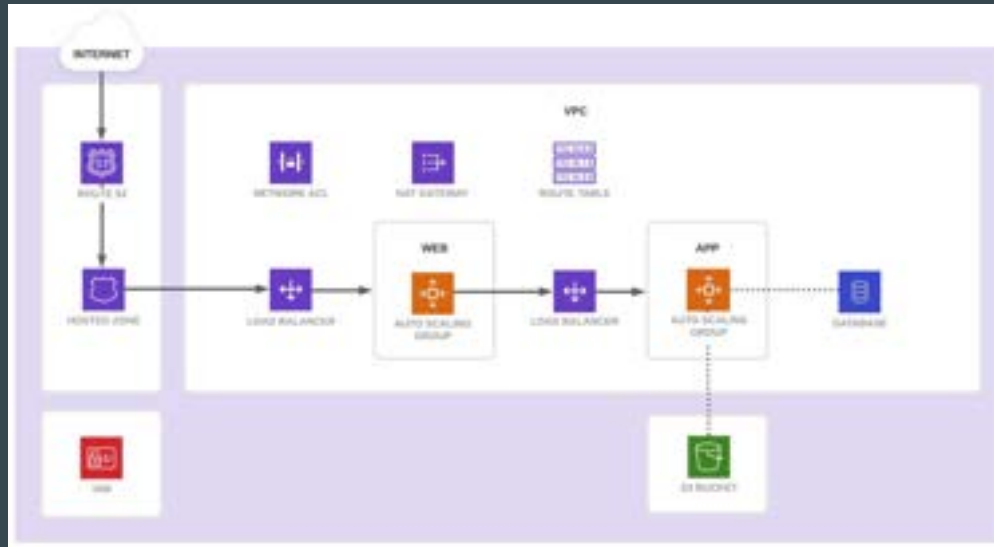
A minimal recommended module following the standard structure is shown below

```
$ tree minimal-module/  
.  
├── README.md  
├── main.tf  
├── variables.tf  
└── outputs.tf
```

# Scope the Requirements for Module Creation

A team wants to provision their infrastructure using Terraform.

The following architecture diagram depicts the desired outcome.





# Planning a Module Structure

In this scenario, a team of Terraform producers, who write Terraform code from scratch, will build a collection of modules to provision the infrastructure and applications.

The members of the team in charge of the application will consume these modules to provision the infrastructure they need.

# Final Module Output

After reviewing the consumer team's requirements, the producer team has broken up the application infrastructure into the following modules:

Network, Web, App, Database, Routing, and Security.



# **Multiple Provider Configuration**

# Understanding the Requirement

There can be a requirement that multiple resource types in the same TF file need to be deployed in separate regions.

```
resource "aws_instance" "myec2" {  
  ami          = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}  
  
resource "aws_security_group" "allow_tls" {  
  name = "staging_firewall"  
}
```

Singapore Region

Mumbai Region

## Setting the Base

At this stage, we have been dealing with single-provider configuration.

In the below code, both resources will be created in Singapore region.

```
provider "aws" {  
    region = "ap-southeast-1"  
}  
  
resource "aws_instance" "myec2" {  
    ami           = "ami-08a0d1e16fc3f61ea"  
    instance_type = "t2.micro"  
}  
  
resource "aws_security_group" "allow_tls" {  
    name = "staging_firewall"  
}
```

# Alias Meta-Argument

Each provider can have one default configuration, and **any number of alternate configurations** that include an extra name segment (or "alias").

```
resource "aws_instance" "myec2" {  
  ami          = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}
```



```
provider "aws" {  
  region = "ap-southeast-1"  
}
```

```
provider "aws" {  
  alias = "mumbai"  
  region = "ap-south-1"  
}
```

```
resource "aws_security_group" "allow_tls" {  
  name = "staging_firewall"  
}
```



```
provider "aws" {  
  alias = "usa"  
  region = "us-east-1"  
}
```

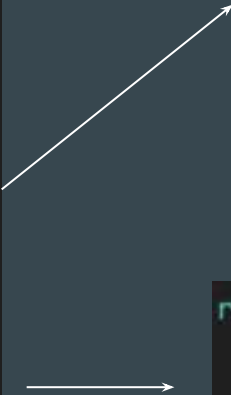
# Final Output Using Alias

By using the provider meta-argument, you can select an alternate provider configuration for a resource.

```
provider "aws" {  
  region = "ap-southeast-1"  
}
```

```
provider "aws" {  
  alias   = "mumbai"  
  region = "ap-south-1"  
}
```

```
provider "aws" {  
  alias = "usa"  
  region = "us-east-1"  
}
```



```
resource "aws_instance" "myec2" {  
  provider = aws.mumbai  
  ami      = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}
```

```
resource "aws_security_group" "allow_tls" {  
  provider = aws.usa  
  name     = "staging_firewall"  
}
```

---

# Publishing Modules

Publish Modules to Terraform Registry

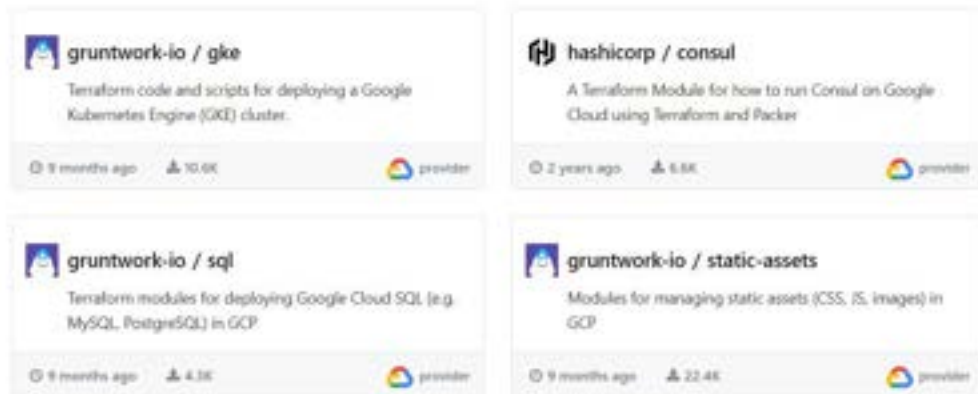
---



# Overview of Publishing Modules

Anyone can publish and share modules on the Terraform Registry.

Published modules support versioning, automatically generate documentation, allow browsing version histories, show examples and READMEs, and more.



# Requirements for Publishing Module

| Requirement               | Description                                                                                                                                                                |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GitHub                    | The module must be on GitHub and must be a public repo. This is only a requirement for the public registry.                                                                |
| Named                     | Module repositories must use this three-part name format<br>terraform-<PROVIDER>-<NAME>                                                                                    |
| Repository description    | The GitHub repository description is used to populate the short description of the module.                                                                                 |
| Standard module structure | The module must adhere to the standard module structure.                                                                                                                   |
| x.y.z tags for releases   | The registry uses tags to identify module versions. Release tag names must be a semantic version, which can optionally be prefixed with a v. For example, v1.0.4 and 0.9.2 |

# Standard Module Structure

The standard module structure is a file and directory layout that is recommended for reusable modules distributed in separate repositories

```
$ tree minimal-module/
```

```
.
├── README.md
├── main.tf
├── variables.tf
└── outputs.tf
```

```
$ tree complete-module/
```

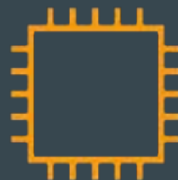
```
.
├── README.md
├── main.tf
├── variables.tf
├── outputs.tf
├── ...
├── modules/
│   ├── nestedA/
│   │   ├── README.md
│   │   ├── variables.tf
│   │   ├── main.tf
│   │   └── outputs.tf
│   ├── nestedB/
│   │   └── .../
│   └── .../
├── examples/
│   ├── exampleA/
│   │   └── main.tf
│   ├── exampleB/
│   │   └── .../
│   └── .../
```

# **Terraform Workspace**

# Setting the Base

An infrastructure created through Terraform is **tied to the** underlying Terraform configuration and a state file.

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```



EC2 Instance



terraform.tfstate

# What If?

What if we have multiple state file for single Terraform configuration?

Can we manage different env's through it separately?

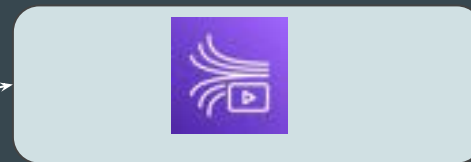
```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```



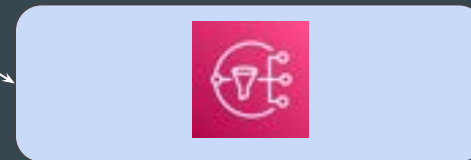
State File 1



State File 2



Environment 1

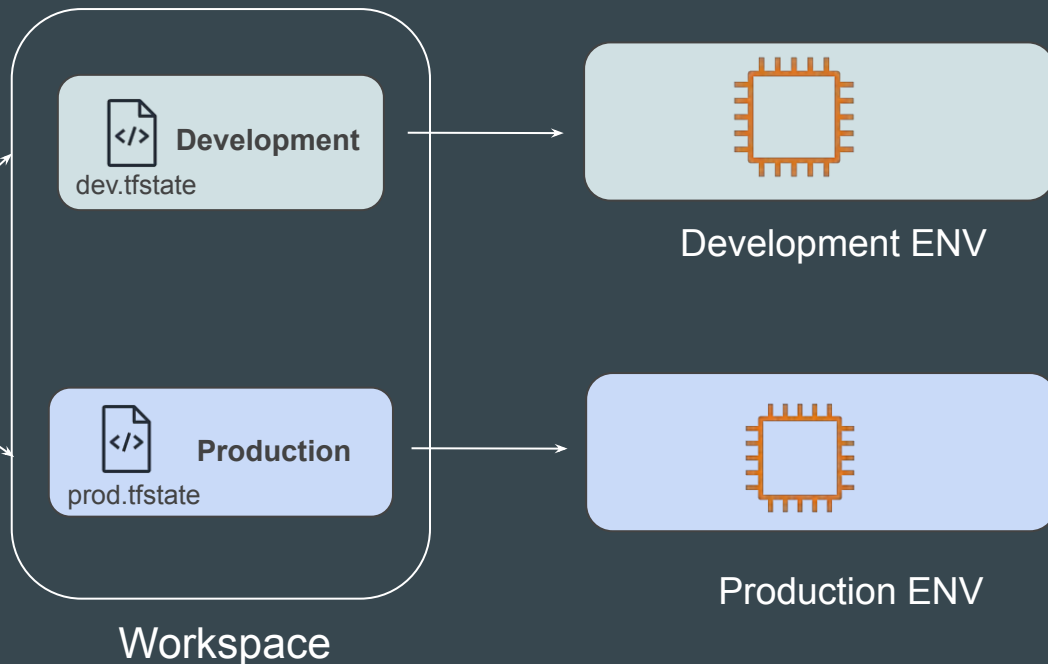


Environment 2

# Introducing Terraform Workspace

Terraform workspaces enable us to **manage multiple set of deployments from the same sets of configuration file.**

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
}
```



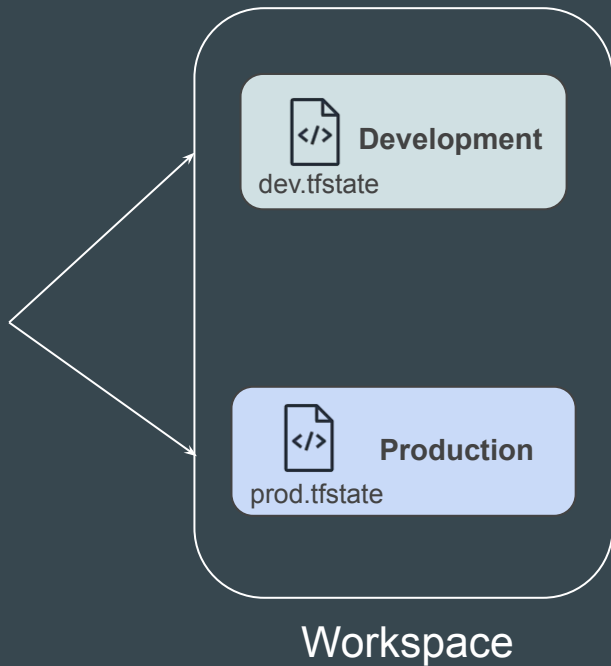
# Flexibility with Workspace

Depending on the workspace being used, the value to a specific argument in your Terraform code can also change.

```
resource "aws_instance" "myec2" {  
  ami          = "ami-08a0d1e16fc3f61ea"  
  instance_type = local.instance_types[terraform.workspace]  
}
```



| Environment | instance_type |
|-------------|---------------|
| Development | t2.micro      |
| Production  | m5.large      |





# **Team Collaboration**

# Local Changes are not always good

Until now, we've been working with Terraform code stored on our local machines.



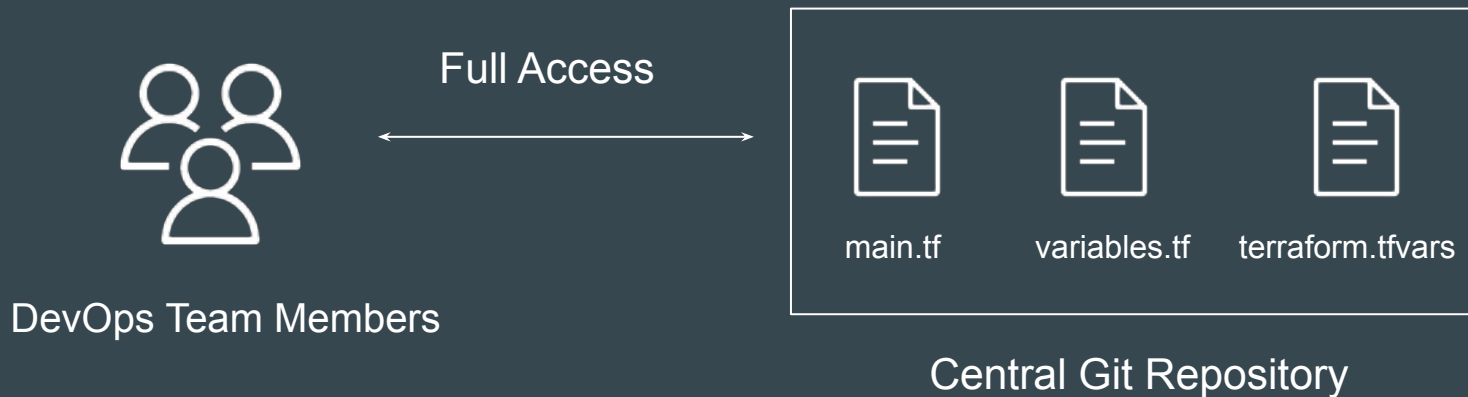
# Disadvantages of a Local-Only Approach

1. If your laptop fails, you risk losing both your Terraform code and, critically, the state file.
2. Team collaboration is difficult—other members can't access or update the infrastructure code.
3. No versioning of changes is visible.

# Integration with Git Repository - Essential

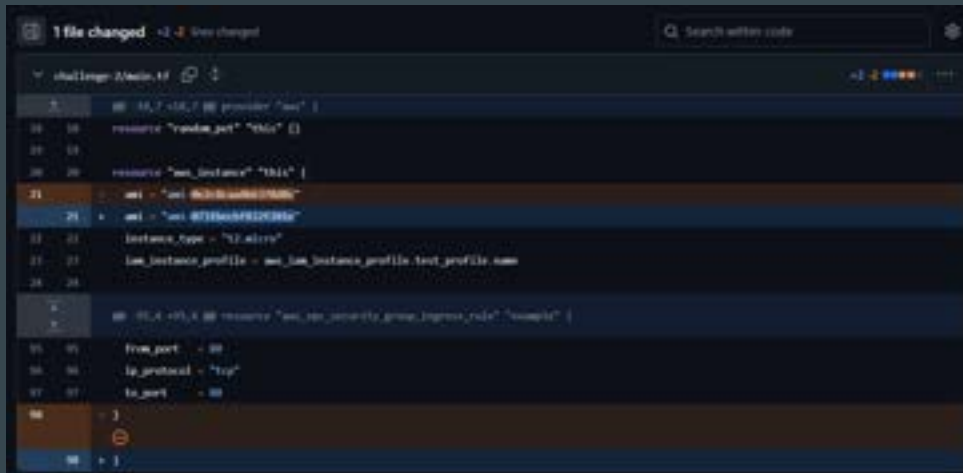
It is **very important** that all of your Terraform code is stored in a Git repository.

All DevOps Team members can have access to Git repository.



# Benefits of Git Repository

1. Centralized access for all team members
2. Full version history and change tracking
3. Code review and approval workflows
4. Integration with CI/CD for validation and plans



The screenshot shows a code editor with a dark theme. At the top, a status bar indicates "1 file changed" and "2 lines changed". A search bar is visible on the right. The main area displays a JSON file named "challenge-2/main.tf". The code is as follows:

```
{
  "provider": "aws"
}

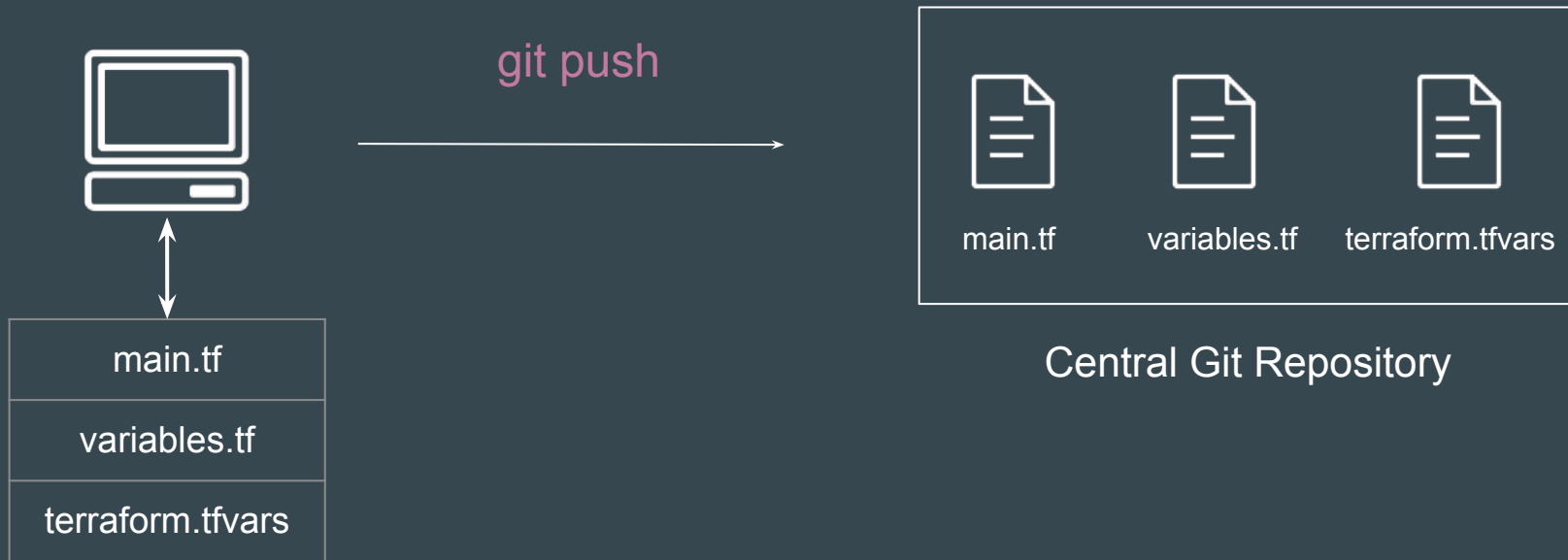
resource "random_pet" "this" {}

resource "aws_instance" "this" {
  ami           = "ami-0c338382e44e2bb9c"
  instance_type = "t2.micro"
  iam_instance_profile = aws_iam_instance_profile.test_profile.name
}

resource "aws_s3_policy_group_ingress_rule" "example" {
  from_port = 80
  to_port   = 80
  ip_protocol = "tcp"
}
```

# Important Step To-Do

Always commit and push your changes to the Git repository so all team members see the latest updates.



## Point to Note - To be Discussed in Next Lectures

While it's good practice to commit your Terraform code files to Git, you shouldn't commit every file.

`terraform.tfstate` file should **NOT** be added to your Git repository.

`.terraform` folder should **NOT** be added to your Git repository.



Central Git Repository

# Relax and Have a Meme Before Proceeding

me: i'll do it at 6

time: 6:05

me: wow looks like i gotta wait til 7 now





# **Security Risks of Storing Terraform State Files in Git**

# Setting the Base

Terraform State File (terraform.tfstate) can include secrets in plain text (e.g., passwords, tokens, etc)

```
() terraform.tfstate X
() terraform.tfstate > [ ]resources > {} 0 > [ ]instances > {} 0 > {} attributes
7   "resources": [
8     {
13      "instances": [
14        {
15          "attributes": {
16            "multi_az": false,
17            "nchar_character_set_name": "",
18            "network_type": "IPV4",
19            "option_group_name": "default:mysql-8-0",
20            "parameter_group_name": "default.mysql8.0",
21            "password": "foobarbaz#321",
22            "password_wo": null,
23            "password_wo_version": null,
24            "performance_insights_enabled": false,
25            "performance_insights_kms_key_id": "",
26            "performance_insights_retention_period": 0,
```

# Why Not Commit to Git

Committing terraform.tfstate file to Git **risks accidental exposure** if the repository is public, shared, or compromised, leading to data breaches.



# **Terraform and .gitignore**

## Revising A Point Discussed in Earlier Video

While it's good practice to commit your Terraform code files to Git, you shouldn't commit every file.

`terraform.tfstate` file should **NOT** be added to your Git repository.

`.terraform` folder should **NOT** be added to your Git repository.



Central Git Repository

# Files You Should Avoid Publishing to Git Repository

| Do Not Commit                                  | Description                                                                                                                          |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| .terraform                                     | Your .terraform directory, where Terraform downloads providers and child modules.                                                    |
| terraform.tfvars                               | Any .tfvars files that contain sensitive information.                                                                                |
| Terraform.tfstate and terraform.tfstate.backup | Avoid committing terraform.tfstate and backup file.                                                                                  |
| crash.log                                      | If terraform crashes, the logs are stored to a file named crash.log                                                                  |
| terraform.tfstate.lock.info                    | Terraform creates and deletes this file automatically when you run a terraform apply command and contains info about your state lock |
| Saved Plan Files                               | Saved plan files that you create when you include the -out flag when you run terraform plan.                                         |

# Overview of gitignore

The **.gitignore** file is a text file that tells Git which files or folders to ignore in a project.

|                   |
|-------------------|
| <b>.gitignore</b> |
| terraform.tfstate |
| .terraform/       |
| crash.log         |



# Always Commit the Following

Always commit:

1. All Terraform code files
2. Your `.terraform.lock.hcl` dependency lock file
3. A `.gitignore` file that excludes the files
4. A `README.md` to describe the code, input variables, and outputs

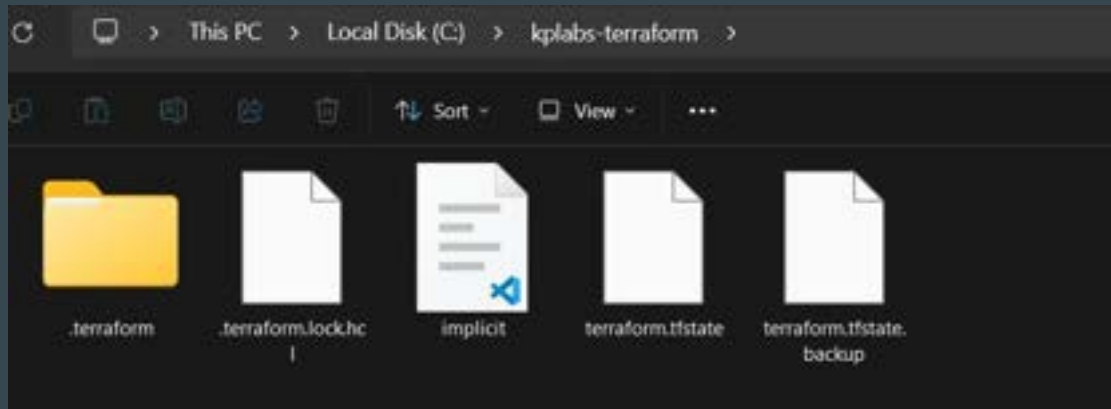


# **Terraform Backends**

# Setting the Base

Backends primarily determine where Terraform stores its state file.

When you do not specify a backend block in your Terraform configuration, Terraform uses the **default local backend** that stores terraform.tfstate in the same project folder.



# Challenges with Local Backend

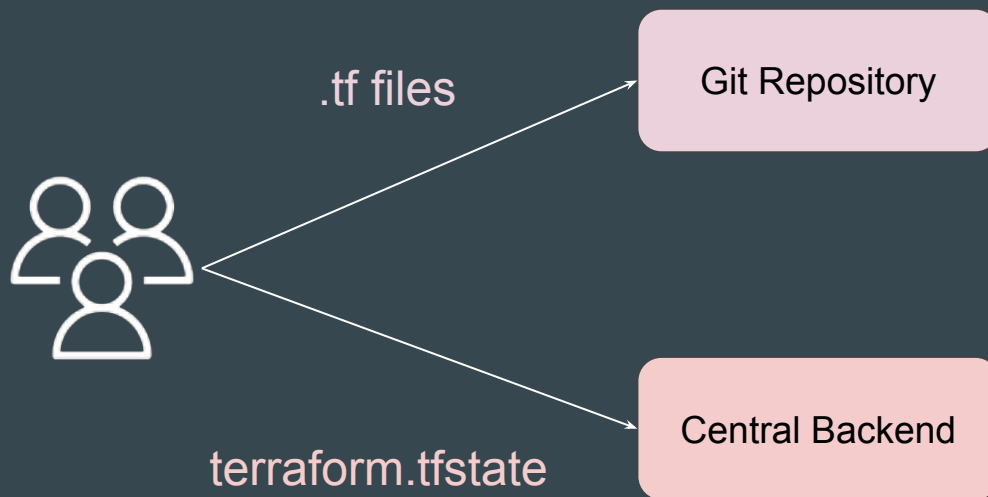
Nowadays, Terraform projects are often handled collaboratively by entire teams.

Storing the state file locally (e.g., on a developer's laptop) hinders effective team collaboration



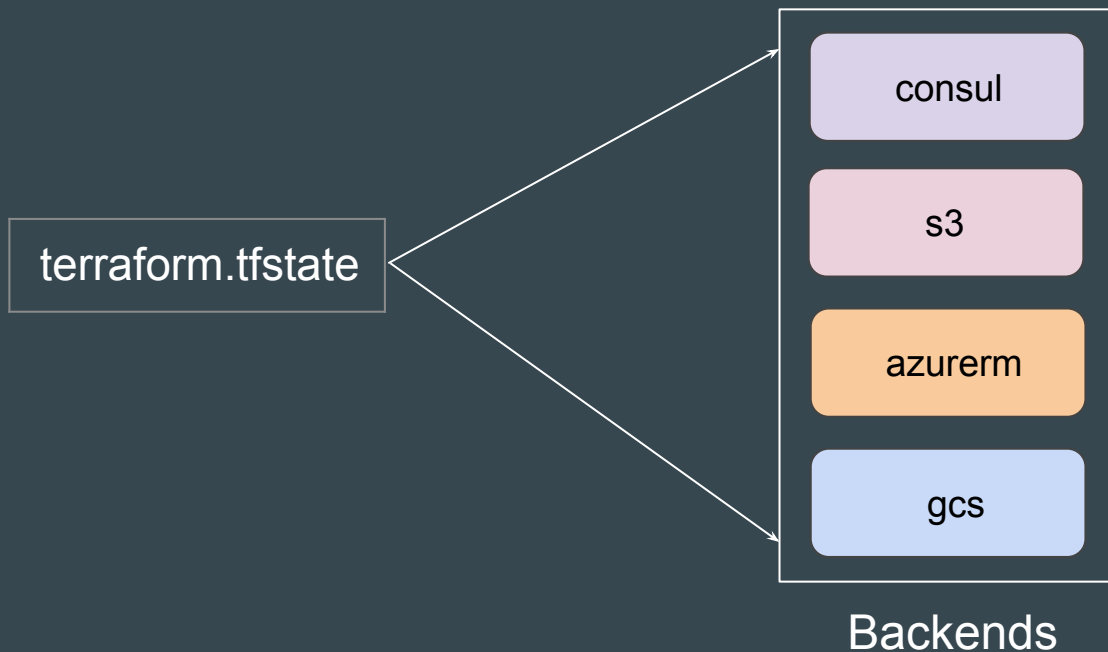
# Recommended Architecture

1. The Terraform Code is stored in a Git Repository.
2. The State file is stored in a Central backend.



# Supported Backends in Terraform

Terraform supports wide variety of remote backends to store state file information.



# Explicit Local Backend Configuration

The **local backend** stores state on the local filesystem, locks that state using system APIs, and performs operations locally.

```
terraform {  
  backend "local" {  
    path = "relative/path/to/terraform.tfstate"  
  }  
}
```

# Important Note - Remote Backends

Accessing state in a remote service generally requires some kind of access credentials

Some backends act like plain "remote disks" for state files; others support state locking functionality.

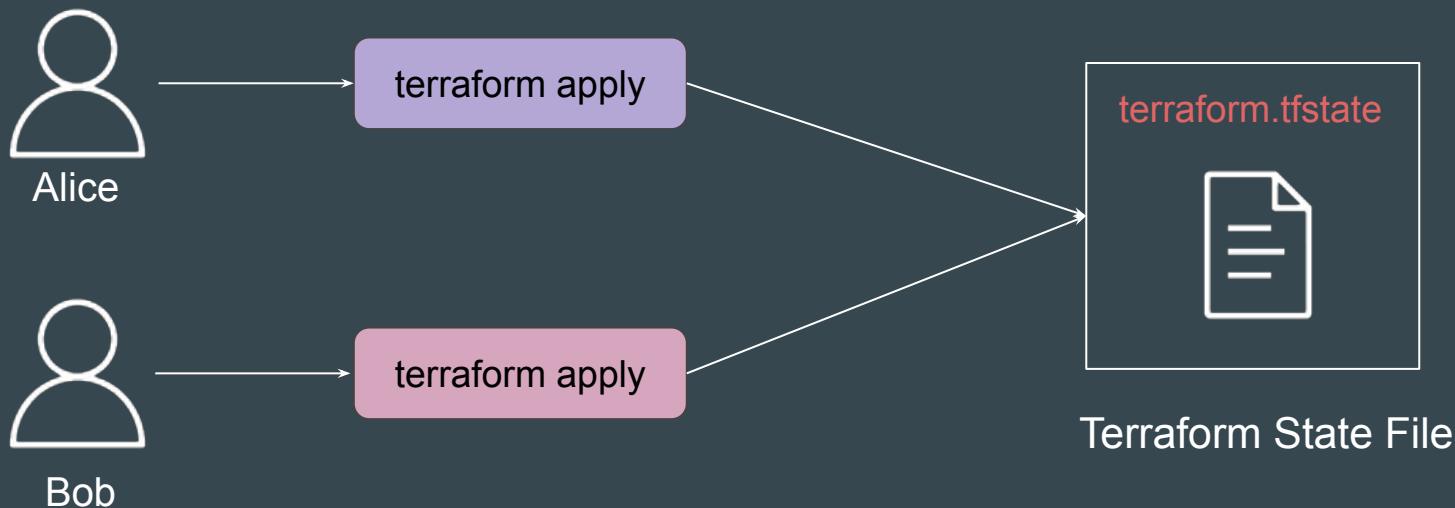


# State Locking



## Setting the Base

Running "terraform apply" simultaneously by multiple team members on same project can cause Terraform to make concurrent changes to the state file. This can result in **state file corruption** and inconsistencies.



# Introducing State Locking

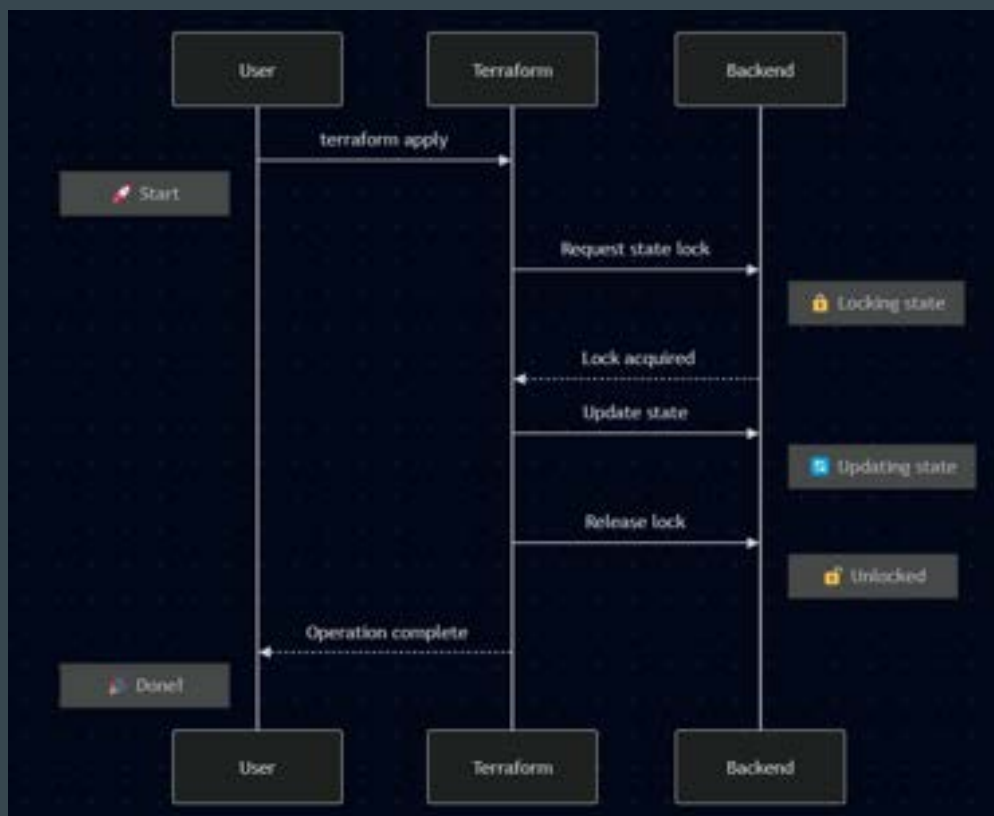
**State locking** is a mechanism that prevents multiple operations from making concurrent changes to your infrastructure state file, which could lead to corruption or inconsistent state.



State File Locked

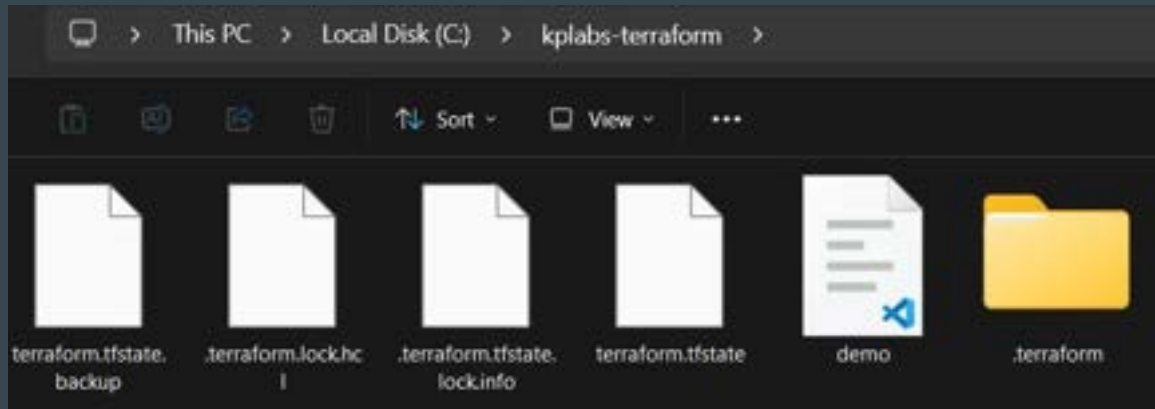
# Process of State Locking

1. Before performing any write operation, Terraform attempts to acquire a "lock" on the state file.
2. If the lock is successfully acquired, Terraform proceeds with the operation.
3. Once the operation is complete , Terraform releases the lock, allowing other processes to acquire it.



# State Locking with Local Backend

If using local state (local backend), Terraform uses a **.lock** file in the working directory for locking.



# What Happens if state file is Locked?

When a state lock is held, any attempt by another process to operate will fail with an error.

```
C:\kplabs-terraform>terraform plan
```

```
Error: Error acquiring the state lock
```

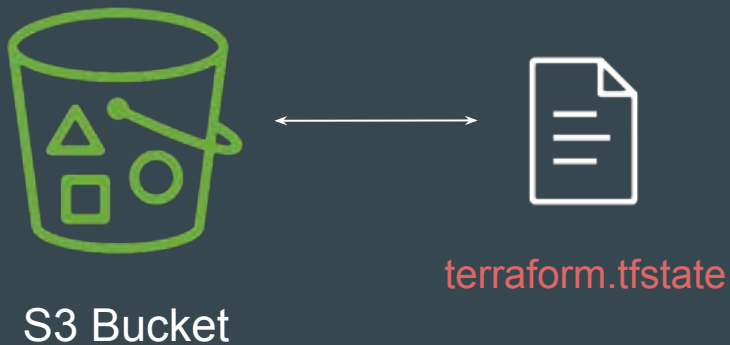
```
Error message: Failed to read state file: The state file could not be read: read terraform.tfstate: The process cannot access the file because another process has locked a portion of the file.
```

```
Terraform acquires a state lock to protect the state from being written by multiple users at the same time. Please resolve the issue above and try again. For most commands, you can disable locking with the "-lock=false" flag, but this is not recommended.
```

# **S3 Backend**

## Setting the Base

The S3 backend allows Terraform to store state file in a specific S3 bucket.





# Example Configuration

**Note:** Terraform should have required access to the S3 Bucket to be able to push and pull state file to/from S3 bucket.

```
s3-backend.tf > ...
terraform {
  backend "s3" {
    bucket = "kplabs-tf-bucket"
    key    = "terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true
  }
}
```

# Additional Recommendation - Bucket Versioning

It is **highly recommended that you enable Bucket Versioning** on the S3 bucket to allow for state recovery in the case of accidental deletions and human error.

# **Terraform State Management**

## Setting the Base

As your Terraform usage becomes more advanced, there are some cases where you may need to modify the Terraform state.

It is **NOT** recommended to modify the state file manually.



# State Management

The **terraform state** command is used for advanced state management

| Sub-Commands     | Description                                                      |
|------------------|------------------------------------------------------------------|
| list             | List resources within terraform state file.                      |
| mv               | Moves item with terraform state.                                 |
| pull             | Manually download and output the state from remote state.        |
| push             | Manually upload a local state file to remote state.              |
| rm               | Remove items from the Terraform state                            |
| show             | Show the attributes of a single resource in the state.           |
| replace-provider | Used to replace the provider for resources in a Terraform state. |

## Sub-Command 1 - List

The `terraform state list` command is used to list resources within a Terraform state.

Useful if you want to quickly view all resources managed by Terraform.

```
C:\kplabs-terraform>terraform state list  
aws_iam_user.dev  
aws_security_group.dev
```

## Sub-Command 2 - Show

The `terraform state show` command is used to show the attributes of a single resource in the state.

Useful for debugging and understanding the current attributes of a resource.

```
C:\kplabs-terraform>terraform state show aws_iam_user.dev
# aws_iam_user.dev:
resource "aws_iam_user" "dev" {
  arn                  = "arn:aws:iam::042025557788:user/kplabs-user-01"
  force_destroy        = false
  id                   = "kplabs-user-01"
  name                 = "kplabs-user-01"
  path                 = "/"
  permissions_boundary = null
  tags                 = {}
  tags_all             = {}
  unique_id            = "AIDAQTSKLD40ALWEWB2AW"
}
```

## Sub-Command 3 - pull

The `terraform state pull` command is used to pull the state from a remote backend and output it to stdout.

Useful to view or backup the current state stored in a remote backend.

```
C:\kplabs-terraform>terraform state pull
{
  "version": 4,
  "terraform_version": "1.9.1",
  "serial": 6,
  "lineage": "78f3ac79-1cb9-e724-964b-37bb3dc649a8",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_iam_user",
      "name": "dev",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:iam::042025557788:user/kplabs-user-01",
            "force_destroy": false,
            "id": "kplabs-user-01",
            "name": "kplabs-user-01",
            "path": "/",
            "permissions_boundary": "",

```



## Sub-Command 4 - rm

The `terraform state rm` command is used to remove items from the state.

Use this when you need to remove a resource from Terraform's state management without destroying it.

```
C:\kplabs-terraform>terraform state rm aws_security_group.dev  
Removed aws_security_group.dev  
Successfully removed 1 resource instance(s).
```

## Sub-Command 5 - mv

The `terraform state mv` command is used to move an item in the state to a different address.

```
C:\kplabs-terraform>terraform state mv aws_security_group.dev aws_security_group.prod
Move "aws_security_group.dev" to "aws_security_group.prod"
Successfully moved 1 object(s).
```

## Sub-Command 6 - replace-provider

The `terraform state replace-provider` command is used to replace the provider for resources in a Terraform state.

```
C:\kplabs-terraform>terraform state replace-provider hashicorp/local kplabs.in/internal/local
Terraform will perform the following actions:

  ~ Updating provider:
    registry.terraform.io/hashicorp/local
  + kplabs.in/internal/local

Changing 1 resources:

  local_file.foo

Do you want to make these changes?
Only 'yes' will be accepted to continue.

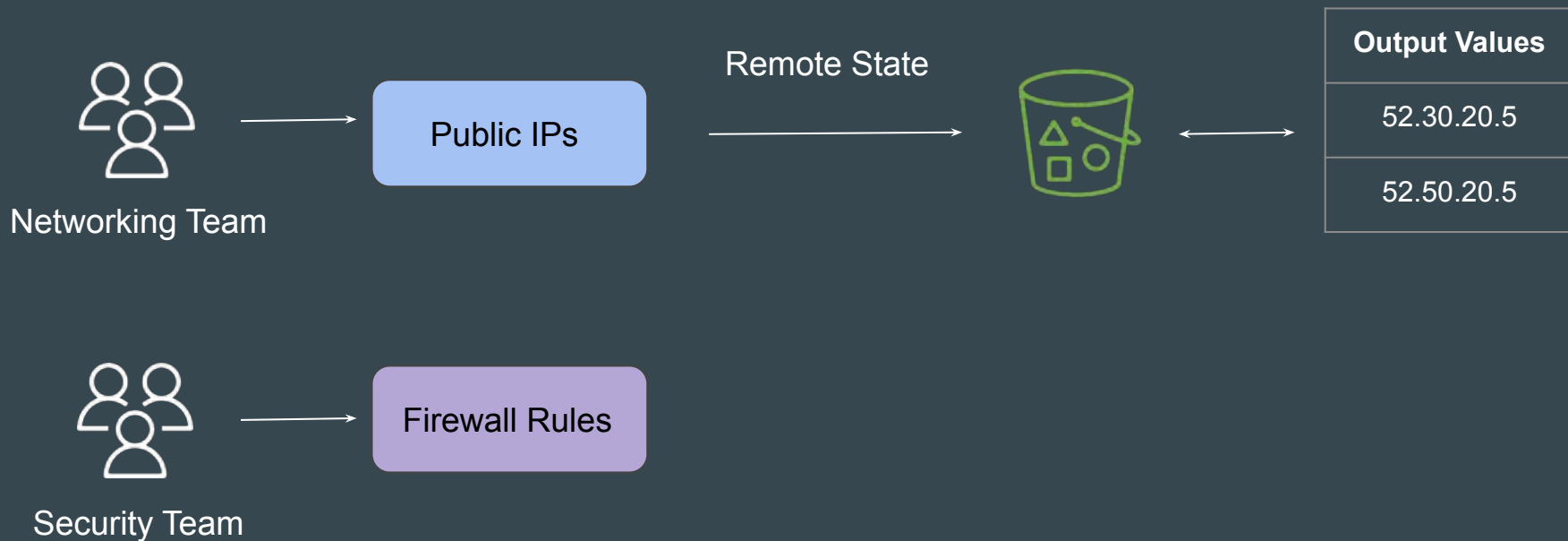
Enter a value: yes

Successfully replaced provider for 1 resources.
```

# **Remote State Data Source**

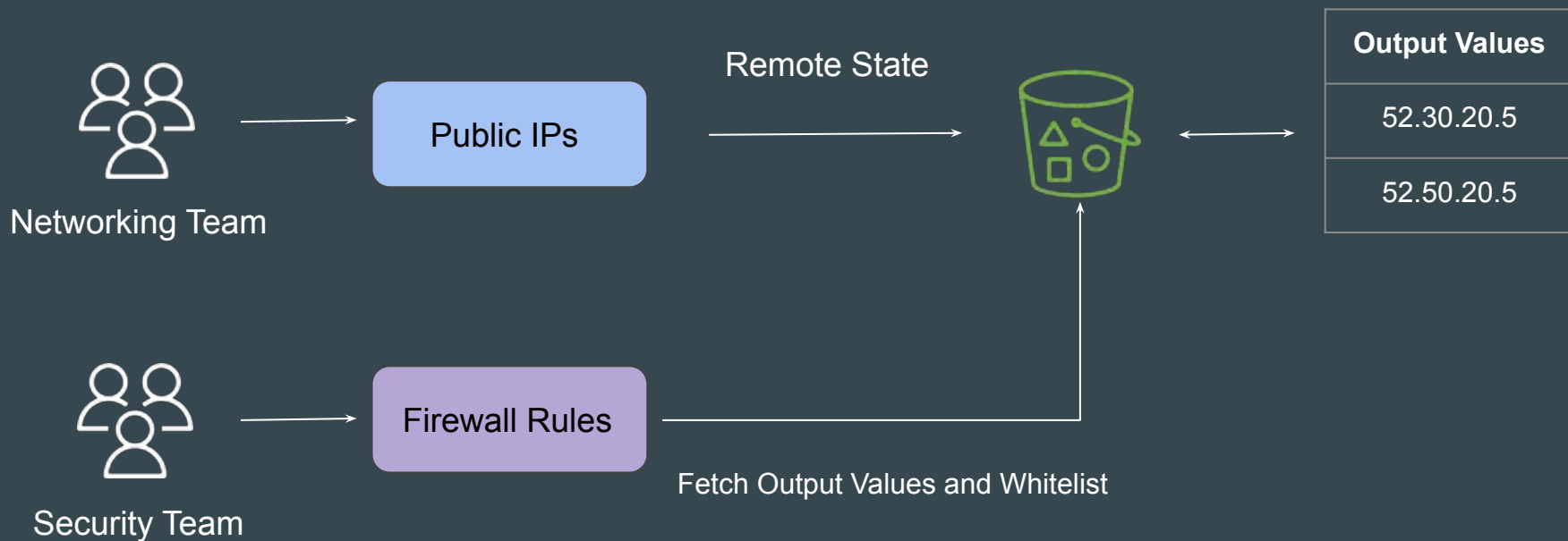
# Setting up the Base

In larger enterprises, there can be multiple different teams working on different aspects of a infrastructure resource



# Understanding the Challenge

Security Team wants that all the IP addresses added as part of Output Values in tfstate file of Networking Team project should be whitelisted in Firewall.



## What Needs to be Achieved

1. The code from Security Team project should connect to the terraform.tfstate file managed by the Networking team.
2. The code should fetch all the IP addresses mentioned in the output values in the state file.
3. The code should whitelist these IP addresses in Firewall rules.

## Practical Workflow Steps

1. Create two folders for networking-team and security-team
2. Create Elastic IP resource in Networking Team and Store the State file in S3 bucket. Output values should have information of EIP.
3. In Security Team, use Terraform Remote State data source to connect to the tfstate file of Networking Team.
4. Use the Remote State to fetch EIP and whitelist it in Security Group rule.



# Introducing Remote State Data Source

The `terraform_remote_state` data source allows us to fetch output values from a specific state backend

```
data "terraform_remote_state" "eip" {  
  backend = "s3"  
  config = {  
    bucket = "kplabs-team-networking-bucket"  
    key    = "eip.tfstate"  
    region = "us-east-1"  
  }  
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "${data.terraform_remote_state.eip.outputs.eip_addr}/32"  
  from_port         = 443  
  ip_protocol       = "tcp"  
  to_port           = 443  
}
```

Step 1 - Define Remote State Source

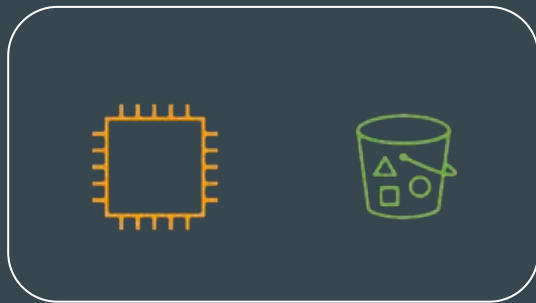
Step 2 - Define Data to Fetch

# Terraform Import

# Typical Challenge

It can happen that all the resources in an organization are created manually.

Organization now wants to start using Terraform and manage these resources via Terraform.

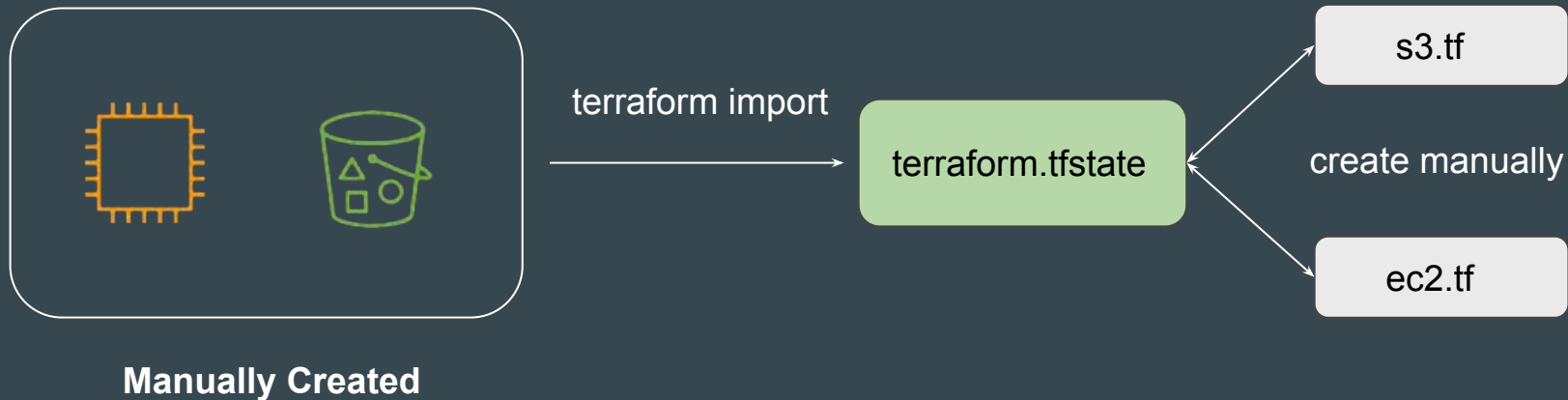


**Manually Created**

## Earlier Approach

In the older approach, Terraform import would create the state file associated with the resources running in your environment.

Users still had to write the tf files from scratch.



# Newer Approach

In the newer approach, **terraform import** can automatically create the terraform configuration files for the resources you want to import.



## Point to Note

**Terraform 1.5** introduces automatic code generation for imported resources.

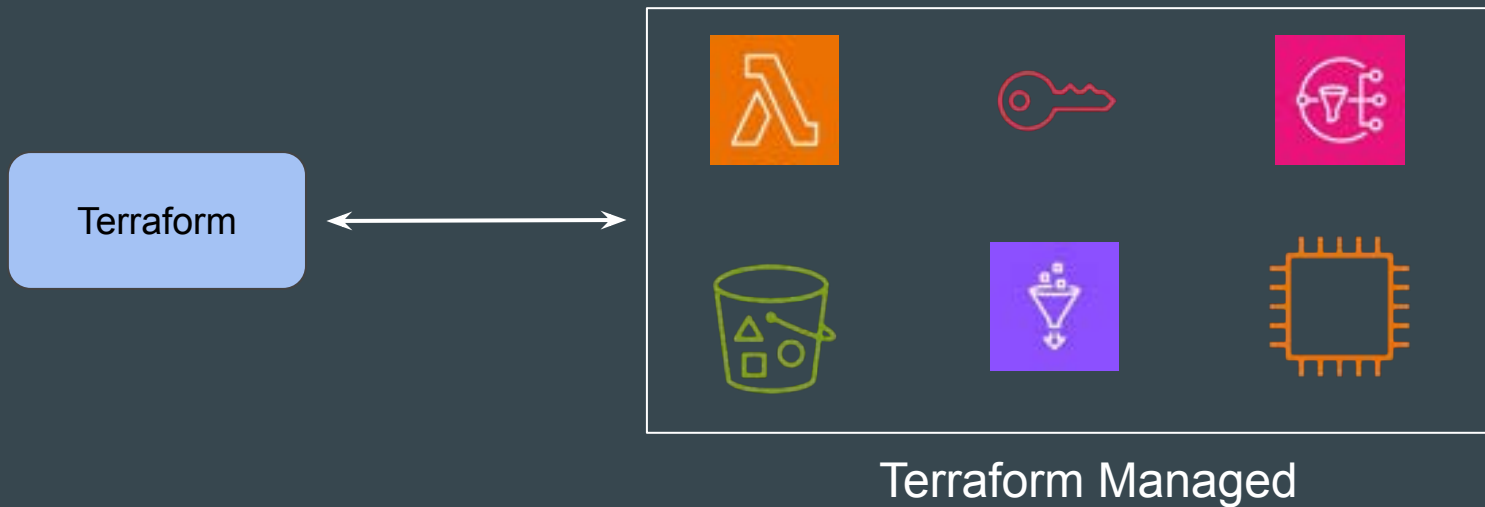
This dramatically reduces the amount of time you need to spend writing code to match the imported

This feature is not available in the older version of Terraform.

**Removed Block**

## Setting the Base

Imagine you have an EC2 instance defined in your Terraform code. You decide you still want the EC2 instance to exist in AWS, but you no longer want Terraform to manage it (perhaps another team is taking over)





# The Old Workflow

|        |                                                                                          |
|--------|------------------------------------------------------------------------------------------|
| Step 1 | You would delete the resource "aws_instance" "example" { ... } block from your .tf file. |
| Step 2 | You had to run a CLI command: <code>terraform state rm aws_instance.example</code>       |

# Disadvantage of Old Workflow

| Problems                    | Description                                                                     |
|-----------------------------|---------------------------------------------------------------------------------|
| Imperative, not Declarative | This is a manual command run by a human. It is not recorded in your code (Git). |
| CI/CD Issues                | Hard to automate safely as part of CI/CD pipeline                               |

# Introducing Removed Block

The **removed block** allows you to declare your intention to remove a resource from the Terraform state.

It turns the imperative terraform state rm command into a **declarative configuration block**.

```
/*
resource "aws_instance" "myec2" {
  ami = "ami-00c39f71452c08778"
  instance_type = "t2.micro"
}
*/

removed {
  from = aws_instance.myec2

  lifecycle {
    destroy = false
  }
}
```

# The Workflow

1. When you run `terraform plan`, Terraform notices the removed block with `destroy=false`
2. Terraform will plan a 'forget' action, and on apply it will remove the object from state.
3. The actual EC2 instance in AWS remains untouched.



# **Multiple Provider Configuration**

# Understanding the Requirement

There can be multiple resource types in same project and you want to deploy them in different set of AWS regions.

```
resource "aws_instance" "myec2" {  
  ami          = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}  
  
resource "aws_security_group" "allow_tls" {  
  name = "staging_firewall"  
}
```

Singapore Region

Mumbai Region

# Setting the Base

At this stage, we have been dealing with single provider configuration.

In below code, both resources will be created in Singapore region.

```
provider "aws" {  
    region = "ap-southeast-1"  
}  
  
resource "aws_instance" "myec2" {  
    ami           = "ami-08a0d1e16fc3f61ea"  
    instance_type = "t2.micro"  
}  
  
resource "aws_security_group" "allow_tls" {  
    name = "staging_firewall"  
}
```



# Alias Meta-Argument

Each provider can have one default configuration, and **any number of alternate configurations** that include an extra name segment (or "alias").

```
resource "aws_instance" "myec2" {  
  ami          = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}
```



```
provider "aws" {  
  region = "ap-southeast-1"  
}
```

```
provider "aws" {  
  alias  = "mumbai"  
  region = "ap-south-1"  
}
```

```
resource "aws_security_group" "allow_tls" {  
  name = "staging_firewall"  
}
```



```
provider "aws" {  
  alias = "usa"  
  region = "us-east-1"  
}
```

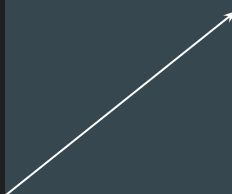
# Final Output Using Alias

By using the provider meta-argument, you can select an alternate provider configuration for a resource.

```
provider "aws" {  
  region = "ap-southeast-1"  
}
```

```
provider "aws" {  
  alias   = "mumbai"  
  region = "ap-south-1"  
}
```

```
provider "aws" {  
  alias = "usa"  
  region = "us-east-1"  
}
```



```
resource "aws_instance" "myec2" {  
  provider = aws.mumbai  
  ami      = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}
```



```
resource "aws_security_group" "allow_tls" {  
  provider = aws.usa  
  name     = "staging_firewall"  
}
```

# **Sensitive Parameter**

# Setting the Base

By default, Terraform will show the values associated with defined attributes in the CLI output during plan, apply operations for most of the resources.

```
C:\kplabs-terraform> terraform plan

Terraform used the selected providers to generate the following execution plan.
following symbols:
+ create

Terraform will perform the following actions:

# local_file.foo will be created
+ resource "local_file" "foo" {
+   content           = "supersecretpassword!"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1      = (known after apply)
+   content_sha256    = (known after apply)
+   content_sha512    = (known after apply)
+   directory_permission = "0777"
+   file_permission   = "0777"
+   filename          = "password.txt"
+   id                = (known after apply)
}
```



# What to Expect

We should design our Terraform code in such way that no sensitive information is available and shown out of the box in CLI Output, Logs, etc.



# Basics of Sensitive Parameter

Adding sensitive parameter ensures that you do not accidentally expose this data in CLI output, log output

```
variable "sensitive_content" {  
  sensitive = true  
  default = "supersecretpassword"  
}  
  
resource "local_file" "foo" {  
  content = var.sensitive_content  
  filename = "password.txt"  
}
```



```
Terraform will perform the following actions:  
  
# local_file.foo will be created  
+ resource "local_file" "foo" {  
  + content           = (sensitive value)  
  + content_base64sha256 = (known after apply)  
  + content_base64sha512 = (known after apply)  
  + content_md5         = (known after apply)  
  + content_sha1        = (known after apply)  
  + content_sha256       = (known after apply)  
  + content_sha512       = (known after apply)  
  + directory_permission = "0777"  
  + file_permission      = "0777"  
  + filename            = "password.txt"  
  + id                  = (known after apply)  
}
```

# Sensitive Values AND Output Values

If you try to reference sensitive value in output values, Terraform will immediately give you an error.

```
variable "sensitive_content" {  
  sensitive = true  
  default = "supersecretpassword"  
}  
  
resource "local_file" "foo" {  
  content = var.sensitive_content  
  filename = "password.txt"  
}  
  
output "content" {  
  value = local_file.foo.content  
}
```



**Error:** Output refers to sensitive values

on local\_file.tf line 12:  
12: output "content" {

To reduce the risk of accidentally exporting sensitive data that  
any root module output containing sensitive data be explicitly marked

If you do intend to export this data, annotate the output value with  
sensitive = true

# Sensitive Values AND Output Values

If you still want sensitive content to be available in “output” of state file but should not be visible in CLI Output, Logs, following approach can be used.

```
variable "sensitive_content" {  
  sensitive = true  
  default = "supersecretpassword"  
}  
  
resource "local_file" "foo" {  
  content = var.sensitive_content  
  filename = "password.txt"  
}  
  
output "content" {  
  value = local_file.foo.content  
  sensitive = "true"  
}
```



Changes to Outputs:

+ content = (sensitive value)

You can apply this plan to save these new output values to the Terraform state

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

content = <sensitive>



# Important Point to Note

Sensitive parameter will NOT protect / redact information from State file.

```
variable "sensitive_content" {  
  sensitive = true  
  default = "supersecretpasswd"  
}  
  
resource "local_file" "foo" {  
  content = var.sensitive_content  
  filename = "password.txt"  
}
```

Configuration File

```
"resources": [  
  {  
    "mode": "managed",  
    "type": "local_file",  
    "name": "foo",  
    "provider": "provider[\\\"registry.terraform.io/hashicorp/local\\\"]",  
    "instances": [  
      {  
        "schema_version": 0,  
        "attributes": {  
          "content": "supersecretpasswd",  
          "content_base64": null,  
          "content_base64sha256": "7Bj9buJUR+8nQAGN/nDad3"
```

State File

# Benefits of Mature Providers

Various providers like AWS will automatically consider the password argument for any database instance as sensitive and will redact it as a sensitive value

```
resource "aws_db_instance" "default" {  
  allocated_storage = 10  
  db_name           = "mydb"  
  engine            = "mysql"  
  engine_version    = "8.0"  
  instance_class     = "db.t3.micro"  
  username          = "foo"  
  password           = "foobazbar"  
  parameter_group_name = "default.mysql8.0"  
  skip_final_snapshot = true  
}
```



```
C:\kplabs-terraform>terraform apply -auto-approve
```

```
Terraform will perform the following actions:
```

```
# aws_db_instance.default will be created  
+ resource "aws_db_instance" "default" (  
  + address                               = (known after apply)  
  + allocated_storage                     = 10  
  + password                             = (sensitive value)  
  + performance_insights_enabled         = false  
  + performance_insights_kms_key_id      = (known after apply)  
  + performance_insights_retention_period = (known after apply)  
  + port                                 = (known after apply)
```

---

# Overview of Vault

HashiCorp Certified: Vault Associate

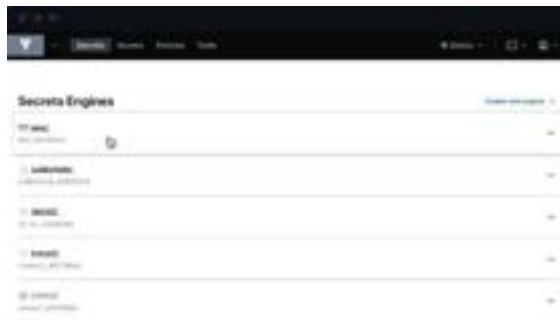
---

# Let's get started

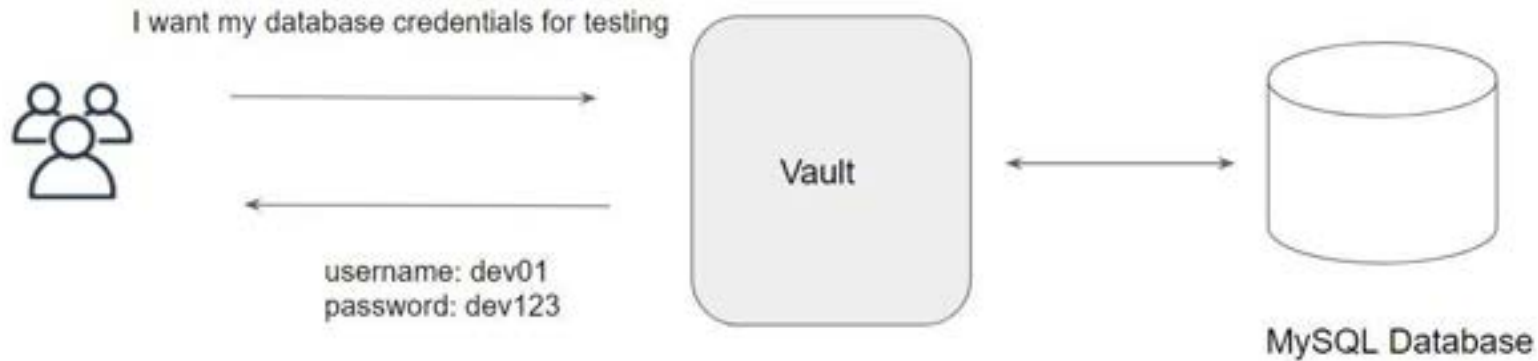
HashiCorp Vault allows organizations to securely store secrets like tokens, passwords, certificates along with access management for protecting secrets.

One of the common challenges nowadays in an organization is “Secrets Management”

Secrets can include, database passwords, AWS access/secret keys, API Tokens, encryption keys and others.



# Dynamic Secrets



# Life Becomes Easier

Once Vault is integrated with multiple backends, your life will become much easier and you can focus more on the right work.

Major aspect related to Access Management can be taken over by vault.



---

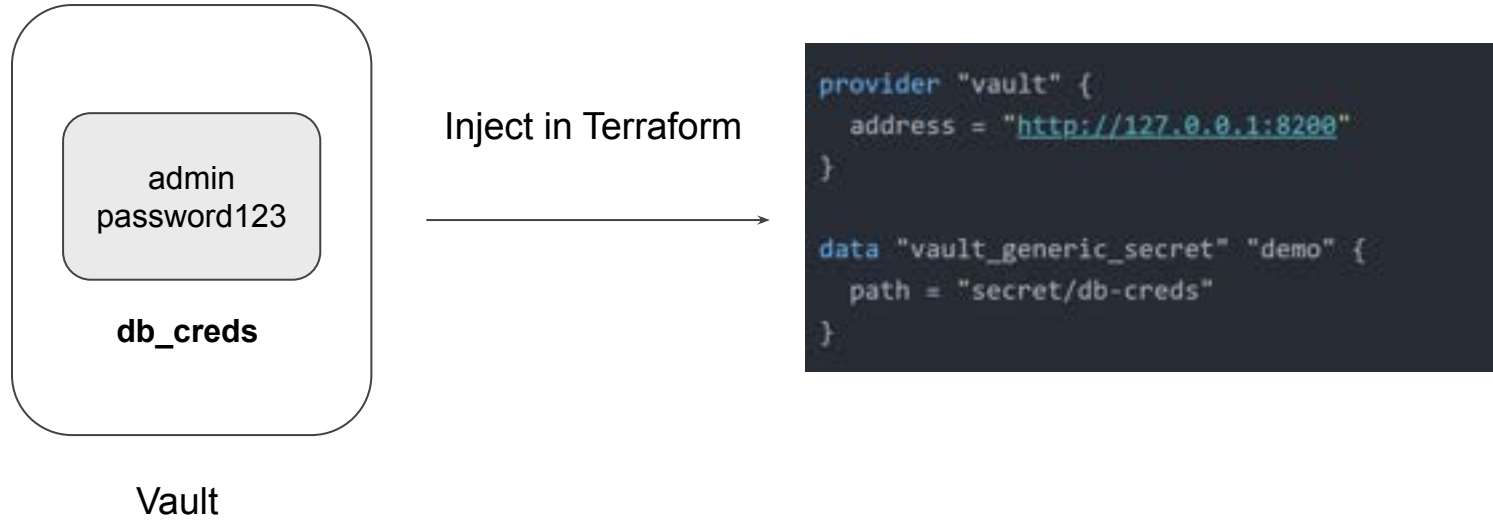
# Vault Provider

[Back to Providers](#)

---

# Vault Provider

The Vault provider allows Terraform to read from, write to, and configure HashiCorp Vault.





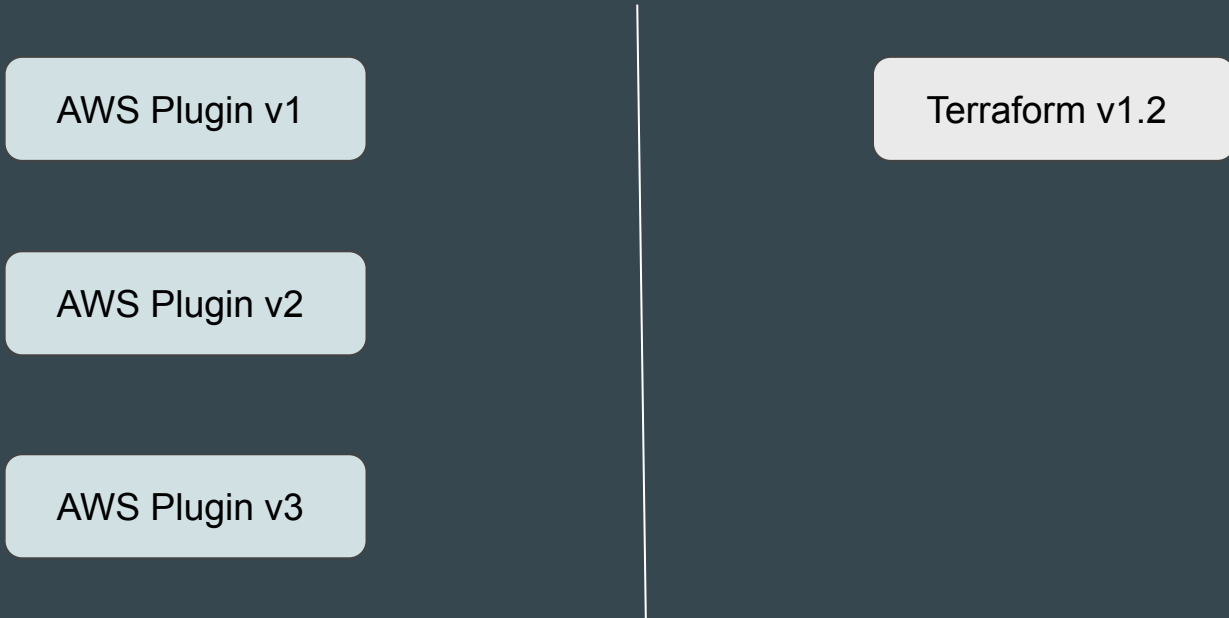
# Important Note

Interacting with Vault from Terraform causes any secrets that you read and write to be persisted in both Terraform's state file.

# **Dependency Lock File**

# Revising the Basics

Provider Plugins and Terraform are managed independently and have different release cycle.



# Understanding the Challenge

The AWS code written in Terraform is working perfectly well with AWS Plugin v1

It can happen that same code might have some issues with newer AWS plugins.

AWS Plugin v1

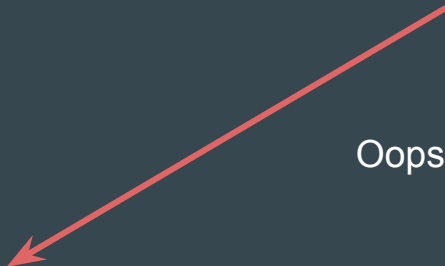


Terraform v1.2

```
resource "aws_instance" "web" {  
  ami           = ami-123  
  instance_type = "t2.micro"  
}
```

AWS Plugin v2

AWS Plugin v3



Oops, Many Issues!

# Version Dependencies

Version constraints within the configuration itself determine which versions of dependencies are potentially compatible.

After selecting a specific version of each dependency Terraform remembers the decisions it made in a dependency lock file so that it can (by default) make the same decisions again in future.

```
5 terraform.lock.hcl
1  # This file is maintained automatically by "terraform init".
2  # Manual edits may be lost in future updates.
3
4  provider "registry.terraform.io/hashicorp/aws" {
5    version      = "4.62.0"
6    constraints = "~> 4.0"
7    hashes = [
8      "h1:TBuxeLRMrChrzgDhAyil3KITJhuNrL+X58QRC+FTmFQ=",
9      "zh:12059dc2b639797b9facb6397ac6aec563891634be8e5aadf3a4
10     "zh:1b3515d70b6998359d0a6d3b3c287940ab2e5c59cd02f95c7d9d
```



```
demo.tf
demo.tf > ...
1  terraform {
2    required_providers {
3      aws = {
4        source = "hashicorp/aws"
5        version = "~> 4.0"
6      }
7    }
8  }
9
10 # Configure the AWS Provider
11 provider "aws" {
12   region = "us-east-1"
13 }
```

# Default Behaviour

What happens if you update the TF file with version that does not match the terraform.lock.hcl?

```
5 terraform.lock.hcl
1  # This file is maintained automatically by "terraform init".
2  # Manual edits may be lost in future updates.
3
4  provider "registry.terraform.io/hashicorp/aws" {
5    version      = "4.62.0"
6    constraints = "~> 4.0"
7    hashes = [
8      "h1:TBuxeLRMrChrzgDhAy1l3HITJhuNrL+X58QRC+FTmFQ=",
9      "zh:12059dc2b639797b9facb6397ac6aec563891634be8e5aadf3a4",
10     "zh:1b3515d70b6998359d0a6d3b3c287940ab2e5c59cd02f95c7d9d"
```

```
demo1 > ...
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "4.60"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}
```

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/aws: 1
configured version constraint 4.60.0; must use terraform init -upgrade to allow
```

# Upgrading Option

If there is a requirement to use newer or downgrade a provider, can override that behavior by adding the **-upgrade** option when you run **terraform init**, in which case Terraform will disregard the existing selections

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init -upgrade
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

- Finding hashicorp/aws versions matching "4.60.0"...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

```
Terraform has made some changes to the provider dependency selections recorded in the .terraform.lock.hcl file. Review those changes and commit them to your version control system if they represent changes you intended to make.
```

## Points to Note

When installing a particular provider for the first time, Terraform will pre-populate the hashes value with any checksums that are covered by the provider developer's cryptographic signature, which usually covers all of the available packages for that provider version across all supported platforms.

```
└─ terraform.lock.hcl
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version      = "4.60.0"
  constraints = "4.60.0"
  hashes = [
    "h1:M9Oxusbiz/HW7zF+jLddXqdpzsfZ38Fa2S+Yaquad2g=",
    "zh:1853d6bc89e289ac36c13485e8ff877c1be8485e22f545bb32c7a30f1d1856e8",
    "zh:4321d145969e3b7ede62fe51bee248a15fe398643f21df9541eef85526bf3641",
    "zh:4c01189cc6963abfe724e6b289a7c06d2de9c395011d8d54efa8fe1aac444e2e",
    "zh:5934db7baa2eec0f9acb9c7f1c3dd3b3fe1e67e23dd4a49e9fe327832967b32b",
  ]
}
```



## Points to Note

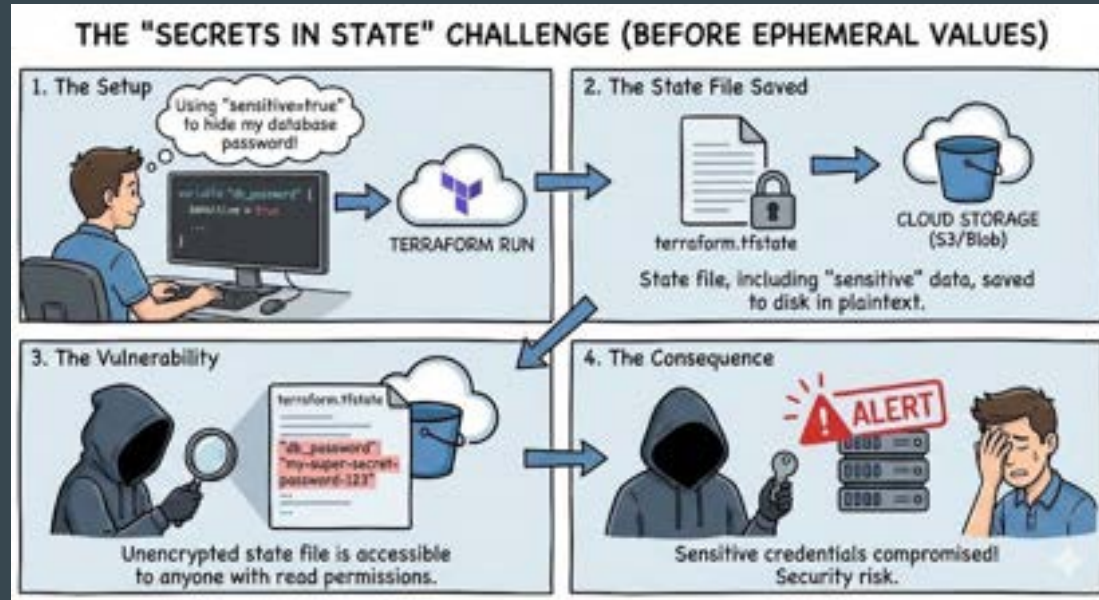
At present, the dependency lock file tracks only provider dependencies.

Terraform does not remember version selections for remote modules, and so Terraform will always select the newest available module version that meets the specified version constraints.

## **Ephemeral Values and Write-Only Arguments**

# Understanding the Challenge

By adding `sensitive = true` for sensitive data like passwords only redacted the value from the CLI output. The actual value was still written in plaintext (JSON) inside the `terraform.tfstate` file



# Ephemeral Blocks

The ephemeral block defines resources that are essentially temporary.

Ephemeral resources have a unique lifecycle, and Terraform does not store information about ephemeral resources in state or plan files

```
1  ephemeral "random_password" "db_password" {  
2    length      = 16  
3  }
```

## Write-only arguments

Write-only arguments let you securely pass temporary values to Terraform's managed resources during an operation without persisting those values to state or plan files.

```
ephemeral "random_password" "db_password" {  
  length      = 16  
}  
  
resource "aws_db_instance" "default" {  
  allocated_storage    = 10  
  db_name              = "mydb"  
  engine              = "mysql"  
  engine_version      = "8.0"  
  instance_class      = "db.t3.micro"  
  username            = "foo"  
  password_wo         = ephemeral.random_password.db_password.result  
  skip_final_snapshot = true  
  password_wo_version = 1  
}
```



## Points to Note - Part 1

To use write-only arguments, **you must use Terraform v.1.11 or later** and use a **resource that supports write-only arguments**.

write-only arguments accept both ephemeral and non-ephemeral values.

## Points to Note - Part 2

Terraform **does not store write-only arguments in state files**, so Terraform has no way of knowing if a write-only argument value has changed.

Terraform also **cannot create plan diffs for write-only arguments** because it does not store those values in plan files.

**Providers typically include version arguments alongside write-only arguments.** Terraform stores version arguments in state, and can track if a version argument changes.

# Trigger an Update of Write-only Argument

To trigger an update of a write-only argument, increment the version argument's value in your configuration:

When you **increment the password\_wo\_version argument**, Terraform notices that change in its plan and notifies the aws provider. The aws provider then uses the new password\_wo value to update the aws\_db\_instance resource.



```
resource "aws_db_instance" "default" {
  allocated_storage    = 10
  db_name              = "mydb"
  engine              = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  username            = "foo"
  password_wo         = ephemeral.random_password.db_password.result
  skip_final_snapshot = true
  password_wo_version  = 2
}
```

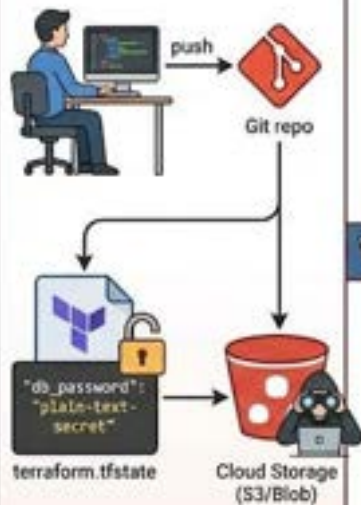


## Points to Note

The implementation of write-only arguments and their version arguments is provider-specific, so consult the Registry for more details about your specific provider.

## HOW EPHEMERAL VALUES SECURE YOUR INFRASTRUCTURE (ORGANIZATIONAL BENEFITS)

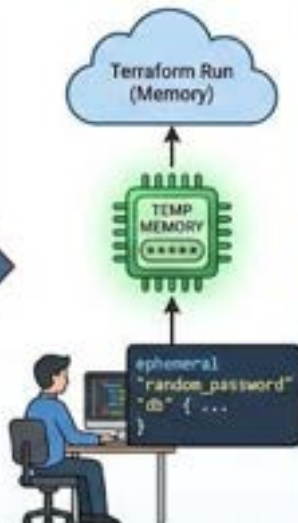
### 1. The "Before" Risk: Persistent Secrets



Secrets are stored in the state file, accessible to many.

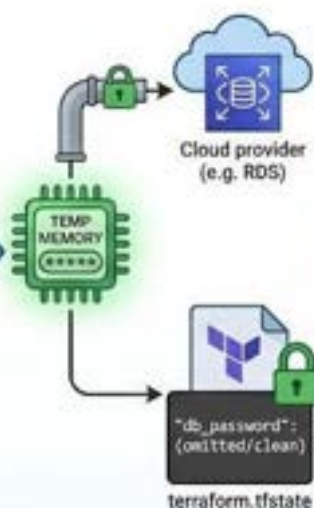
VS

### 2. The "After" Solution: Ephemeral Generation



Secrets are generated only in memory, never written to disk.

### 3. Secure Handoff & Clean State



Value is used for provisioning and immediately discarded. State file remains clean.

### 4. Organizational Benefits: Compliance & Security



Eliminates state file vulnerability, simplifies audits, and improves security posture.

# **HashiCorp Cloud Platform - Terraform**

# Setting the Base

Till now, we have been managing Terraform through CLI

Although the CLI approach works well, it's important to remember that there is also a GUI-based alternative also available through HCP Terraform.

```
root@esl-1c1ab01:~# terraform -help
2024-08-14T09:01:48.844Z [INFO] terraform version: 1.9.5
2024-08-14T09:01:48.844Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-14T09:01:48.844Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-14T09:01:48.844Z [DEBUG] using github.com/hashicorp/terraform-svchost v1.1.1
2024-08-14T09:01:48.844Z [DEBUG] using github.com/hashicorp/go-chy v1.14.4
2024-08-14T09:01:48.844Z [INFO] go runtime version: go1.21.5
2024-08-14T09:01:48.844Z [INFO] CLI args: [string]"terraform", string]"-help"]
2024-08-14T09:01:48.844Z [TRACE] stdout is a terminal of width 120
2024-08-14T09:01:48.844Z [TRACE] stderr is a terminal of width 120
2024-08-14T09:01:48.844Z [TRACE] stdin is a terminal
2024-08-14T09:01:48.844Z [DEBUG] attempting to open CLI config file: /root/.terraformrc
2024-08-14T09:01:48.844Z [DEBUG] file doesn't exist, but doesn't need to. ignoring.
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-14T09:01:48.844Z [DEBUG] ignoring non-existing provider search directory /usr/lib/terraform/plugins
2024-08-14T09:01:48.844Z [INFO] CLI command args: [string]"plan"]
```



# HCP Terraform

**HCP Terraform** manages Terraform runs in a consistent and reliable environment with various features like access controls, private registry for sharing modules, policy controls and others.

The screenshot displays the HCP Terraform console interface. At the top, a notification states 'mykplate triggered a run from UI a few seconds ago'. Below this, a 'Plan finished' status is shown with a green checkmark, indicating the run completed 'a few seconds ago'. A summary bar shows 'Resources: 1 to add, 0 to change, 0 to destroy'. A progress bar indicates '1 to create'. The console lists the resource 'aws\_instance.myec2' with an AWS icon. Below the resource list, a 'Download Sentinel mocks' button is visible. A 'Cost estimation finished' section follows, also with a green checkmark and 'a few seconds ago' timestamp. It shows 'Resources: 1 of 1 estimated: \$8.35/mo -> \$8.35'. At the bottom, a table provides a detailed cost breakdown for the 'aws\_instance.myec2' resource.

| RESOURCE     | NAME  | HOURLY COST | MONTHLY COST | MONTHLY DELTA |
|--------------|-------|-------------|--------------|---------------|
| aws_instance | myec2 | \$0.092     | \$8.352      | +\$8.352      |

# **HCP Terraform - Pricing**

# Not Everything is Free

HCP Terraform is not entirely free. Depending on the usage and features needed, there are multiple pricing plans that are available.

| Essentials                                                                                                                       | Standard                                                                                                                         | Premium                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| STARTING AT<br><b>\$0.10</b><br>per month per resource                                                                           | STARTING AT<br><b>\$0.47</b><br>per month per resource                                                                           | STARTING AT<br><b>\$0.99</b><br>per month per resource                                                                              |
| Cloud                                                                                                                            | Cloud                                                                                                                            | Cloud                                                                                                                               |
| For professional individuals or teams adopting infrastructure as code provisioning.<br>Get started with a \$500 HCP trial credit | For enterprises standardizing and managing infrastructure automation and lifecycle.<br>Get started with a \$500 HCP trial credit | For enterprises to maximize their IT investments with a secure, self-service workflow.<br>Get started with a \$500 HCP trial credit |
| <a href="#">Sign up</a>                                                                                                          | <a href="#">Sign up</a>                                                                                                          | <a href="#">Sign up</a>                                                                                                             |
| *Rated hourly at \$0.00013 per hour                                                                                              | Includes HCP Waypoint.<br>*Rated hourly at \$0.00064 per hour                                                                    | Includes IBM HCP Waypoint.<br>*Rate hourly at \$0.00135 per hour                                                                    |

# Important Features Availability

| Feature                           | Essential               | Standard  | Premium   | Enterprise |
|-----------------------------------|-------------------------|-----------|-----------|------------|
| Audit Logging                     | No                      | Yes       | Yes       | Yes        |
| Drift Detection                   | No                      | Yes       | Yes       | Yes        |
| Continuous validation             | No                      | Yes       | Yes       | Yes        |
| Policy as code (Sentinel and OPA) | 1 Policy set 5 Policies | Unlimited | Unlimited | Yes        |
| Air Gap Installation              | No                      | No        | No        | Yes        |
| Audit Trails API                  | No                      | Yes       | Yes       | No         |

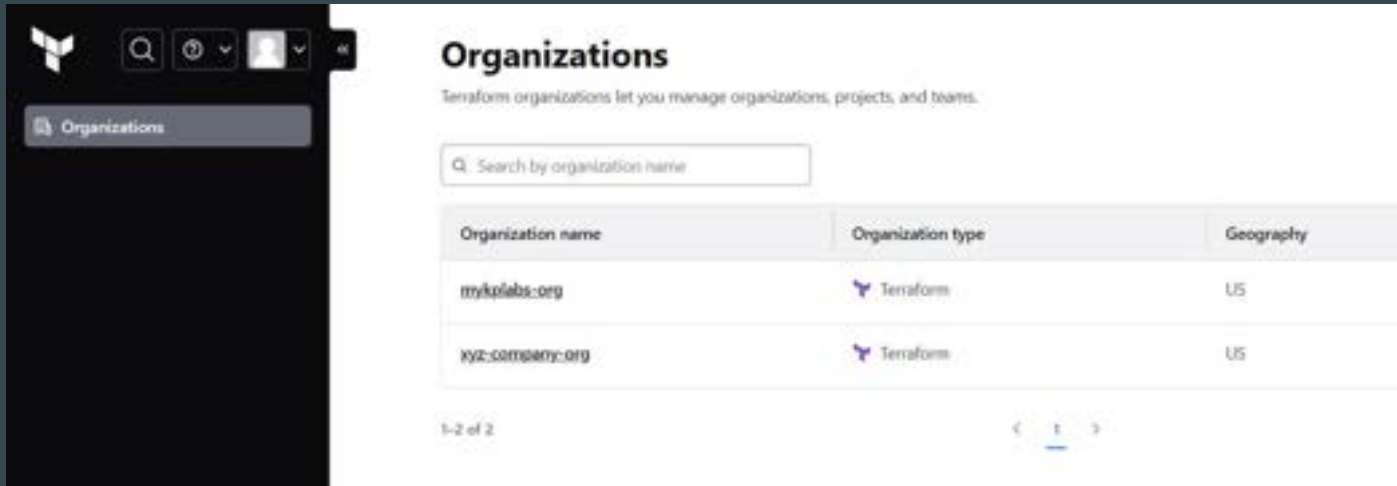


|                         |                                                                              |
|-------------------------|------------------------------------------------------------------------------|
| Pay-as-You-Go (PAYG)    | On-demand, usage-based billing with no long-term commitment                  |
| Flex                    | Single and multi-year plans available with preferred pricing for maximum ROI |
| Enterprise Self Managed | Fully customizable plans for maximum control                                 |

# **HCP Terraform - Basic Structure**

# 1 - Organizations

Organizations are a shared space for one or more teams to collaborate on workspaces.



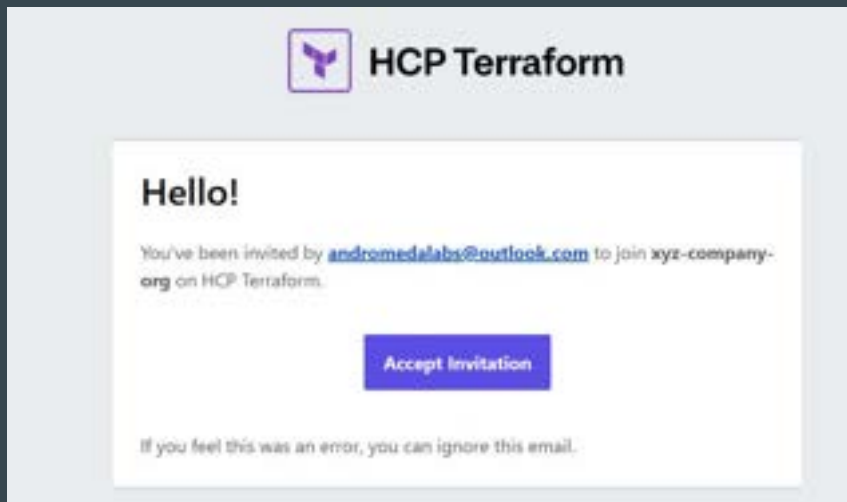
The screenshot shows the Terraform Organizations management interface. On the left is a dark sidebar with the Terraform logo and navigation icons. The main content area is titled "Organizations" and includes a subtitle: "Terraform organizations let you manage organizations, projects, and teams." Below the title is a search bar labeled "Search by organization name". A table lists the organizations, with columns for "Organization name", "Organization type", and "Geography". Two organizations are listed: "mykplebs.org" and "xyz-company.org", both of type "Terraform" and located in the "US". At the bottom left of the table area, it says "1-2 of 2". At the bottom right, there are pagination controls showing "1" as the current page.

| Organization name | Organization type | Geography |
|-------------------|-------------------|-----------|
| mykplebs.org      | Terraform         | US        |
| xyz-company.org   | Terraform         | US        |

# Points to Note - Organizations

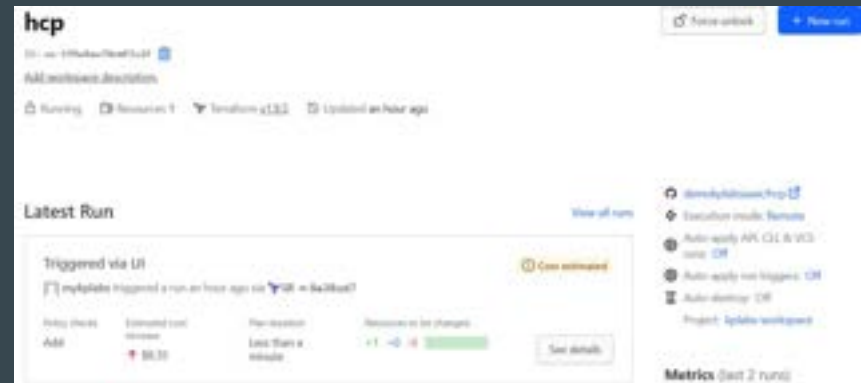
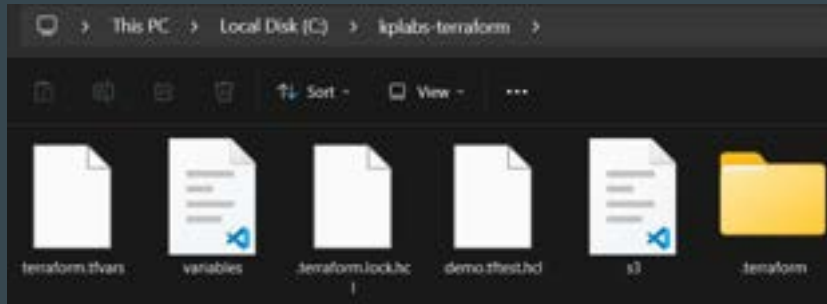
HCP Terraform manages plans and billing at the organization level.

Each HCP Terraform user can belong to multiple organizations, which might subscribe to different billing plans.



## 2 - Workspace

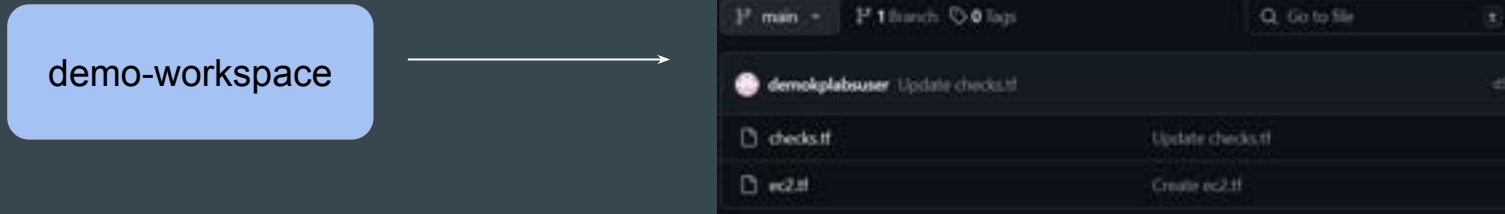
HCP Terraform manages infrastructure collections with workspaces instead of directories



# Workspace & Configuration Files

The Terraform configuration file (sample.tf) is not directly uploaded to a workspace.

Instead, workspace is connected to GitHub repository where it can fetch code from.



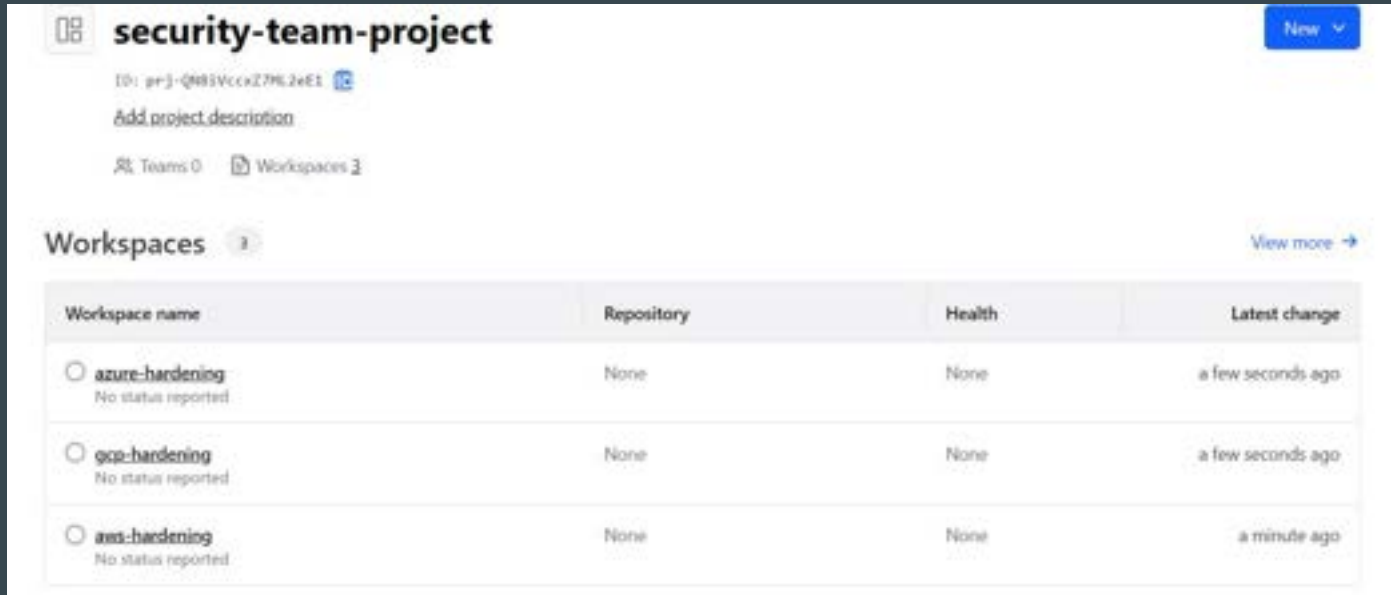
GitHub Repo

# Workspace vs Directories

| Component               | Local Terraform                                                          | HCP Terraform                                                              |
|-------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Terraform configuration | On disk                                                                  | In linked version control repository, or periodically uploaded via API/CLI |
| Variable values         | As <code>.tfvars</code> files, as CLI arguments, or in shell environment | In workspace                                                               |
| State                   | On disk or in remote backend                                             | In workspace                                                               |
| Credentials and secrets | In shell environment or entered at prompts                               | In workspace, stored as sensitive variables                                |

## 3 - Projects

HCP Terraform projects let you organize your workspaces into groups.



The screenshot shows the HCP Terraform interface for a project named 'security-team-project'. The project ID is 'prj-Q483VccxZ7M2eE1'. Below the project name, there is a link to 'Add project description'. At the bottom, it shows 'Teams 0' and 'Workspaces 3'. The 'Workspaces' section is expanded, showing a table with three workspaces: 'azure-hardening', 'gcp-hardening', and 'aws-hardening'. Each workspace has a radio button, a status of 'No status reported', a repository of 'None', a health of 'None', and a latest change time.

**security-team-project** New ▾

ID: prj-Q483VccxZ7M2eE1 

[Add project description](#)

Teams 0 Workspaces 3

**Workspaces** 3 View more →

| Workspace name                                                     | Repository | Health | Latest change     |
|--------------------------------------------------------------------|------------|--------|-------------------|
| <input type="radio"/> <b>azure-hardening</b><br>No status reported | None       | None   | a few seconds ago |
| <input type="radio"/> <b>gcp-hardening</b><br>No status reported   | None       | None   | a few seconds ago |
| <input type="radio"/> <b>aws-hardening</b><br>No status reported   | None       | None   | a minute ago      |



## Point to Note - Projects

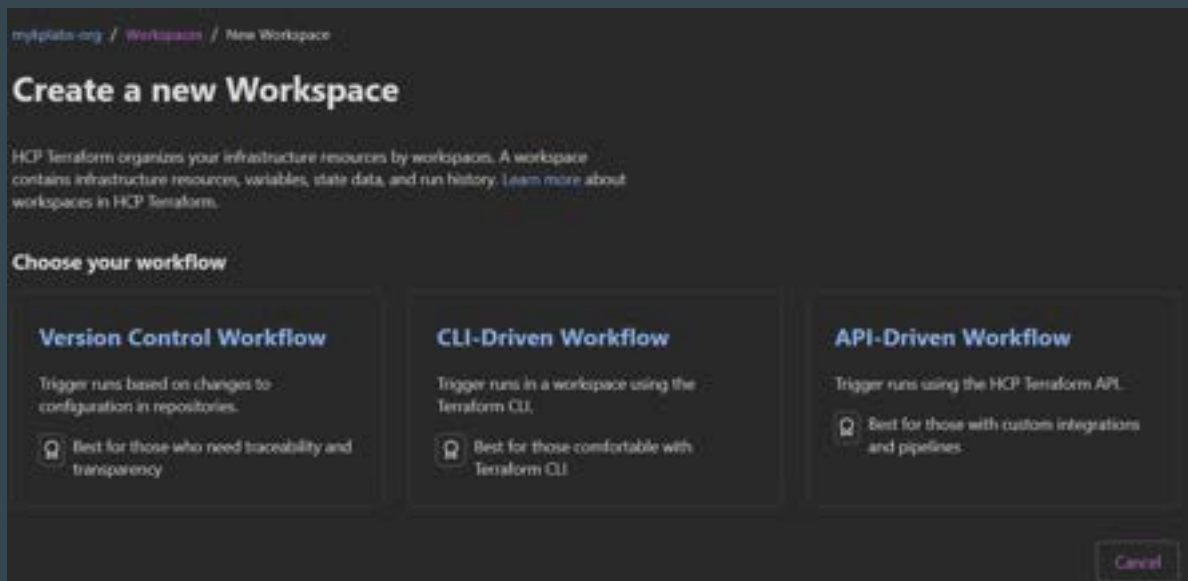
You can structure your projects based on your organization's resource usage and ownership patterns, such as teams, business units, or services.

With HCP Terraform Standard Edition, you can give teams access to groups of workspaces using projects.

# **The CLI-driven Run Workflow**

# Setting the Base

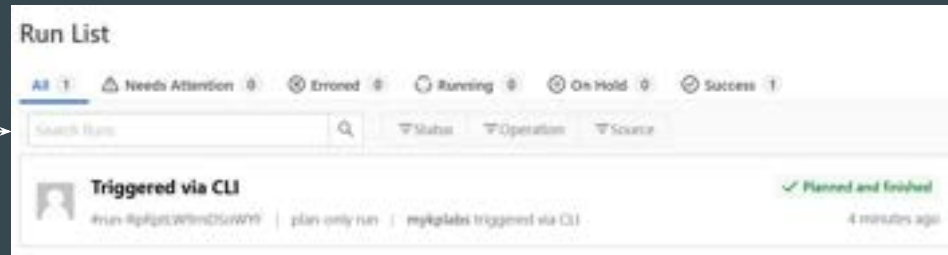
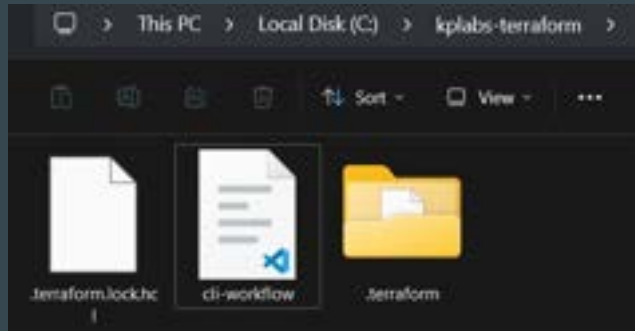
Whenever we create a new Workspace in HCP, following are the 3 types of workflow modes that are available.



# Basics of CLI Driven Workflow

In this approach, the working directory on your workstation is linked with HCP Workspace.

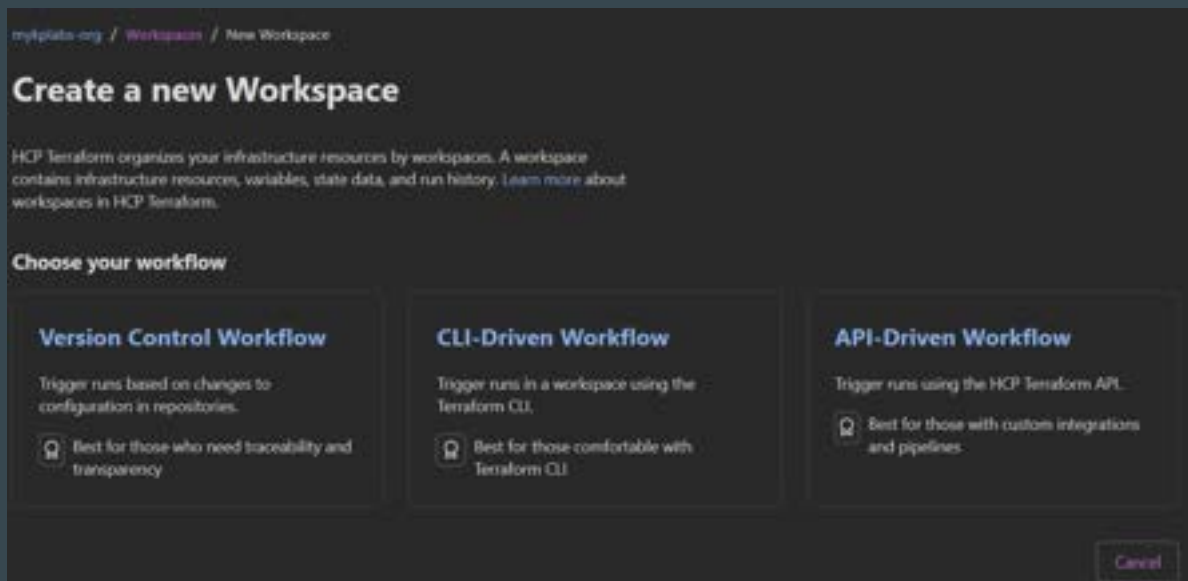
The code file can be present in your laptop, and plan/apply operations can also be initiated from local workstation.



# **The CLI-driven Run Workflow - Practical**

# Setting the Base

Whenever we create a new Workspace in HCP, following are the 3 types of workflow modes that are available.



# Step 1 - Setup Cloud Integration

You have to add code block within your .tf file to setup cloud integration.

This code will contain details about your HCP organization and workspace name.

```
terraform {  
  cloud {  
  
    organization = "mykplabs-org"  
  
    workspaces {  
      name = "remote-operation-workspace"  
    }  
  }  
}
```

## Step 2 - Terraform Login

Once your cloud integration code block has been added, next step is to run the **terraform login** command.

```
C:\kplabs-terraform>terraform login
Terraform will request an API token for app.terraform.io using your browser.

If login is successful, Terraform will store the token in plain text in
the following file for use by subsequent commands:
  C:\Users\zealv\AppData\Roaming\terraform.d\credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

  Enter a value: yes

-----

Terraform must now open a web browser to the tokens page for app.terraform.io.

If a browser does not open this automatically, open the following URL to proceed:
  https://app.terraform.io/app/settings/tokens?source=terraform-login
```



## Step 3 - Initialize

Run the `terraform init` command to initialize

```
C:\kplabs-terraform>terraform init
Initializing HCP Terraform...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.70.0...
- Installed hashicorp/aws v5.70.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

HCP Terraform has been successfully initialized!

You may now begin working with HCP Terraform. Try running "terraform plan" to
see any changes that are required for your infrastructure.

If you ever set or change modules or Terraform Settings, run "terraform init"
again to reinitialize your working directory.
```

## Step 4 - Run the Plan / Apply Operations

Once initialized, the terraform “plan”, and “apply” commands when entered through CLI will run in HCP Terraform with output streamlined in terminal.

```
C:\kplabs-terraform>terraform plan
Running plan in HCP Terraform. Output will stream here. Pressing Ctrl-C
will stop streaming the logs, but will not stop the plan running remotely.

Preparing the remote plan...

To view this run in a browser, visit:
https://app.terraform.io/app/mykplabs-org/remote-operation-workspace/runs/run-RpRptLW9rnDSowYF

Waiting for the plan to start...

Terraform v1.9.7
on linux_amd64
Initializing plugins and modules...

Terraform used the selected providers to generate the following execution plan. Resource actions
following symbols:
  + create
```

---

# Sentinel

Terraform Cloud In Detail

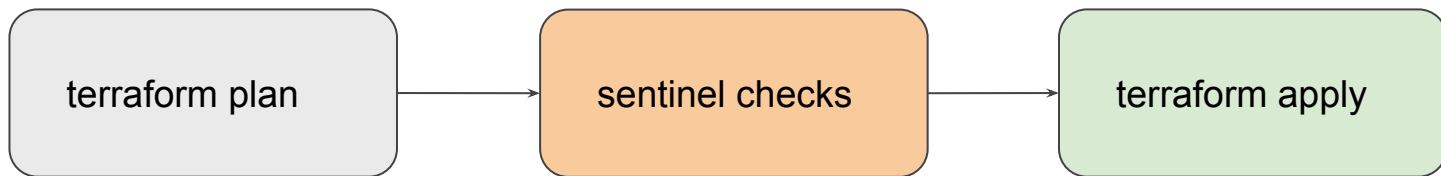
---

# Overview of the Sentinel

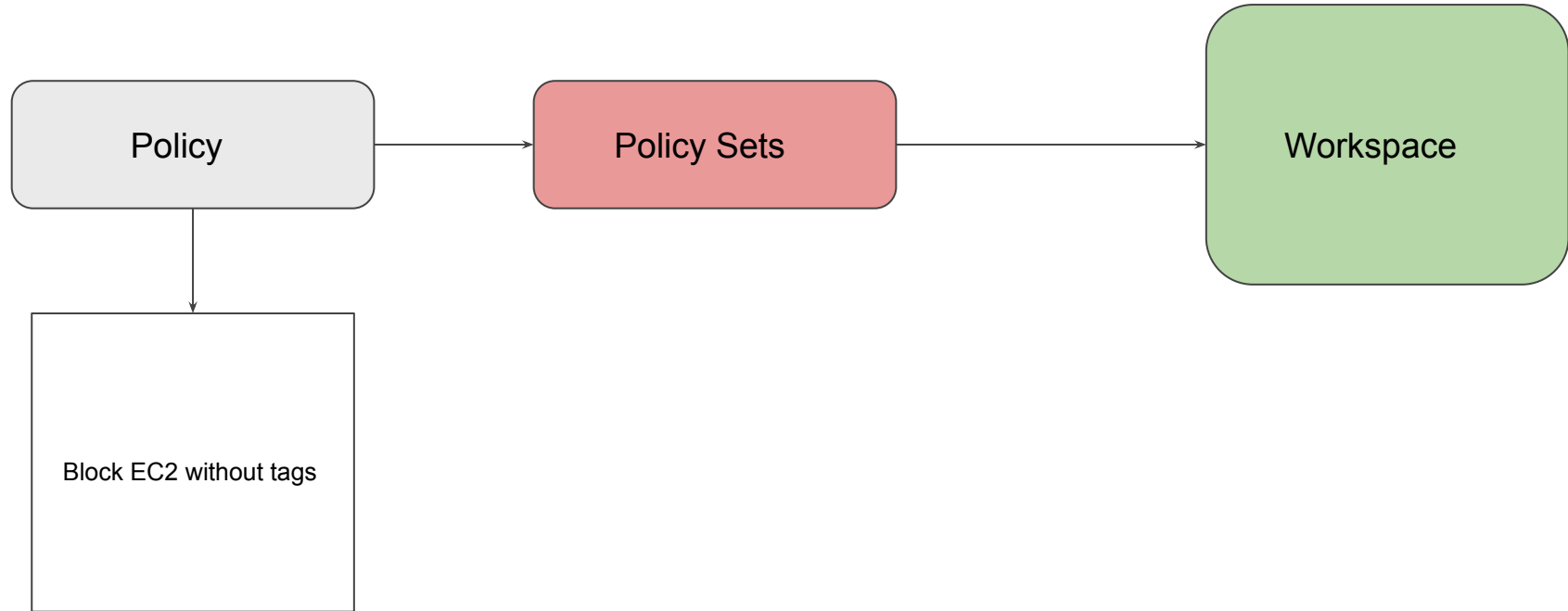
Sentinel is a policy-as-code framework integrated with the HashiCorp Enterprise products.

It enables fine-grained, logic-based policy decisions, and can be extended to use information from external sources.

Note: Sentinel policies are paid feature



# High Level Structure



---

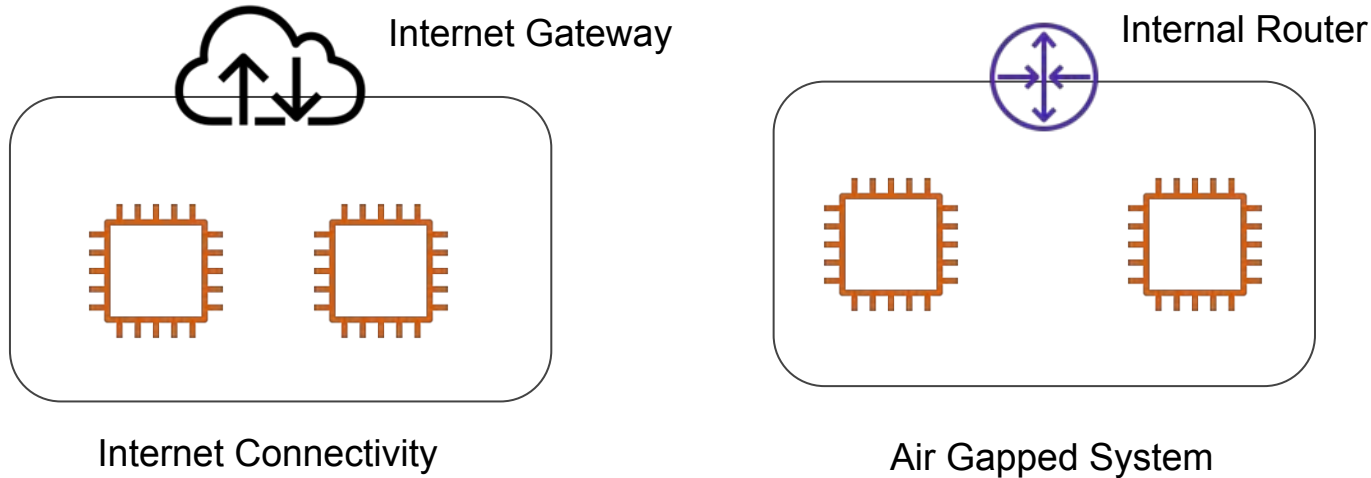
# Air Gapped Environments

Installation Methods

---

# Understanding Concept of Air Gap

An air gap is a network security measure employed to ensure that a secure computer network is physically isolated from unsecured networks, such as the public Internet.



# Usage of Air Gapped Systems

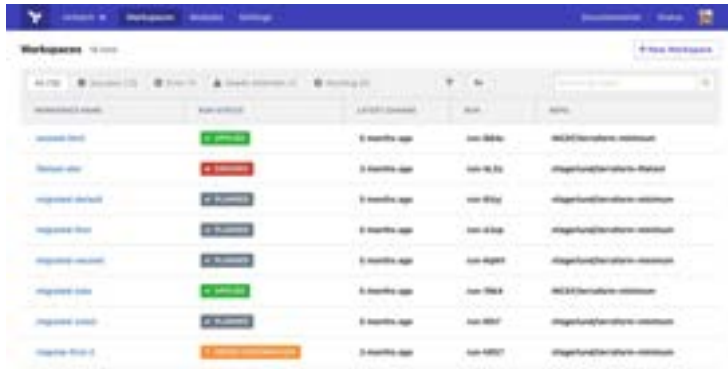
Air Gapped Environments are used in various areas. Some of these include:

- Military/governmental computer networks/systems
- Financial computer systems, such as stock exchanges
- Industrial control systems, such as SCADA in Oil & Gas fields



# Terraform Enterprise Installation Methods

Terraform Enterprise installs using either an online or air gapped method and as the names infer, one requires internet connectivity, the other does not



Terraform Enterprise

Air Gap Install



Isolated Server

Choose your installation type



Online



Airgapped

Please choose an installation type to continue.

Continue »

## Provide path or upload airgap bundle

Provide absolute path on this server to archive file

e.g. /usr/local/share/airgap/airgap

Continue >

Select file for upload

Upload Airgap Bundle

To upload an app bundle, file must have a **bundle** extension

Back

# Relax and Have a Meme Before Proceeding



Cole  
@its\_cmillz6

when you're sleeping and your alarm  
didn't ring yet but the amount of  
sleep you're getting is suspicious



BABY @HORCHATACUM - 20h



# **HCP Terraform - Private Registry**

# Setting the Base

HCP Terraform's **private registry** works similarly to the public Terraform Registry and helps you share Terraform providers and Terraform modules across your organization.

The screenshot shows the Terraform Private Registry interface for a module named 'private-module'. The page includes a header with the module name, version (v1.0.0), and a 'Source' link. Below the header, there are tabs for 'Readme', 'Inputs', 'Outputs', 'Dependencies', and 'Resources'. The 'Readme' tab is active, displaying the module's description: 'This module generates a random "pet name" (e.g. aws-ec2-instance-1) using the random provider. It is designed as a zero-cost, zero-auth module for testing the HCP Terraform Private Registry workflow. It requires no cloud credentials (AWS, Azure, GCP), making it safe to run in any environment.' The 'Usage' section shows a code snippet for the module's configuration. On the right side, there is a 'Testing' section with a 'Configure Tools' button, and a 'Usage Instructions' section with a 'Copy and paste into your Terraform configuration' button.

**private-module** Private Source

Published by vey update Provider Update Version 1.0.0 Change Published a minute ago Source Download module Branch main Commit 6d9761247

[Readme](#) [Inputs](#) [Outputs](#) [Dependencies](#) [Resources](#)

## Terraform Petname Module

This module generates a random "pet name" (e.g. aws-ec2-instance-1) using the random provider.

It is designed as a **zero-cost, zero-auth** module for testing the HCP Terraform Private Registry workflow. It requires no cloud credentials (AWS, Azure, GCP), making it safe to run in any environment.

### Usage

```
module "petname" {  
  # Replace v1.0.0, 0000 with your actual HCP Terraform Organization name  
  source = "app.terraform.io/v1.0.0,0000/petname/module"  
  version = "1.0.0"  
  
  # Optional inputs  
  seed = 4  
}
```

### Testing

[Configure Tools](#)

[Manage Module for Organization](#)

[Open in Designer](#)

[Publish New Version](#)

### Usage Instructions

Copy and paste into your Terraform configuration and set values for the input variables. Or, [download a configuration](#) for ready-to-use module and workspace outputs as inputs.

```
module "petname-module" {  
  source = "app.terraform.io/v1.0.0,0000/petname/module"  
  version = "1.0.0"  
}
```

# Calling the Private Module

To call the Private Registry Module from your workspace, you have to specify the correct private registry module source URL.

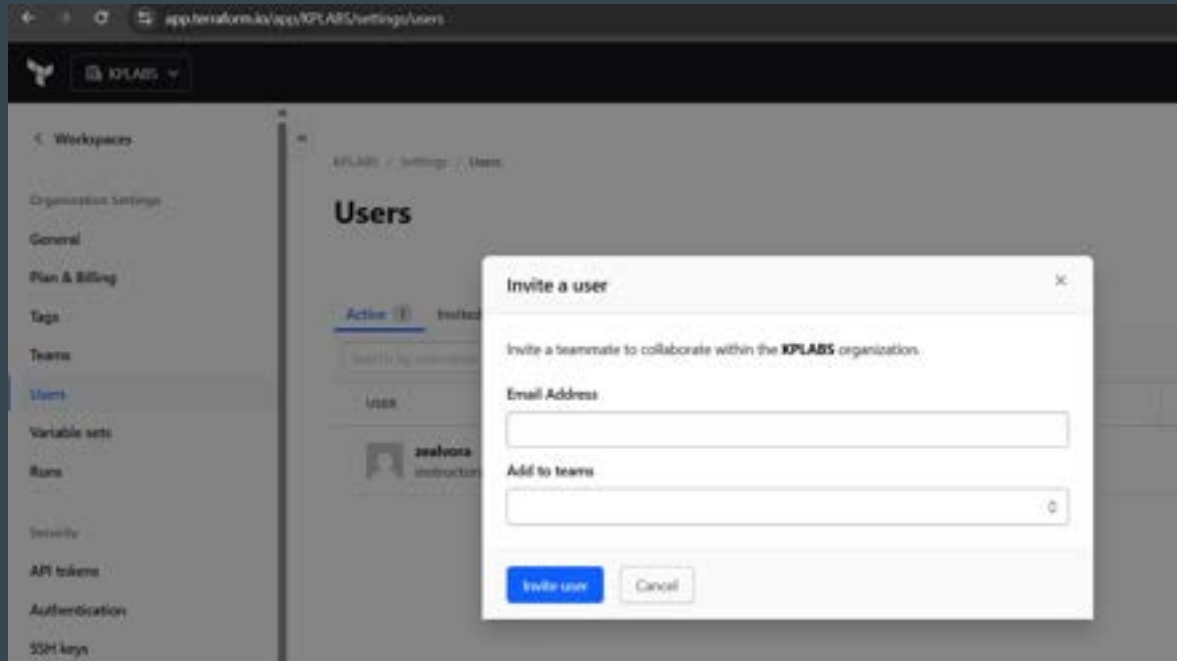
```
main.tf > ...  
module "private-module" {  
    source = "app.terraform.io/org-kplabs/private-module/kplabs"  
    version = "1.0.0"  
    word_count = 4  
}  
  
output "final_pet_name" {  
    value = module.private-module  
}
```

# **HCP Terraform - Teams**



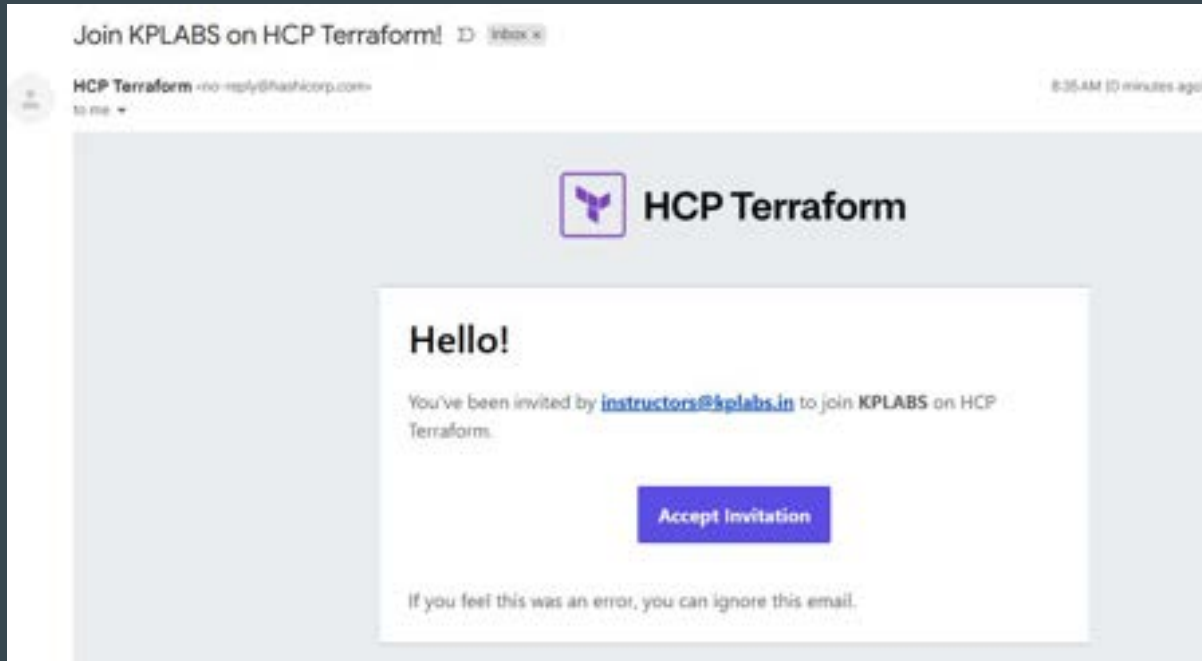
# HCP Terraform - Users

You can invite new users to HCP Terraform Organization to collaborate on projects.



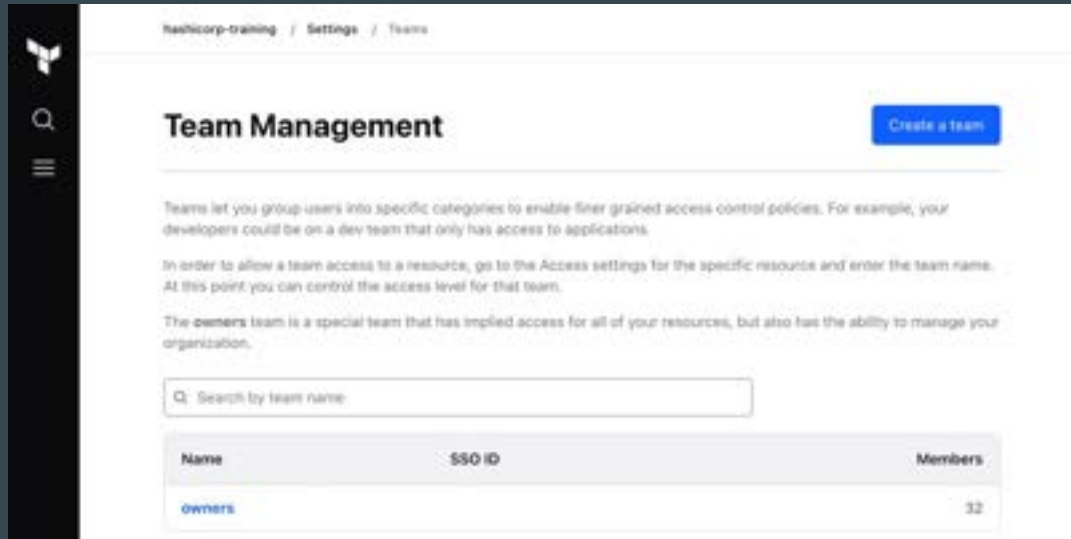
# Reference Screenshot

Reference screenshot of user receiving the invitation to join.



# HCP Terraform - Teams

Teams are groups of HCP Terraform users within an organization.



The screenshot shows the 'Team Management' page in the HCP Terraform console. The breadcrumb trail at the top reads 'hashicorp-training / Settings / Teams'. On the left is a dark sidebar with the HashiCorp logo and navigation icons. The main content area has a 'Team Management' header with a 'Create a team' button. Below the header, there is explanatory text about teams and access control, followed by a search bar labeled 'Search by team name:'. At the bottom, a table lists the existing teams.

| Name   | SSO ID | Members |
|--------|--------|---------|
| owners |        | 32      |

## Point to Note

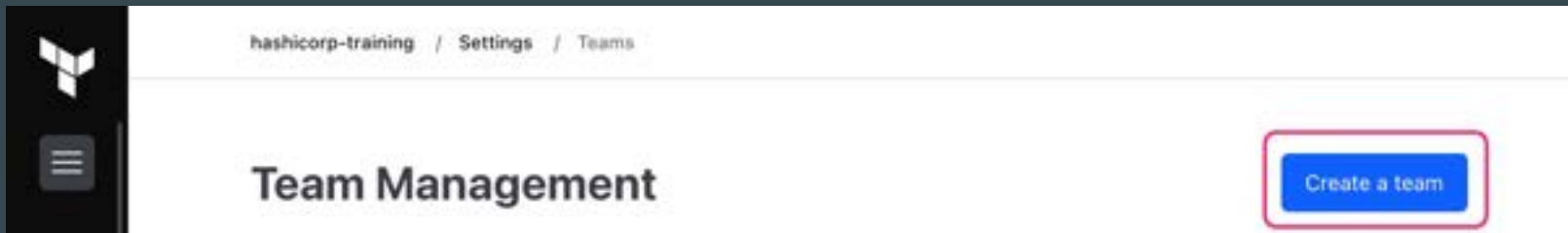
The owners team is the default team of an HCP Terraform organization.

## **Manage permissions in HCP Terraform**

## Create a new team

The owners team is the default team of an HCP Terraform organization.

This team has every available permission in the organization, so it is important to create restricted team access before adding new members.





## Team: Dev-Team

### Team name

Dev-Team

### SSO team ID

Optional. SSO team ID is used only for customizing SAML SSO identification.

SSO team ID

Create team

## Organization Access

### Project permissions

#### ☒ None

Does not grant members any explicit access to projects. Permissions they inherit depend on individual projects.

#### ☐ View all projects

Allow the team to view all projects in this organization.

#### ☐ Manage all projects

Grants members the ability to view, edit, delete, and assign team access to all projects in this organization, as well as the ability to create new workspaces for any project.

### Workspace permissions

#### ☒ None

Does not grant members any explicit access to workspaces. Permissions they will be granted on individual workspaces.

#### ☐ View all workspaces

Allow the team to view all workspaces in this organization.

#### ☐ Manage all workspaces

Grants members the ability to view, edit, delete, and assign team access to all workspaces in this organization, as well as the ability to create new workspaces in the default project.

### Settings permissions

#### ☐ Manage policies

Allow members to create, edit, test, fail and delete the organization's policies.

#### ☐ Manage policy overrides

Allow members to override built-in mandatory policy checks.

#### ☐ Manage rule tasks

Allow members to create, update, and delete rule tasks on per-organization.

#### ☐ Manage vendor control settings

Allow members to manage the organization's AWS Provider and SSO keys.

#### ☐ Manage membership

Allow members to add and remove users from the organization, and to manage the membership of teams. This permission allows members to assign themselves to other visible teams, and should only be granted to trusted users.

### Private registry permissions

#### ☐ Manage private registry

##### ☐ Manage modules

Allow members to publish and delete modules in the organization's private registry.

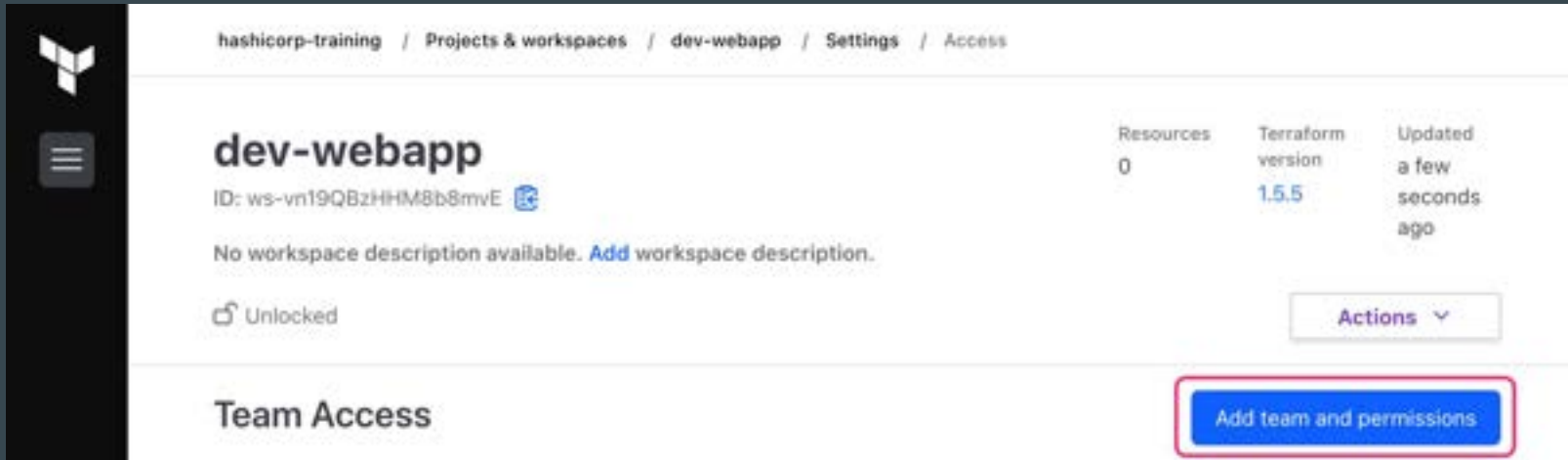
##### ☐ Manage providers

Allow members to publish and delete providers in the organization's private registry.

Revoke team organization access

# Workspace Level Permissions

You can configure Workspace level permissions for Teams



The screenshot shows the 'dev-webapp' workspace settings in the HashiCorp Terraform UI. The breadcrumb navigation at the top reads: hashicorp-training / Projects & workspaces / dev-webapp / Settings / Access. The workspace name 'dev-webapp' is prominently displayed, followed by its ID 'ws-vn19QBzHHM8b8mvE' and a small icon. Below this, a message states 'No workspace description available. Add workspace description.' with a link to 'Add workspace description'. To the left of this message is a lock icon and the text 'Unlocked'. On the right side, there are three status indicators: 'Resources' with a value of '0', 'Terraform version' with a value of '1.5.5', and 'Updated' with a value of 'a few seconds ago'. Below these indicators is an 'Actions' button with a dropdown arrow. At the bottom of the page, under the 'Team Access' section, there is a blue button labeled 'Add team and permissions' which is highlighted with a red rectangular border.

hashicorp-training / Projects & workspaces / dev-webapp / Settings / Access

## dev-webapp

ID: ws-vn19QBzHHM8b8mvE 

No workspace description available. [Add workspace description.](#)

 Unlocked

Resources: 0    Terraform version: 1.5.5    Updated: a few seconds ago

Actions ▾

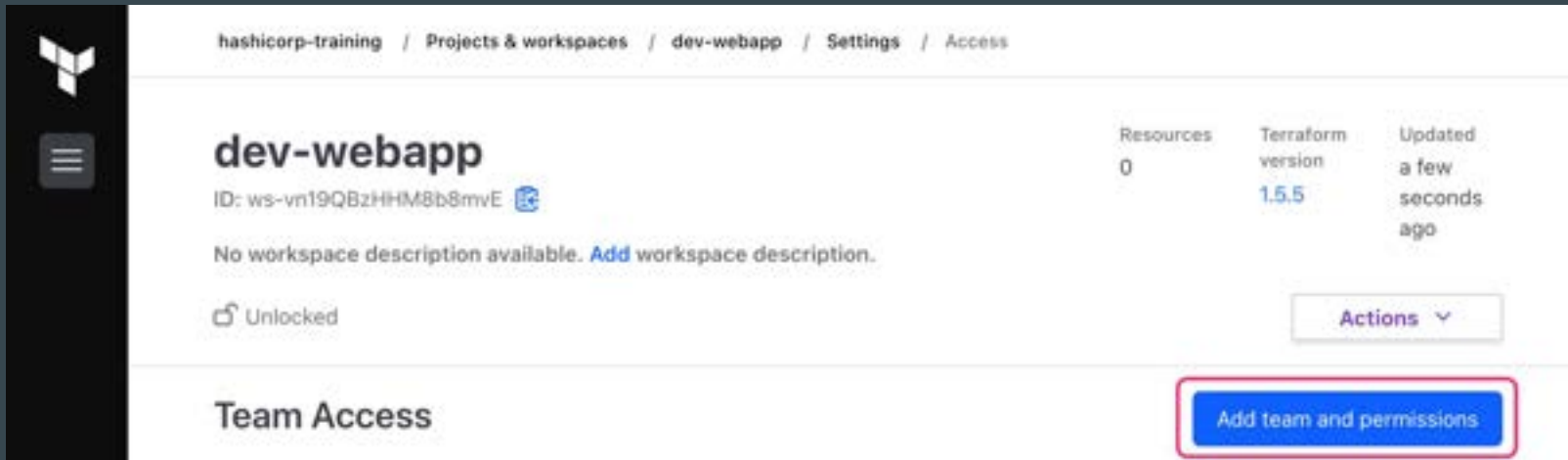
### Team Access

[Add team and permissions](#)



# Workspace Level Permissions

You can configure Workspace level permissions for Teams



The screenshot shows the HashiCorp Terraform web interface. On the left is a dark sidebar with the HashiCorp logo and a menu icon. The main content area has a breadcrumb trail: `hashicorp-training` / `Projects & workspaces` / `dev-webapp` / `Settings` / `Access`. The workspace name **dev-webapp** is prominently displayed, followed by its ID: `ws-vn19QBzHHM8b8mvE` and a small icon. Below this, it states "No workspace description available. [Add workspace description.](#)". A status indicator shows a lock icon and the text "Unlocked". To the right, a summary table provides key details:

| Resources | Terraform version | Updated           |
|-----------|-------------------|-------------------|
| 0         | 1.5.5             | a few seconds ago |

Below the table is an "Actions" button with a dropdown arrow. At the bottom, the "Team Access" section is visible, featuring a blue button labeled "Add team and permissions" which is highlighted with a red rectangular border.

## dev-webapp

Go to [dev-webapp](#) (current)

See workspace description available. [Add workspace description](#).

Updated

Members 8  
Environments 10/2  
Created a day ago

Actions

### Add team access

Grant a team permission on this workspace.

Standard permission groups (Admin, Write, Plan, Read) contain different capabilities for shared settings. [Learn more](#)

Team: **dev-team**

✓ **dev-team**

Permission group: **dev-team**

☐ **Read**  
Default permission for reading workspace.

☐ **Plan**  
Read permission plus the ability to create runs.

☒ **Write**  
Read, plan, and write permissions.

☐ **Admin**  
Full control of the workspace.

☐ **Custom**  
Create a custom permission set for this team.

[Assign permissions](#)

Cancel

#### Custom Permissions

Create a custom permission set for this workspace.

#### Run Access

- ☐ **Read**  
Can read and generate information on the workspace's runs, including logs, status, results, all policy details, and job templates.
- ☐ **Plan**  
Can create plans and templates in runs, in addition to all abilities of the read permission.
- ☒ **Write**  
Can create, modify, or delete runs, in addition to all abilities of the plan permission.

#### Variables

- ☐ **No access**  
No access to the workspace's variables.
- ☐ **Read**  
Can view the workspace's variables.
- ☒ **Read and write**  
Can view and edit the workspace's variables.

#### Stack access

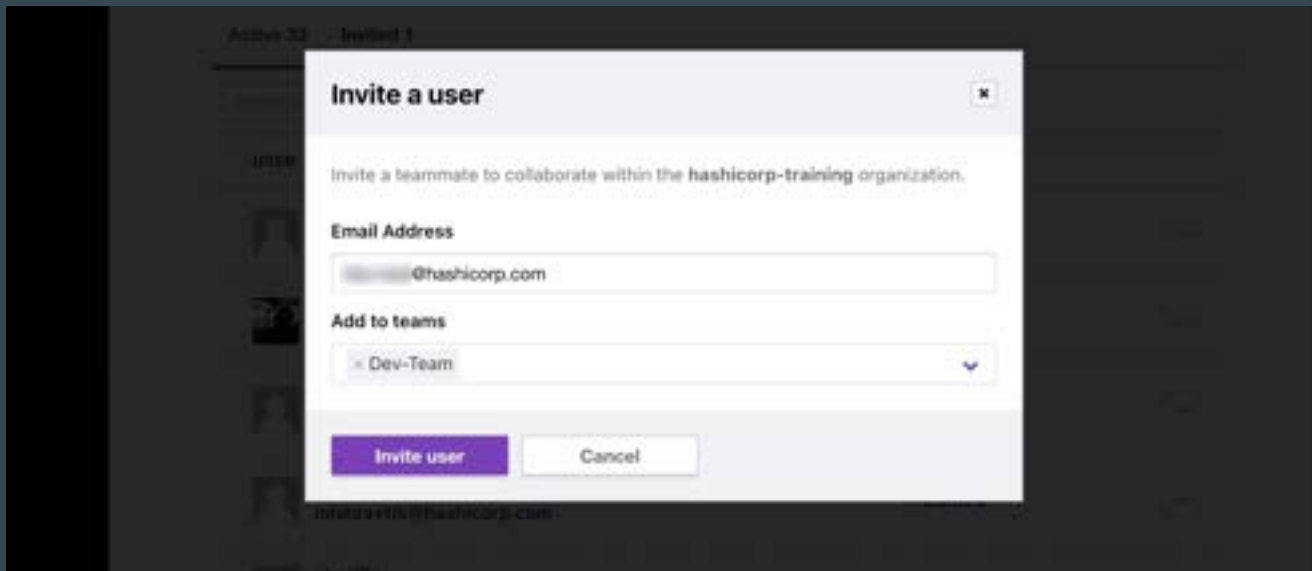
- ☐ **No access**  
No access to stack resources associated with this workspace.
- ☐ **Read outputs only**  
Can run [stack outputs](#) to retrieve workspace's stack outputs.
- ☐ **Read**  
Can view the workspace's stack resources.
- ☒ **Read and write**  
Can view and modify workspace stack resources. This permission is only required when the workspace [auto-deletes](#) or set up "tags".

#### Other controls

- ☒ **Download build artifacts**  
Can download build artifacts. [Add](#) for any of the workspace's runs, permission by S3/2. Requires the environment setting all settings.
- ☐ **Manage Workspace Run Tasks**  
Allow members to create and manage templates of this workspace.
- ☒ **Lockdown workspace**  
Can securely lock and unlock the workspace. This permission is required when the workspace [auto-deletes](#) or set up "tags".

# Invite a user to your organization and team

To collaborate with your team members in HCP Terraform, you need to grant them access to the same HCP Terraform organization.



The screenshot shows a modal dialog titled "Invite a user" with a close button (X) in the top right corner. The dialog contains the following elements:

- A message: "Invite a teammate to collaborate within the hashicorp-training organization."
- A label "Email Address" above a text input field containing "@hashicorp.com".
- A label "Add to teams" above a dropdown menu showing "Dev-Team" with a downward arrow.
- Two buttons at the bottom: "Invite user" (purple) and "Cancel" (white with a grey border).

The background of the application is dimmed, showing a sidebar with "Active 32" and "Invited 1" counts, and a list of users with avatars and email addresses like "mindaevn@hashicorp.com".

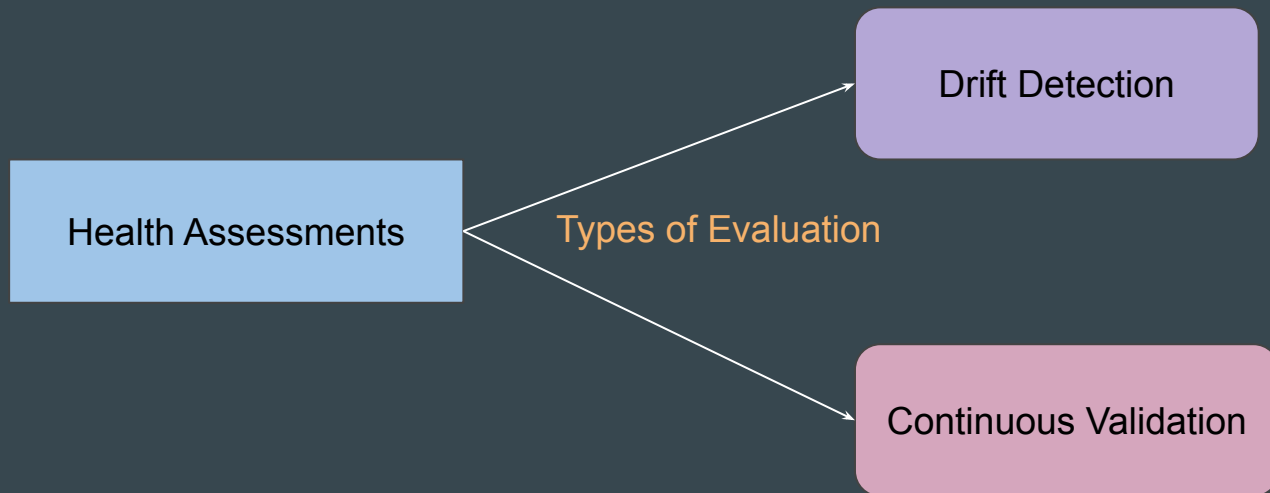
## Point to Note

The owners team is the default team of an HCP Terraform organization.

# **HCP Terraform - Health Assessments**

# Setting the Base

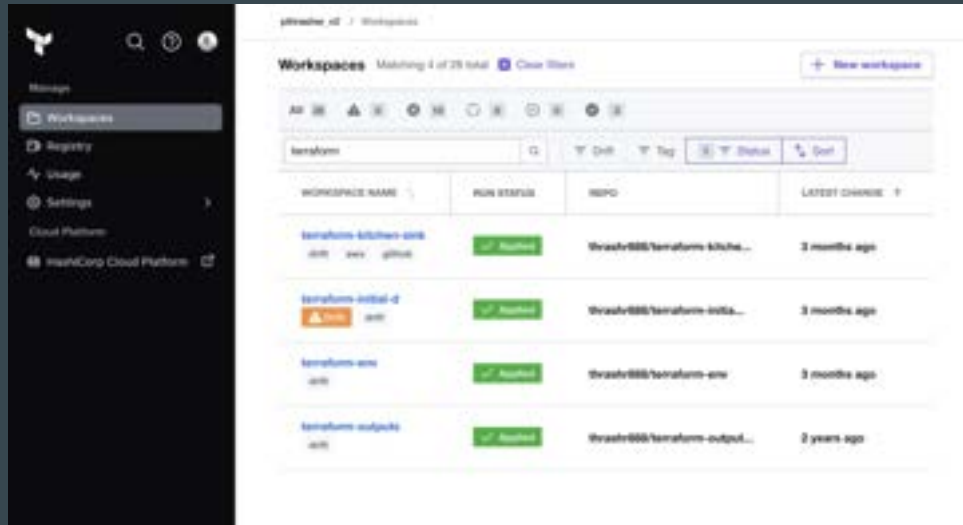
HCP Terraform can perform **automatic health assessments in a workspace** to assess whether its real infrastructure matches the requirements defined in its Terraform configuration.



# Drift Detection

Drift detection determines whether your real-world infrastructure matches your Terraform configuration.

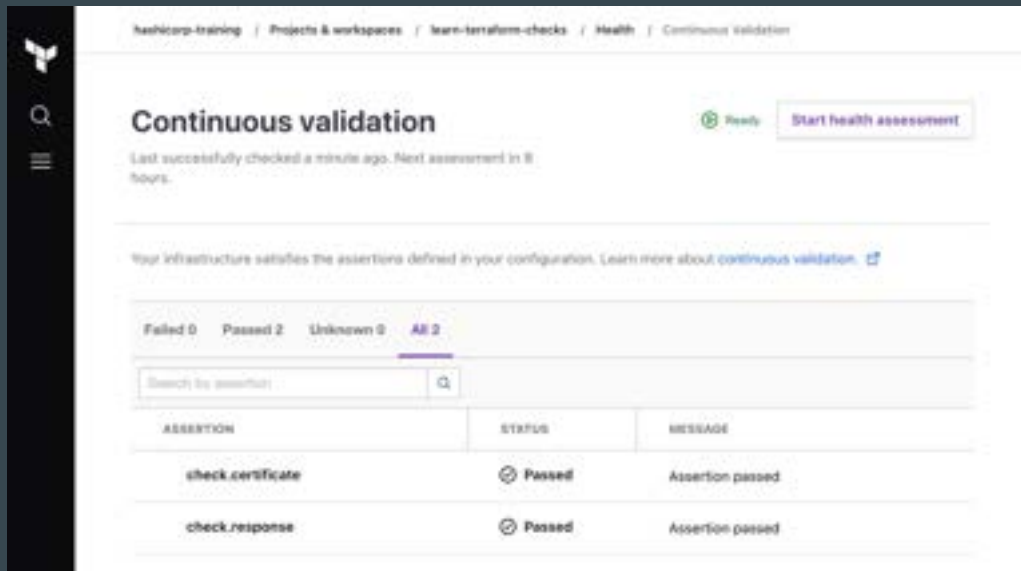
Configuration drift occurs when changes are made outside Terraform's regular process (manual changes)



# Continuous Validation

Continuous validation determines whether custom conditions in the workspace's configuration continue to pass after Terraform provisions the infrastructure.

For example, you can monitor whether your website returns an expected status code.



The screenshot displays the 'Continuous validation' page in the Terraform UI. The breadcrumb trail at the top reads: 'hashicorp-training / Projects & workspaces / learn-terraform-checks / Health / Continuous validation'. The main heading is 'Continuous validation', followed by a status indicator 'Ready' and a 'Start health assessment' button. A message states: 'Last successfully checked a minute ago. Next assessment in 8 hours.' Below this, a summary line says: 'Your infrastructure satisfies the assertions defined in your configuration. Learn more about [continuous validation](#).' A summary bar shows 'Failed 0', 'Passed 2', 'Unknown 0', and 'All 2'. A search bar is present with the text 'Search by assertion'. A table lists the assertions:

| ASSERTION         | STATUS | MESSAGE          |
|-------------------|--------|------------------|
| check_certificate | Passed | Assertion passed |
| check_response    | Passed | Assertion passed |



## Example - Continuous Validation

The following example uses the HTTP Terraform provider and a scoped data source within a check block to assert the Terraform website returns a 200 status code, indicating it is healthy.

```
check "health_check" {  
  data "http" "terraform_io" {  
    url = "https://kplabs.in"  
  }  
  
  assert {  
    condition = data.http.terraform_io.status_code == 200  
    error_message = "KPLABS returned an unhealthy status code"  
  }  
}
```

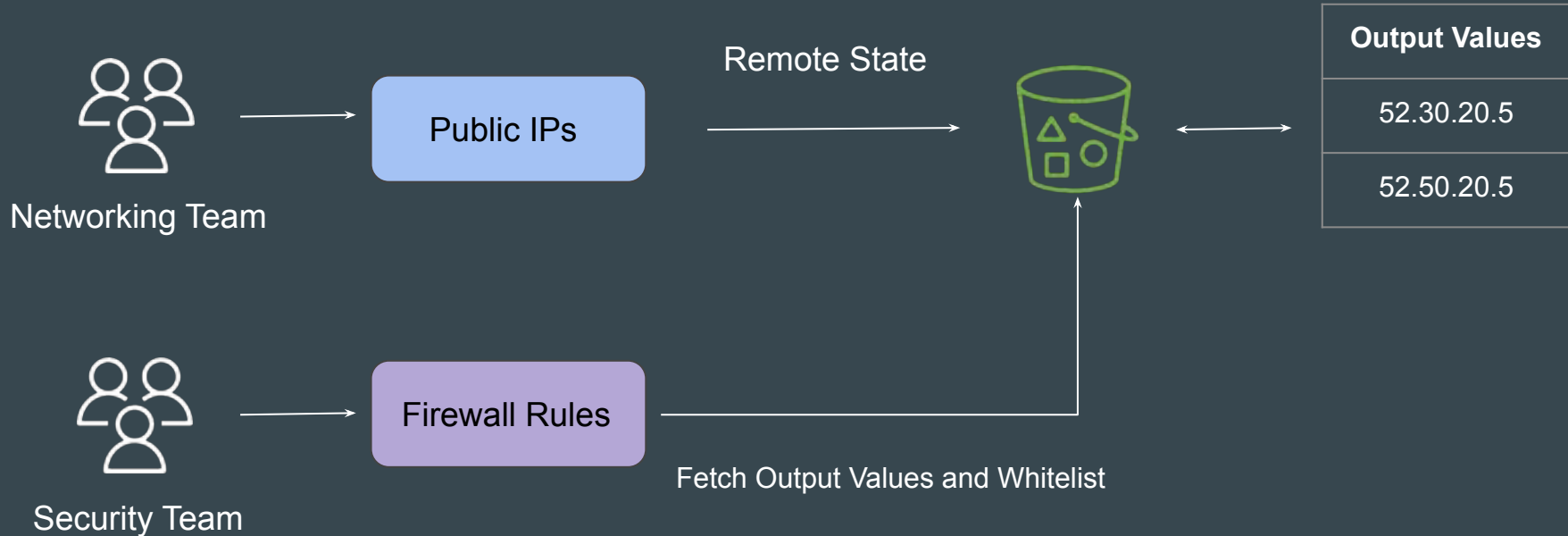
## Point to Note

Health assessments are available in HCP Terraform Standard and Premium editions.

# **HCP Terraform - Run Triggers**

# Setting the Base

Network project contains state file whose outputs contains latest IP addresses that needs whitelisting in security project.



# Possible Solution - Remote State Data Source

The `terraform_remote_state` data source allows us to fetch output values from a specific state backend

```
data "terraform_remote_state" "eip" {  
  backend = "s3"  
  config = {  
    bucket = "kplabs-team-networking-bucket"  
    key    = "eip.tfstate"  
    region = "us-east-1"  
  }  
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = "${data.terraform_remote_state.eip.outputs.eip_addr}/32"  
  from_port         = 443  
  ip_protocol       = "tcp"  
  to_port           = 443  
}
```

Step 1 - Define Remote State Source

Step 2 - Define Data to Fetch

# Challenge with Remote State Data Source

If you updated the Networking space(e.g., new Public IP), the Application space wouldn't know about it. The infrastructure was decoupled, but the process was manual.

# Introducing Run Triggers

If you need to update your workspace every time when a source workspace changes, you can use Run Triggers.

**network-project** Force-unlock New run

ID: [ws-8618427709fa270](#)  
Add workspace description

Running Resources 0 Tags 0 Sentinel v1.54.2 Updated a few seconds ago

### Update main.tf

CURRENT Planned

| Pending confirmation | Resources to be changed | Actions     |
|----------------------|-------------------------|-------------|
| Less than a minute   | +0 -0 -0                | 0 to invoke |

**Run Details** a few seconds ago denicolahosseini triggered a run from GitHub

**Plan finished** a few seconds ago Resources: 0 to add, 0 to change, 0 to destroy | Actions: 0 to invoke Run Triggers, 0 runs to queue

Started a few seconds ago — Finished a few seconds ago

**No changes** Your infrastructure matches the configuration

Sentinel v1.54.2 Download run log

**Outputs** 1 planned to change

Download Sentinel mocks Sentinel mocks can be used for testing your Sentinel policies

Runs will automatically be queued in the following workspaces:  
[security-project](#)

# Important Configuration - 1

In the network-project workspace, you need to enable “Remote state sharing” and designate the workspace with which you want to share the state.

## Remote state sharing

Choose whether this workspace should share state with the entire organization, or only with specific approved workspaces. The `terraform_remote_state` data source relies on state sharing to access workspace outputs.

- ☐ Share with all workspaces in this organization
- ☒ Share with specific workspaces

security-project

1 selected ↕



# Important Configuration - 2

Within the security-project workspace, you need to configure the “Run Triggers” settings and designate the appropriate connected namespace.

## Run Triggers

Run triggers allow you to connect this workspace to one or more source workspaces. These connections allow runs to queue automatically in this workspace on successful apply of runs in any of the source workspaces.

### Auto-apply

Sets this workspace to automatically apply changes for successful runs. If disabled, runs require operator approval.

☒ Auto-apply run triggers ⓘ

### Connected workspaces

[Connect workspace](#)

| Name            | Privileges                                       |
|-----------------|--------------------------------------------------|
| network-project | <span>Connected</span> <span>✕ Disconnect</span> |

## Data Source - tfe\_outputs

HashiCorp recommends using the `tfe_outputs` data source in the HCP Terraform/Enterprise Provider to access remote state outputs in HCP Terraform or Terraform Enterprise.

The `tfe_outputs` data source is more secure because it does not require full access to workspace state to fetch outputs.

```
data "tfe_outputs" "foo" {
  organization = "my-org"
  workspace   = "my-workspace"
}

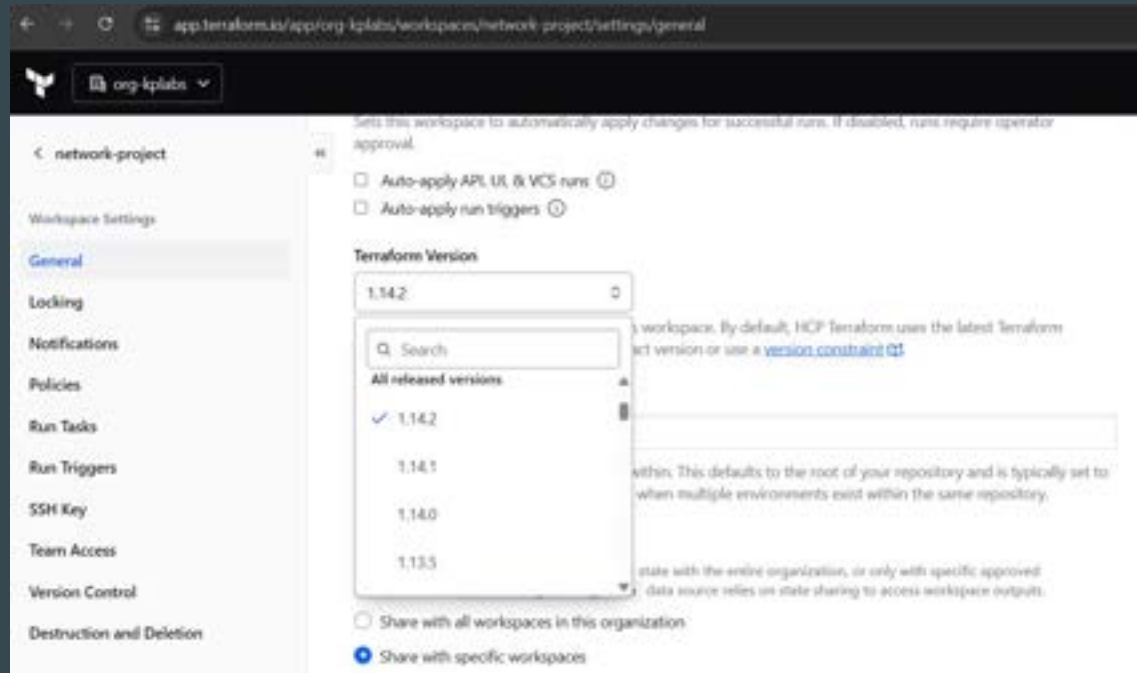
resource "random_id" "vpc_id" {
  keepers = {
    bar = data.tfe_outputs.foo.values.bar
  }

  byte_length = 8
}
```

# **HCP Terraform - Choosing Version**

# Setting the Base

Each HCP Terraform workspace has an assigned Terraform version that it uses for all remote operations in the workspace.



# **Terraform Challenges**

# Key Observations

At this stage, we have been learning core concepts of Terraform step by step.

Whenever learning a new technology, small set of practical projects are always useful to grasp the practical aspects of a technology.



# Introducing Terraform Challenges

With Terraform Challenges, we aim to reduce the gap between learning and gaining practical experience.

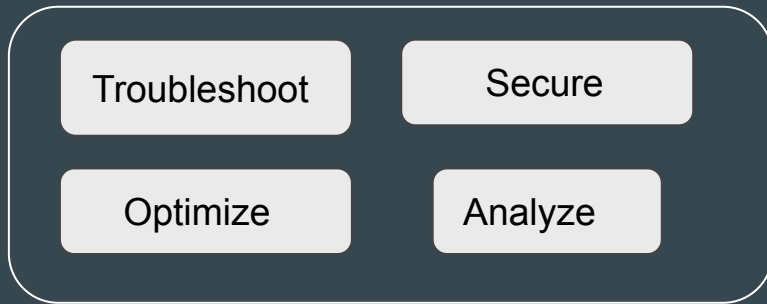


# About the Challenges

Each Challenge will test you in different areas of Terraform that will help you gain some kind of hands-on experience.



Awesome Students



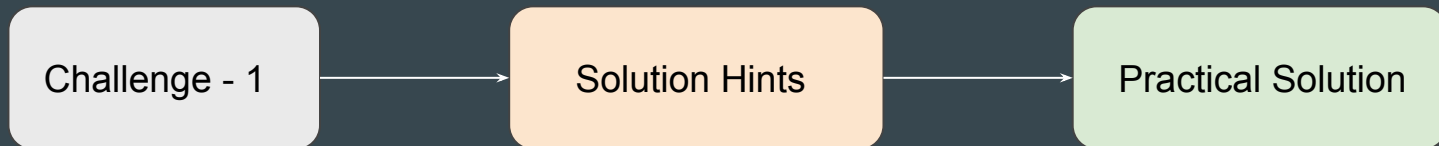
Terraform



# Workflow Steps

We will have multiple sets of challenges.

After each challenge video, we will have a **Solution Hints** video and then the **Practical Solution** video.



# **Terraform Challenge 1**

# Understanding the Challenge

A Developer at Sample Small Corp had created a Terraform File for creating certain resources.

The code was written a few years back based on the old Terraform version.

```
provider "aws" {  
  version = "~> 2.54"  
  region  = "us-east-1"  
  access_key = "AKIAIOSF00NN7EXAMPLE"  
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"  
}  
  
provider "digitalocean" {}  
  
terraform {  
  required_version = "0.12.31"  
}  
  
resource "aws_eip" "kplabs_app_ip" {  
  vpc      = true  
}
```

## What you need to do?

1. Create Infrastructure using the provided code (without modifications).
2. Verify if the code works in the latest version of Terraform and Provider .
3. Modify and Fix the code so that it works with latest version of Terraform.
4. Feel free to edit the code as you like.

# **TF Challenge 1 - Solution Discussion and Hints**

## Hint 1 - Create Infrastructure with Base Code

Based on the initial code given to you, use appropriate version of binaries to ensure infrastructure gets created successfully.

```
terraform_0.12.31  
terraform_0.12.30  
terraform_0.12.29  
terraform_0.12.28  
terraform_0.12.27  
terraform_0.12.26  
terraform_0.12.25  
terraform_0.12.24  
terraform_0.12.23  
terraform_0.12.22
```

## Hint 2 - Access/Secret Keys

There are hardcoded AWS Access/Secret keys with the code.

This MUST be fixed.

```
provider "aws" {  
  version = "~> 2.54"  
  region  = "us-east-1"  
  access_key = "AKIAIOSFODNN7EXAMPLE"  
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxrFc1YEXAMPLEKEY"  
}
```



## Hint 3 - Provider Block

Provider Block is used to define provider version along with 3rd party providers.

Instead, use the new `required_provider` block to define provider and constraints.

```
provider "aws" {  
  version = "~> 2.54"  
  region  = "us-east-1"  
  access_key = "AKIAIOSFODNN7EXAMPLE"  
  secret_key = "wJalrXUtnFEMI/K7MDENG/t  
}  
  
provider "digitalocean" {}
```

```
terraform {  
  required_providers {  
    mycloud = {  
      source = "mycorp/mycloud"  
      version = "~> 1.0"  
    }  
  }  
}
```



## Hint 4 - Terraform Core Version Requirement

Since the challenge states that latest version of Terraform should be used, you can plan to remove the `required_version` block from the code.

```
terraform {  
  |   required_version = "0.12.31"  
}
```

## Hint 5 - Code Upgrade

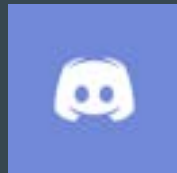
Does the resource block of “aws\_eip” work with the latest version of Terraform?

It can happen that latest AWS provider requires some changes in the aws\_eip resource block. Incorporate these changes to ensure EIP gets created.

```
resource "aws_eip" "kplabs_app_ip" {  
  vpc      = true  
}
```

# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)



[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **Terraform Challenge 2**

# Understanding the Challenge

A sample code has been provided to you that creates certain resources.

You are required to optimize the code following the Best Practices.

```
resource "aws_security_group" "security_group_payment_app" {
  name           = "payment_app"
  description    = "Application Security Group"
  depends_on    = [aws_eip.example]

  # Below ingress allows HTTPS from DEV VPC
  ingress {
    from_port     = 443
    to_port       = 443
    protocol      = "tcp"
    cidr_blocks   = ["172.31.0.0/16"]
  }

  # Below ingress allows APIs access from DEV VPC
  ingress {
    from_port     = 8080
    to_port       = 8080
  }
}
```

## Conditions to Meet

1. Ensure the code is working and resource gets created.
2. Do NOT delete the existing terraform.lock.hcl file. File is free to be modified based on requirements.
3. Demonstrate ability to modify variable “splunk” from 8088 to 8089 without modifying the Terraform code.

## **TF Challenge 2 - Solution Discussion and Hints**

## Hint 1 - Indentation

Indentation issues are present in the code.

Make sure that code is properly indented.

```
# Below ingress allows HTTPS from DEV VPC
ingress {
    from_port      = 443
    to_port        = 443
    protocol       = "tcp"
    cidr_blocks    = ["172.31.0.0/16"]
}
```



## Hint 2 - Using Variables and TFVars

Many values are hard-coded as part of the code.

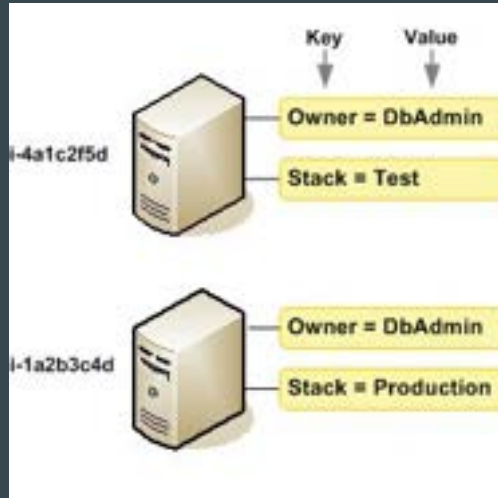
This makes it difficult to modify if code base becomes larger.

```
# Below ingress allows HTTPS from DEV VPC
ingress {
    from_port      = 443
    to_port        = 443
    protocol        = "tcp"
    cidr_blocks     = ["172.31.0.0/16"]
}
```

## Hint 3 - Using Tags

It is important that resources are properly tagged

This will make it easier to identify the resource among all others.



## Hint 4 - Variable Precedence

Consider using appropriate variable precedence to override variables from Terraform code.

## Hint 5 - Right Folder Structure

Having right naming convention for files is important.

Bad Structure: Everything in one single file named main.tf

Good Structure: providers.tf , variables.tf , ec2.tf and so on.

# **Terraform Challenge 3**

# Understanding the Requirements

You will be provided with a variable named `instance_config`

The variable type is `map`.

```
variable "instance_config" {  
  type = map  
  default = {  
    instance1 = { instance_type = "t2.micro", ami = "ami-03a6eaae9938c858c" }  
    instance2 = { instance_type = "t2.micro", ami = "ami-053b0d53c279acc90" }  
  }  
}
```

## Conditions to Meet

1. Based on the values specified in map, EC2 instances should be created accordingly.
2. If key/value is removed from map, EC2 instances should be destroyed accordingly.

## **TF Challenge 3 - Hints**



## Hint 1 - Loops

The requirement indicates that based on key/value specified in map, the resources should be created and destroyed accordingly.

We need to use some kind of loops to achieve this.

```
variable "instance_config" {  
  type = map  
  default = {  
    instance1 = { instance_type = "t2.micro", ami = "ami-03a6eaae9938c858c" }  
    instance2 = { instance_type = "t2.micro", ami = "ami-053b0d53c279acc90" }  
  }  
}
```

## Hint 2 - for\_each

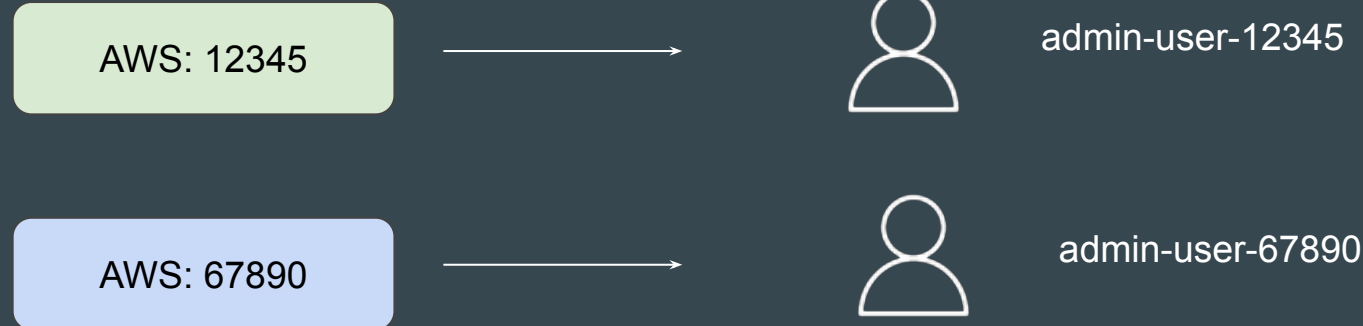
If a resource block includes a `for_each` argument whose value is a **map** or a **set** of strings, Terraform creates one instance for each member of that map or set.

# **Terraform Challenge 4**

# Requirement - 1

Clients wants a code that can create IAM user in AWS account with following syntax:

admin-user-{account-number-of-aws}



## Requirement - 2

Client wants to have a logic that will show names of ALL users in AWS account in the output.



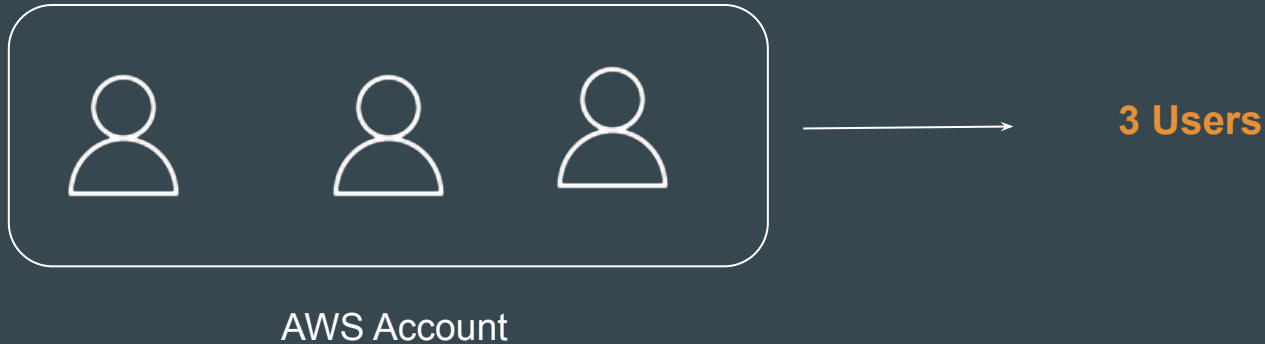
AWS Account



```
+ users = [  
  + "Ankit",  
  + "DemoUser",  
  + "cloudformation-user",  
  + "demo-user",  
  + "kplabs-demo-user",  
  + "terraform",  
]
```

## Requirement - 3

Along with list of users in AWS, client also wants Terraform to show Total number of users in AWS.



## **TF Challenge 4 - Solution Hints**

## Hint 1 - Data Sources

Data Sources allows us to dynamically fetch information from the infrastructure resource or other state backends.

You can try to dynamically fetch information like AWS Account ID, User names using Data Sources.



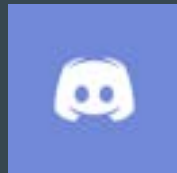
## Hint 2 - Functions

To calculate number of users is outside scope of Data Source.

You need to make use of Terraform Function that can calculate total number of users and output it.

# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)

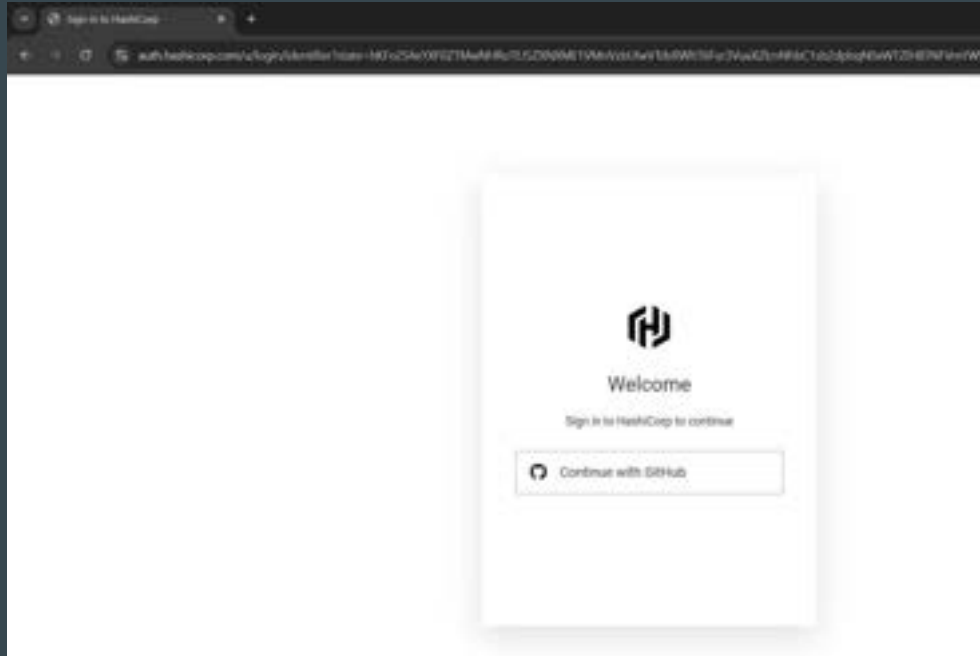


[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **Exam appointment rules and requirements**

# Requirement 1 - GitHub

Exam portals are attached to your **GitHub account**. You will be prompted to login to GitHub when first creating your Exam Portal account.



# Identification Requirements

You must have a **unexpired government-issued physical ID** card to enter your appointment. Digital IDs are not allowed.

All **names on your ID** must exactly match the names in your HashiCorp Certification Exam Portal account



## Point to Note

Your account must have ALL names that are on your ID.

If you have a **middle name**, or more than one given or surname, these must ALL be accounted for in your account.

The names in your account must be in the same language and characters as the names on your ID.

## Few Recommendations

If text in ID are in small font, it might look blurry in the camera.

Keep a backup ID (preferably passport) as text are much larger than traditional identity proofs.

# System Requirements

You will take your HashiCorp certification exam on your computer through an online exam delivery and proctoring system called **Certiverse**.

Your system must comply with all system requirements listed on Certiverse's Knowledge Base and pass the Network Test





# Important Points to Note

HashiCorp recommends using **Google Chrome browser** for exams.

You may only use **one monitor** during your exam appointment.

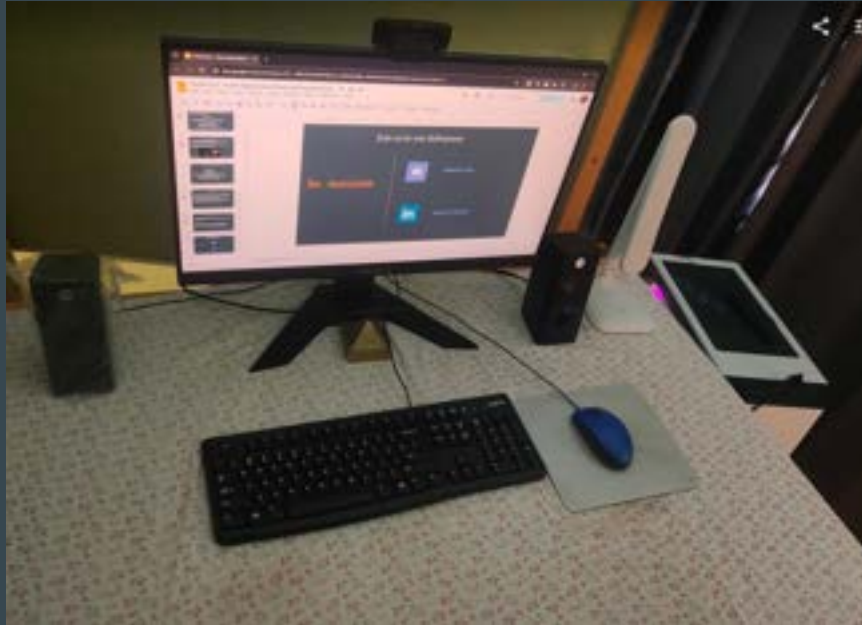
If you are using an external monitor, your laptop screen must be closed.

## Requirement 3 - Physical Space

You will have to scan your exam environment to show compliance with these rules prior to accessing your exam.

| Room Requirements                                                                  |
|------------------------------------------------------------------------------------|
| No one else is permitted to be in your testing room for the duration of your exam. |
| Be sure your space is adequately lit, so the proctor can see you and your space.   |
| Your desk and work area must be clear.                                             |
| Any electronics in the room must not be operational.                               |
| Background noise must be as limited as possible.                                   |
| No phones, smartwatches, or other similar devices are allowed in the room.         |

# Example - My Physical Desk



# Important Behavior Requirements

| Behavior Requirements                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------|
| You may not leave your seat.                                                                                                              |
| If your exam allows a break (Professional-level exams), follow all proctor prompts regarding your break.                                  |
| Talking or communicating outside of the exam environment is not allowed during your appointment, including reading or mouthing questions. |
| Beverages must be in clear containers with no writing.                                                                                    |

## Points to Note

To make your check in easy, **remove as much clutter as you find** reasonable.

Your proctor will be looking for places **recording devices** could be hidden; the fewer of these present, the easier your check in will be.

If your proctor has a question about a part of your room, they may ask to examine the area via the webcam to protect exam security

# Scheduling Requirements

If you need to **cancel or reschedule your appointment**, you must do so at least **48 hours in advance**.

Cancellation with less than 48 hours notice or not appearing for your appointment without documentation may result in loss of your exam fees.

## Point to Note

Make sure there are **no notification** that appear on your screen while giving the exams.

Always **verify the updated guidelines released by HashiCorp** for the exams to ensure you get the latest update before sitting for the exams.

# **Exam Preparation - Part 1**



# Providers in AWS

A provider is responsible for understanding API interactions and exposing resources.

When we run **terraform init**, plugins required for the provider are automatically downloaded and saved locally to a **.terraform** directory.

```
C:\Users\bealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

## Interesting Question

Is provider block {..} mandatory to be added as part of your Terraform configuration? Yes/No

```
provider "aws" {  
    region = "us-east-1"  
}
```

## Two Pointers from Documentation

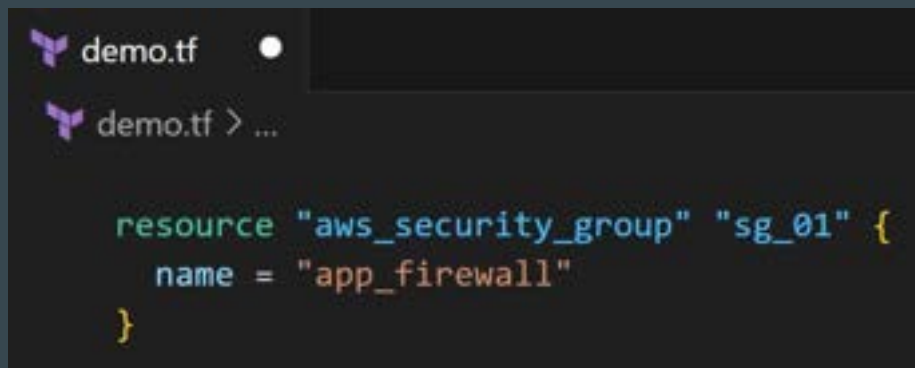
All Terraform configurations **must** declare which providers they require so that Terraform can install and use them.

A provider block may be omitted if its contents would otherwise be empty.

## Concluding this Question

If you plan to explicitly add some contents to provider {} like region, credentials, then defining this block is required.

Otherwise, even if you skip, the terraform apply will work fine.



```
demo.tf
demo.tf > ...

resource "aws_security_group" "sg_01" {
  name = "app_firewall"
}
```

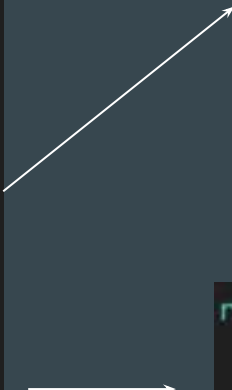
# Alias in Providers

**alias** can be used for using the same provider with different configurations for different resources

```
provider "aws" {  
  region = "ap-southeast-1"  
}
```

```
provider "aws" {  
  alias = "mumbai"  
  region = "ap-south-1"  
}
```

```
provider "aws" {  
  alias = "usa"  
  region = "us-east-1"  
}
```



```
resource "aws_instance" "myec2" {  
  provider = aws.mumbai  
  ami      = "ami-08a0d1e16fc3f61ea"  
  instance_type = "t2.micro"  
}
```

```
resource "aws_security_group" "allow_tls" {  
  provider = aws.usa  
  name     = "staging_firewall"  
}
```

## Point to Note - Providers

It is a good practice to store the credentials outside of Terraform configuration, such as in Environment Variables.

# Terraform Settings

Terraform Settings are used to configure project-specific Terraform behaviours, such as requiring a minimum Terraform version to apply to your configuration.

```
terraform {  
  required_version = "2.0"  
  
  required_providers {  
    aws = {  
      version = "5.54.1"  
      source  = "hashicorp/aws"  
    }  
  }  
}
```

| Options That Can be Defined   |
|-------------------------------|
| Required Terraform Version    |
| Required Provider and Version |
| BackEnd Configuration         |
| Experimental Features         |

## Point to Note

You cannot define configuration related to regions, Access/Secret keys inside `required_provider` block.

For these, you have to use a `provider {}` block.



# Versioning Constraint

Version Constraint allows you to specify mix of multiple operators to select a suitable version of Terraform and Provider Plugins.

| Operators and Examples           | Description                       |
|----------------------------------|-----------------------------------|
| <code>&gt;=1.0</code>            | Greater than equal to the version |
| <code>&lt;=1.0</code>            | Less than equal to the version    |
| <code>~&gt;2.0</code>            | Any version in the 2.X range.     |
| <code>&gt;=2.10,&lt;=2.30</code> | Any version between 2.10 and 2.30 |

# Provider Tiers

There are 3 primary type of provider tiers in Terraform.

| Provider Tiers | Description                                                                                  |
|----------------|----------------------------------------------------------------------------------------------|
| Official       | Owned and Maintained by HashiCorp.                                                           |
| Partner        | Owned and Maintained by Technology Company that maintains direct partnership with HashiCorp. |
| Community      | Owned and Maintained by Individual Contributors.                                             |

# Terraform Init

The **terraform init** command initializes a working directory.

Initialization includes Installing Provider Plugins, Backend initialization, Copy Source Module, etc.

This is the first command that should be run after writing a new Terraform configuration. It is safe to run multiple times.

```
C:\Users\beal\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

# Terraform Init Upgrade

The `terraform init -upgrade` installs the latest module and provider versions allowed within configured constraints.

If you have latest provider plugins installed and if you define new version constraints that matches different version, you will have to run the `init -upgrade`

```
C:\kplabs-terraform>terraform plan
```

```
Error: Inconsistent dependency lock file
```

```
The following dependency selections recorded in the lock file are inconsistent with the current configuration:
```

```
- provider registry.terraform.io/hashicorp/aws: locked version selection 5.54.1 doesn't match the updated version  
ints "5.50.1"
```

```
To update the locked dependency selections to match a changed configuration, run:
```

```
terraform init -upgrade
```

# Terraform Plan

The **terraform plan** command is used to create an execution plan.

The infrastructure is not modified as part of this plan.

The **state file is not modified** even when it detects drift in real-world and current infrastructure.

```
C:\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following
following symbols:
+ create

Terraform will perform the following actions:

# aws_security_group.sg_01 will be created
+ resource "aws_security_group" "sg_01" {
+   arn              = (known after apply)
+   description      = "Managed by Terraform"
+   egress           = (known after apply)
+   id               = (known after apply)
+   ingress          = (known after apply)
+   name             = "app_firewall"
+   name_prefix      = (known after apply)
+   owner_id         = (known after apply)
```

## Saving Plan to File

You can use the optional `-out=FILE` option to save the generated plan to a file on disk, which you can later execute by passing the file to terraform apply as an extra argument.

This ensures consistent infrastructure as defined in the plan.

`terraform plan -out ec2.plan`



```
Saved the plan to: ec2.plan
```

```
To perform exactly these actions, run the following command to apply:  
terraform apply "ec2.plan"
```

# Terraform Apply

`terraform apply` command is used to apply the changes required to reach the desired state of the configuration.

The state file gets modified in this command.

Name of state file = `terraform.tfstate`

Terraform Apply can change, destroy and provision resources but cannot import any resource.

# Terraform Destroy

`terraform destroy` command is used to destroy the Terraform-managed infrastructure.

`terraform destroy` command is not the only command through which infrastructure can be destroyed.



# Terraform Format

`terraform fmt` command is used to rewrite Terraform configuration files to a canonical format and style. It will directly perform “write” operation and not “read”

For use-case, where the all configuration written by team members needs to have a proper style of code, `terraform fmt` can be used.

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "test-ec2"  
  }  
}
```

Before



```
resource "aws_instance" "myec2" {  
  ami           = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "test-ec2"  
  }  
}
```

After

# terraform fmt options

You have to keep a note of two important flags for terraform fmt command

| Command    | Description                                                                                                     |
|------------|-----------------------------------------------------------------------------------------------------------------|
| -check     | Check if the input is formatted. Files are not modified.                                                        |
| -recursive | Also process files in subdirectories. By default, only the given directory (or current directory) is processed. |

## **Exam Preparation - Part 2**

# Terraform Validate

`terraform validate` command validates the configuration files in a directory.

Requires an initialized working directory with any referenced plugins and modules installed.

`terraform plan` uses implied validation check.

```
C:\kplabs-terraform>terraform validate  
Success! The configuration is valid.
```

# Terraform Refresh

`terraform refresh` command reads the current settings from all managed remote objects and updates the Terraform state to match.

This won't modify your real remote objects, but it will modify the Terraform state.

This command is deprecated, because its default behavior is unsafe.

# Resource Blocks

A resource block declares a resource of a **given type** ("aws\_instance") with a **given local name** ("web").

Resource type and Name together serve as an identifier for a given resource and so must be unique.

Address of the following resource is: aws\_instance.web

```
resource "aws_instance" "web" {  
  ami          = ami-123  
  instance_type = "t2.micro"  
}
```

# Important Terminology

```
resource "aws_instance" "myec2" {  
  ami          = "ami-123"  
  instance_type = "t2.micro"  
}
```



|              |                             |
|--------------|-----------------------------|
| aws_instance | Resource Type               |
| myec2        | Local name for the resource |
| ami          | Argument Name               |
| ami-123      | Argument value              |

# Data Types in Terraform

| Data Types | Description                                                                           |
|------------|---------------------------------------------------------------------------------------|
| string     | a sequence of Unicode characters representing some text, like "hello".                |
| number     | A Numeric value                                                                       |
| bool       | a boolean value, either true or false                                                 |
| list       | a sequence of values, like ["us-west-1a", "us-west-1c"]                               |
| set        | a collection of unique values that do not have any secondary identifiers or ordering. |
| map        | a group of values identified by named labels, like {name = "Mabel", age = 52}.        |
| null       | a value that represents absence or omission.                                          |



## Point to Note - Data Types

Array data types are not supported in Terraform

# State Management

The **terraform state** command is used for advanced state management

| Sub-Commands     | Description                                                      |
|------------------|------------------------------------------------------------------|
| list             | List resources within terraform state file.                      |
| mv               | Moves item with terraform state.                                 |
| pull             | Manually download and output the state from remote state.        |
| push             | Manually upload a local state file to remote state.              |
| rm               | Remove items from the Terraform state                            |
| show             | Show the attributes of a single resource in the state.           |
| replace-provider | Used to replace the provider for resources in a Terraform state. |

## Use-Case - Removing Item from State

There are 5 EC2 instances created through Terraform using count =5

The Team wants to destroy all the EC2 instances except the second instance with the resource address of `aws_instance.web[1]`.

1. This is not possible since the instance created through Count.
2. Apply taint on the EC2 instance.
3. Use `terraform state rm aws_instance.web[1]`
4. Use `terraform state mv aws_instance.web[1]`
5. None of the Above

# Debugging in Terraform

Terraform has detailed logs that can be enabled by setting the `TF_LOG` environment variable to any value.

You can set `TF_LOG` to one of the log levels `TRACE`, `DEBUG`, `INFO`, `WARN` or `ERROR` to change the verbosity of the logs.

To persist logged output, you can set `TF_LOG_PATH`

# Terraform Import

Allows importing existing infrastructure to Terraform.

Automatic code generation for imported resources is supported.

You can use **import blocks** to import more than one resource at a time.



# Import Workflow Steps

```
import {  
  to = aws_security_group.mysg  
  id = "sg-07f13feb262ba8b6f"  
}  
  
# terraform plan -generate-config-out="mysg.tf"  
  
# terraform apply -auto-approve
```

# Local Values

Locals are used when you want to avoid repeating the same expression multiple times.

Local values are created by a **locals** block (plural), but you reference them as attributes on an object named **local** (singular)

Local value can reference values from other variables, locals etc.

```
locals {  
  common_tags = {  
    Team = "payments-team"  
  }  
}
```

# Terraform Workspace

Terraform workspaces enable us to manage multiple sets of deployments from the same sets of configuration files.

State File Directory = terraform.tfstate.d

**Not suitable** for isolation for strong separation between workspace (stage/prod)

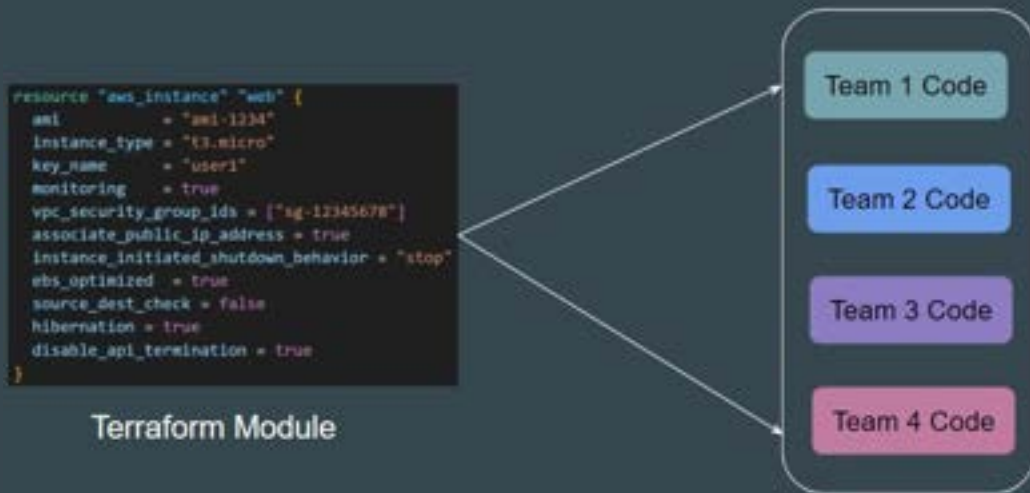
| Use-Case                       | Command                         |
|--------------------------------|---------------------------------|
| Create New Workspace           | terraform workspace new kplabs  |
| Switch to a specific Workspace | terraform workspace select prod |



# Terraform Modules

Terraform Modules allow us to centralize the resource configuration, and it makes it easier for multiple projects to re-use the Terraform code.

Instead of writing code from scratch, we can use multiple ready-made modules available.



# Calling a Module

Module source code can be present in a wide variety of locations including:

GitHub, Local Paths, Terraform Registry, S3 Buckets, HTTP URLs

To reference a module, you need to make use of **module** block and **source**

Terraform uses this during the module installation step of terraform init to download the source code to a directory on local disk so that other Terraform commands can use it.

```
module "ec2" {  
  source = "https://example.com/vpc-module.zip"  
}
```

## Example 1 - Local Paths

Local paths are used to reference to a module that is available in local filesystem.

A local path must begin with either `./` or `../` to indicate that a local path.

Modules sourced from local paths do **NOT** support versions.

```
module "ec2" {  
    source = "../modules/ec2"  
}
```

## Example 2 - Generic Git Repository

Arbitrary Git repositories can be used by prefixing the address with the special `git::` prefix.

```
module "vpc" {  
  source = "git::https://example.com/vpc.git"  
}
```

# Root vs Child Modules

**Root Module** resides in the main working directory of your Terraform configuration. This is the entry point for your infrastructure definition

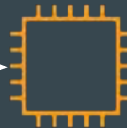
A module that has been called by another module is often referred to as a **child module**.

```
resource "aws_instance" "myec2" {  
  ami = var.ami  
  instance_type = var.instance_type  
}
```

Child Module

```
module "ec2" {  
  source = "../modules/ec2"  
}
```

Root Module



# Module Outputs

A child module can use outputs to expose a subset of its resource attributes to a parent module.

**Format:** <MODULE NAME>.<OUTPUT NAME>

```
resource "aws_instance" "myec2" {  
    ami = var.ami  
    instance_type = var.instance_type  
}  
  
output "instance_id" {  
    value = aws_instance.myec2.id  
}
```



```
module "ec2" {  
    source = "../../modules/ec2"  
}  
  
resource "aws_eip" "lb" {  
    instance = module.ec2.instance_id  
    domain   = "vpc"  
}
```

# Module Versioning

When using modules installed from a module registry, HashiCorp recommends explicitly constraining the acceptable version numbers to avoid unexpected or unwanted changes.

It is **not mandatory** to specify a version argument.

```
module "ec2-instance" {  
  source = "terraform-aws-modules/ec2-instance/aws"  
  version = "5.6.1"  
}
```

# Terraform Registry

- Hosts a broad collection of publicly available Terraform modules.
- Each Terraform module has an associated address.
- A module address has the syntax `hostname/namespace/name/system`

The `hostname/` portion of a module is optional, and if omitted, it defaults to `registry.terraform.io/`.

```
module "ec2-instance" {  
  source = "terraform-aws-modules/ec2-instance/aws"  
  version = "5.6.1"  
}
```



# Functions in Terraform

The Terraform language includes a number of built-in functions that you can use to transform and combine values.

**NO SUPPORT** for User-Defined Functions.

| Function Categories   | Functions Available                           |
|-----------------------|-----------------------------------------------|
| Numeric Functions     | abs, ceil, floor, max, min                    |
| String Functions      | concat, replace, split, join, tolower,toupper |
| Collection Functions  | element, keys, length, merge, sort, slice     |
| Filesystem Functions: | file, filebase64, dirname                     |

# Lookup function

lookup retrieves the value of a single element from a map, given its key. If the given key does not exist, the given default value is returned instead.

```
> lookup({a="ay", b="bee"}, "a", "what?")  
ay  
> lookup({a="ay", b="bee"}, "c", "what?")  
what?
```

# Zipmap function

zipmap constructs a map from a list of keys and a corresponding list of values.

```
←[J> zipmap(["pineapple","oranges","strawberry"], ["yellow","orange","red"])  
{  
  "oranges" = "orange"  
  "pineapple" = "yellow"  
  "strawberry" = "red"  
}
```

# Index function

index finds the element index for a given value in a list.

```
C:\kplabs-terraform>terraform console  
> index(["a", "b", "c"], "b")  
1
```

# Element Function

element retrieves a single element from a list.

Format: `element(list, index)`

```
> element(["a", "b", "c"], 1)
b
```

# Testing Element Function

Code:    Name = element(var.tags,count.index)

```
variable "tags" {  
  type = list  
  default = ["firstec2","secondec2"]  
}
```



The diagram consists of two white arrows originating from the bottom of the variable definition block. One arrow points diagonally down and to the left towards the first code block, and the other points diagonally down and to the right towards the second code block.

```
> element(["firstec2","secondec2"],0)  
"firstec2"
```

```
> element(["firstec2","secondec2"],1)  
"secondec2"
```

# toiset Function

toiset function will convert the list of values to SET

```
> toiset(["a", "b", "c","a"])  
toiset(  
  "a",  
  "b",  
  "c",  
])
```

# TimeStamp Function

timestamp returns a UTC timestamp string in RFC 3339 format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```



# File Function

File function can reduce the overall Terraform code size by loading contents from external sources during terraform operations.

```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": "ec2:*",
        "Effect": "Allow",
        "Resource": "*"
      },
      {
        "Effect": "Allow",
        "Action": "elasticloadbalancing:*",
        "Resource": "*"
      }
    ]
  })
}
```

Before



```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = file("./ec2-policy.json")
}
```

After

# Meta Arguments in Terraform

Terraform allows us to include meta-arguments within the resource block, which allows some details of this standard resource behaviour to be customized on a per-resource basis.

Inside resource block



```
resource "aws_instance" "myec2" {  
    ami = "ami-00c39f71452c08778"  
    instance_type = "t2.micro"  
  
    lifecycle {  
        ignore_changes = [tags]  
    }  
}
```

# Different Meta-Arguments

| Meta-Argument | Description                                                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| depends_on    | Handle hidden resource or module dependencies that Terraform cannot automatically infer.                                                               |
| count         | Accepts a whole number, and creates that many instances of the resource                                                                                |
| for_each      | Accepts a map or a set of strings, and creates an instance for each item in that map or set.                                                           |
| lifecycle     | Allows modification of the resource lifecycle.                                                                                                         |
| provider      | Specifies which provider configuration to use for a resource, overriding Terraform's default behavior of selecting one based on the resource type name |

# Lifecycle Meta-Argument

Some details of the default resource behaviour can be customized using the special nested lifecycle block within a resource block body:

```
resource "aws_instance" "myec2" {  
  ami = "ami-0f34c5ae932e6f0e4"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "HelloEarth"  
  }  
  
  lifecycle {  
    ignore_changes = [tags]  
  }  
}
```

# Arguments Available for LifeCycle Block

There are four argument available within lifecycle block.

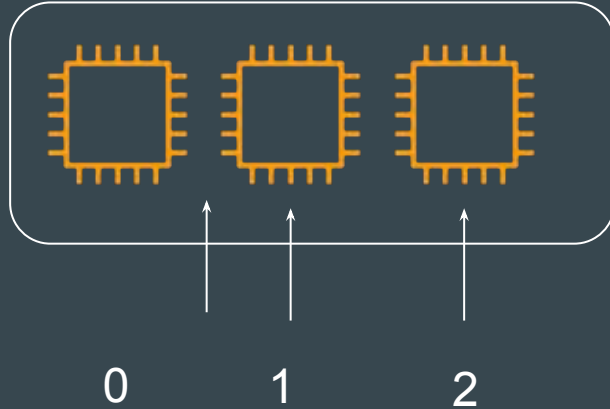
| Arguments             | Description                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| create_before_destroy | New replacement object is created first, and the prior object is destroyed after the replacement is created.         |
| prevent_destroy       | Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource |
| ignore_changes        | Ignore certain changes to the live resource that does not match the configuration.                                   |
| replace_triggered_by  | Replaces the resource when any of the referenced items change                                                        |

# Count and Count Index

The count argument accepts a whole number, and creates that many instances of the resource.

count.index — The distinct index number (starting with 0) corresponding to this instance.

```
resource "aws_instance" "myec2" {  
  ami = "ami-00c39f71452c08778"  
  instance_type = "t2.micro"  
  count = 3  
}
```



# **Exam Preparation - Part 3**

## Find the Issue - Use-Cases

You can expect a use case in exam with a sample Terraform code, and you must find what should be removed as part of Terraform best practice.

```
terraform {  
  backend "s3" {  
    bucket = "mybucket"  
    key     = "path/to/my/key"  
    region = "us-east-1"  
    access_key = "AKIAIOSFODNN7EXAMPLE"  
    aecret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEK"  
  }  
}
```

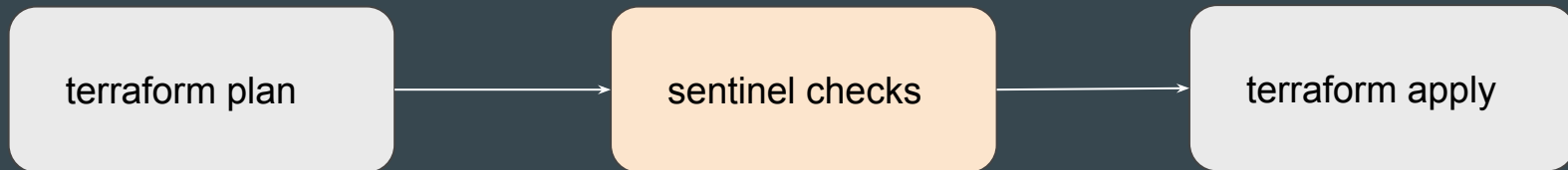


# Sentinel

Sentinel is an embedded policy-as-code framework integrated with the HashiCorp Enterprise products. Sentinel is a proactive service.

Can be used for various use-cases like:

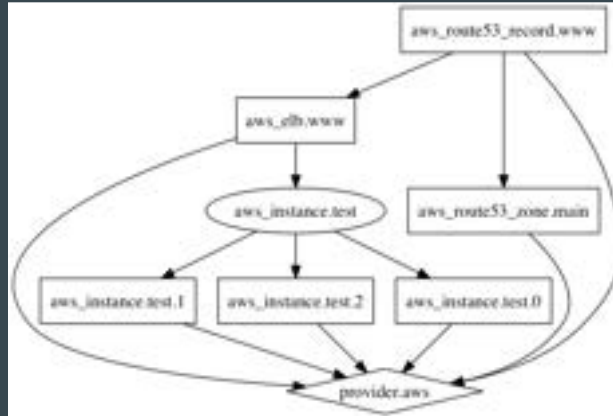
- Verify if EC2 instance has tags.
- Verify if the S3 bucket has encryption enabled.



# Terraform Graph

Terraform graph refers to a **visual representation of the dependency relationships** between resources defined in your Terraform configuration.

The output of terraform graph is in the **DOT format**, which can easily be converted to an image.



# Input Variables

Terraform input variables are used to pass certain values from outside of the configuration

| Name     | Value           |
|----------|-----------------|
| vpn_ip   | 101.0.62.210/32 |
| app_port | 8080            |

Variable File

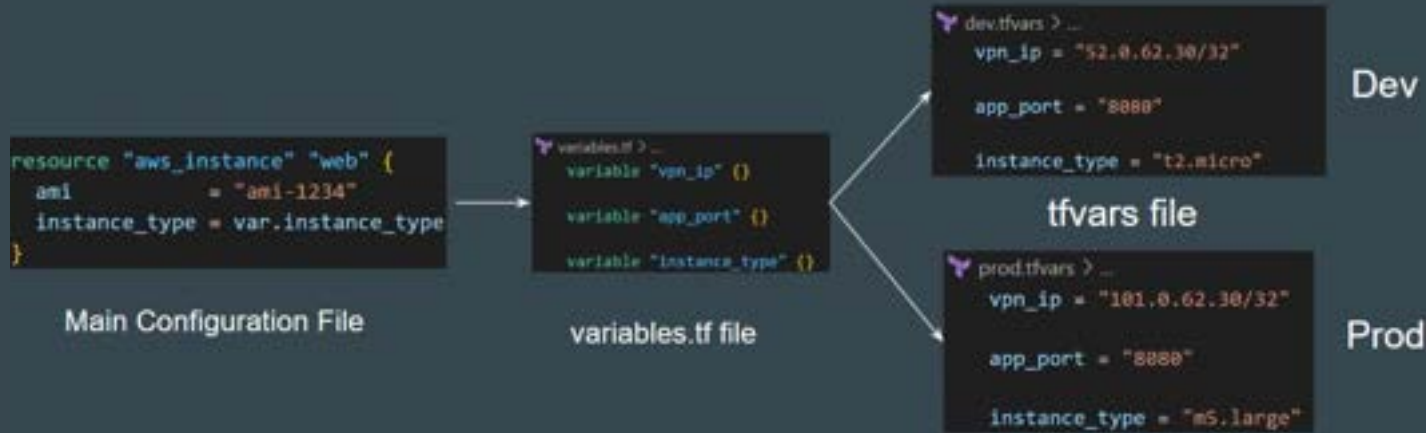
```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {  
  security_group_id = aws_security_group.allow_tls.id  
  cidr_ipv4         = var.vpn_ip  
  from_port         = var.app_port  
  ip_protocol       = "tcp"  
  to_port           = var.app_port  
}
```



# Terraform TFVARS

terraform.tfvars file can be used to define value to all the variables.

This approach leads to easier setup for multi-project deployments.



# Selecting tfvars File

If you have multiple variable definitions file (\*.tfvars) file, you can manually define the file to use during the command line.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"
```

```
Terraform used the selected providers to generate the following execution plan.  
following symbols:
```

```
+ create
```

```
Terraform will perform the following actions:
```

```
# aws_instance.web will be created
```

```
+ resource "aws_instance" "web" {  
  + ami              = "ami-1234"  
  + arn              = (known after apply)  
  + associate_public_ip_address = (known after apply)  
  + availability_zone = (known after apply)  
  + cpu_core_count    = (known after apply)  
  + cpu_threads_per_core = (known after apply)  
  + disable_api_stop   = (known after apply)  
  + disable_api_termination = (known after apply)  
  + ebs_optimized      = (known after apply)
```

# Declaring Variable Values

When variables are declared in your configuration, they can be set in a number of ways:

1. Variable Defaults.
2. Variable Definition File (\*.tfvars)
3. Environment Variables
4. Setting Variables in the Command Line.

```
variables.tf > ...  
  
variable "app_port" {  
    default = "8080"  
}
```

```
prod.tfvars > ...  
vpn_ip = "101.0.62.30/32"  
  
app_port = "8080"  
  
instance_type = "m5.large"
```

# Setting Variable in Command Line

To specify individual variables on the command line, use the `-var` option when running the `terraform plan` and `terraform apply` commands:

```
C:\Users\zealiv\kplabs-terraform>terraform plan -var="instance_type=m5.large"

Terraform used the selected providers to generate the following execution plan. Resource
following symbols:
  - create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-1234"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + get_password_data      = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "m5.large"
```

# Setting Variable through Environment Variables

Terraform searches the environment of its own process for environment variables named `TF_VAR_` followed by the name of a declared variable.

```
C:\Users\zealv\kplabs-terraform>echo %TF_VAR_instance_type%
t2.large

C:\Users\zealv\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following execution plan
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                        = "ami-1234"
  + arn                       = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone          = (known after apply)
```



# Variable Definition Precedence

Terraform loads variables in the following order, with later sources taking precedence over earlier ones:

1. Environment variables
2. The terraform.tfvars file, if present.
3. The terraform.tfvars.json file, if present.
4. Any \*.auto.tfvars or \*.auto.tfvars.json files, processed in lexical order of their filenames.
5. Any -var and -var-file options on the command line

## Variables with undefined values

If you have variables with undefined values, it will **NOT** directly result in an error.

Terraform will ask you to supply the value associated with them.

```
C:\Users\zealv\kplabs-terraform>terraform plan
var.instance_type
  Enter a value:
```

# Not Allowed Variable Names

We cannot use all words within variable names.

Terraform reserves some additional names that can no longer be used as input variable names for modules. These reserved names are:

- count
- depends\_on
- for\_each
- lifecycle
- providers
- source

```
Error: Invalid variable name
```

```
on demo.tf line 6, in variable "count":  
6: variable "count" {
```

```
The variable name "count" is reserved due to its special meaning
```

## Points to Note - State File

Terraform state file generally stores details about the resources that it manages.

Various aspects like “Input Variables” are not stored.

Output value will be stored in state file but not the description

```
variable "instance_type" {  
  type =    string  
  default = "t2.micro"  
  description = "Default EC2 Instance type for Module"  
}  
  
output "variable" {  
  value = var.instance_type  
  description = "Instance Type of EC2"  
}
```

# Terraform Console

Terraform Console provides an interactive environment specifically **designed to test functions** and experiment with expressions before integrating them into your main code.

```
C:\Users\zealv\kplabs-terraform>terraform console  
> max(10,20,30)  
30
```

# Dependency Lock File

Terraform dependency lock file allows us to lock to a specific version of the provider. Name is terraform.lock.hcl. For tracking provider dependencies.

If a particular provider already has a selection recorded in the lock file, Terraform will always re-select that version for installation, even if a newer version has become available.

You can override that behavior by running `terraform init -upgrade`

# Dependencies - Implicit

With **implicit dependency**, Terraform can automatically find references of the object, and create an implicit ordering requirement between the two resources.

In the following screenshot, Terraform will create EC2 first before EIP.

```
resource "aws_eip" "lb" {  
  domain    = "vpc"  
  instance = aws_instance.myec2.id  
}  
  
resource "aws_instance" "myec2" {  
  ami           = "ami-123"  
  instance_type = "t2.micro"  
}
```

# Dependencies - Explicit

Explicitly specifying a dependency is only necessary when a resource relies on some other resource's behavior but doesn't access any of that resource's data in its arguments.

Uses the `depends_on` meta-argument

```
resource "aws_iam_role" "example" {  
  name = "example"  
  assume_role_policy = "..."  
}  
  
resource "aws_instance" "example" {  
  ami           = "ami-a1b2c3d4"  
  instance_type = "t2.micro"  
  depends_on = [aws_iam_role_policy.example]  
}
```



# Data Sources

Data sources allow Terraform to **use / fetch** information defined outside of Terraform

```
data "aws_instances" "example" {}
```

```
data "local_file" "foo" {  
  filename = "${path.module}/demo.txt"  
}
```

# Terraform Enterprise

Terraform Enterprise provides several added advantages compared to Terraform Cloud.

Some of these include:

- Single Sign-On
- Auditing
- Private Data Center Networking
- Clustering

Team & Governance features are not available for Terraform Cloud Free (Paid)

## Remote Backend

The remote backend stores Terraform state and may be used to run operations in Terraform Cloud.

When using full remote operations, operations like `terraform plan` or `terraform apply` can be executed in Terraform Cloud's run environment, with log output streaming to the local terminal.

The remote backend was the primary implementation of HCP Terraform's CLI-driven run workflow for Terraform versions 0.11.13 through 1.0.x. We recommend using the native cloud integration for Terraform versions 1.1 or later, as it provides an improved user experience and various enhancements.

# Points to Note

HCP = HashiCorp Cloud Platform

Secure Variable Storage is available in Terraform Enterprise and Cloud but not in the normal version of Terraform.

Terraform Cloud / Enterprise comes with a Private Module registry which allows organizations to restrict access based on requirements.

Terraform Cloud provides the feature of Remote State storage.

Encryption of state file is available

## Points to Note

In a HCP workspace linked to a VCS repository, runs start automatically when you merge or commit changes to version control.

A workspace is linked to one branch of a VCS repository and ignores changes to other branches.

To protect secret values in HCP, you can mark any Terraform or environment variable as sensitive data by clicking its Sensitive checkbox that is visible during editing. Marking a variable as sensitive makes it write-only and prevents all users (including you) from viewing its value

# Sensitive Values in HCP

To protect secret values in HCP, you can mark any Terraform or environment variable as sensitive data by clicking its Sensitive checkbox that is visible during editing.

Marking a variable as sensitive makes it write-only and prevents all users (including you) from viewing its value

The screenshot shows the 'Variable set scope' configuration page in the HashiCorp Control Plane. It includes sections for 'Add new variable', 'Variables', and 'Select variable category'. The 'Add new variable' section has a 'Name' field with the value 'my\_secret' and a 'Value' field with the value 'secret'. The 'Sensitive' checkbox is checked. The 'Description' field is empty. The 'Add variable' button is at the bottom left.

Variable set scope

☒ Add new variable  
Use new and existing variables in the expression set across this variable set.

☐ Apply to specific projects and environments

Variables

You can add any number of [Terraform](#) and [Environment](#) variables. Variables are used in expressions for all other and global operations in the specified environments.

| Name      | Value  | Category    |
|-----------|--------|-------------|
| my_secret | secret | Environment |

Select variable category

☒ Terraform variable  
These variables are used in the Terraform configuration. They are used in the configuration to set the value of other variables.

☐ Environment variable  
These variables are used in the Terraform configuration to set the value of other variables.

Name: my\_secret Value: secret Sensitive: ☒

Description (Optional)

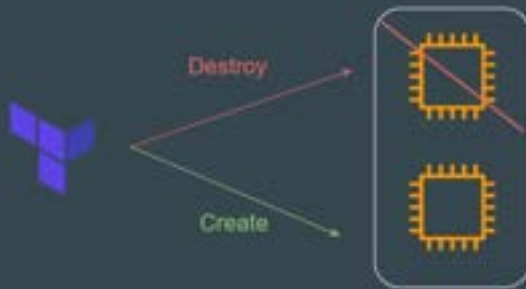
Add variable Cancel

## Recreating the Resource

The **-replace** option with terraform apply to force Terraform to replace an object even though there are no configuration changes that would require it.

```
terraform apply -replace="aws_instance.web"
```

A similar kind of functionality was achieved using the **terraform taint** command in older versions of Terraform. Not recommended now.



# Benefits of IAC Tool

There are three primary benefits of Infrastructure as Code tools:

Automation, Versioning, and Reusability.

Various IAC Tools Available in the market:

- Terraform
- CloudFormation
- Azure Resource Manager
- Google Cloud Deployment Manager



# terraform output

The terraform output command is used to extract the value of an output variable from the state file.

```
variable "instance_type" {  
  type =    string  
  default = "t2.micro"  
  description = "Default EC2 Instance type for Module"  
}  
  
output "instance_type" {  
  value = var.instance_type  
  description = "Instance Type of EC2"  
}
```



```
C:\kplabs-terraform>terraform output  
instance_type = "t2.small"
```

# Module Source and Git Branches

By default, Terraform will clone and use the default branch (referenced by HEAD) in the selected repository.

You can override this using the ref argument.

Format: ?ref=<version-number>

```
module "vpc" {  
  source = "git::https://example.com/vpc.git?ref=v1.2.0"  
}
```

# Splat Expressions

Splat Expression allows us to get a list of all the attributes.

Resources that use the `for_each` argument will appear in expressions as a map of objects, so you can't use splat expressions with those resources.

```
resource "aws_iam_user" "lb" {  
  name = "iamuser.${count.index}"  
  count = 3  
  path = "/system/"  
}  
  
output "arns" {  
  value = aws_iam_user.lb[*].arn  
}
```

# Important Documentation Reference - Splat

A *splat expression* provides a more concise way to express a common operation that could otherwise be performed with a `for` expression.

If `var.list` is a list of objects that all have an attribute `id`, then a list of the ids could be produced with the following `for` expression:

```
[for o in var.list : o.id]
```



This is equivalent to the following *splat expression*:

```
var.list[*].id
```



## Point to Note

Will this code block display the names of all the IAM usernames created?

Answer = NO

```
resource "aws_iam_user" "lb" {  
  name = "iamuser.${count.index}"  
  count = 3  
  path = "/system/"  
}  
  
output "arns" {  
  value = aws_iam_user.lb[*].name  
}
```

# Legacy Splat Expression

Earlier versions of the Terraform language had a slightly different version of splat expressions, which Terraform continues to support for backward compatibility.

The legacy "attribute-only" splat expressions use the sequence `.*`, instead of `[*]`:

```
resource "aws_iam_user" "lb" {
  name = "iamuser.${count.index}"
  count = 3
  path = "/system/"
}

output "arns" {
  value = aws_iam_user.lb.*.name
}
```

# Fetching Values from List

To fetch the `instance_type` value of `m5.xlarge` from the list, you can reference to the key of 1

`var.size[1]`

```
variable "size" {  
  type = list  
  default = ["m5.large", "m5.xlarge", "t2.medium"]  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-082b5a644766e0e6f"  
  instance_type = var.size[1]  
}
```

# Fetching Values from Map

To reference the “t2.small” instance type from the below map, the following approaches need to be used:

```
var.types["ap-south-1"]
```

```
variable "types" {  
  type = map  
  default = {  
    us-east-1 = "t2.micro"  
    us-west-2 = "t2.nano"  
    ap-south-1 = "t2.small"  
  }  
}  
  
resource "aws_instance" "myec2" {  
  ami = "ami-082b5a644766e0e6f"  
  instance_type = var.types["ap-south-1"]  
}
```



# Dealing with Larger Infrastructure

Cloud Providers have set a certain rate limit, so Terraform can only request a certain number of resources over a period of time.

It is important to break larger configurations into multiple smaller configurations that can be independently applied.

Alternatively, you can make use of `-refresh=false` and `target` flag for a workaround (not recommended)

# BackEnds

Backends primarily determine where Terraform stores its state.

By default, Terraform implicitly uses a backend called local to store state as a local file on disk.

If required, you can store state file in other backend as well.

```
terraform {  
  backend "s3" {  
    bucket = "mybucket"  
    key    = "path/to/my/key"  
    region = "us-east-1"  
  }  
}
```

# Points to Note - Initializing Backend

When configuring a backend for the first time (moving from no defined backend to explicitly configuring one), Terraform will give you the option to migrate your state to the new backend.

This lets you adopt new backends without losing any existing state.

# Local Backend

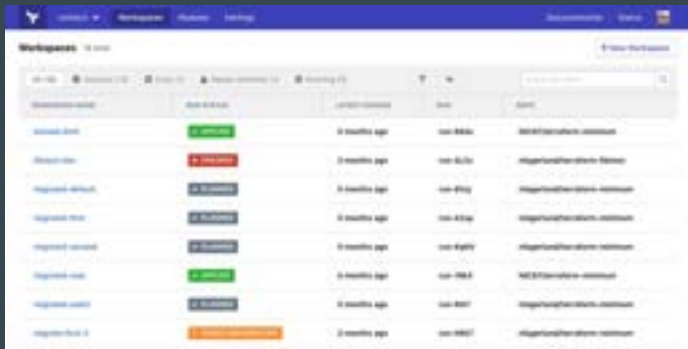
The local backend stores the state on the local filesystem, locks that state using system APIs, and performs operations locally.

By default, Terraform uses the "local" backend, which is the normal behavior of Terraform you're used to

```
terraform {  
  backend "local" {  
    path = "relative/path/to/terraform.tfstate"  
  }  
}
```

# Air Gapped Environments

An air gap is a network security measure employed to ensure that a secure computer network is physically isolated from unsecured networks, such as the public Internet.



Terraform Enterprise

Air Gap Install



Isolated Server

## Choose your installation type



Please choose an installation type to continue.

Continue >

# Requirements for Publishing Module in Registry

| Core Requirements                    | Description                                                                                                                                                                                         |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GitHub                               | The module must be on GitHub and must be a public repo. This is only a requirement for the public registry and not for private registry.                                                            |
| Named<br>terraform-<PROVIDER>-<NAME> | Example:<br><br>terraform-aws-ec2-instance                                                                                                                                                          |
| Repository description               | The GitHub repository description is used to populate the short description of the module.                                                                                                          |
| Standard module structure            | The module must adhere to the standard module structure. This allows the registry to inspect your module and generate documentation, track resource usage, parse submodules and examples, and more. |
| x.y.z tags for releases              | For example, v1.0.4 and 0.9.2                                                                                                                                                                       |

# Comments in Terraform

A comment is a text note added to the source code to provide explanatory information, usually about the function of the code.

```
/* This is Terraform Backend  
State file stored in S3.  
*/  
  
terraform {  
  backend "s3" {  
    bucket = "mybucket"  
    key    = "path/to/my/key"  
    region = "us-east-1"  
  }  
}  
  
# This is single line comment  
// This is single line comment
```



# Dynamic Blocks

Dynamic Block allows us to dynamically construct repeatable nested blocks, which are supported inside resource, data, provider, and provisioner blocks.

Overuse of Dynamic blocks can make configuration hard to read and maintain.

```
variable "sg_ports" {
  type      = list(number)
  description = "list of ingress ports"
  default   = [8200, 8201, 8300, 9200, 9500]
}

resource "aws_security_group" "dynamicsg" {
  name = "dynamic-sg"

  dynamic "ingress" {
    for_each = var.sg_ports
    iterator = port
    content {
      from_port = port.value
      to_port   = port.value
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

## Miscellaneous Pointers

GitHub is not the supported backend type in Terraform.

API and CLI access for Terraform Cloud can be managed through API tokens that can be generated from Terraform Cloud UI.

Terraform uses Parallelism to reduce the time it takes to create the resource. By default, this value is set to 10

# Code Formatting Recommended Practices

Indent two spaces for each nesting level

When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs:

```
ami           = "abc123"  
instance_type = "t2.micro"
```

# Miscellaneous Pointers

Terraform does not require **go** as a prerequisite.

Terraform providers are NOT always installed through the internet. There is a different offline approach for air-gapped systems.

Terraform and Terraform Provider **NEED NOT** have the same major version for compatibility.

# Sensitive Parameter

Adding sensitive parameter ensures that you do not accidentally expose this data in CLI output, log output.

The sensitive value will be present as part of the state file.

```
variable "sensitive_content" {  
  sensitive = true  
  default = "supersecretpassword"  
}  
  
resource "local_file" "foo" {  
  content = var.sensitive_content  
  filename = "password.txt"  
}
```



```
Terraform will perform the following actions:  
  
# local_file.foo will be created  
+ resource "local_file" "foo" {  
  + content = (sensitive value)  
  + content_base64sha256 = (known after apply)  
  + content_base64sha512 = (known after apply)  
  + content_md5 = (known after apply)  
  + content_sha1 = (known after apply)  
  + content_sha256 = (known after apply)  
  + content_sha512 = (known after apply)  
  + directory_permission = "0777"  
  + file_permission = "0777"  
  + filename = "password.txt"  
  + id = (known after apply)  
}
```

# Actions Forbidden When State File is Locked

When the state file is locked, the following actions are forbidden:

Running terraform apply or any other command that modifies the state file (e.g. terraform plan, terraform destroy, etc.)

Running terraform refresh, which updates the state file to reflect the current state of the infrastructure

terraform state [push, rm, mv]

# Miscellaneous Pointers

1. terraform.tfstate will NOT always match the current state infrastructure.

If you are making use of the GIT repository for committing terraform code, the .gitignore should be configured to ignore certain terraform files that might contain sensitive data.

Some of these can include:

terraform.tfstate file (this can include sensitive information)

\*.tfvars (may contain sensitive data like passwords)

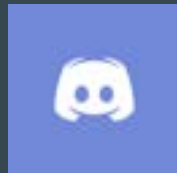
## Points to Note - State

1. If supported by your backend, Terraform will lock your state for all operations that could write state.
2. Not all backends support locking functionality.
3. Terraform has a force-unlock command to manually unlock the state if unlocking failed. *terraform force-unlock LOCK\_ID [DIR]*



# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)

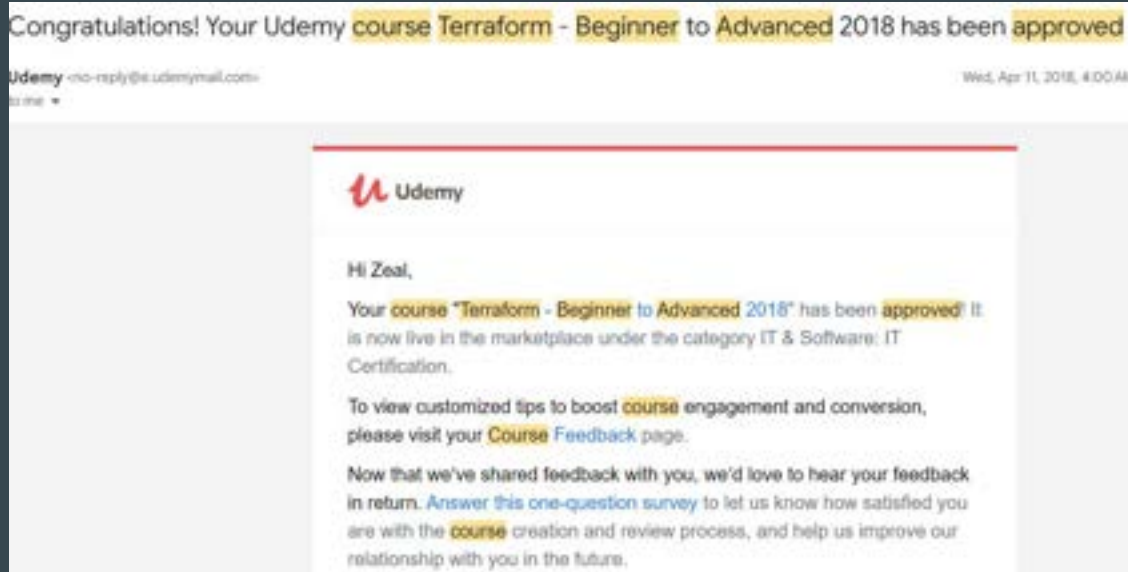


[kplabs.in/linkedin](https://kplabs.in/linkedin)

# **What's Next - Terraform Professional**

# Our First Terraform Course

We had launched our first Terraform course in 2018.



# Setting the Base

This course goes above and beyond to explain all the topics of Terraform Associate certification in a practical way that will help you in real world use-cases.

## HashiCorp Certified: Terraform Associate

All in One course for learning Terraform and gaining the official Terraform Associate Certification (003).

Bestseller

4.7 ★★★★★

(26,413 ratings)

123,030 students

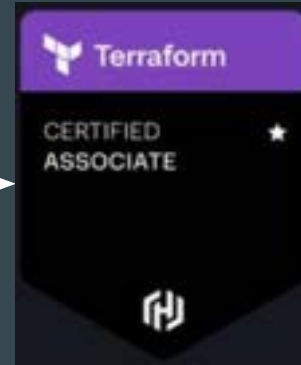
Created by [Zeal Vora](#)

# Terraform Associate Exam

**Terraform Associate** certification is primarily an MCQ-based exam and is relatively easier to pass even by memorizing the concepts.



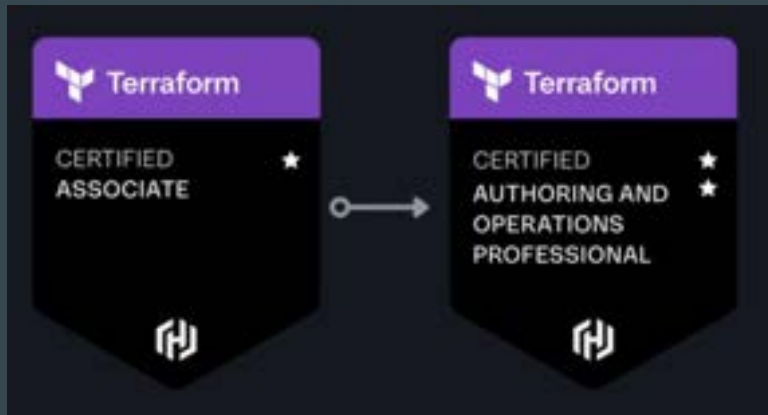
|                                  |
|----------------------------------|
| What are Providers in Terraform? |
| Option A                         |
| Option B                         |
| Option C                         |



Sample Question

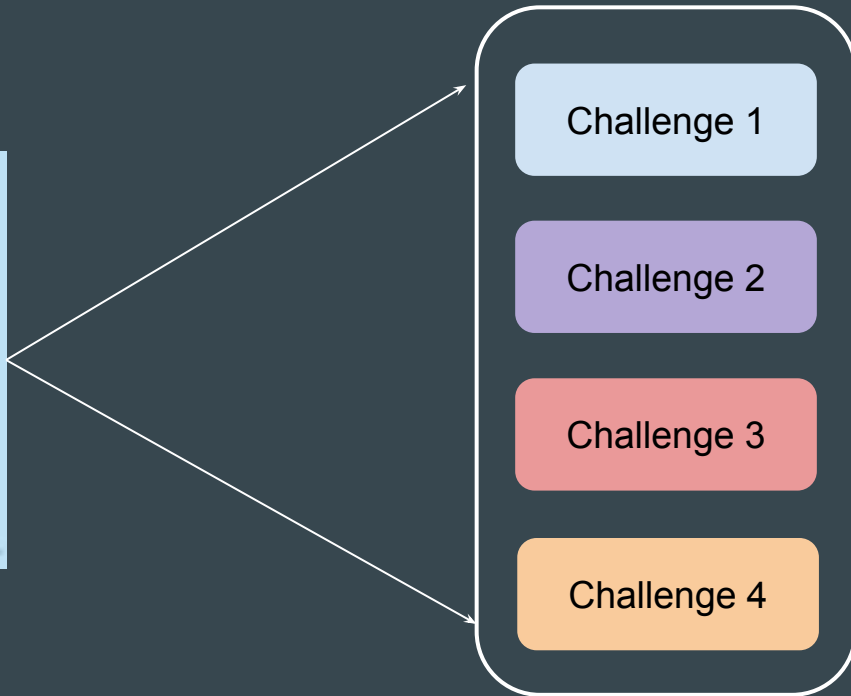
# Introducing Terraform Professional Certification

Terraform Authoring and Operations Professional exam assesses both **advanced configuration** authoring and a **deep understanding** of Terraform workflows.



# Lab Based Exams

Terraform Professional exam is a **4-hour intensive lab-based exam** where you will have to solve multiple scenarios presented to you.



# Our Terraform Professional Course

We are the first one to launch dedicated course on Terraform Professional certification exam.

This course covers many advance concepts in Terraform.



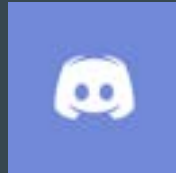


# Access Terraform Professional Course

<https://kplabs.in/tfpro>

# Join us in our Adventure

## Be Awesome



[kplabs.in/chat](https://kplabs.in/chat)



[kplabs.in/linkedin](https://kplabs.in/linkedin)